
Discrete Optimization

Alexandra Schwartz

TU Dresden
Fakultät Mathematik
Institut für Numerische Mathematik

`alexandra.schwartz@tu-dresden.de`

Lecture Notes
Summer Term 2022

Last changed: July 14, 2023

Preface

This lecture is an introduction to discrete optimization. We consider some famous examples of discrete optimization problems and introduce basic terminology in Chapter 1. Then we briefly recall important results and algorithms for linear programs in Chapter 2, before we focus on polyhedra for a while in Chapter 3 and develop useful representation. Afterwards, we turn our attention to (mixed-)integer linear programs in Chapter 4 and discuss several well known solution algorithms. Then we consider several important optimization problems in graphs in Chapter 5 and develop efficient solution algorithms, which utilize the graph structure. Finally, we take a brief look at heuristic methods in Chapter 6.

The contents of these lecture notes can be found in most books on discrete or integer optimization such as [3, 4, 2] as well as other lecture notes. Their only claim to originality is that they contain exactly the topics covered in my lecture and are written in the notation I use.

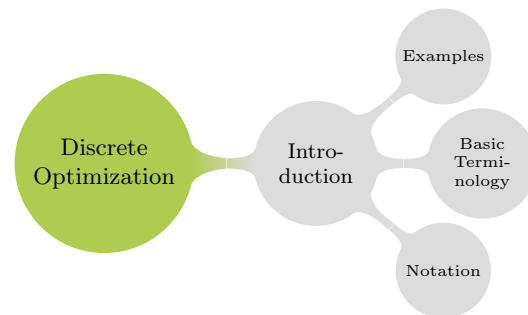


Contents

Preface	ii
1 Introduction	1
1.1 Examples	1
1.2 Basic Terminology	6
1.3 Notation	9
1.4 Exercises	10
2 Repetition: Linear Programs	11
2.1 Duality and Optimality Conditions	12
2.2 Primal and Dual Simplex Method	17
2.2.1 Primal Simplex Method	17
2.2.2 Dual Simplex Method	21
2.3 Reoptimization	25
2.4 Exercises	28
3 Polyhedral Geometry	29
3.1 Hulls and Dimension of Sets	31
3.2 Outer Representation of Polyhedra	37
3.3 Inner Representation of Polyhedra	50
3.4 Exercises	58
4 (Mixed-)Integer Linear Programs	61
4.1 Integer Linear Programs and Auxiliary Problems	62
4.1.1 The LP-Relaxation	62
4.1.2 The Integer Hull	65
4.1.3 Integral Polyhedra	70
4.2 Cutting Planes	77
4.2.1 Some Ideas for Generating Cuts	78
4.2.2 Cutting Plane Method	83
4.3 Branch&Bound Method	91
4.3.1 General Branch&Bound Method	92
4.3.2 Branch&Bound Method for (Mixed-)Integer Linear Programs	95
4.4 Branch&Cut Method	101

4.5	Application to the Traveling Salesperson Problem	103
4.5.1	Reduced Costs	104
4.5.2	Tailoring the Branch&Bound Method	105
4.6	Exercises	110
5	Optimization in Graphs	114
5.1	Repetition: Basic Terminology	114
5.1.1	Trees and Spanning Trees	119
5.1.2	Breadth-First Search and Depth-First-Search	121
5.2	Shortest Paths	124
5.2.1	Repetition: Dijkstra's Algorithm	126
5.2.2	Repetition: Algorithm of Bellman and Ford	128
5.2.3	Algorithm of Floyd and Warshall	131
5.3	Flow Problems in Networks	137
5.3.1	Maximum Flows and Minimum Cuts	142
5.3.2	Algorithm of Ford and Fulkerson	146
5.3.3	Algorithm of Goldberg-Tarjan	153
5.4	Exercises	161
6	Heuristics	163
6.1	Greedy Methods and Matroids	164
6.2	Local Search	172
6.3	Genetic Algorithms	175
6.4	Exercises	178

1 Introduction



In this chapter, we start by having a look at some prominent examples from the area of discrete optimization in Section 1.1. Then we introduce the general and some useful terminology in Section 1.2. Finally, some notation used throughout the lecture notes is collected in Section 1.3.

1.1 Examples

Before we introduce the class of discrete optimization problems formally, let us first have a look at some famous examples of such problems.

The Knapsack Problem A logistics company has to load a container. They have a number of items $i = 1, \dots, n$ to ship and each good i has an associated weight $w_i > 0$ and an associated importance $c_i > 0$, e.g. how fast it needs to reach its destination. However, the container can only handle a maximum load $W > 0$, which does not allow to pack all items into the container at once. How should the cargo company decide which items to load into the container?

If we introduce binary decision variables

$$x_i = \begin{cases} 1 & \text{if item } i \text{ is loaded into the container,} \\ 0 & \text{else,} \end{cases}$$

then the cargo company might be interested in solving the following optimization problem:

$$\max_{x_i} \sum_{i=1}^n c_i x_i \quad (\text{total importance of the load})$$

$$\begin{aligned} \text{s.t.} \quad & \sum_{i=1}^n w_i x_i \leq W \quad (\text{capacity constraint}), \\ & x_i \in \{0, 1\} \quad \forall i = 1, \dots, n \quad (\text{decision variables}). \end{aligned}$$

To formulate this problem a bit more compact, we can introduce the vectors $c = (c_1, \dots, c_n)^T$ and $w = (w_1, \dots, w_n)^T$ as well as the variable vector $x = (x_1, \dots, x_n)^T$. Furthermore, it is also possible to consider scenarios, where more than one copy of one good i exists and thus we can choose $x_i \in \mathbb{N}_0$. Or one can think of scenarios, where goods are arbitrarily divisible, e.g. salt. Thus, instead of working with $\{0, 1\}^n$ as feasible set for x , we can also work with a more general set $D \subseteq [0, \infty)^n$. This leads to the more general formulation of the *knapsack problem* (*Rucksack-Problem*) as

$$\max_x \quad c^T x \quad \text{s.t.} \quad w^T x \leq W, \quad x \in D.$$

Some special cases of this problem are

- *continuous (kontinuierliches) knapsack problem* with $D = [0, \infty)^n$,
- *integer (ganzzahliges) knapsack problem* with $D = \mathbb{N}_0^n$,
- *0-1 knapsack problem* with $D = \{0, 1\}^n$.

In case there is more than one constraint, the problem is called *multi-dimensional knapsack problem*. In our example, it might make sense to include a volume constraint in addition to the weight constraint.

The Facility Location Problem A company has customers $j = 1, \dots, m$ in different locations. To serve these customers, e.g. to deliver pizza, the company wants to open new stores. For these stores, they have already identified a number of possible locations $i = 1, \dots, n$. Building a new store in location i requires an investment of $c_i > 0$. Delivering the whole demand of customer j from location i generates costs of $d_{ij} > 0$. Where should the company build new stores?

In order to model this situation, we can introduce the following variables:

$$\begin{aligned} x_i &= \begin{cases} 1 & \text{if a store is built in location } i, \\ 0 & \text{else,} \end{cases} \\ y_{ij} &= \text{the fraction of the demand of customer } j \text{ delivered from location } i. \end{aligned}$$

Assuming that the company wants to minimize costs, we can then write down the following optimization problem:

$$\min_{x_i, y_{ij}} \quad \sum_{i=1}^n c_i x_i + \sum_{i=1}^n \sum_{j=1}^m d_{ij} y_{ij} \quad (\text{total cost})$$

$$\begin{aligned}
 \text{s.t.} \quad & \sum_{i=1}^n y_{ij} = 1 \quad \forall j = 1, \dots, m && \text{(whole demand of each customer is satisfied),} \\
 & x_i \in \{0, 1\} \quad \forall i = 1, \dots, n && \text{(decision variables),} \\
 & y_{ij} \geq 0 \quad \forall i = 1, \dots, n, j = 1, \dots, m && \text{(nonnegativity constraint),} \\
 & y_{ij} \leq x_i \quad \forall i = 1, \dots, n, j = 1, \dots, m && \text{(delivery only from built stores).}
 \end{aligned}$$

Here, we minimize over the sum of the building and delivery costs. The first constraint ensures that the demand of all customers is satisfied, which implicitly also ensures that at least one store is built. The last two constraints ensure that the customers do not act as resellers and that deliveries only occur from locations, where a store is being built.

This version of the optimization problem is called the *uncapacitated facility location problem* (etwa: *nicht kapazitiertes Standortsplanungsproblem*), because the stores built in each location are assumed to be able to deliver arbitrary amounts. It is further an example of a *mixed-integer problem* (*gemischt-ganzzahliges Problem*), because some variables have to be integer while others are continuous.

If it is not possible to split the demand of a customer into parts that can be delivered from different locations, e.g. in case of a pizza delivery, we should replace the constraint $y_{ij} \geq 0$ by the constraint $y_{ij} \in \{0, 1\}$ to ensure that each customer gets their whole delivery from one location. Then the problem becomes a purely discrete optimization problem.

In many situations, it is more realistic to include a capacity constraint for each location, i.e. to limit the amounts of goods that can be delivered from there. In this case, we have to change our model a little bit, because at the moment the demand of each customer is normalized to 1 and the actual size is hidden in the delivery costs. The resulting problem is then called *capacitated facility location problem*.

The Traveling Salesperson Problem A group of tourists wants to visit the cities $S_1, \dots, S_n$ starting in S_1 . They want to visit each city exactly once and return to S_1 at the end of their tour. In order to save time, they want to minimize the total time spent traveling from one city to the next.

In order to model this, consider the set

$$E = \{(i, j) \mid \text{there exists a direct connection from } S_i \text{ to } S_j\}$$

of all possible connections. Note that $(i, j) \in E$ does not imply $(j, i) \in E$, i.e. connections can be unidirectional. For each connection (i, j) let $c_{ij} > 0$ describe the time needed to travel from S_i to S_j .

If we introduce binary decision variables

$$x_{ij} = \begin{cases} 1 & \text{connection } (i, j) \text{ is used on the tour,} \\ 0 & \text{else,} \end{cases}$$

for all connections $(i, j) \in E$, then we can try to model the problem as follows:

$$\begin{aligned}
 \min_{x_{ij}} \quad & \sum_{(i,j) \in E} c_{ij} x_{ij} \quad (\text{total travel time}) \\
 \text{s.t.} \quad & \sum_{i=1}^n x_{ij} = 1 \quad \forall j = 1, \dots, n \quad (\text{each city is reached exactly once}), \\
 & \sum_{j=1}^n x_{ij} = 1 \quad \forall i = 1, \dots, n \quad (\text{each city is left exactly once}), \\
 & x_{ij} \in \{0, 1\} \quad \forall i, j = 1, \dots, n \quad (\text{decision variables}).
 \end{aligned}$$

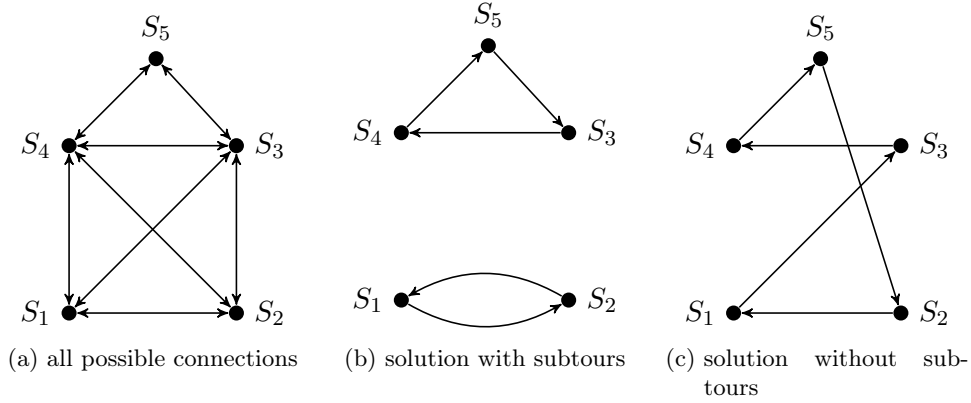


Figure 1.1: Illustration of the possibility of subtours in the traveling salesperson problem

However, if we use this formulation, it might happen that we do not end up with one roundtour, but instead several smaller and disjoint roundtours, see Figure 1.1 for an illustration. In order to prevent this, we need to include additional constraints, so called *subtour elimination constraints*. One possibility to achieve this, is to demand that every true subset of cities, that is not empty or the set of all cities, is left at least once. We actually use this type of condition already to ensure that every individual city is reached and left exactly once. To model this, let us use the notation

$$V := \{1, \dots, n\}.$$

Then we can write down the additional constraints

$$\sum_{(i,j) \in E: i \in U, j \in V \setminus U} x_{ij} \geq 1 \quad \forall U \subset V \text{ with } 2 \leq |U| \leq n-2.$$

With these constraints, we can formulate the problem, which is called *traveling salesperson problem* (*Handlungsreisendenproblem*) as a discrete optimization problem:

$$\begin{aligned}
 \min_{x_{ij}} \quad & \sum_{(i,j) \in E} c_{ij} x_{ij} \quad (\text{total travel time}) \\
 \text{s.t.} \quad & \sum_{i=1}^n x_{ij} = 1 \quad \forall j = 1, \dots, n \quad (\text{each city is reached exactly once}), \\
 & \sum_{j=1}^n x_{ij} = 1 \quad \forall i = 1, \dots, n \quad (\text{each city is left exactly once}), \\
 & \sum_{(i,j) \in E: i \in U, j \in V \setminus U} x_{ij} \geq 1 \quad \forall U \subset V : 2 \leq |U| \leq n-2 \quad (\text{subtour elimination}).
 \end{aligned}$$

Unfortunately, the number of subtour elimination constraints needed in this formulation grows exponentially with the number n of cities. But there also exist other possible formulations, that require less constraints.

The Linear Assignment Problem Assume that the department of mathematics has to plan courses $i = 1, \dots, n$ for the next term and has teachers $j = 1, \dots, n$ available to teach these courses. Every teacher j has a certain qualification $c_{ij} > 0$ to teach course i and should teach exactly one course. Which teacher should be assigned to which course?

If we introduce binary decision variables

$$x_{ij} = \begin{cases} 1 & \text{teacher } j \text{ teaches course } i, \\ 0 & \text{else,} \end{cases}$$

then we obtain the following optimization problem

$$\begin{aligned}
 \max_{x_{ij}} \quad & \sum_{i=1}^n \sum_{j=1}^n c_{ij} x_{ij} \quad (\text{total qualification}) \\
 \text{s.t.} \quad & \sum_{i=1}^n x_{ij} = 1 \quad (\text{only one course per teacher}) \\
 & \sum_{j=1}^n x_{ij} = 1 \quad (\text{every course is assigned}), \\
 & x_{ij} \in \{0, 1\} \quad \forall i, j = 1, \dots, n \quad (\text{decision variables}).
 \end{aligned}$$

This problem is called *linear assignment problem* (*lineares Zuordnungsproblem*) and is a linear program with binary variables x_{ij} .

The Quadratic Assignment Problem In an office n employees $i = 1, \dots, n$ have to be assigned to n offices $k = 1, \dots, n$. Employee i has to meet employee j in person $c_{ij} > 0$ times per day. For all offices k, l we know the distance $d_{kl} > 0$ between the two rooms. The offices should be assigned such that the total walking distance of all employees per day is as low as possible.

To model this, we introduce the decision variable

$$x_{ik} = \begin{cases} 1 & \text{if employee } i \text{ is assigned office } k, \\ 0 & \text{else.} \end{cases}$$

Then we can model the situation with the following optimization problem:

$$\begin{aligned} \min_{x_{ik}} \quad & \sum_{i,j=1}^n \sum_{k,l=1}^n 2c_{ij}d_{kl}x_{ik}x_{jl} \quad (\text{total walking distance of all employees } i) \\ \text{s.t.} \quad & \sum_{k=1}^n x_{ik} = 1 \quad \forall i = 1, \dots, n \quad (\text{employee } i \text{ is assigned one room}), \\ & \sum_{i=1}^n x_{ik} = 1 \quad \forall k = 1, \dots, n \quad (\text{room } k \text{ is assigned only once}), \\ & x_{ik} \in \{0, 1\} \quad \forall i, k = 1, \dots, n \quad (\text{decision variables}). \end{aligned}$$

In contrast to the previous examples, this *quadratic assignment problem* (*quadratisches Zuordnungsproblem*) is not a linear program with discrete variables but instead a quadratic program with discrete variables, because in the objective function we consider products $x_{ik}x_{jl}$ of variables.

An alternative formulation of this problem is possible by using the fact that we have to assign n persons to n rooms. If we number both the people and the persons from 1 to n , then we are looking for a permutation $\pi : \{1, \dots, n\} \rightarrow \{1, \dots, n\}$, that assigns each person i the room $\pi(i)$. We are then minimizing the objective function

$$f(\pi) = \sum_{i=1}^n \sum_{j=1}^n 2c_{ij}d_{\pi(i)\pi(j)}$$

over all possible permutations.

1.2 Basic Terminology

In general an *optimization problem* (*Optimierungsproblem*) in $\mathbb{R}^n$ is of the form

$$\min_x f(x) \quad \text{s.t.} \quad x \in X$$

with

- *variables (Variablen)* $x \in \mathbb{R}^n$,
- an *objective function (Zielfunktion)* $f : \mathbb{R}^n \rightarrow \mathbb{R}$,
- and a *feasible set (zulässige Menge)* $X \subseteq \mathbb{R}^n$, which is often described by *constraints (Restriktionen)*, for example

$$X = \{x \in \mathbb{R}^n \mid g(x) \leq 0, \quad h(x) = 0\}.$$

In case $X = \mathbb{R}^n$, the problem is called an *unconstrained (unrestringiertes)* optimization problem, otherwise a *constrained (restringiertes)* optimization problem.

The goal is to minimize the objective function f on the feasible set X , i.e. to find a *feasible point (zulässiger Punkt)* $x^* \in X$ with the property

$$f(x^*) \leq f(x) \quad \forall x \in X.$$

Such a feasible point is then called a *global minimum (globales Minimum)*. In continuous optimization, we often also work with the weaker concept of *local minima (lokale Minima)*, that only need to satisfy $x^* \in X$ and

$$f(x^*) \leq f(x) \quad \forall x \in X \cap B_r(x^*)$$

with some positive radius $r > 0$.

Without loss of generality, it suffices to consider only minimization problems. Every maximization problem

$$\max_x f(x) \quad \text{s.t.} \quad x \in X$$

can be transformed into the equivalent minimization problem

$$\min_x -f(x) \quad \text{s.t.} \quad x \in X.$$

The local and global extrema of both problems are attained in the same points, but the minima of one problem are the maxima of the other problem due to the changed sign of the objective function.

In contrast to continuous optimization, in *discrete optimization (Diskrete Optimierung)* we are interested in optimizations problems whose feasible set consists only isolated points, e.g. only of integer points. The feasible set of a discrete optimization problem is thus often of the form

$$X := \{x \in \mathbb{R}^n \mid g(x) \leq 0, \quad h(x) = 0\} \cap \mathbb{Z}^n.$$

Since all feasible points of a discrete optimization problem are isolated, concepts such as local optima or stationary points commonly used in continuous optimization are less useful. For example one easily verifies that every feasible point of a discrete problem is a local minimum according to the definition given above.

Some classes of optimization problems are collected below. A few of these we have already seen in the examples.

- *linear program (LP, lineares Program)*

$$\min_x c^T x \quad \text{s.t.} \quad Ax = b, \quad x \geq 0$$

- *quadratic program (QP, quadratisches Program)*

$$\min_x \frac{1}{2} x^T Q x + c^T x \quad \text{s.t.} \quad Ax = b, \quad x \geq 0$$

- *nonlinear program (NLP, nichtlineares Program)*

$$\min_x f(x) \quad \text{s.t.} \quad g(x) \leq 0, \quad h(x) = 0$$

- *integer linear program (ILP, ganzzahliges lineares Program)*

$$\min_x c^T x \quad \text{s.t.} \quad Ax = b, \quad x \geq 0, \quad x \in \mathbb{Z}^n$$

- *mixed-integer linear program (MILP, gemischt-ganzzahliges lineares Program)*

$$\min_x c^T x \quad \text{s.t.} \quad Ax = b, \quad x \geq 0, \quad x \in \mathbb{Z}^{n_1} \times \mathbb{R}^{n_2}$$

- *discrete optimization problem (DO, diskretes Optimierungsproblem)*

$$\min_x f(x) \quad \text{s.t.} \quad g(x) \leq 0, \quad h(x) = 0, \quad x \in D$$

with a discrete set $D \subset \mathbb{R}^n$, e.g. $D = \mathbb{Z}^n$ or $D = \{0, 1\}^n$.

- *mixed-integer nonlinear program (MINLP, gemischt-ganzzahliges nichtlineares Program)*

$$\min_x f(x) \quad \text{s.t.} \quad g(x) \leq 0, \quad h(x) = 0, \quad x \in \mathbb{Z}^{n_1} \times \mathbb{R}^{n_2}$$

In this lecture, we mostly consider (mixed-)integer linear programs. A common assumption here is that the problem data c, A, b consists of rational numbers. Besides the fact that computers only work with rational numbers and that we can in general not expect problems with irrational data to have integer solutions, this assumption is also needed for some theoretical results. We will come back to this point later.

1.3 Notation

- We work with the real numbers $\mathbb{R}$, the natural numbers $\mathbb{N}$ starting with 1, the integer numbers $\mathbb{Z}$ and the rational numbers $\mathbb{Q}$.
- When rounding to an integer, we use the *floor* and *ceiling functions*

$$\begin{aligned}\lfloor x \rfloor &:= \max\{z \in \mathbb{Z} \mid z \leq x\}, \\ \lceil x \rceil &:= \min\{z \in \mathbb{Z} \mid z \geq x\}.\end{aligned}$$

- We use several norms for vectors in $\mathbb{R}^n$. In case a norm $\|x\|$ is not further specified, it is an arbitrary norm on $\mathbb{R}^n$. Some norm we use regularly are

$$\begin{aligned}\|x\|_2 &:= \sqrt{\sum_{i=1}^n x_i^2} && \text{(Euclidean norm),} \\ \|x\|_1 &:= \sum_{i=1}^n |x_i| && \text{(Manhattan norm),} \\ \|x\|_\infty &:= \max_{i=1,\dots,n} |x_i| && \text{(maximum norm).}\end{aligned}$$

With respect to a given norm $B_r(x^*)$ denotes the open ball with center x^* and radius $r > 0$.

- Let $A \in \mathbb{R}^{m \times n}$ be a matrix. Then we denote the individual elements of the matrix A as $a_{ij} \in \mathbb{R}$. We denote the i -th row of A as A_{i*} and the j -th columns as A_{*j} . For subsets $I \subseteq \{1, \dots, m\}$ and $J \subseteq \{1, \dots, n\}$ we denote the corresponding submatrix as $A_{IJ} \in \mathbb{R}^{|I| \times |J|}$.
- Let $A \in \mathbb{R}^{m \times n}$ be a matrix. Then we note the *kernel* (*Kern*) or *nullspace* of A as

$$\ker A = \{x \in \mathbb{R}^n \mid Ax = 0\}$$

and the *rank* (*Rang*) of A , i.e. the maximum number of linear independent rows or columns of A (or the dimension of the image of A) as $\text{rank } A$. We occasionally use the relation

$$n = \text{rank}(A) + \dim(\ker A).$$

- For two sets $X, Y \subseteq \mathbb{R}^n$ and a scalar $c \in \mathbb{R}$ we define the *Minkowski sum* of X and Y and the multiple of X as

$$\begin{aligned}X + Y &:= \{x + y \mid x \in X, y \in Y\}, \\ c \cdot X &:= \{c \cdot x \mid x \in X\}.\end{aligned}$$

Here, we use the convention $X + \emptyset = \emptyset$ and $c \cdot \emptyset = \emptyset$. In case one of the two sets consists only of one element, e.g. $X = \{x\}$, we also use the notation $x + Y := \{x\} + Y$.

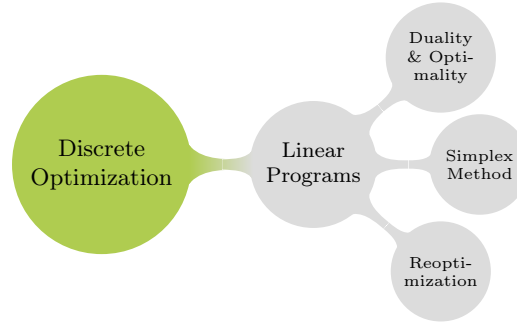
- For a set $X \subseteq \mathbb{R}^n$ we define the *interior* (*Inneres*), *closure* (*Abschluss*) and *boundary* (*Rand*) as

$$\begin{aligned} \text{int } X &:= \{x \in X \mid \exists r > 0 \mid B_r(x) \subseteq X\}, \\ \overline{X} &:= \{y \in \mathbb{R}^n \mid \exists (x^k)_k \rightarrow_X y\}, \\ \partial X &:= \overline{X} \setminus \text{int } X. \end{aligned}$$

A point $y \in \mathbb{R}^n$ is called an *accumulation point* (*Häufungspunkt*) of X , if for all $r > 0$ the ball $B_r(y)$ contains at least one point $x \in X \setminus \{y\}$. A point $x \in X$ is called an *isolated point* (*isolierter Punkt*) of X , if there exists an $r > 0$ such that $B_r(x) \cap X = \{x\}$.

1.4 Exercises

2 Repetition: Linear Programs



In this chapter, we only consider linear programs in *normal form*

$$\min_x c^T x \quad \text{s.t.} \quad Ax = b, x \geq 0 \quad (2.1)$$

with $c \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. here, the feasible set is given by the polyhedron

$$P^=(A, b) := \{x \in \mathbb{R}^n \mid Ax = b, x \geq 0\}.$$

Since this problem has a linear objective function and linear constraints, it is a convex optimization problem. We are thus looking for global minima, i.e. points $x^* \in P^=(A, b)$ with

$$c^T x^* \leq c^T x \quad \forall x \in P^=(A, b).$$

It suffices to consider linear programs of this form, because all linear programs can be transformed into normal form using the following transformations:

- *Slack variables*: Linear inequality constraints can be transformed into equality constraints by introducing an auxiliary variable, called a *slack variable* (*Schlupfvariable*).

$$a^T x \leq \beta \quad \Longleftrightarrow \quad a^T x + s = \beta, s \geq 0.$$

- *Splitting variables*: Free variables, i.e. variables without a sign constraint, can be split into two sign-constrained variables.

$$x_i \in \mathbb{R} \quad \Longleftrightarrow \quad x_i = x_i^+ - x_i^-, x_i^+, x_i^- \geq 0.$$

- *Changing signs:* If the objective function is supposed to be maximized or there are inequalities of the form $a^T x \geq \beta$, we can modify the respective terms by multiplying them with -1 .

In Section 2.1 we briefly recall the dual problem and important optimality conditions for linear programs. Then we recall the primal and dual simplex method in Section 2.2. To close this chapter on linear programs, we discuss how we can efficiently reoptimize after the linear program has changed in Section 2.3.

2.1 Duality and Optimality Conditions

To find a lower bound for the optimal function value of (2.1) we consider vectors $y \in \mathbb{R}^m$ with the property $A^T y \leq c$. For each such vector y and for all points x feasible for (2.1) we then have

$$\underbrace{c^T}_{\geq y^T A} \underbrace{x}_{\geq 0} \geq y^T A x = y^T b.$$

The solution of the linear program

$$\max_y \quad b^T y \quad \text{s.t.} \quad A^T y \leq c, \quad y \in \mathbb{R}^m \quad (2.2)$$

thus provides the largest lower bound for the optimal function value of (2.1), that we can obtain with this approach. We call (2.2) the *dual problem* to (2.1) and the original problem (2.1) the *primal problem*. With our derivation of the dual problem we have proven the following result:

Lemma 2.1. (Weak Duality) Let x be feasible for the primal problem (2.1) and y be feasible for the dual problem (2.2). Then it holds that

$$b^T y \leq c^T x.$$

In case of equality it follows that x is a global solution of the primal problem and y is a global solution of the dual problem.

The dual problem thus is a useful tool when working with linear programs. Since the dual problem is itself a linear program, we can rewrite it into normal form and consider the dual problem of the dual problem. However, it turns out that the resulting linear program is equivalent to the original primal problem again. This relation is often abbreviated as: the dual of the dual problem is the primal problem.

So far, we have only stated the dual problem for linear programs in normal form. This is sufficient, because every linear program can be rewritten in normal form. But with

some exercise one can also write down the dual problem directly without transforming the primal problem into normal form first.

Remark 2.2 (Optional: Direct path to the dual problem). *Consider the following example:*

$$\begin{aligned} \min_{x \in \mathbb{R}^4} c^T x \quad \text{s.t.} \quad & \begin{pmatrix} - & a_1^T & - \end{pmatrix} x = b_1, & \quad | & y_1 \\ & \begin{pmatrix} - & a_2^T & - \end{pmatrix} x \leq b_2, & \quad | & y_2 \\ & \begin{pmatrix} - & a_3^T & - \end{pmatrix} x \geq b_3, & \quad | & y_3 \\ & x_1 \geq 0, \quad x_2 \leq 0, \quad x_3, x_4 \in \mathbb{R}. \end{aligned}$$

This problem has three linear constraints, i.e. the dual problem has three variables y_1, y_2, y_3 . The objective function of the dual problem is the right hand side of the linear constraints, i.e. $b = (b_1, b_2, b_3)$. The linear constraint of the dual problem consists of the matrix $A^T = (a_1, a_2, a_3)$ and the right hand side c . So far, we know the dual problem has the following form:

$$\begin{aligned} \max_{y \in \mathbb{R}^3} b^T y \quad \text{s.t.} \quad & \begin{pmatrix} | & | & | \\ a_1 & a_2 & a_3 \\ | & | & | \end{pmatrix} y \quad ? \quad \begin{pmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{pmatrix}, \\ & \begin{pmatrix} y_1 \\ y_2 \\ y_3 \end{pmatrix} \quad ? \quad 0. \end{aligned}$$

To identify the sign constraints on y and the type of linear constraints in the dual problem, we have to take a closer look at the primal problem. The type of linear constraint in the primal problem determines the sign constraint on the corresponding dual variable y_i : An equality constraint results in a free variable $y_i \in \mathbb{R}$, a $\leq$ -inequality results in a variable $y_i \leq 0$ and a $\geq$ -inequality results in a variable $y_i \geq 0$. So now we know

$$\begin{aligned} \max_{y \in \mathbb{R}^3} b^T y \quad \text{s.t.} \quad & \begin{pmatrix} | & | & | \\ a_1 & a_2 & a_3 \\ | & | & | \end{pmatrix} y \quad ? \quad \begin{pmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{pmatrix}, & \begin{array}{l} | \quad x_1 \\ | \quad x_2 \\ | \quad x_3 \\ | \quad x_4 \end{array} \\ & y_1 \in \mathbb{R}, \quad y_2 \leq 0, \quad y_3 \geq 0. \end{aligned}$$

To determine the type of linear constraint in the dual problem, we associate each of these constraints with the corresponding primal variable x_j . The sign constraint on x_j then, similarly to the previous step but with a sign change, tells us the type of linear constraint

we need: For $x_j \in \mathbb{R}$ we obtain an equality constraint, for $x_j \geq 0$ we obtain a $\leq$ -inequality and for $x_j \leq 0$ we obtain a $\geq$ -inequality. This results in

$$\max_{y \in \mathbb{R}^3} b^T y \quad \text{s.t.} \quad \begin{pmatrix} | & | & | \\ a_1 & a_2 & a_3 \\ | & | & | \end{pmatrix} y \begin{matrix} \leq \\ \geq \\ = \\ = \end{matrix} \begin{pmatrix} c_1 \\ c_2 \\ c_3 \\ c_4 \end{pmatrix}, \quad \begin{matrix} | & x_1 \\ | & x_2 \\ | & x_3 \\ | & x_4 \end{matrix}$$

$$y_1 \in \mathbb{R}, y_2 \leq 0, y_3 \geq 0.$$

A closer look reveals that all signs are chosen such that for all x feasible for the primal problem and all y feasible for the dual problem we obtain the inequality known from weak duality

$$c^T x = \sum_{j=1}^n c_j x_j \geq \sum_{j=1}^n (A_{*,j}^T y) x_j = y^T A x = \sum_{i=1}^m y_i (A_{i,*} x) \geq \sum_{i=1}^m y_i b_i = b^T y.$$

Weak duality tells us that for all x feasible for the primal problem and all y feasible for the dual problem we have

$$b^T y \leq c^T x.$$

If we have equality for two feasible points x, y , those are solutions of the primal and dual problem. For linear problems, one can also prove the opposite direction, i.e. that the primal problem has a solution if and only if the dual problem has a solution and that, in this case, the optimal objective function values are equal. This is the essence of strong duality, which can be proven using the Farkas Lemma.

Theorem 2.3. (Strong Duality) For the primal problem (2.1) and the dual problem (2.2) the following statements are equivalent:

- (a) The primal and the dual problem both have a solution x^* and y^* .
- (b) The primal problem has a solution x^* .
- (c) The dual problem has a solution y^* .
- (d) There exist x^*, y^* feasible for the primal and dual problem with $c^T x^* = b^T y^*$.
- (e) The primal and the dual problem both have feasible points x and y .
- (f) The primal problem has a feasible point x and is bounded below.
- (g) The dual problem has a feasible point y and is bounded above.

The primal linear program (2.1) thus always belongs to one of the following three cases

- *Infeasible*: $P^=(A, b) = \emptyset$
- *Unbounded (below)*: $P^=(A, b)$ is unbounded and there exists a sequence $(x^k)_k \subset P^=(A, b)$ with $c^T x^k \rightarrow -\infty$.
- *Has a solution*: $P^=(A, b) \neq \emptyset$ and there exists a global minimum $x^* \in P^=(A, b)$.

Analogously the dual problem is either infeasible, unbounded (above) or has a solution. Only the following combinations are possible:

	primal infeasible	primal has a solution	primal unbounded
dual infeasible	yes	no	yes
dual has a solution	no	yes	no
dual unbounded	yes	no	no

For a pair of primal/dual feasible points x, y with $c^T x = b^T y$ it follows from

$$c^T x = \sum_{x_j > 0} \underbrace{c_j}_{\geq A_{*,j}^T y} \cdot \underbrace{x_j}_{> 0} + \sum_{x_j = 0} \underbrace{c_j}_{\geq A_{*,j}^T y} \cdot 0 = y^T A^T x = y^T b,$$

that $(A^T y)_j = c_j$ has to hold for all j with $x_j > 0$. This condition combined with primal and dual feasibility results in the so-called KKT conditions for linear programs.

Theorem 2.4. (KKT Conditions for LPs) For the primal problem (2.1) and the dual problem (2.2) the following are equivalent:

- x^* solves the primal problem.
- y^* solves the dual problem.
- (x^*, y^*) solve the *KKT system*:

$$\begin{aligned} Ax = b, \quad x \geq 0, & \quad (\text{primal feasibility}) \\ A^T y \leq c, & \quad (\text{dual feasibility}) \\ \forall j = 1, \dots, n : x_j = 0 \text{ oder } (A^T y)_j = c_j, & \quad (\text{complementarity}) \end{aligned}$$

The following observation is useful to compute solutions of the primal problem.

Lemma 2.5. If the primal problem (2.1) has a solution, there also exists a solution in an *extreme point* (*Extrempunkt*) of $P^=(A, b)$, i.e. in a point $x^* \in P^=(A, b)$, that cannot be written as a true convex combination of two points from $P^=(A, b)$.

A few remarks on this result:

- It only says that — if a solution of (2.1) exists — there also exists a solution in an extreme point of $P^=(A, b)$. It *does not* claim that solutions always exist or that all solutions are extreme points of $P^=(A, b)$.
- The result is only true for polyhedra of the form $P^=(A, b)$. For a counterexample with a different form of feasible set see e.g.

$$\min_{x \in \mathbb{R}^2} x_1 \quad \text{s.t.} \quad x_1 \geq 0, \quad x_2 \in \mathbb{R}.$$

Here, the polyhedron does not have any extreme points, but the linear problem still has a nonempty solution set.

- Extreme points can be defined for general convex sets. For polyhedra, we later define vertices (Ecken) as the 0-dimensional faces (Seitenflächen) of the polyhedron and see that the extreme points of a polyhedron are exactly its vertices. This is the reason why we use the terms “extreme point” and “vertex” interchangeably for polyhedra.

But keep in mind that vertices are only defined for polyhedra and not for general convex sets.

We know that the extreme points of the polyhedron $P^=(A, b)$ can be characterized as follows.

Lemma 2.6. For a polyhedron of the form $P^=(A, b)$ the following are equivalent:

- (a) x^* is an extreme point of $P^=(A, b)$.
- (b) $x^* \in P^=(A, b)$ and for the set $I := \{i \mid x_i^* > 0\}$ the matrix $A_{*,I}$ has full column rank.

This characterization of extreme point is only correct for polyhedra of the form $P^=(A, b)$. For polyhedra of the form $P(A, b) = \{x \mid Ax \leq b\}$ we will derive a different characterization of extreme points later.

The extreme points of $P^=(A, b)$ can thus be computed using bases and basic solutions. And since a polyhedron can have only finitely many different bases/basic solutions, we can then iterate over those (in a smart order) to find a solution of the primal problem.

Definition 2.7. Consider the polyhedron $P^=(A, b) = \{x \mid Ax = b, x \geq 0\}$ and assume that the matrix $A \in \mathbb{R}^{m \times n}$ has *full row rank* m .

- A set $B \subseteq \{1, \dots, n\}$ with $|B| = m$ and $A_B := A_{*,B} \in \mathbb{R}^{m \times m}$ regular is called *basis* (*Basis*). The set $N := \{1, \dots, n\} \setminus B$ is called *nonbasis* (*Nichtbasis*) and the matrix A_B the *basic matrix* (*Basismatrix*).
- If B is a basis, the vector $x = (x_B, x_N)$ with $x_B := A_B^{-1}b$ and $x_N := 0$ is called the corresponding *basic solution* (*Basislösung*).
- If B is a basis and $x_B = A_B^{-1}b \geq 0$, then B is called a *primal feasible basis* (*primal zulässige Basis*).
- If even $x_B = A_B^{-1}b > 0$ holds, the basic solution is called *nondegenerate* (*nichtdegeneriert*).
- If B is a basis and $A_N^T A_B^{-T} c_B \leq c_N$, the basis is called *dual feasible* (*dual zulässig*).

If the matrix A has full row rank, the following holds: A point x^* is an extreme point of $P^=(A, b)$ if and only if there exists a primal feasible basis B , such that x^* is the corresponding basic solution. If the matrix A does not have full row rank, then no bases exist. But the polyhedron $P^=(A, b)$ can still have extreme points.

If a basic solution is degenerate, i.e. has more than $n - m$ zero components, there can be more than one primal feasible basis B , that results in the same basic solution x^* . This effect can cause trouble in the simplex method.

2.2 Primal and Dual Simplex Method

Since we know that — if the primal problem (2.1) has a solution — it also has a solution in an extreme point and that — if the matrix A has full row rank — we can characterize these finitely many extreme points with bases, we can now formulate the basic idea of the simplex method as follows: Find a feasible basic solution. If this basic solution is not optimal, jump to a neighboring basic solution with a better objective function value. The details how to find an initial feasible basic solution, how to check for optimality and how to find a neighboring basic solution are described below.

2.2.1 Primal Simplex Method

We will describe a way to obtain an initial primal feasible basis at the end of this section. For now, assume that a primal feasible basis B is known and compute the corresponding basic solution $\hat{x} = (\hat{x}_B, \hat{x}_N)$ with

$$\hat{x}_B = A_B^{-1}b, \quad \hat{x}_N = 0.$$

For all other feasible points $x = (x_B, x_N)$ with $x_N = 0 = \hat{x}_N$ we know

$$Ax = A_B x_B + A_N x_N = A_B x_B = b$$

and thus $x_B = \hat{x}_B$, i.e. $x = \hat{x}$. If we want to leave the basic solution $\hat{x}$, we consequently have to choose nonbasic variables $x_N \neq 0$. This then results in the following change of the basic variables x_B :

$$Ax = A_B x_B + A_N x_N = b \implies x_B = A_B^{-1}b - A_B^{-1}A_N x_N = \hat{x}_B - A_B^{-1}A_N x_N.$$

Then we can express the resulting change in the objective function depending on x_N as

$$c^T x = c_B^T x_B + c_N^T x_N = c_B^T \hat{x}_B + (c_N - A_N^T A_B^{-T} c_B)^T x_N. \quad (2.3)$$

Let us define the two auxiliary variables¹

$$y := A_B^{-T} c_B \quad \text{and} \quad z_N := c_N - A_N^T y.$$

We have defined the variable y such that it has the property

$$A_B^T y = c_B,$$

i.e. it satisfies the complementarity condition together with the primal variable $\hat{x} = (\hat{x}_B, 0)$. If additionally we have

$$A_N^T y \leq c_N \iff z_N \geq 0,$$

then y is feasible for the dual problem and we have found solutions $\hat{x}$ und y of the primal and dual problem, see Theorem 2.4. Otherwise, there exists at least one index $j \in N$, for which the *reduced costs* (*reduzierte Kosten*) z_j are negative. Increasing the corresponding nonbasic variable x_j to a positive value then improves the objective function value due to (2.3).

In order to decrease the objective function value as much as possible, we want to choose x_j as large as possible. Since we still have to remain feasible for the primal problem, we choose $x_N = \gamma e_j$ with $\gamma > 0$ as large as possible such that we still have

$$x_B = \hat{x}_B - A_B^{-1} A_N \gamma e_j = \hat{x}_B - \gamma A_B^{-1} A_{*,j} \geq 0.$$

We introduce a new auxiliary variable²

$$w_B = A_B^{-1} A_{*,j}.$$

¹For every nonbasic variable x_j with $j \in N$ we define corresponding reduced costs z_j . We do not define reduced costs for the basic variables. But if you prefer to have a full vector $z = (z_B, z_N) \in \mathbb{R}^n$, you can set z_B arbitrarily.

²For each basic variable x_i with $i \in B$ we define a corresponding auxiliary variable w_i . For the nonbasic variables we do not define these auxiliary variables. But if you prefer to have a full vector $w = (w_B, w_N) \in \mathbb{R}^n$, you can set w_N arbitrarily.

In case $w_B \leq 0$ we can choose $\gamma > 0$ arbitrarily large and thus decrease the objective function value arbitrarily. The linear program then is unbounded. Otherwise the largest value we can choose for γ is

$$\gamma = \min \left\{ \frac{\hat{x}_i}{w_i} \mid i \in B, w_i > 0 \right\}.$$

Let $i \in B$ be an index, where this minimum is attained. Then one can show that the new point

$$x_B^+ := \hat{x}_B - \gamma w_B, \quad x_N^+ = \gamma e_j$$

is a basic solution with the basis

$$B^+ = (B \setminus \{i\}) \cup \{j\}, \quad N^+ = (N \setminus \{j\}) \cup \{i\}.$$

This results in the primal simplex method, see Algorithm 2.8.

Algorithm 2.8. (Primal Simplex Method)

Input: $A \in \mathbb{R}^{m \times n}$ with full row rank, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$ and a primal feasible basis B .

Output: A solution x^* or the information that the problem is unbounded.

Compute the basic solution $x = (A_B^{-1}b, 0)$.

Compute the complementary variable $y = A_B^{-T}c_B$.

Compute the reduced costs $z_N = c_N - A_N^T y$.

while there exists a $j \in N$ with $z_j < 0$ **do**

 Choose an index $j \in N$ with $z_j < 0$.

 Compute $w_B = A_B^{-1}A_{*,j}$.

if $w_B \leq 0$ **then**

return The LP is unbounded.

end if

 Compute

$$\gamma = \min \left\{ \frac{x_i}{w_i} \mid i \in B, w_i > 0 \right\}$$

 and choose an index $i \in B$, where the minimum is attained.

 Compute the new basis and basic solution

$$x_B \leftarrow x_B - \gamma w_B$$

$$x_N \leftarrow \gamma e_j$$

$$N \leftarrow (N \setminus \{j\}) \cup \{i\}$$

$$B \leftarrow (B \setminus \{i\}) \cup \{j\}$$

```

    Compute the complementary variable  $y = A_B^{-T} c_B$ .
    Compute the reduced costs  $z_N = c_N - A_N^T y$ .
end while
return  $x^* = (x_B, 0)$  is a solution.

```

Some remarks on the primal simplex method:

- If all occurring basic solutions are nondegenerate, the objective function value decreases in every iteration and an optimal solution is found after finitely many iterations. The number of iterations can be exponential (with respect to the dimension of the problem), but is generally not too large in practice³.
- In case of degenerate basic solutions, the method can start to cycle (kreiseln) and represent the same basic solution with alternating bases. This can be prevented by suitable rules for choosing the nonbasic variable j and basic variable i such as always choosing the one with the smallest possible index.
- In every iteration we have to solve two linear equations to compute y and w_B . If we implement the basis B as an ordered tuple $B = (B_1, \dots, B_m)$ and insert the entering index j in the spot where the leaving index i was before, then the matrix A_B only changes in one column from one iteration to the next. One can thus compute a LR-decomposition of A_B once and then update this decomposition inexpensively. (For stability purposes one should recompute the LR decomposition occasionally.)
- To find a primal feasible initial basis, we can first solve the following auxiliary problem with the simplex method:

$$\min_{x,s} \sum_{j=1}^m s_j \quad \text{s.t.} \quad Ax + s = b, \quad x \geq 0, s \geq 0.$$

This is called *phase 1* (*Phase 1*) of the simplex method. In case $b \geq 0$ (which can always be achieved by multiplying individual equalities with -1), we know that $x = 0, s = b$ is a primal feasible basic solution for the auxiliary problem. If in a solution (x^*, s^*) of the auxiliary problem we have $s^* \neq 0$, then the original problem

³There also exist solution algorithms for linear programs with polynomial runtime such as the ellipsoid method and the interior point method. However, while the interior point method also performs well in practice, the ellipsoid method does not. For this reason, linear programs are typically solved with either a simplex method or an interior point method, see e.g. the documentation of `linprog` in MATLAB.

is infeasible. If instead $s^* = 0$, then x^* is a primal feasible basic solution of the original problem, for which a basis B can be computed efficiently.

Using the approach described above requires the solution of two linear programs. one to determine the primal feasible base and one to minimize the objective function. An alternative approach called the *big-M method* (*big-M-Methode*) tries to combine both steps into one by solving the problem

$$\min_{x,s} f(x) + M \sum_{j=1}^m s_j \quad \text{s.t.} \quad Ax + s = b, \quad x \geq 0, s \geq 0$$

with a constant $M > 0$. One can show that, in case the original problem has a solution x^* , it is possible to choose M large enough such that $(x^*, 0)$ is a solution of the big-M problem. And since the big-M problem has the same feasible set as the phase 1 problem, it is easy to determine a primal feasible initial basis.

2.2.2 Dual Simplex Method

Consider again the primal problem

$$\min_x c^T x \quad \text{s.t.} \quad Ax = b, \quad x \geq 0$$

and assume that the matrix A has full row rank. Instead of solving the primal problem, we can also solve the corresponding dual problem

$$\max_y b^T y \quad \text{s.t.} \quad A^T y \leq c.$$

This can have the following benefits: The number of iterations of the simplex method depends on the number of extreme points of $P^=(A, b)$, which typically grows with the number of constraints m . In case of a linear program with few variables and many constraints solving the dual problem thus can be faster. Additionally, e.g. in discrete optimization, we sometimes does not only have to solve one LP, but a sequence of LPs, which are modified in a special way. For some such modifications, it is easier to “warm-start” the dual simplex method, see Section 2.3.

To solve the dual problem, we first reformulate it into *dual standard form*

$$\max_{y,s} b^T y \quad \text{s.t.} \quad A^T y + s = c, \quad s \geq 0$$

by introducing slack variables $s \in \mathbb{R}^n$. If we want to solve this problem with a simplex-type method, we have to handle that the variables y do not have a sign constraint. We could split the variables $y = y^+ - y^-$ into a positive and negative part and apply the primal simplex method. But this increases the number of variables unnecessarily. Instead we want to apply a simplex-type method directly to the dual problem.

To understand what bases for the dual problem look like, define $D = (A^T, I) \in \mathbb{R}^{n \times (m+n)}$. Then a basis is a set $H \subseteq \{1, \dots, n+m\}$ with $|H| = n$, such that the matrix $D_H \in \mathbb{R}^{n \times n}$ is regular. If we assume that A has full row rank, the first m columns of D are linearly independent. One can show that it suffices to consider bases H containing all first m columns, i.e. of the form

$$H = \{1, \dots, m\} \cup (N + m)$$

with $N \subset \{1, \dots, n\}$ and $|N| = |H| - m = n - m$. The corresponding nonbasis is then of the form

$$M = \{1, \dots, m+n\} \setminus H = B + m$$

with $B \subset \{1, \dots, n\}$ and $|B| = n + m - |H| = m$. The corresponding basic matrix and nonbasic matrix are of the form

$$D_H = (A^T, I_{*,N}) = \begin{pmatrix} A_N^T & I_N \\ A_B^T & 0 \end{pmatrix} \quad \text{and} \quad D_M = I_{*,B} = \begin{pmatrix} 0 \\ I_B \end{pmatrix}.$$

Thus, the basic matrix D_H is regular if and only if the matrix A_B is regular.

Analogously to primal feasible bases we have to understand which bases H generate dual feasible basic solutions. To this end, we rewrite the constraints as

$$\begin{aligned} (A^T, I) \begin{pmatrix} y \\ s \end{pmatrix} = c, \quad s \geq 0 &\iff A_B^T y + s_B = c_B, \quad s_B \geq 0, \\ &A_N^T y + s_N = c_N, \quad s_N \geq 0, \\ &\iff y = A_B^{-T}(c_B - s_B), \quad s_B \geq 0, \\ &s_N = c_N - A_N^T A_B^{-T}(c_B - s_B), \quad s_N \geq 0, \end{aligned}$$

If (y, s) is a basic solution with basis H , then we know $s_B = 0$ and the feasibility conditions reduce to

$$s_N = c_N - A_N^T A_B^{-T} c_B \geq 0.$$

This was exactly our definition of a dual feasible basis B . Since all relevant bases H can be characterized via the set B , instead of working with H , we instead use the sets B, N already known from the primal simplex method. But keep in mind that in the dual simplex method the set B is the nonbasis.

Now we can apply the simplex method to the dual problem. In the first step, we have to compute the dual variable x to the basic solution (y, s) such that the complementarity condition is satisfied. Due to $s_N \geq 0$ and $s_B = 0$, we compute this complementary dual variable as

$$x_N = 0, \quad x_B = A_B^{-1} b$$

Using this, we can compute the reduced costs, i.e. the effect of the nonbasic variables s_B on the objective function, as

$$z_B = -x_B$$

Recall that the dual problem is a maximization problem. If the reduced costs are $z_B = -x_B \leq 0$, the objective function value cannot be increased any further and we have found a solution. This is the moment, when x is both complementary to (y, s) and primal feasible, i.e. the KKT conditions are satisfied. Otherwise choose $j \in B$ with $z_j > 0$, i.e. with $x_j < 0$. To determine how far we can increase s_j we define the auxiliary vectors

$$w := A_B^{-T} e_j \quad \text{and} \quad v_N := -A_N^T w.$$

If we increase s_j by γ , the basic variables change to

$$\begin{pmatrix} y^+ \\ s_N^+ \end{pmatrix} = \begin{pmatrix} y \\ s_N \end{pmatrix} - \gamma \begin{pmatrix} w \\ v_N \end{pmatrix}.$$

Since the basic variables y do not have a sign constraint, we only have to ensure

$$s_N^+ = s_N - \gamma v_N \geq 0.$$

In case $v_N \leq 0$, we can choose $\gamma \geq 0$ arbitrarily large, meaning the dual problem is unbounded (and the primal problem infeasible). Otherwise, we choose

$$\gamma = \min \left\{ \frac{s_i}{v_i} \mid i \in N, v_i > 0 \right\}$$

and an index $i \in N$, where the minimum is attained. Then we update analogously to the primal simplex method and obtain the dual simplex method, see Algorithm 2.9.

Algorithm 2.9. (Dual Simplex Method)

Input: $A \in \mathbb{R}^{m \times n}$ with full row rank, $b \in \mathbb{R}^m$, $c \in \mathbb{R}^n$ and a dual feasible basis B .

Output: A solution x^* of the primal problem or the information that the primal problem is infeasible.

Compute the basic solution $y = A_B^{-T} c_B$, $s_N = c_N - A_N^T y$, $s_B = 0$.

Compute the complementary variable $x_B = A_B^{-1} b$.

while there exists $j \in B$ with $x_j < 0$ **do**

 Choose an index $j \in B$ with $x_j < 0$.

 Compute $w = A_B^{-T} e_j$ and $v_N = -A_N^T w$.

if $v_N \leq 0$ **then**

return The dual problem is unbounded and the primal problem is infeasible.

end if

Compute

$$\gamma = \min \left\{ \frac{s_i}{v_i} \mid i \in N, v_i > 0 \right\}$$

and choose an index $i \in N$, where the minimum is attained.

Compute the new basis and basic solution

$$\begin{aligned} y &\leftarrow y - \gamma w \\ s_N &\leftarrow s_N - \gamma v_N \\ s_B &\leftarrow \gamma e_j \\ B &\leftarrow (B \setminus \{j\}) \cup \{i\} \\ N &\leftarrow (N \setminus \{i\}) \cup \{j\} \end{aligned}$$

Compute the complementary variable $x_B = A_B^{-1}b$.

end while

In the primal simplex method, we started with a primal feasible, but suboptimal point x , remained primal feasible in all iterations and terminated, once we reached optimality. We recognized that x became optimal, because the complementary variable y became dual feasible.

In the dual simplex method, we work with a dual feasible variable y and a complementary (and thus superoptimal), but primal infeasible point x . We terminate, once x is primal feasible.

As we have seen in the derivation of the primal and dual simplex method, we rely heavily on the fact that we have an equation

$$Ax = b \quad \text{or} \quad A^T y + s = c,$$

which allows us to determine basic solutions $(x_B, 0)$ or $(y, s_N, 0)$. The remaining constraints $x \geq 0$ or $s \geq 0$ are then taken care of when determining the stepsize γ . This observation can be used to develop variants of the simplex method for linear programs of the form

$$\min_x \quad c^T x \quad \text{s.t.} \quad Ax = b, \quad l \leq x \leq u,$$

where in addition to the known linear equation $Ax = b$ we have *box constraints* $l \leq x \leq u$ with $l \in (\mathbb{R} \cup \{-\infty\})^n$ and $u \in (\mathbb{R} \cup \{\infty\})^n$ such that $l \leq u$. In a primal simplex method for this type of problem, we would then have the basic variables $x_i \in [l_i, u_i]$ for $i \in B$ and two types of nonbasic variables $x_j = l_j$ for $j \in N_l$ and $x_j = u_j$ for $j \in N_u$.

2.3 Reoptimization

So far, we have considered only one given optimization problem

$$\min_x c^T x \quad \text{s.t.} \quad Ax = b, \quad x \geq 0 \quad (2.4)$$

with given data $c \in \mathbb{R}^n$, $A \in \mathbb{R}^{m \times n}$ with full row rank m and $b \in \mathbb{R}^m$ and computed a solution thereof with the primal or dual simplex method. In case this problem has a solution, both methods determine solutions x^* of the primal problem and (y^*, s^*) of the dual problem together with a basis B , which is primal and dual feasible due to

$$x_B^* = A_B^{-1}b \geq 0 \quad \text{and} \quad s_N^* = c_N - A_N^T A_B^{-T} c_B = c_N - A_N^T y^* \geq 0.$$

However, sometimes the optimization problem (2.4) changes, after we have solved it. Some possible reasons for this are:

- A constraint has been overlooked.
- A parameter was wrong or has changed.
- We are using the big-M method and have minimized the objective function $c^T x + M e^T s$, but need to increase the value of $M > 0$.
- We are solving a discrete optimization problem with the branch&bound method, see Section 4.3, and thus have to adjust the bounds on variables frequently.
- We are solving a discrete optimization problem with the cutting plane method, see Section 4.2, and thus have to insert new inequality constraints frequently.

Of course it would be good, if we did not have to start completely from scratch in this case, but instead could use the solution of (2.4) somehow to “warmstart” the simplex method for the modified problem.

Changes in the objective function Let us begin by considering a change in the objective function, i.e. let us replace the vector c by a new vector $\tilde{c} \in \mathbb{R}^n$. Since this does not affect the feasible set of the linear program, the solution x^* of (2.4) is still a primal feasible basic solution with basis B .

However, the reduced costs change to

$$\tilde{z}_N = \tilde{c}_N - A_N^T A_B^{-T} \tilde{c}_B.$$

In case $\tilde{z}_N \geq 0$, the old solution x^* remains a solution of the modified problem. Otherwise, we can use the primal feasible basis B and corresponding basic solution x^* to start the primal simplex method.

One can show that the optimal value function

$$p(c) := \inf_x \{c^T x \mid Ax = b, x \geq 0\}$$

depends continuously on the vector c in the interior of its domain

$$D_p = \{c \in \mathbb{R}^n \mid p(c) > -\infty\}.$$

Thus, a small change in c , which does not cause the problem to become unbounded, leads only to a small change in $p(c)$. In case the new solution $\tilde{x}$ also only changes a little compared to the old solution x^* , it can be computed efficiently by starting from x^* .

Changes in the right-hand side Let us consider a change in the right-hand side next, i.e. let us replace the vector b by a new vector $\tilde{b} \in \mathbb{R}^m$. Since this does not affect the feasible set of the dual problem, the solution (y^*, s^*) of the dual problem corresponding to (2.4) is still a dual feasible basic solution with basis B .

However the complementary variable changes to

$$\tilde{x}_B = A_B^{-1} \tilde{b}.$$

In case $\tilde{x}_B \geq 0$, the basis B is still optimal, the dual solution (y^*, s^*) remains unchanged and the primal solution changes to $\tilde{x} = (\tilde{x}_B, 0)$. Otherwise, we can use the dual feasible basis B and the corresponding basic solution (y^*, s^*) to start the dual simplex method.

Analogously to the previous case, the optimal value function

$$d(b) := \sup_y \{b^T y \mid A^T y \leq c\}$$

depends continuously on the vector b in the interior of its domain. Thus, it makes sense to start the reoptimization in (y^*, s^*) , because we hope to reach the new solution in few iterations.

Changes in the matrix If we change entries in the matrix A to $\tilde{A}$, we have to differentiate two cases:

In case we change entries of the basis matrix A_B , both x_B and (y_B, s_N) are affected. Thus, it is hard to salvage information from the prior solution. In the worst case, B is not even a basis anymore, because the changed entries can cause the matrix A_B to become singular.

If all changed entries are elements of the nonbasis matrix A_N , then x^* remains a feasible basic solution and can be used to start the primal simplex method. The reduced costs change to

$$\tilde{z}_N = c_N - \tilde{A}_N^T A_B^{-T} c_B.$$

If the reduced costs still satisfy $\tilde{z}_N \geq 0$, then (y^*, s^*) remains dual feasible and thus x^* remains a solution of the primal problem.

Adding a new variable If we add a new variable $x_{n+1} \geq 0$ with coefficient $\tilde{c}_{n+1}$ in the objective function and column $\tilde{a}_{n+1}$ in the matrix, then B is still a primal feasible basis, but the nonbasis changes to

$$\tilde{N} = N \cup \{n+1\}.$$

Note that in the dual problem this new variable x_{n+1} results in an additional constraint and corresponding slack variable

$$\tilde{a}_{n+1}^T y + s_{n+1} = \tilde{c}_{n+1}, \quad s_{n+1} \geq 0.$$

The reduced costs for an nonbasic variables in N remain unchanged and the new variable has the reduced costs

$$z_{n+1} = \tilde{c}_{n+1} - \tilde{a}_{n+1}^T A_B^{-T} c_B.$$

If $z_{n+1} \geq 0$, then (y^*, s^*, s_{n+1}^*) with $s_{n+1}^* = z_{n+1}$ is feasible for the dual problem and x^* remains a solution of the primal problem. Otherwise, we can use B and x^* to start the primal simplex method. In this case the new variable x_{n+1} enters the basis in the first iteration.

Adding a new inequality constraint If we add a new inequality constraint $\tilde{a}^T x \leq \tilde{\beta}$, we also have to introduce a new variable x_{n+1} and thus our problem changes to

$$\min_{x, x_{n+1}} \underbrace{\begin{pmatrix} c \\ 0 \end{pmatrix}^T}_{=: \tilde{c}} \underbrace{\begin{pmatrix} x \\ x_{n+1} \end{pmatrix}}_{=: \tilde{x}} \quad \text{s.t.} \quad \underbrace{\begin{pmatrix} A & 0 \\ a^T & 1 \end{pmatrix}}_{=: \tilde{A}} \underbrace{\begin{pmatrix} x \\ x_{n+1} \end{pmatrix}}_{=: \tilde{b}} = \underbrace{\begin{pmatrix} b \\ \beta \end{pmatrix}}_{=: \tilde{b}}, \quad \begin{pmatrix} x \\ x_{n+1} \end{pmatrix} \geq 0.$$

We thus need a basis with $m+1$ elements. Let us consider $\tilde{B} := B \cup \{n+1\}$ and leave N unchanged. Then the corresponding basis matrix is

$$\tilde{A}_{\tilde{B}} = \begin{pmatrix} A_B & 0 \\ a_B^T & 1 \end{pmatrix}$$

and thus regular, i.e. $\tilde{B}$ is a basis matrix. One directly verifies that the corresponding inverse matrix is

$$\tilde{A}_{\tilde{B}}^{-1} = \begin{pmatrix} A_B^{-1} & 0 \\ -a_B^T A_B^{-1} & 1 \end{pmatrix}.$$

We cannot guarantee $x_{n+1} = \beta - a_B^T x_B^* \geq 0$, meaning we do not know, if $\tilde{B}$ is a primal feasible basis. But we observe

$$s_N = \tilde{c}_N - \tilde{A}_N^T \tilde{A}_{\tilde{B}}^{-T} \tilde{c}_{\tilde{B}} = c_N - (A_N^T \quad a_N) \begin{pmatrix} A_B^{-T} & -A_B^{-T} a_B \\ 0 & 1 \end{pmatrix} \begin{pmatrix} c_B \\ 0 \end{pmatrix} = s_N^* \geq 0.$$

Consequently $\tilde{B}$ is a dual feasible basis and we can start the dual simplex method with $(y^*, y_{m+1}^*, s^*, s_{n+1}^*)$ with $y_{m+1}^* = s_{n+1}^* = 0$. The corresponding complementary variable is

$$\tilde{x}_{\tilde{B}} = \tilde{A}_{\tilde{B}}^{-1} \tilde{b} = \begin{pmatrix} x_B^* \\ \beta - a_B^T x_B^* \end{pmatrix}.$$

In case $\tilde{x}_{n+1} \geq 0$, the basis $\tilde{B}$ is also primal feasible and $(x^*, 0)$ is a solution of the modified problem.

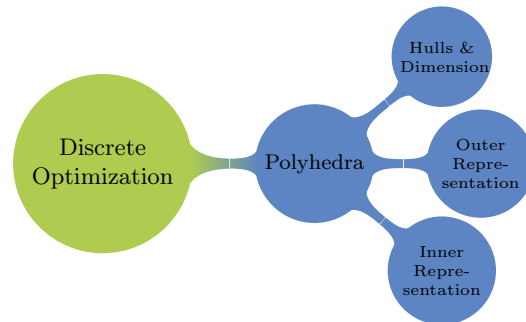
A summary of these results can be found in Table 2.1. These results also underline why the dual simplex method is of particular interest for solving discrete linear programs.

Table 2.1: Reoptimization: Changes and suitable algorithms

changed component	simplex method	initial feasible basis	initial basic solution
objective function c	primal	B	x^*
right-hand side b	dual	B	(y^*, s^*)
nonbasis matrix A_N	primal	B	x^*
new variable x_{n+1}	primal	B	$(x^*, 0)$
new inequality $a^T x \leq \beta$	dual	$B \cup \{n+1\}$	$(y^*, 0, s^*, 0)$

2.4 Exercises

3 Polyhedral Geometry



One of our main goals in this lecture is to solve (mixed-) integer linear programs, whose feasible set is the intersection of a polyhedron with some integrality condition. A common approach to do this is to minimize the objective function on the whole polyhedron first. If the resulting solution also satisfies the integrality conditions, we have found a solution of the (mixed-)integer problem and are done. Otherwise, we need to find a way to separate the solution from the feasible points in the polyhedron, that also satisfy the integrality conditions.

To this end, we want to have a closer look at polyhedra in this chapter. We begin by introducing several types of hulls and a dimension for sets in Section 3.1. Then we derive an outer representation of polyhedra as intersections of halfspaces and affine subspaces in Section 3.2. Finally, we develop an inner representation of polyhedra as the sum of a convex hull and a conical hull in Section 3.3.

Before we get started, let us recall the names of some important types of polyhedra, see also Figure 3.1 for some illustrations.

Definition 3.1.

- (a) A set $P \subseteq \mathbb{R}^n$ is called a *polyhedron* (*Polyeder*), if it can be described with finitely many linear inequality constraints, i.e. if there exist vectors $a_1, \dots, a_m \in \mathbb{R}^n$ and scalars $\beta_1, \dots, \beta_m \in \mathbb{R}$ such that

$$P = \{x \in \mathbb{R}^n \mid a_i^T x \leq \beta_i \quad \forall i = 1, \dots, m\}.$$

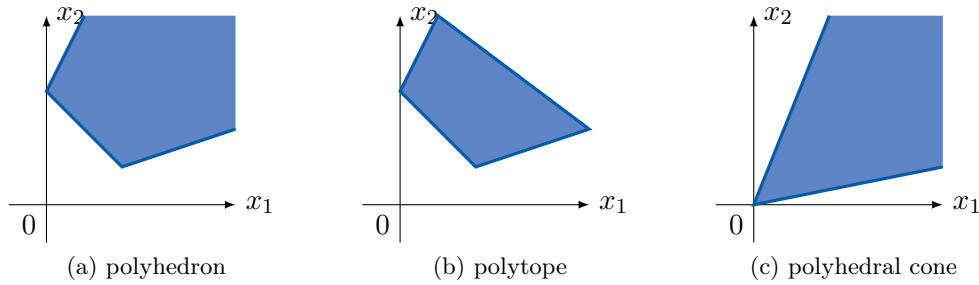


Figure 3.1: Illustration of some types of polyhedra

(b) A polyhedron of the form

$$P(A, b) = \{x \in \mathbb{R}^n \mid Ax \leq b\}$$

with a matrix $A \in \mathbb{R}^{m \times n}$ and a vector $b \in \mathbb{R}^m$ is said to have *natural form* (*natürliche Form*).

(c) A polyhedron of the form

$$P^=(A, b) = \{x \in \mathbb{R}^n \mid Ax = b, \quad x \geq 0\}$$

with a matrix $A \in \mathbb{R}^{m \times n}$ and a vector $b \in \mathbb{R}^m$ is said to have *normal form* (*Normalform*) or *standard form* (*Standardform*).

(d) A bounded polyhedron is called a *polytope* (*Polytop*).

(e) A polyhedron that is also a cone, i.e. that has the property

$$x \in P \implies cx \in P \quad \text{for all } c \geq 0,$$

is called a *polyhedral cone* (*polyedrischer Kegel*).

(f) A polyhedron is called *rational* (*rational*), if it can be described with rational data, i.e. has a representation of the form $P = P(A, b)$ with $A \in \mathbb{Q}^{m \times n}$ and $b \in \mathbb{Q}^m$.

Some remarks on these definitions:

- A rational polyhedron has to have at least one rational representation. But it can also have irrational representations, e.g.

$$P = \{x \in \mathbb{R} \mid x \leq 0\} = \{x \in \mathbb{R} \mid \sqrt{2}x \leq 0\}.$$

- When solving integer linear programs, we typically assume that the feasible set is a rational polyhedron and then also assume that we are working with a rational representation of the polyhedron.
- One can show that polyhedral cones can be represented in natural form as $P = P(A, 0)$, see Exercise 3.1.

3.1 Hulls and Dimension of Sets

When you consider a full-dimensional polyhedron in $\mathbb{R}^3$, then its surface consists of flat polyhedra, edges and vertices. All of these surface components are again polyhedra, but of different dimensions, see Figure 3.2 for an illustration. When we want to describe the surface of a polyhedron, which in our case often is the boundary of a feasible set, we want to do this with as few constraints as possible, i.e. we should work with the large surface polyhedra. In contrast, when we are looking for solutions of linear programs, we know that these are typically attained in vertices, which are the 0-dimensional surface components. This motivates why it is important to understand the structure of polyhedra and it also illustrates that we need to define some type of dimension for polyhedra.

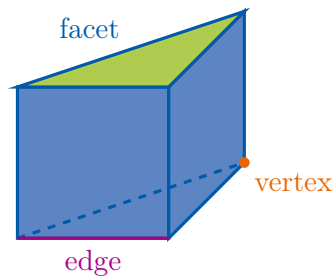


Figure 3.2: Surface of a 3-dimensional polyhedron

In this section, we want to define a dimension for arbitrary nonempty sets $X \subseteq \mathbb{R}^n$. To do so, we need some concepts from linear algebra that will also be useful later.

Definition 3.2.

- (a) A subset $U \subseteq \mathbb{R}^n$ is called a *subspace* (*Unterraum*), if $0 \in U$ and

$$\alpha x + \beta y \in U \quad \text{for all } x, y \in U \text{ and all } \alpha, \beta \in \mathbb{R}.$$

- (b) If $U \subseteq \mathbb{R}^n$ is a subspace, then the maximum number of linearly independent vectors in U is called the *dimension* (*Dimension*) of U and abbreviated as $\dim U$.

- (c) For a nonempty set $X \subseteq \mathbb{R}^n$ the *linear hull* (*lineare Hülle*) of X is defined as the smallest subspace including X , i.e.

$$\text{lin } X := \bigcap \{U \subseteq \mathbb{R}^n \text{ subspace} \mid X \subseteq U\}.$$

In this definition, we have used an outer characterization of the linear hull as the intersection of all subspaces including the set X . One can verify that it is also possible to work with an inner characterization

$$\text{lin } X = \left\{ \sum_{k=1}^m c_k x^k \mid m \in \mathbb{N}, \quad c_1, \dots, c_m \in \mathbb{R}, \quad x^1, \dots, x^m \in X \right\},$$

which says that the linear hull of X is the set of all possible finite (!) linear combinations of elements of X .

Since $\text{lin } X$ is a subspace, we could use the dimension of $\text{lin } X$ to define a dimension of arbitrary sets X . However, this would have the effect that shifting a set X could change its dimension, see Figure 3.3 for an illustration.

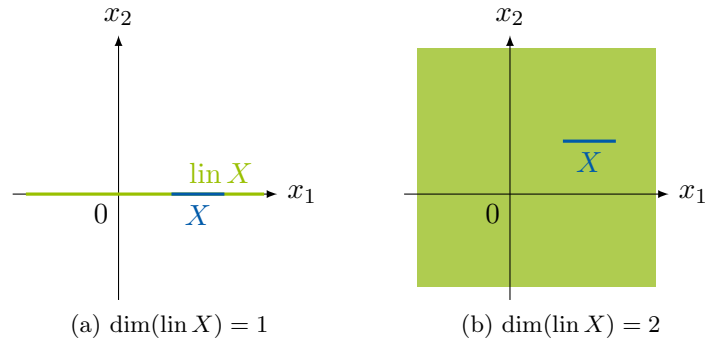


Figure 3.3: Illustration how shifting a set X can change the dimension of its linear hull

For this reason, we do not use the linear hull to define the dimension of sets but instead work with the affine hull. To this end, we need to introduce affine subspaces, which can be considered as “shifted” subspaces.¹

¹One can show that the subspace U related to an affine subspace A is independent from the choice of $a^* \in A$, see Figure 3.4 for an illustration.

Definition 3.3.

- (a) A set $A \subset \mathbb{R}^n$ is called an *affine subspace* (*affiner Unterraum*), if the set

$$U := A - a^* \quad \text{with some } a^* \in A$$

is a subspace.

- (b) The *dimension* of an affine subspace $A \subseteq \mathbb{R}^n$ with $a^* \in A$ is defined as

$$\dim A := \dim(A - a^*).$$

- (c) For a nonempty set $X \subseteq \mathbb{R}^n$ the *affine hull* (*affine Hülle*) of X is defined as the smallest affine subspace including X , i.e.

$$\text{aff } X := \bigcap \{A \subseteq \mathbb{R}^n \text{ affine subspace} \mid X \subseteq A\}.$$

- (d) For a nonempty set $X \subseteq \mathbb{R}^n$ the *dimension* of X is defined as

$$\dim X := \dim(\text{aff } X).$$

For empty sets one occasionally defines $\dim \emptyset := -1$.

Our definition of the affine hull is again an outer definition based on the intersection of supersets. One can also derive an inner characterization of the affine hull as

$$\text{aff } X = \left\{ \sum_{k=1}^m c_k x^k \mid m \in \mathbb{N}, \quad c_1, \dots, c_m \in \mathbb{R} \text{ with } \sum_{k=1}^m c_k = 1, \quad x^1, \dots, x^m \in X \right\}.$$

This characterization of the affine hull looks similar to the characterization of the convex hull of X , but note that the coefficients c_k can also be negative in the affine hull.

One can verify that, if we define the dimension of nonempty sets X as above, $\dim X$ remains unchanged, if we shift, rotate or scale the set X (with a nonzero scalar), see also Figure 3.5.

We have already introduced the linear hull and affine hull of arbitrary nonempty sets. Analogously, one can define the *convex hull* (*konvexe Hülle*) of a nonempty set X as the smallest convex set including the set X and the *conical hull* (*konische Hülle*) as the smallest convex² cone including the set X . A special case, which we need frequently in

²By definition, the conical hull of X is not the smallest cone including the set X , but the smallest convex cone including X . So it should be called the *convex conical hull* but typically is just called the *conical hull*.

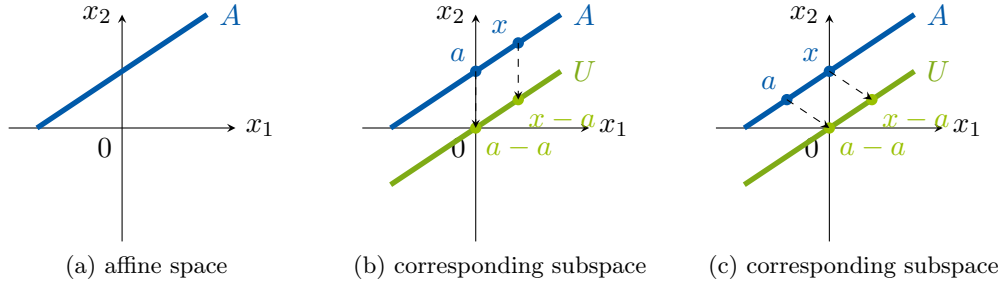


Figure 3.4: Illustration of an affine space and the corresponding subspace

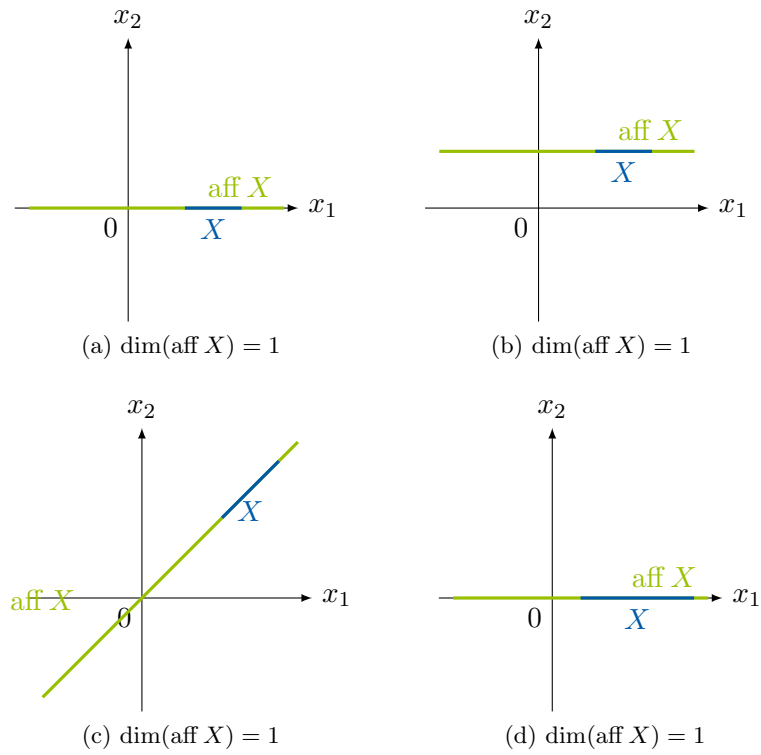


Figure 3.5: Illustration how shifting, scaling or rotating a set X does not change the dimension of its affine hull

Section 3.3, are such hulls of sets X consisting of finitely many points $x^1, \dots, x^m \in \mathbb{R}^n$. In this case, one can show that a very compact notation of these sets is possible. Please beware that X is a matrix and not a set in the following result.

Lemma 3.4. Let $x^1, \dots, x^m \in \mathbb{R}^n$ be given vectors and consider the matrix $X \in \mathbb{R}^{n \times m}$, whose columns are $X_{*,k} = x^k$ für $k = 1, \dots, m$. Then the following holds, where we use $e = (1, \dots, 1)^T \in \mathbb{R}^m$:

- (a) The convex hull of $x^1, \dots, x^m$ is given by

$$\text{conv } X := \{Xc \mid c \in \mathbb{R}^m, c \geq 0, e^T c = 1\}.$$

- (b) The conic hull of $x^1, \dots, x^m$ is given by

$$\text{cone } X := \{Xc \mid c \in \mathbb{R}^m, c \geq 0\}.$$

- (c) The affine hull of $x^1, \dots, x^m$ is given by

$$\text{aff } X := \{Xc \mid c \in \mathbb{R}^m, e^T c = 1\}.$$

- (d) The linear hull of $x^1, \dots, x^m$ is given by

$$\text{lin } X := \{Xc \mid c \in \mathbb{R}^m\}.$$

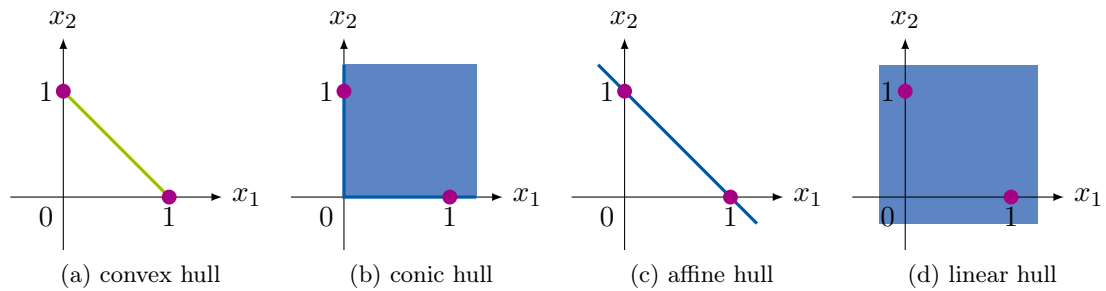


Figure 3.6: Illustration of the different types of hulls of two points

Now recall that we are not only interested in full-dimensional polyhedra, i.e. $P \subseteq \mathbb{R}^n$ with $\dim P = n$, but also in lower-dimensional polyhedra, because these form the surface of a polyhedron. If we consider a polyhedron $P \subset \mathbb{R}^n$ with $\dim P < n$, then $\text{aff } P \neq \mathbb{R}^n$

and thus we automatically know that the *interior* (*Innere*) of P

$$\text{int } P := \{x \in P \mid \exists r > 0 : B_r(x) \subseteq P\}$$

has to be empty. The reason for this is that we determine the interior of P with regards to the superset $\mathbb{R}^n$ and do not take into account that P actually lives in the smaller space $\text{aff } P$. This observation motivates the following definition.

Definition 3.5. Consider a nonempty set $X \subseteq \mathbb{R}^n$.

(a) The *relative interior* (*das relative Innere*) of X is defined as

$$\text{rint } X := \{x \in X \mid \exists r > 0 : B_r(x) \cap \text{aff } X \subseteq X\}.$$

(b) The *relative boundary* (*der relative Rand*) of X is defined as

$$\text{rbd } X := \overline{X} \setminus \text{rint } X.$$

The definition of the relative interior immediately implies

$$\text{int } X \subseteq \text{rint } X \subseteq X \subseteq \overline{X}.$$

However, please beware that not all of the rules, which we know for the interior, also hold for the relative interior.

Example 3.6. Consider the sets $X = [0, 1]^2 \subset \mathbb{R}^2$ and $Y = \{0\} \times [0, 1] \subset \mathbb{R}^2$. Then we have

$$\begin{aligned} \text{aff } X &= \mathbb{R}^2 \quad \text{and} \quad \text{rint } X = \text{int } X = (0, 1)^2, \\ \text{aff } Y &= \{0\} \times \mathbb{R} \quad \text{and} \quad \text{rint } Y = \{0\} \times (0, 1), \quad \text{int } Y = \emptyset. \end{aligned}$$

Thus, we have the inclusions $Y \subseteq X$ and $\text{int } Y \subseteq \text{int } X$ but $\text{rint } Y \not\subseteq \text{rint } X$. The reason why the latter inclusion does not hold, is that X and Y have different affine hulls.

For nonconvex sets we know that the difference between a set, its (relative) interior and its closure can be very large. For example for the set $\mathbb{Q} \subset \mathbb{R}$ we have

$$\text{int } \mathbb{Q} = \text{rint } \mathbb{Q} = \emptyset \quad \text{and} \quad \overline{\mathbb{Q}} = \mathbb{R}.$$

In contrast, for nonempty convex sets $X \subseteq \mathbb{R}^n$ one can show that

$$\text{rint } X \neq \emptyset \quad \text{and} \quad \dim(\text{rint } X) = \dim X = \dim \overline{X}.$$

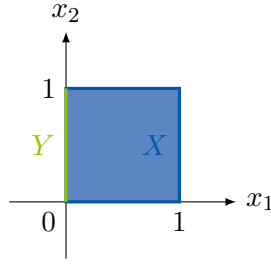


Figure 3.7: Illustration of Example 3.6

3.2 Outer Representation of Polyhedra

In the last section, we have introduced several types of hulls for arbitrary nonempty sets X and have seen that we can either construct these hulls from the outside by intersecting certain supersets of X or from the inside by using specific linear combinations of elements of the set X .

Let us now come back to polyhedra $P \subseteq \mathbb{R}^n$. We already know that it is possible to construct polyhedra from the outside as an intersection of halfspaces. In this section, our goal is to describe polyhedra with as few constraints/halfspaces as possible and to understand the effect of these constraints. In contrast, in Section 3.3 we will construct polyhedra from the inside.

We have defined polyhedra $P \subseteq \mathbb{R}^n$ as those sets, that can be described with finitely many linear constraints, i.e. have a representation of the form

$$P = P(A, b) = \{x \in \mathbb{R}^n \mid Ax \leq b\}$$

with $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.

Each linear constraint $a^T x \leq \beta$ with $a \in \mathbb{R}^n$, $\beta \in \mathbb{R}$ defines a *hyperplane* (*Hyperebene*) and two *halfspaces* (*Halbräume*)

$$\begin{aligned} H(a, \beta) &:= \{x \in \mathbb{R}^n \mid a^T x = \beta\}, \\ H^-(a, \beta) &:= \{x \in \mathbb{R}^n \mid a^T x \leq \beta\}, \\ H^+(a, \beta) &:= \{x \in \mathbb{R}^n \mid a^T x \geq \beta\}. \end{aligned}$$

Thus, we can equivalently define a polyhedron as the intersections of finitely many halfspaces. To describe these halfspaces, let us introduce the following notions.

Definition 3.7. Let $X \subseteq \mathbb{R}^n$ be nonempty, closed and convex and $a \in \mathbb{R}^n$, $\beta \in \mathbb{R}$.

- (a) The inequality $a^T x \leq \beta$ is called a *valid inequality* (*gültige Ungleichung*) for X , if

$$X \subseteq H^-(a, \beta).$$

- (b) The hyperplane $H(a, \beta)$ is called a *supporting hyperplane* (*Stützhyperebene*) of X , if $a^T x \leq \beta$ is a valid inequality for X and

$$H \cap X \neq \emptyset.$$

Some remarks on this definition:

- We will switch between working with an inequality $a^T x \leq \beta$, the corresponding halfspace $H^-(a, \beta)$ and the corresponding hyperplane $H(a, \beta)$ frequently.
- We want to describe polyhedra as the intersection of halfspaces. For this, only halfspaces corresponding to valid inequalities can be used, because otherwise the intersection would not contain the whole polyhedron.
- We are interested in efficient descriptions, i.e. with as few halfspaces as possible. Thus, only supporting hyperplanes are interesting, because valid inequalities, that do not correspond to a supporting hyperplane, can be further tightened, i.e. replaced by a sharper constraint, see Figure 3.8 for an illustration. However, beware that this argument only holds for closed sets.
- We work with closed halfspaces. Since arbitrary intersections of closed sets remain closed, this means we can only describe closed sets as intersections of halfspaces.

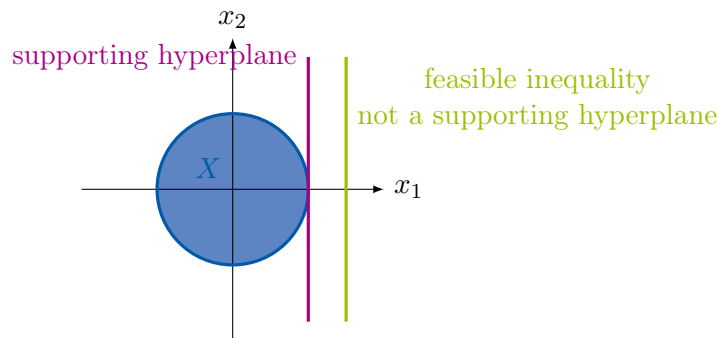


Figure 3.8: Illustration of feasible inequalities and supporting hyperplanes

One can prove that every nonempty, closed and convex set X can be represented as the intersection of all halfspaces corresponding to supporting hyperplanes. However, for arbitrary nonempty, closed and convex sets X , this construction can require the use of infinitely many supporting hyperplanes. As an example for this consider the closed unit

ball $\overline{B_1(0)}$, see also Figure 3.8. In contrast to this, nonempty polyhedra, which are closed and convex sets, require only finitely many (supporting) hyperplanes.

Valid inequalities can be inactive in all points $x \in X$, active in some points $x \in X$ or active in all points $x \in X$.

Example 3.8. Consider an arbitrary nonempty, closed and convex set $X \subseteq \mathbb{R}^n$.

- Then $0^T x \leq 1$ is a valid inequality but not active in any point $x \in X$.
- In contrast $0^T x \leq 0$ is a valid inequality that is active in all points $x \in X$ and thus a supporting hyperplane since $X \neq \emptyset$.

Of special interest to us are valid inequalities – supporting hyperplanes to be exact – that are active only in a proper subset of the polyhedron P , because those can be used to define surface components of P .

Definition 3.9. Let $P \subseteq \mathbb{R}^n$ be a polyhedron.

- (a) If $a^T x \leq \beta$ is a valid inequality for P , then the set

$$F := \{x \in P \mid a^T x = \beta\}$$

is called a *face* (*Seitenfläche*) of P induced by this inequality.

- (b) A face F of the polyhedron P with $F \notin \{\emptyset, P\}$ is called a *proper* (*eigentlich*) face of P .

Since a face F is the intersection of the polyhedron P with the two halfspaces $H^+(a, \beta)$, $H^-(a, \beta)$, every face is a subset of P and itself a polyhedron. As illustrated by Example 3.8 every polyhedron P has the faces $F = \emptyset$ and $F = P$. Since those do not include any additional information compared to the original polyhedron P , we often exclude these extreme cases and only consider proper faces.

Example 3.10. Consider the polyhedron

$$P = \{x \in \mathbb{R}^2 \mid x_1 \geq 0, \quad x_2 \geq 0, \quad x_1 \leq 2, \quad x_2 \leq 2, \quad x_1 + x_2 \leq 3\},$$

see Figure 3.9 for an illustration. This polyhedron has 12 faces: the 5 vertices, the 5 edges, the whole polyhedron P and the empty set.

Now consider a proper face such as the vertex $F_v = \{(2, 0)^T\}$. Then we can describe this face as

$$F_v = \{x \in P \mid x_1 = 2, \quad x_2 = 0\},$$

i.e. those points in P , where two of the inequality constraints are active. Similarly if we consider the edge $F_e = \{2\} \times [0, 1]$, we can describe this face as

$$F_e = \{x \in P \mid x_1 = 2\}.$$

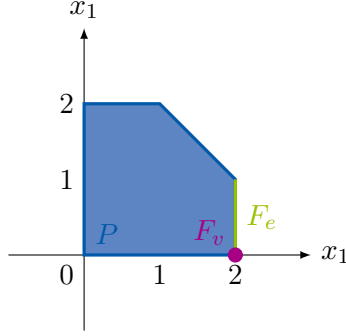


Figure 3.9: Illustration of Example 3.10

It seems that we can generate the faces of $P = P(A, b)$ by turning some of the inequality constraints in $Ax \leq b$ into equality constraints. This would of course be more efficient than having to find additional valid inequalities $a^T x \leq \beta$ in order to describe the faces.

Based on these observations, we introduce the following terminology.

Definition 3.11. Let $P = P(A, b) \subseteq \mathbb{R}^n$ be a polyhedron with $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.

- (a) For a face F of the polyhedron P we define the *set of active constraints* (*Menge der aktiven Restriktionen*) in F as

$$\text{eq}(F) := \{i = 1, \dots, m \mid A_{i,*}x = b_i \quad \forall x \in F\}.$$

We also use the notation

$$\text{ineq}(F) := \{1, \dots, m\} \setminus \text{eq}(F).$$

- (b) The *set of implicit equations* (*Menge der impliziten Gleichungen*) of P is defined as

$$\text{eq}(P) = \{i = 1, \dots, m \mid A_{i,*}x = b_i \quad \forall x \in P\}$$

- (c) For a subset $I \subseteq \{1, \dots, m\}$ we call

$$\text{face}(I) := \{x \in P \mid A_{I,*}x = b_I\}$$

the *face induced by I* (*induzierte Seitenfläche*).

So for a face F we collect the indices of those inequality constraints $A_{i,*}x \leq b_i$, that are active on the whole face F , in the set $\text{eq}(F)$. The remaining inequalities $\text{ineq}(F)$ can also be active in some points in F , but not all points in F .

Conversely, we can also start with an index set I and use it to define a face via $\text{face}(I)$. The set $\text{face}(I)$ is in fact a face of the polyhedron P , because it is induced by the valid inequality

$$\left(\sum_{i \in I} A_{i,*} \right)^T x \leq \sum_{i \in I} b_i.$$

The implicit equations $\text{eq}(P)$ are those inequalities, which are active in the whole polyhedron P . Since every face F of P is a subset of P , the inequalities $\text{eq}(P)$ are also active on the whole face F . We thus always have $\text{eq}(P) \subseteq \text{eq}(F)$ and will later see that this inclusion is strict for proper faces.

From linear algebra we know that the solution set of the linear equation

$$A_{\text{eq}(P),*}x = b_{\text{eq}(P)}$$

is either empty (in which case $P = \emptyset$) or an affine subspace of dimension

$$\dim(\ker A_{\text{eq}(P),*}) = n - \text{rank } A_{\text{eq}(P),*}.$$

Since the polyhedron P is a subset of this affine subspace, the implicit equations $\text{eq}(P)$ thus impact the dimension of the polyhedron P . This observation is made more precise in the following result.

Theorem 3.12. Let $P = P(A, b) \subseteq \mathbb{R}^n$ be a nonempty polyhedron with $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. Then the following statements are true:

(a) There exists at least one point $x^* \in P$ with

$$A_{\text{eq}(P),*}x^* = b_{\text{eq}(P)} \quad \text{and} \quad A_{\text{ineq}(P),*}x^* < b_{\text{ineq}(P)}.$$

(b) For every point $x^* \in P$ satisfying (a) we have

$$\begin{aligned} \text{aff } P &= x^* + \ker A_{\text{eq}(P),*}, \\ \dim P &= \dim(\ker A_{\text{eq}(P),*}) = n - \text{rank } A_{\text{eq}(P),*}. \end{aligned}$$

Proof. (a) In case $\text{ineq}(P) = \emptyset$ we have $P = \{x \in \mathbb{R}^n \mid A_{\text{eq}(P),*}x = b_{\text{eq}(P)}\}$ and thus every point $x^* \in P$ satisfies all conditions.

In case $\text{ineq}(P) \neq \emptyset$ we know by definition of $\text{ineq}(P)$ that for all $i \in \text{ineq}(P)$ there exists an $x^i \in P$ with $A_{i,*}x^i < b_i$. Then we can define x^* as the convex combination

$$x^* =: \frac{1}{|\text{ineq}(P)|} \sum_{i \in \text{ineq}(P)} x^i.$$

This immediately implies $x^* \in P$ and $A_{\text{eq}(P),*}x^* = b_{\text{eq}(P)}$. Furthermore, for all $j \in \text{ineq}(P)$ we have

$$A_{j,*}x^* = \frac{1}{|\text{ineq}(P)|} \left(\underbrace{A_{j,*}x^j}_{< b_j} + \sum_{i \in \text{ineq}(P) \setminus \{j\}} \underbrace{A_{j,*}x^i}_{\leq b_j} \right) < b_j.$$

This shows that x^* satisfies all conditions.

- (b) Let $x^* \in P$ be a point satisfying (a). Then by choice of x^* there exists $\varepsilon > 0$ such that for all $y \in \ker A_{\text{eq}(P),*}$ with $\|y\| \leq \varepsilon$ we have

$$\begin{aligned} A_{\text{eq}(P),*}(x^* + y) &= \underbrace{A_{\text{eq}(P),*}x^*}_{= b_{\text{eq}(P)}} + \underbrace{A_{\text{eq}(P),*}y}_{= 0} = b_{\text{eq}(P)}, \\ A_{\text{ineq}(P),*}(x^* + y) &= \underbrace{A_{\text{ineq}(P),*}x^*}_{< b_{\text{ineq}(P)}} + \underbrace{A_{\text{ineq}(P),*}y}_{\text{small}} \leq b_{\text{ineq}(P)} \end{aligned}$$

This shows the inclusions

$$\{x^* + y \mid y \in \ker A_{\text{eq}(P),*} \text{ with } \|y\| \leq \varepsilon\} \subseteq P \subseteq \{x^* + y \mid y \in \ker A_{\text{eq}(P),*}\}.$$

Computing the affine hull of all three sets thus yields the inclusions

$$\text{aff}(x^* + \ker A_{\text{eq}(P),*}) \subseteq \text{aff } P \subseteq \text{aff}(x^* + \ker A_{\text{eq}(P),*})$$

and thus (b) follows. □

In Theorem 3.12 (a) we have shown the existence of points $x^* \in P = P(a, b)$ with

$$A_{\text{ineq}(P),*}x^* < b_{\text{ineq}(P)}.$$

In Exercise 3.5 we show that this condition in fact characterizes the relative interior points of P and generalize the result to relative interior points of faces F of P . This observation implies that the relative interior points of P are those, where only the active constraints $\text{eq}(P)$ of P are active but no other constraints. This also implies that the relative interior points of P are not contained in any proper face of P .

Our next step is to translate the previous theorem from the whole polyhedron P to its faces F . To this end we can use that faces of P are again polyhedra.

Theorem 3.13. Let $P = P(A, b) \subseteq \mathbb{R}^n$ be a nonempty polyhedron with $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. Further let F be a nonempty face of P . Then the following holds:

- (a) $\text{aff } F = \{x \in \mathbb{R}^n \mid A_{\text{eq}(F),*}x = b_{\text{eq}(F)}\}$
- (b) $\dim F = \dim(\ker A_{\text{eq}(F),*}) = n - \text{rank } A_{\text{eq}(F),*}$
- (c) $F = \text{face}(\text{eq}(F)) = \{x \in \mathbb{R}^n \mid A_{\text{eq}(F),*}x = b_{\text{eq}(F)}, \quad A_{\text{ineq}(F),*}x \leq b_{\text{ineq}(F)}\}$

Proof. Let $a^T x \leq \beta$ be a valid inequality inducing the face F , i.e.

$$\begin{aligned} F &= P \cap \{x \in \mathbb{R}^n \mid a^T x = \beta\} \\ &= \{x \in \mathbb{R}^n \mid A_{\text{eq}(F),*}x = b_{\text{eq}(F)}, \quad A_{\text{ineq}(F),*}x \leq b_{\text{ineq}(F)}, \quad a^T x = \beta\}. \end{aligned}$$

Note that in this representation of F for all $i \in \text{ineq}(F)$ there exists at least one $x \in F$ with $A_{i,*}x < b_i$. Applying Theorem 3.12 to the polyhedron F yields the existence of a point $x^* \in F$ with

$$A_{\text{eq}(F),*}x^* = b_{\text{eq}(F)}, \quad A_{\text{ineq}(F),*}x^* < b_{\text{ineq}(F)}, \quad a^T x^* = \beta.$$

and

$$\text{aff } F = x^* + \ker \begin{pmatrix} A_{\text{eq}(F),*} \\ a^T \end{pmatrix}.$$

All of the claims follow, if we can show that

$$\ker \begin{pmatrix} A_{\text{eq}(F),*} \\ a^T \end{pmatrix} = \ker A_{\text{eq}(F),*}.$$

Here, the inclusion

$$\ker \begin{pmatrix} A_{\text{eq}(F),*} \\ a^T \end{pmatrix} \subseteq \ker A_{\text{eq}(F),*}$$

always holds. Now assume that there was a vector $y^* \neq 0$ with $y^* \in \ker A_{\text{eq}(F),*}$ but $a^T y^* \neq 0$. Then we can assume w.l.o.g. that $a^T y^* > 0$. For all $\varepsilon > 0$ sufficiently small the choice of x^* implies $x^* + \varepsilon y^* \in P$ but we have

$$a^T(x^* + \varepsilon y^*) = \underbrace{a^T x^*}_{=\beta} + \varepsilon \underbrace{a^T y^*}_{>0} > \beta.$$

This contradicts that $a^T x \leq \beta$ is a valid inequality.

Consequently we have

$$\text{aff } F = x^* + \ker \begin{pmatrix} A_{\text{eq}(F),*} \\ a^T \end{pmatrix} = x^* + \ker A_{\text{eq}(F),*}$$

and thus also $\dim F = \dim(\ker A_{\text{eq}(F),*}) = n - \text{rank } A_{\text{eq}(F),*}$. Since the face F is nonempty, it also follows that

$$\begin{aligned} F &= \{x \in \mathbb{R}^n \mid A_{\text{eq}(F),*}x = b_{\text{eq}(F)}, \quad A_{\text{ineq}(F),*}x \leq b_{\text{ineq}(F)}, \quad a^T x = \beta\} \\ &= \{x \in \mathbb{R}^n \mid A_{\text{eq}(F),*}x = b_{\text{eq}(F)}, \quad A_{\text{ineq}(F),*}x \leq b_{\text{ineq}(F)}\} = \text{face}(\text{eq}(F)). \end{aligned}$$

□

Since for every face F of a polyhedron P we have $F \subseteq P$ and thus $\text{eq}(P) \subseteq \text{eq}(F)$ it follows that $\dim F \leq \dim P$.

The result above shows that we do not need additional valid inequalities $a^T x \leq \beta$ in order to characterize faces F of P but instead can use the set of active constraints $\text{eq}(F)$.

Beware: Every subset $I \subseteq \{1, \dots, m\}$ induces a face $F := \text{face}(I)$ of $P = P(A, b)$. By construction this face satisfies

$$I \cup \text{eq}(P) \subseteq \text{eq}(F)$$

but the set $\text{eq}(F)$ of active constraints can be larger than the set I or even the set $I \cup \text{eq}(P)$. The formulas in Theorem 3.13 are only true for the set $\text{eq}(F)$.

In proofs we occasionally consider faces $F_1 \subseteq F_2$ of a polyhedron P and have to decide if the face F_1 is a proper subset of the face F_2 or if the two faces are equal. To do so, the following observation can be useful.

Lemma 3.14. Let $P = P(A, b) \subseteq \mathbb{R}^n$ be a nonempty polyhedron with $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. Further let F_1, F_2 be two nonempty faces of P with $F_1 \subseteq F_2$. Then

$$F_1 = F_2 \quad \Longleftrightarrow \quad \dim F_1 = \dim F_2.$$

Proof. Due to $F_1 \subseteq F_2$ we have $\dim F_1 \leq \dim F_2$. In case $\dim F_1 \neq \dim F_2$ we obviously have $F_1 \neq F_2$.

Now consider the case $\dim F_1 = \dim F_2$. By Theorem 3.13 this implies

$$\dim(\ker A_{\text{eq}(F_1),*}) = \dim(\ker A_{\text{eq}(F_2),*}).$$

Since the nullspace of a matrix is a vector space and we know $\text{eq}(F_2) \subseteq \text{eq}(F_1)$ this implies

$$\ker A_{\text{eq}(F_1),*} = \ker A_{\text{eq}(F_2),*}.$$

For an arbitrary point $x^* \in F_1 \subseteq F_2$ we thus have

$$F_1 = \{x^* + y \in P \mid y \in \ker A_{\text{eq}(F_1),*}\} = \{x^* + y \in P \mid y \in \ker A_{\text{eq}(F_2),*}\} = F_2.$$

□

From linear optimization we know that, if a linear program with feasible set $P = P^=(A, b)$ has a solution, it also has a solution in an extreme point. For extreme points of polyhedra we also used the term vertex. Now we are in a situation to define both terms properly and analyze their relation.

Definition 3.15. Let $P \subseteq \mathbb{R}^n$ be a polyhedron.

- (a) A point $x^* \in P$ is called an *extreme point* (*Extrempunkt*) of P , if it cannot be represented as a proper convex combination of two elements of P , i.e.

$$\forall x, y \in P \text{ with } x \neq y \quad \forall c \in (0, 1) : \quad cx + (1 - c)y \neq x^*.$$

- (b) A point $x^* \in P$ is called a *vertex* (*Ecke*) of P , if $\{x^*\}$ is a face of P .

To reiterate: We can define extreme points for arbitrary convex sets. But vertices are defined as the 0-dimensional faces of P and thus are only defined for polyhedra.

With our preparations we can now finally prove that the extreme points of a polyhedron are exactly its vertices and additionally provides a characterization of vertices for polyhedra of the form $P = P(A, b)$, see Exercise 3.6. We already know a similar characterization of vertices for polyhedra of the form $P = P^=(A, b)$ from the simplex method, see Exercise 3.7.

Faces of a polyhedron with dimension 0 are called vertices. Faces with dimension 1 are called *edges*. Each edge F belongs to one of the following three cases:

- If an edge F is a (straight) line (Gerade), then it is called an *extreme line* (*Extremalgerade*).
- If an edge F is a half-line or ray (Halbgerade), then it is called an *extreme ray* (*Extremalstrahl*).
- If $F = [x, y]$ is an interval, then the two vertices $x \neq y$ are called *adjacent* (*adjazent*).

In the simplex method, we start each iteration in a vertex and follow an edge, which is either an interval or an extrem ray since $P = P^=(A, b)$ does not contain whole lines. We follow an edge, which decreases the objective function value. In case this edge is an extrem ray, the problem is unbounded. Otherwise, we follow the edge to the vertex on its other end and start a new iteration.

In Section 3.3 we will show that we can represent polyhedra as the convex hull of finitely many points plus the conical hull of finitely many directions. Except for some special cases, we can use the extreme points and extreme rays in this construction.

In this section, however, we are interested in constructing polyhedra “from the outside”, i.e. as the intersection of halfspaces. And we would like to use as few halfspaces as

possible in order to obtain a compact description of the polyhedra. To this end, we first introduce the following terminology.

Definition 3.16. Let $P = P(A, b) \subseteq \mathbb{R}^n$ be a nonempty polyhedron with $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$.

- (a) An inequality $j \in \{1, \dots, m\}$ is called *redundant* (*redundant*), if

$$P = \{x \in \mathbb{R}^n \mid A_{i,*}x \leq b_i \quad \forall i \in \{1, \dots, m\} \setminus \{j\}\},$$

i.e. if one can eliminate the inequality $A_{j,*}x \leq b_j$ without changing the polyhedron P . Otherwise, the inequality is called *irredundant* (*irredundant*).

- (b) The description $P = \{x \in \mathbb{R}^n \mid Ax \leq b\}$ is called *irredundant*, if all inequalities are irredundant.

- (c) For subsets $I \subseteq \text{eq}(P)$ and $J \subseteq \text{ineq}(P)$ the description

$$P = \{x \in \mathbb{R}^n \mid A_{I,*}x = b_I, \quad A_{J,*}x \leq b_J\}$$

of the polyhedron P is called *irredundant*, if the matrix $A_{I,*}$ has full row rank and all inequalities $j \in J$ are irredundant.

So an irredundant description $P = \{x \in \mathbb{R}^n \mid Ax \leq b\}$ cannot contain any redundant inequalities. We have already seen that we cannot only describe a polyhedron in natural form as above, but also in the form

$$P = \{x \in \mathbb{R}^n \mid A_{\text{eq}(P),*}x = b_{\text{eq}(P)}, \quad A_{\text{ineq}(P),*}x \leq b_{\text{ineq}(P)}\}.$$

From this description, we can obviously eliminate all redundant inequalities $j \in \text{ineq}(P)$. Additionally, we can eliminate all equations $i \in \text{eq}(P)$, for which the row $A_{i,*}$ is linearly dependent from the other rows in $A_{\text{eq}(P),*}$, because this does not change the solution set of the linear equation. This motivates part (c) of the previous definition.

Example 3.17. Consider the polyhedron

$$P_1 = \{x \in \mathbb{R}^2 \mid x_1 \leq 1, \quad x_2 \leq 1, \quad x_1 + x_2 \leq 2\}.$$

In this case the constraint $x_1 + x_2 \leq 2$ is redundant, because it is implied by the other two constraints and can thus be eliminated. An irredundant description of this polyhedron would thus be

$$P_1 = \{x \in \mathbb{R}^2 \mid x_1 \leq 1, \quad x_2 \leq 1\}.$$

Now consider the polyhedron

$$P_2 = \{x \in \mathbb{R}^2 \mid x_1 \leq 1, \quad x_2 \leq 1, \quad -x_1 + -x_2 \leq -2\} = \{(1, 1)\}.$$

In this case we have $\text{eq}(P_2) = \{1, 2, 3\}$, but

$$P_2 = \{x \in \mathbb{R}^2 \mid x_1 = 1, \quad x_2 = 1, \quad -x_1 + -x_2 = -2\}$$

is not yet an irredundant description, because the three linear equations are linearly dependent. In order to obtain a irredundant description, one can in this case use any two out of the three equations, e.g..

$$P_2 = \{x \in \mathbb{R}^2 \mid x_1 = 1, \quad x_2 = 1\}.$$

Our goal is to find an irredundant description of $P = P(A, b)$ of the form

$$P = \{x \in \mathbb{R}^n \mid A_{I,*}x = b_I, \quad A_{J,*}x \leq b_J\}$$

with $I \subseteq \text{eq}(P)$ and $J \subseteq \text{ineq}(P)$. When choosing the set I , we can work with any subset $I \subseteq \text{eq}(P)$ such that the resulting matrix $A_{I,*}x = b_I$ has full row rank. But which inequalities $j \in \text{ineq}(P)$ should we include in the set J ? The following results provides an answer.

Theorem 3.18. Let $P = P(A, b) \subseteq \mathbb{R}^n$ be a nonempty polyhedron with $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. Then an inequality $j \in \text{ineq}(P)$ is irredundant if and only if there exists a proper face F with

$$\text{eq}(F) = \text{eq}(P) \cup \{j\}.$$

Proof. First let the inequality $j \in \text{ineq}(P)$ be redundant and consider a proper face F of P . Since the inequality j is redundant, F is then also a proper face of

$$\tilde{P} := \{x \in \mathbb{R}^n \mid A_{\text{eq}(P),*}x = b_{\text{eq}(P)}, \quad A_{\text{ineq}(P) \setminus \{j\},*}x \leq b_{\text{ineq}(P) \setminus \{j\}}\} = P.$$

Since F is a proper face of $P = \tilde{P}$, it follows that there exists at least one $k \in \text{ineq}(P) \setminus \{j\}$ with $k \in \text{eq}(F)$. This shows that there cannot exist a proper face F of P with $\text{eq}(F) = \text{eq}(P) \cup \{j\}$.

Conversely, let the inequality $j \in \text{ineq}(P)$ now be irredundant, meaning

$$\tilde{P} := \{x \in \mathbb{R}^n \mid A_{i,*}x \leq b_i \quad \forall i \in \{1, \dots, m\} \setminus \{j\}\} \supsetneq P.$$

Due to $j \in \text{ineq}(P)$ we know that $F := \text{face}(\{j\})$ is a proper face of P with $\text{eq}(F) \supseteq \text{eq}(P) \cup \{j\}$. We show that this face F satisfies $\text{eq}(F) = \text{eq}(P) \cup \{j\}$. To this end we use that according to Theorem 3.12 there exists a point $x^* \in P$ with

$$A_{\text{eq}(P),*}x^* = b_{\text{eq}(P)}, \quad A_{\text{ineq}(P),*}x^* < b_{\text{ineq}(P)}.$$

Further, consider an arbitrary $\tilde{x} \in \tilde{P} \setminus P$, i.e. with

$$A_{\text{eq}(P),*}\tilde{x} = b_{\text{eq}(P)}, \quad A_{\text{ineq}(P) \setminus \{j\},*}\tilde{x} \leq b_{\text{ineq}(P) \setminus \{j\}}, \quad A_{j,*}\tilde{x} > b_j.$$

Then there exists a $c \in (0, 1)$ such that $\bar{x} := cx^* + (1 - c)\tilde{x}$ satisfies

$$A_{\text{eq}(P),*}\bar{x} = b_{\text{eq}(P)}, \quad A_{\text{ineq}(P) \setminus \{j\},*}\bar{x} < b_{\text{ineq}(P) \setminus \{j\}}, \quad A_{j,*}\bar{x} = b_j.$$

This shows that there exists a point $\bar{x} \in F$, where only the inequalities $\text{eq}(P)$ and j are active and consequently $\text{eq}(F) \subseteq \text{eq}(P) \cup \{j\}$. Combining both inequalities we have proven that the proper face F satisfies $\text{eq}(F) = \text{eq}(P) \cup \{j\}$. $\square$

We will see next that the irredundant inequalities are closely related to the largest proper faces of P .

Definition 3.19. Let $P \subseteq \mathbb{R}^n$ be a nonempty polyhedron. A proper face F of P , which is not a subset of any other proper face of P is called a *facet* (*Facette*) of P .

Recall that the polyhedron P is itself always a face of P , which obviously includes all other faces of P . For this reason we only consider proper faces in the definition of a facet.

Recall that we can characterize vertices of P either as the extreme points of P or as the faces of dimension 0. Similarly, we can characterize facets either as defined above or as the faces of dimension $\dim P - 1$.

Lemma 3.20. Let $P = P(A, b) \subseteq \mathbb{R}^n$ be a nonempty polyhedron with $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. Then the following holds:

- (a) If F is a facet of P , then every inequality $i \in \text{eq}(F) \cap \text{ineq}(P)$ induces the facet, i.e. $F = \text{face}(\{i\})$.
- (b) If F_1, F_2 are two different facets, i.e. $F_1 \neq F_2$, then $\text{eq}(F_1) \cap \text{eq}(F_2) = \text{eq}(P)$.
- (c) A nonempty face F of P is a facet of P if and only if $\dim F = \dim P - 1$.

Proof. (a) For all inequalities $i \in \text{eq}(F) \cap \text{ineq}(P)$ we know that $G := \text{face}(\{i\})$ is a proper face of P , which includes F . Since F is a facet, this implies $F = G = \text{face}(\{i\})$.

- (b) Since F_1, F_2 are faces of P , we know $\text{eq}(P) \subseteq \text{eq}(F_1) \cap \text{eq}(F_2)$. Now assume that there was an $i \in \text{ineq}(P)$, which also satisfies $i \in \text{eq}(F_1) \cap \text{eq}(F_2)$. Then (a) implies $F_1 = \text{face}(\{i\}) = F_2$ in contradiction to the assumption $F_1 \neq F_2$.

- (c) Let F be a nonempty face of P with $\dim F = \dim P - 1$. Then every face G with $F \subseteq G \subseteq P$ either satisfies $\dim G = \dim F$ or $\dim G = \dim P$ and thus by Lemma 3.14 either $G = F$ or $G = P$. This shows that F is a facet.

Conversely let F be a facet of P . Then we know from (a) that every $i \in \text{eq}(F) \cap \text{ineq}(P)$ induces F . Since F is a proper face of P the set $\text{eq}(F) \cap \text{ineq}(P)$ is not empty. If there exist $j, k \in \text{eq}(F) \cap \text{ineq}(P)$ with $j \neq k$, then we know from Theorem 3.18 that the inequality k is redundant if and only if there exists a proper face G of P with

$$\text{eq}(G) = \text{eq}(P) \cup \{k\}.$$

Due to $\text{eq}(F) \supsetneq \text{eq}(P) \cup \{k\}$ the face $G \neq F$ would then be a proper face including F , which contradicts the fact that F is a facet. Thus, if $\text{eq}(F) \cap \text{ineq}(P)$ contains more than one element, all but one of these inequalities are redundant and can be eliminated. We can thus assume w.l.o.g. $\text{eq}(F) \cap \text{ineq}(P) = \{j\}$.

Now assume for contradiction that $\dim F \leq \dim P - 2$. Since by Theorem 3.13 we have $\dim F = n - \text{rank } A_{\text{eq}(F),*}$ this would imply

$$\text{rank } A_{\text{eq}(F),*} \geq \text{rank } A_{\text{eq}(P),*} + 2.$$

But since $\text{eq}(F) = \text{eq}(P) \cup \{j\}$ this is not possible. □

According to Lemma 3.20 (a) the description of a polyhedron P needs to include at least one inequality $j \in \text{ineq}(P)$ for each facet of P . This implies that $P = P(A, b)$ can have at most

$$|\text{ineq}(P)| = m - |\text{eq}(P)|$$

different facets. And by Theorem 3.18 combined with Lemma 3.20 (c) an inequality $j \in \text{ineq}(P)$ is redundant if and only if it induces a facet.

Putting all pieces together we have shown that a description of P of the form

$$P = \{x \in \mathbb{R}^n \mid A_{\text{eq}(P),*}x = b_{\text{eq}(P)}, \quad A_{\text{ineq}(P),*}x \leq b_{\text{ineq}(P)}\}$$

is redundant if and only if

- the matrix $A_{\text{eq}(P),*}$ has $n - \dim P$ linearly independent rows
- and $\text{ineq}(P)$ consists of exactly one inducing inequality per facet.

This is an outer description of the polyhedron P , because we describe P as the intersection of the affine subspace $\{x \in \mathbb{R}^n \mid A_{\text{eq}(P),*}x = b_{\text{eq}(P)}\}$ with the halfspaces $\{x \in \mathbb{R}^n \mid A_{j,*}x \leq b_j\}$ for all $j \in \text{ineq}(P)$.

3.3 Inner Representation of Polyhedra

In this section we want to derive an inner description of a polyhedron P . To be precise we want to show that for every nonempty polyhedron $P \subseteq \mathbb{R}^n$ there exist matrices $V \in \mathbb{R}^{n \times p}$ and $D \in \mathbb{R}^{n \times q}$ such that

$$P = \text{conv } V + \text{cone } D.$$

The important property here is that the number of columns p, q in both matrices is finite, i.e. P can be written as the convex combination of finitely many points $V_{*,j} \in \mathbb{R}^n$ plus the conical hull of finitely many directions $D_{*,j} \in \mathbb{R}^n$. In case $p = 0$ the convex hull is not needed and P is a cone. In case $q = 0$ the conical hull is not needed and P is a polytope.

To derive this representation of polyhedra we need the following result, which we do not prove here.

Theorem 3.21. Let $P \subseteq \mathbb{R}^{n_1+n_2}$ be a polyhedron. Then the orthogonal projection of P onto $\mathbb{R}^{n_1}$, i.e. the set

$$Q := \left\{ x \in \mathbb{R}^{n_1} \mid \exists y \in \mathbb{R}^{n_2} : \begin{pmatrix} x \\ y \end{pmatrix} \in P \right\}$$

is also a polyhedron.

This theorem is a special case of a more general result, which says that the projection of a polyhedron along an arbitrary direction onto another polyhedron is again a polyhedron. We prove Theorem 3.21 for the special case $n_2 = 1$ in the following remark.

Remark 3.22 (Fourier-Motzkin Elimination). Let $A \in \mathbb{R}^{m \times n}$, $b \in \mathbb{R}^m$ and $a \in \mathbb{R}^m$ be given and consider the polyhedron

$$P := \left\{ \begin{pmatrix} x \\ y \end{pmatrix} \in \mathbb{R}^{n+1} \mid Ax + ay \leq b \right\}.$$

We want to eliminate the variable y , i.e. to compute the projection

$$Q := \left\{ x \in \mathbb{R}^n \mid \exists y \in \mathbb{R} : \begin{pmatrix} x \\ y \end{pmatrix} \in P \right\}.$$

To this end we define the three index sets

$$\begin{aligned} I_0 &:= \{i = 1, \dots, m \mid a_i = 0\}, \\ I_+ &:= \{i = 1, \dots, m \mid a_i > 0\}, \\ I_- &:= \{i = 1, \dots, m \mid a_i < 0\}. \end{aligned}$$

Now consider an arbitrary point $(x, y) \in P$. Then we know

$$\begin{aligned} A_{i,*}x &\leq b_i & \forall i \in I_0, \\ A_{i,*}x &\leq b_i - a_i y & \forall i \in I_+, \\ A_{i,*}x &\leq b_i - a_i y & \forall i \in I_-. \end{aligned}$$

In case $I_+ = \emptyset$ one can show

$$P = \left\{ \begin{pmatrix} x \\ y \end{pmatrix} \in \mathbb{R}^{n+1} \mid A_{I_0,*}x \leq b_{I_0}, \quad y \geq \max_{i \in I_-} \frac{A_{i,*}x - b_i}{-a_i} \right\}$$

and thus

$$Q = \{x \in \mathbb{R}^n \mid A_{I_0,*}x \leq b_{I_0}\}.$$

Analogously, in case $I_- = \emptyset$ one can show

$$P = \left\{ \begin{pmatrix} x \\ y \end{pmatrix} \in \mathbb{R}^{n+1} \mid A_{I_0,*}x \leq b_{I_0}, \quad y \leq \min_{i \in I_+} \frac{A_{i,*}x - b_i}{-a_i} \right\}$$

and thus

$$Q = \{x \in \mathbb{R}^n \mid A_{I_0,*}x \leq b_{I_0}\}.$$

It remains to consider the case, where both $I_+ \neq \emptyset$ and $I_- \neq \emptyset$. For each pair of inequalities $(i, j) \in I_+ \times I_-$ we can then multiply the first inequality with $-a_j > 0$ and the second with $a_i > 0$ and add both inequalities to obtain

$$(a_i A_{j,*} - a_j A_{i,*})x \leq a_i b_j - a_j b_i.$$

With this we have shown the inclusion

$$\begin{aligned} Q \subseteq \{x \in \mathbb{R}^n \mid & A_{i,*}x \leq b_i \quad \forall i \in I_0, \\ & (a_i A_{j,*} - a_j A_{i,*})x \leq a_i b_j - a_j b_i \quad \forall (i, j) \in I_+ \times I_-\} =: M. \end{aligned}$$

To show the opposite inclusion, consider an arbitrary point $x \in M$. Then for all $y \in \mathbb{R}$ we have

$$A_{I_0,*}x + a_{I_0}y = A_{I_0,*}x \leq b_{I_0}.$$

The remaining inequalities are satisfied if

$$\begin{aligned} A_{I_+,*}x + a_{I_+}y \leq b_{I_+} & \iff y \leq \min_{i \in I_+} \frac{b_i - A_{i,*}x}{a_i}, \\ A_{I_-,*}x + a_{I_-}y \leq b_{I_-} & \iff y \geq \max_{j \in I_-} \frac{-b_j + A_{j,*}x}{-a_j}. \end{aligned}$$

A $y \in \mathbb{R}$ satisfying these conditions exists if and only if for all $(i, j) \in I_+ \times I_-$ we have

$$\frac{-b_j + A_{j,*}x}{-a_j} \leq \frac{b_i - A_{i,*}x}{a_i} \iff (a_i A_{j,*} - a_j A_{i,*})x \leq a_i b_j - a_j b_i.$$

And this is ensured due to $x \in M$. This shows the opposite inclusion $M \subseteq Q$ and thus the equality $Q = M$.

We have shown that the projection Q can be described by finitely many linear inequality constraints and therefore is a polyhedron.

The Fourier-Motzkin elimination we used in the previous remark to compute the projection Q of the polyhedron P is constructive in the sense that it provides us with a description of the polyhedron Q . This description of Q requires $|I_0| + |I_+| \cdot |I_-|$ inequality constraints, out of which generally many are redundant. One can show that iterative application of this elimination process allows one to also compute the projection in case $n_2 > 1$.

With Theorem 3.21 we can now prove the following result, which is extremely useful to show that certain sets are polyhedra.

Theorem 3.23. Let $P = P(A, b) \subseteq \mathbb{R}^n$ be a polyhedron and $f : \mathbb{R}^n \rightarrow \mathbb{R}^p$ be an affine map, i.e. $f(x) = Bx + c$ with $B \in \mathbb{R}^{p \times n}$ and $c \in \mathbb{R}^p$. Then the set

$$f(P) = \{Bx + c \mid x \in P\}$$

also is a polyhedron.

Proof. We can rewrite the image set as

$$f(P) = \{y \in \mathbb{R}^p \mid \exists x \in \mathbb{R}^n : Ax \leq b, y = Bx + c\}.$$

And thus $f(P)$ is the projection of the set

$$\left\{ \begin{pmatrix} y \\ x \end{pmatrix} \in \mathbb{R}^{m+n} \mid \begin{pmatrix} 0 & A \\ I & -B \\ -I & B \end{pmatrix} \begin{pmatrix} y \\ x \end{pmatrix} \leq \begin{pmatrix} b \\ c \\ -c \end{pmatrix} \right\}$$

onto $\mathbb{R}^m$ and thus by Theorem 3.21 a polyhedron. □

This immediately implies that the following sets are also polyhedra.

Corollary 3.24.

- (a) Let $P_1, P_2 \subseteq \mathbb{R}^n$ be two polyhedra. Then the sum $P_1 + P_2$ also is a polyhedron.
- (b) Let $X \in \mathbb{R}^{n \times m}$ be a matrix. Then the following sets are polyhedra, where again $e = (1, \dots, 1)^T$:

$$\begin{aligned} \text{lin } X &:= \{Xc \mid c \in \mathbb{R}^m\} \\ \text{aff } X &:= \{Xc \mid c \in \mathbb{R}^m, \quad e^T c = 1\} \\ \text{cone } X &:= \{Xc \mid c \geq 0\} \\ \text{conv } X &:= \{Xc \mid c \geq 0, \quad e^T c = 1\} \end{aligned}$$

Proof. (a) For $i = 1, 2$ consider $P_i = \{x \in \mathbb{R}^n \mid A_i x \leq b_i\}$. Then we obtain for the sum

$$P_1 + P_2 = \{x + y \mid x \in P_1, y \in P_2\} = \left\{x + y \mid \begin{pmatrix} A_1 & 0 \\ 0 & A_2 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} \leq \begin{pmatrix} b_1 \\ b_2 \end{pmatrix}\right\},$$

i.e. $P_1 + P_2$ is the image of a polyhedron under the linear map $f(x, y) = x + y$ and thus by Theorem 3.23 a polyhedron.

- (b) All four sets are images of a polyhedron under the linear map $f(c) = Xc$ and thus by Theorem 3.23 polyhedra. □

With the previous result we have shown that every set of the form

$$P = \text{conv } V + \text{cone } D$$

with $V \in \mathbb{R}^{n \times p}$ and $D \in \mathbb{R}^{n \times q}$ is a polyhedron.

To show the opposite direction, i.e. that every polyhedron can be written in this form, we need two tools known from linear optimization, namely the Farkas Lemma and polar cones.

Lemma 3.25. (Farkas Lemma) Consider a matrix $A \in \mathbb{R}^{m \times n}$ and a vector $b \in \mathbb{R}^m$. Then exactly one of the following two systems has a solution:

- (a) $Ax = b, \quad x \geq 0$
- (b) $A^T y \leq 0, \quad b^T y > 0$

Here (a) corresponds to the case $b \in \text{cone } A$ meaning b is an element of the cone spanned by the columns of A . If this is not the case, then we can separate the point b strictly from the closed convex set $\text{cone } A$ with a hyperplane, whose normal vectors is y , i.e. we are in case (b).

We can use the Farkas lemma to compute the polar cones to some important sets.

Definition 3.26. Let $X \subseteq \mathbb{R}^n$ be a nonempty set. Then

$$X^\circ := \{w \in \mathbb{R}^n \mid w^T x \leq 0 \quad \forall x \in X\}$$

is called the *polar cone*³ (*Polarkegel*) of X .

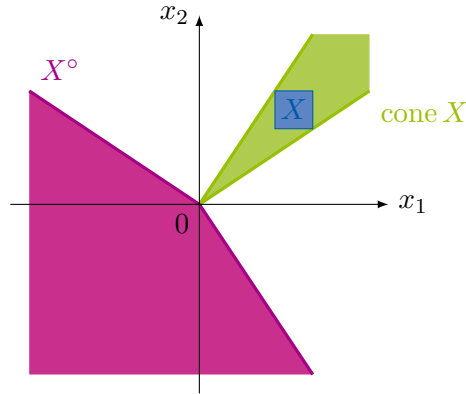


Figure 3.10: Illustration of the polar cone of a set X

Figure 3.10 provides an illustration of the polar cone. Note that by definition the polar cone contains those directions w , whose angle to directions $d \in \text{cone } X$ is at least 90° .

The polar cone has several useful properties, which are collected in the following result.

Lemma 3.27. Let $X, Y \subseteq \mathbb{R}^n$ be two nonempty sets. Then the following holds:

- (a) X° is a nonempty, closed and convex cone.
- (b) $X \subseteq Y$ implies $Y^\circ \subseteq X^\circ$.
- (c) $(X \cup Y)^\circ = X^\circ \cap Y^\circ$.
- (d) $X^\circ = (\text{cone } X)^\circ$.
- (e) $X^{\circ\circ} := (X^\circ)^\circ \supseteq \overline{\text{cone } X}$.

In fact one can even show equality in (e). This then implies that $X^{\circ\circ} = X$ holds if and only if X is a closed convex cone.

Beware: The polar cone of intersections $X \cap Y$ is difficult to compute. In general, we have $(X \cap Y)^\circ \neq X^\circ \cup Y^\circ$.

With the Farkas lemma we can show that for a matrix $A \in \mathbb{R}^{m \times n}$ the following two cones are polar to each other, see Exercise 3.8 for a more general formulation:

$$(\text{cone } A^T)^\circ = P(A, 0) \quad \text{and} \quad P(A, 0)^\circ = \text{cone } A^T. \quad (3.1)$$

Using this result and Corollary 3.24 we can now show that all polyhedral cones can be represented as the conical hull of finitely many directions.

Corollary 3.28. A set $P \subseteq \mathbb{R}^n$ is a polyhedral cone, i.e. $P = P(A, 0)$ with some matrix $A \in \mathbb{R}^{m \times n}$, if and only if there exists a matrix $D \in \mathbb{R}^{n \times q}$ with $P = \text{cone } D$.

Proof. If $P = \text{cone } D$, then P is a polyhedral cone according to Corollary 3.24.

For the opposite direction let $P = P(A, 0)$ be a polyhedral cone with some matrix $A \in \mathbb{R}^{m \times n}$. Then we know from (3.1) that $P = P(A, 0) = (\text{cone } A^T)^\circ$. Using Lemma 3.24 again, we know that $\text{cone } A^T \subseteq \mathbb{R}^n$ is a polyhedral cone, i.e. that there exists a matrix $D \in \mathbb{R}^{n \times q}$ with $\text{cone } A^T = P(D^T, 0)$. And then we can use (3.1) again to obtain

$$P = P(A, 0) = (\text{cone } A^T)^\circ = P(D^T, 0)^\circ = \text{cone } D.$$

□

With these preparations we can now derive the representation of polyhedra as sum of a convex hull and a conical hull.

Theorem 3.29. (Weyl-Minkowski Theorem) A nonempty set $P \subseteq \mathbb{R}^n$ is a polyhedron, i.e. $P = P(A, b)$ if and only if there exist matrices $V \in \mathbb{R}^{n \times p}$ and $D \in \mathbb{R}^{n \times q}$ with

$$P = \text{conv } V + \text{cone } D.$$

Proof. By Corollary 3.24 every set of the form $\text{conv } V + \text{cone } D$ is a polyhedron.

To prove the opposite direction let $P = P(A, b)$ be a polyhedron and consider the polyhedral cone

$$C := \left\{ \begin{pmatrix} x \\ y \end{pmatrix} \in \mathbb{R}^{n+1} \mid \begin{pmatrix} A & -b \\ 0 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} \leq \begin{pmatrix} 0 \\ 0 \end{pmatrix} \right\}.$$

Then we have $x \in P$ if and only if $(x, 1) \in C$.

Since C is a polyhedral cone, we know from Corollary 3.28 that there exists a matrix $\tilde{D} \in \mathbb{R}^{(n+1) \times \tilde{q}}$ with $C = \text{cone } \tilde{D}$. Since all $(x, y) \in C = \{\tilde{D}c \mid c \geq 0\}$ satisfy $y \geq 0$ it follows that $\tilde{D}_{n+1,*} \geq 0$. By multiplying the columns of $\tilde{D}$ with $\frac{1}{D_{n+1,j}}$ in case $D_{n+1,j} > 0$ and changing the order of the columns, both of which does not affect the conical hull, we can assume w.l.o.g. that $\tilde{D}$ is of the form

$$\tilde{D} = \begin{pmatrix} V & D \\ e^T & 0 \end{pmatrix}$$

with $V \in \mathbb{R}^{n \times p}$ and $D \in \mathbb{R}^{n \times q}$. Then we obtain

$$\begin{aligned} x \in P &\iff \begin{pmatrix} x \\ 1 \end{pmatrix} \in C = \text{cone } \tilde{D} \\ &\iff \exists \lambda \geq 0, \mu \geq 0 \text{ with } x = V\lambda + D\mu, e^T \lambda = 1 \\ &\iff x \in \text{conv } V + \text{cone } D. \end{aligned}$$

□

The important feature of this result is again that we only need finitely many points $V_{*,j}$ and directions $D_{*,j}$ in order to describe polyhedra.

For rational polyhedra, i.e. $P = P(A, b)$ with $A \in \mathbb{Q}^{m \times n}$ and $b \in \mathbb{Q}^m$, one can show that the matrices V and D can also always be chosen rational.

We have already seen that the case, where we do not need $\text{conv } V$, i.e. $P = \text{cone } D$, characterizes polyhedral cones. To close this section, we want to show that the case where we do not need $\text{cone } D$, i.e. $P = \text{conv } V$, characterizes the polytopes.

Corollary 3.30. A nonempty set $P \subseteq \mathbb{R}^n$ is a polytope if and only if there exists a matrix $V \in \mathbb{R}^{n \times p}$ with

$$P = \text{conv } V.$$

Proof. By Corollary 3.24 every set of the form $P = \text{conv } V$ is a polyhedron. It is also bounded because for all $x \in P$ there exists a vector $c \geq 0$ with $e^T c = 1$ and $x = Vc$, which implies

$$\|x\| = \|Vc\| \leq \sum_{j=1}^p c_j \|V_{*,j}\| \leq \max_{j=1,\dots,p} \|V_{*,j}\|.$$

Consequently P is a polytope.

Conversely let P be a polytope. Then by Theorem 3.29 we know

$$P = \text{conv } V + \text{cone } D$$

with some matrices $V \in \mathbb{R}^{n \times p}$ and $D \in \mathbb{R}^{n \times q}$. Here, we can assume w.l.o.g. that $\text{conv } V \neq \emptyset$. (In case $p = 0$ we can always choose $V = 0 \in \mathbb{R}^{n \times 1}$.) Now assume that there was a vector $d \in \text{cone } D$ with $d \neq 0$. Then for all $c > 0$ and all $v \in \text{conv } V$ we have

$$v + cd \in P \quad \text{with} \quad \|v + cd\| \rightarrow \infty \text{ for } c \rightarrow \infty.$$

But this contradicts the assumption that P is a polytope. Consequently we either have $q = 0$ or $\text{cone } D = 0$. In either case we obtain $P = \text{conv } V$. $\square$

Polytopes are sometimes defined as the sets which can be generated as the convex hull of finitely many points. Corollary 3.30 ensures that our definition of polytopes is equivalent to this alternative characterization.

It is possible to connect the inner description of polyhedrons, which we derived in this section, and the outer description, which we derived in the previous section. Every polytope can be described as the convex hull of its finitely many vertices, i.e. of the 0-dimensional faces. A polyhedron, which does not contain whole lines, can be represented as the convex hull of its vertices plus the conical hull of its extreme directions. A polyhedron, which does contain a whole line, can according to Theorem 3.29 still be described as the sum of a convex hull and a conical hull, but vertices and extreme directions do not suffice anymore.

Example 3.31. (a) Consider the polyhedron

$$P = \{x \in \mathbb{R}^2 \mid x_1 \geq 0, x_2 \geq 0, x_1 + x_2 = 1\}.$$

This polyhedron is a polytope with the two vertices $v_1 = (1, 0)^T$ and $v_2 = (0, 1)^T$ and can be written as

$$P = \text{conv}\{v_1, v_2\}.$$

(b) Consider the polyhedron

$$P = \{x \in \mathbb{R}^2 \mid x_1 \geq 0, x_2 \geq 0\}.$$

This polyhedron is a cone with one vertex $v_1 = (0, 0)^T$ and two extreme directions $d_1 = (1, 0)^T$ and $d_2 = (0, 1)^T$ and can be written as

$$P = \text{cone}\{d_1, d_2\}.$$

(c) Consider the polyhedron

$$P = \{x \in \mathbb{R}^2 \mid x_1 \geq 0, x_2 \geq 0, x_1 + x_2 \geq 1\}.$$

This unbounded polyhedron has the two vertices $v_1 = (1, 0)^T$ and $v_2 = (0, 1)^T$ and the two extreme directions $d_1 = (1, 0)^T$ and $d_2 = (0, 1)^T$ and can be written as

$$P = \text{conv}\{v_1, v_2\} + \text{cone}\{d_1, d_2\}.$$

(d) Consider the polyhedron

$$P = \{x \in \mathbb{R}^2 \mid x_1 + x_2 \geq 1\}.$$

This unbounded polyhedron has no vertices and the two extreme directions $d_1 = (1, -1)^T$ and $d_2 = (-1, 1)^T$. But these two extreme directions are not enough to represent the polyhedron. We need one additional direction, e.g. $d_3 = (1, 1)^T$ and a point on the extreme line, e.g. $v_1 = (1, 0)^T$ and can then write

$$P = \text{conv}\{v_1\} + \text{cone}\{d_1, d_2, d_3\}.$$

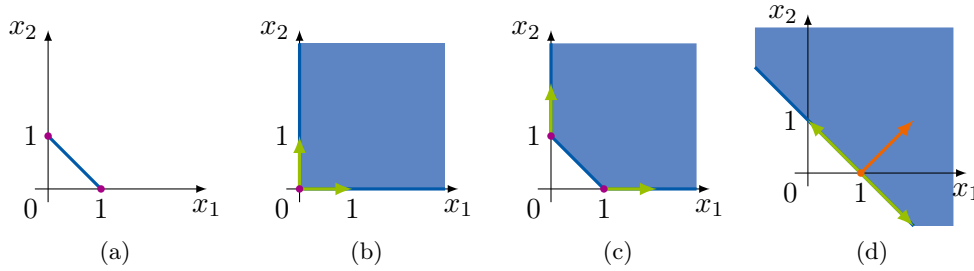


Figure 3.11: Illustration of Example 3.31

Analogously to Theorem 3.29 one can show the following result for rational polyhedra.

Theorem 3.32. $P = P(a, b) \subseteq \mathbb{R}^n$ is a rational polyhedron, i.e. has a representation with $A \in \mathbb{Q}^{m \times n}$ and $b \in \mathbb{Q}^m$, if and only if there exist rational matrices $V \in \mathbb{Q}^{n \times p}$ and $D \in \mathbb{Q}^{n \times q}$ such that

$$P = \text{conv } V + \text{cone } D.$$

3.4 Exercises

Exercise 3.1. Prove that a nonempty polyhedron $P \subseteq \mathbb{R}^n$ is a cone if and only if it has a representation of the form

$$P = P(A, 0) = \{x \in \mathbb{R}^n \mid Ax \leq 0\}$$

with some matrix $A \in \mathbb{R}^{m \times n}$.

Exercise 3.2. Let $A \subseteq \mathbb{R}^n$ be an affine subspace. Prove that then the subspace $U := A - a^*$ is independent from the reference point $a^* \in A$.

This observation ensures that the dimension of an affine subspace is well defined.

Exercise 3.3. Vectors $x^1, \dots, x^m \in \mathbb{R}^n$ are called *affine independent*, if the two conditions

$$\sum_{k=1}^m c_k x^k = 0 \quad \text{and} \quad \sum_{k=1}^m c_k = 0$$

are only satisfied for $c_1 = \dots = c_m = 0$.

Prove that the following statements are equivalent:

- (a) The vectors $x^1, \dots, x^m \in \mathbb{R}^n$ are affine independent.
- (b) The vectors $x^2 - x^1, \dots, x^m - x^1$ are linearly independent.
- (c) The vectors $\begin{pmatrix} x^1 \\ 1 \end{pmatrix}, \dots, \begin{pmatrix} x^m \\ 1 \end{pmatrix}$ are linearly independent.

Exercise 3.4. Let $X \subseteq \mathbb{R}^n$ be a nonempty set. Prove that the affine hull of X can then also be computed as

$$\text{aff } X = \text{lin}(X - X) + x^* \subseteq \text{lin}(X - X) + X$$

for all $x^* \in X$.

Exercise 3.5. Let $P = P(A, b) \subseteq \mathbb{R}^n$ be a nonempty polyhedron with $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. Further let F be a nonempty face of P . Prove that a point $x^* \in F$ is a relative interior point of the face F if and only if

$$\text{eq}(\{x^*\}) = \text{eq}(F) \quad \text{or equivalently} \quad A_{\text{ineq}(F),*} x^* < b_{\text{ineq}(F)}.$$

Exercise 3.6. Let $P = P(A, b) \subseteq \mathbb{R}^n$ be a nonempty polyhedron with $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. Prove that for a point $x^* \in P$ the following are equivalent:

- (a) x^* is a vertex of P .
- (b) x^* is an extreme point of P .
- (c) $\text{rank } A_{\text{eq}(\{x^*\}),*} = n$.

Exercise 3.7. Consider a polyhedron of the form

$$P := P^=(A, b) = \{x \in \mathbb{R}^n \mid Ax = b, x \geq 0\}$$

with $A \in \mathbb{R}^{m \times n}$ and $b \in \mathbb{R}^m$. Prove that $x^* \in P$ is a vertex of P if and only if the columns of the matrix

$$A_{*,J} \quad \text{with} \quad J = \{j = 1, \dots, n \mid x_j^* > 0\}$$

are linearly independent.

Exercise 3.8. For $A \in \mathbb{R}^{n \times m}$ and $B \in \mathbb{R}^{n \times p}$ consider the two polyhedral cones

$$\begin{aligned} C_1 &:= \{y = A\lambda + B\mu \in \mathbb{R}^n \mid \lambda \in \mathbb{R}^m, \lambda \geq 0, \mu \in \mathbb{R}^p\}, \\ C_2 &:= \{x \in \mathbb{R}^n \mid A^T x \leq 0, B^T x = 0\}. \end{aligned}$$

Prove that these cones are polar to each other, i.e. that we have

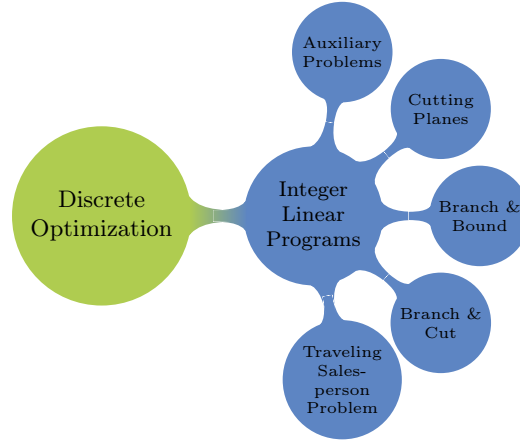
$$C_1^\circ = C_2 \quad \text{and} \quad C_2^\circ = C_1.$$

The cone C_2 has the form of the linearization cone $L_X(x^*)$ used in the optimality conditions for constrained nonlinear optimization problems. And the cone C_1 appears in the gradient equation of the KKT conditions. By proving that the two cones are polar to each other, we have thus proven that

$$\begin{aligned} &\nabla f(x^*)^T d \geq 0 \quad \forall d \in L_X(x^*) \\ \iff &-\nabla f(x^*) \in L_X(x^*)^\circ \\ \iff &x^* \text{ is a KKT point.} \end{aligned}$$

Unfortunately in local minima x^* we typically only know $\nabla f(x^*)^T d \geq 0$ for all feasible directions $d \in T_X(x^*)$, where $T_X(x^*)$ is the tangent cone satisfying $T_X(x^*) \subseteq L_X(x^*)$. This is the reason why we need constraint qualifications in order to ensure that local minima of constrained nonlinear optimization problems are KKT points. One such constraint qualification is that all constraints are linear.

4 (Mixed-)Integer Linear Programs



In this chapter we mostly consider *integer linear programs* (*ganzzahlige lineare Programme*), i.e. optimization problems of the form

$$\min_x c^T x \quad \text{s.t.} \quad x \in P \cap \mathbb{Z}^n \quad (4.1)$$

with a vector $c \in \mathbb{R}^n$ and a polyhedron $P \subseteq \mathbb{R}^n$. Most of the time, we will assume that the problem data is rational, i.e. $c \in \mathbb{Q}^n$ and $P = \{x \in \mathbb{R}^n \mid Ax \leq b\}$ with $A \in \mathbb{Q}^{m \times n}$ and $b \in \mathbb{Q}^m$.

As we already know from linear programs, there exist several common formulations for the feasible polyhedron, such as

$$\begin{aligned} P(A, b) &:= \{x \in \mathbb{R}^n \mid Ax \leq b\}, \\ P^=(A, b) &:= \{x \in \mathbb{R}^n \mid Ax = b, x \geq 0\}. \end{aligned}$$

However, by applying the usual tricks such as introducing slack variables to turn inequality constraints into equality constraints and splitting variables into their positive and negative part, it is possible to transform all polyhedra into either of these two forms. And it is quite straightforward to see that, if a linear program can be described with rational (or integer) data, these transformations preserve that property. Thus, when we assume that a linear program can be described with rational data, it does not really matter in which form the polyhedron is given.

Example 4.1. Consider the linear program

$$\min_x c^T x \quad \text{s.t.} \quad Ax = b, x \geq 0, x \in \mathbb{Z}^n$$

with $c \in \mathbb{Q}^n$, $A \in \mathbb{Q}^{m \times n}$ and $b \in \mathbb{Q}^m$. Then this problem is equivalent to the linear program

$$\min_x c^T x \quad \text{s.t.} \quad \begin{pmatrix} A \\ -A \\ -I \end{pmatrix} x \leq \begin{pmatrix} b \\ -b \\ 0 \end{pmatrix}, x \in \mathbb{Z}^n,$$

which also has rational data.

Conversely, consider the linear program

$$\min_x c^T x \quad \text{s.t.} \quad Ax \leq b, x \in \mathbb{Z}^n$$

with $c \in \mathbb{Q}^n$, $A \in \mathbb{Q}^{m \times n}$ and $b \in \mathbb{Q}^m$. Multiplying A and b with a sufficiently large integer, if necessary, we can assume w.l.o.g. that $A \in \mathbb{Z}^{m \times n}$ and $b \in \mathbb{Z}^m$. Then x^* solves this problem if and only if (x^+, x^-, s) with $x^+ = \max\{x^*, 0\}$, $x^- = \max\{-x^*, 0\}$ and $s = b - Ax^* \in \mathbb{Z}^m$ solves the linear program

$$\min_{x^+, x^-, s} \begin{pmatrix} c \\ -c \\ 0 \end{pmatrix}^T \begin{pmatrix} x^+ \\ x^- \\ s \end{pmatrix} \quad \text{s.t.} \quad (A \quad -A \quad I) \begin{pmatrix} x^+ \\ x^- \\ s \end{pmatrix} = b, \begin{pmatrix} x^+ \\ x^- \\ s \end{pmatrix} \geq 0, \begin{pmatrix} x^+ \\ x^- \\ s \end{pmatrix} \in \mathbb{Z}^n.$$

Before we develop solution algorithms for integer linear programs in Sections 4.2, 4.3 and 4.4, let us first collect some useful properties in Section 4.1. We end this chapter on integer linear programs by applying the solutions algorithms to the traveling salesperson problem in Section 4.5.

4.1 Integer Linear Programs and Auxiliary Problems

In this section, we have a look at two auxiliary problems and discuss how solving these relates to solving the integer linear problem (4.1). We begin with the LP relaxation in Section 4.1.1 and then have a look at the integer hull in Section 4.1.2. Finally, we discuss the situation, where both auxiliary problems coincide, in Section 4.1.3.

4.1.1 The LP-Relaxation

Since we already know how to solve linear programs, an obvious approach to solving the integer linear program (4.1) is to solve the corresponding *LP relaxation* (*LP-Relaxierung*)

$$\min_x c^T x \quad \text{s.t.} \quad x \in P, \tag{4.2}$$

which is a linear program.

For linear programs we know that these are always either infeasible, unbounded or have a solution. For integer linear programs (ILP) with a rational polyhedron P we will see shortly that the same is true, i.e. that they are also either infeasible, unbounded or have a solution. An interesting question is thus, which combinations of these three situations can occur for an ILP and its LP relaxation. One can show that only the following combinations are possible, if P is rational:

	LP rel. infeasible	LP rel. has a solution	LP rel. unbounded
ILP infeasible	yes	yes	yes
ILP has a solution	no	yes	no
ILP unbounded	no	no	yes

Since the feasible set P of the LP relaxation is a superset of the feasible set $P \cap \mathbb{Z}^n$ of the ILP, infeasibility of the LP relaxation immediately implies that the ILP also has to be infeasible.

Along the same lines, if the objective function attains a minimum on the feasible set P of the LP relaxation, it cannot be unbounded on the feasible set $P \cap \mathbb{Z}^n$ of the ILP. However, it is possible that the LP relaxation has a solution, whereas the ILP is infeasible. As an example consider the polyhedron $P = \{0.5\} \subset \mathbb{R}^1$ with $P \cap \mathbb{Z} = \emptyset$. And it is also possible that both the LP relaxation and the ILP attain a solution, e.g. in case $P = [0, 1] \subset \mathbb{R}^1$ with $P \cap \mathbb{Z} = \{0, 1\}$.

In case the LP relaxation is unbounded, the ILP can also be unbounded. As an example consider

$$\min_{x \in \mathbb{R}} x \quad \text{s.t.} \quad x \in \mathbb{R} \cap \mathbb{Z}.$$

And it is possible that the LP relaxation is unbounded but the ILP is infeasible. As an example consider

$$\min_{x \in \mathbb{R}^2} x_1 \quad \text{s.t.} \quad x_2 = 0.5, x \in \mathbb{Z}^2.$$

But if we restrict ourselves to rational problem data, then an unbounded LP relaxation can not occur, if the ILP has a solution.

Theorem 4.2. Consider an integer linear program of the form

$$\min_x c^T x \quad \text{s.t.} \quad x \in P, x \in \mathbb{Z}^n$$

with a rational polyhedron P . If this problem has at least one integer feasible point and the corresponding LP relaxation is unbounded, then the integer linear program is unbounded.

Proof. By Theorem 3.32 there exist rational matrices $V \in \mathbb{Q}^{n \times p}$ and $D \in \mathbb{Q}^{n \times q}$ such that

$$P = \text{conv } V + \text{cone } D.$$

Here, we can always scale D with a positive integer to obtain $D \in \mathbb{Z}^{n \times q}$. We thus know that all $x \in P$ can be written as

$$x = V\lambda + D\mu$$

with $\lambda \geq 0$, $e^T \lambda = 1$ and $\mu \geq 0$. Since the LP relaxation is unbounded, there has to exist at least one direction $D_{*,i}$ with $c^T D_{*,i} < 0$. Otherwise, for all $x \in P$ we would have

$$c^T x = \sum_{j=1}^p \lambda_j c^T V_{*,j} + \sum_{j=1}^q \mu_j c^T D_{*,j} \geq \sum_{j=1}^p \lambda_j c^T V_{*,j} \geq \min_{j=1, \dots, p} c^T V_{*,j}$$

and thus the LP relaxation would be bounded below.

By assumption, the integer problem has at least one feasible point $x^* \in \mathbb{Z}^n$. Then by construction, all points

$$x(\mu_i) := x^* + \mu_i D_{*,i}$$

with $\mu_i \geq 0$ are also feasible for the LP relaxation and satisfy $c^T x(\mu_i) \rightarrow -\infty$ for $\mu_i \rightarrow +\infty$. And since $x^* \in \mathbb{Z}^n$ and $D_{*,i} \in \mathbb{Z}^n$, there are infinitely many integer points on this half-line. This shows that the integer problem also is unbounded. $\square$

Thus, if the ILP is feasible and we consider only rational data A, b , then the solution behavior of the LP relaxation is the same as the solution behavior of the ILP, i.e. either both are unbounded or both have a solution.

If instead we also allow irrational problem data, additional cases are possible, see Example 4.3 for details.

Example 4.3. (a) Consider the ILP

$$\min_{x \in \mathbb{R}^2} x_1 \quad \text{s.t.} \quad x_1 - \sqrt{2}x_2 = 0, \quad x \in \mathbb{Z}^2.$$

Then the corresponding LP relaxation is unbounded, while the ILP has exactly one feasible point and thus a solution.

(b) Consider the ILP

$$\min_x x_1 - \sqrt{2}x_2 \quad \text{s.t.} \quad x_1 - \sqrt{2}x_2 \geq 0, \quad x \geq 0, \quad x \in \mathbb{Z}^2.$$

Then the corresponding LP relaxation has feasible points and is bounded below, i.e. has a solution. The ILP also has feasible points and is bounded below. But the ILP cannot have a solution, because this solution x^* would be the best rational approximation $\frac{x_1^*}{x_2^*} \geq \sqrt{2}$ of $\sqrt{2}$.

If a solution of the LP relaxation is integer, then it also is a solution of the ILP due to the inclusion of the feasible sets $P \cap \mathbb{Z}^n \subseteq P$. However, a solution of the LP relaxation does not have to be integer and can have a strictly smaller objective function value than the minimal objective function value of the ILP. Separating such points from the feasible set of the ILP is then the basic idea of solution algorithms such as the cutting plane method, see Section 4.2.

4.1.2 The Integer Hull

But before we have a look at solution algorithms, let us first have a look at another auxiliary problem. We still want to solve the integer linear program (4.1). Instead of working with the larger set P an alternative and possibly better choice for the feasible set of an auxiliary problem would be the following.

Definition 4.4. Let $P \subseteq \mathbb{R}^n$ be a polyhedron. Then the set

$$P_I := \text{conv}(P \cap \mathbb{Z}^n)$$

is called the *integer hull* (*ganzzahlige Hülle*) of P .

To illustrate the relation between the three feasible sets P , $P \cap \mathbb{Z}^n$ and P_I let us have a look at the following example.

Example 4.5. For the integer linear program

$$\begin{aligned} \min_x c^T x \quad & \text{s.t.} \quad -4x_1 + 6x_2 \leq 9, \\ & x_1 + x_2 \leq 4, \\ & x \geq 0, \quad x \in \mathbb{Z}^2. \end{aligned} \tag{4.3}$$

the three types of feasible sets are illustrated in Figure 4.1.

First, let us note that P_I is not necessarily a polyhedron, if we allow irrational polyhedra P . This is illustrated in the following example.

Example 4.6. Consider the polyhedron

$$P := \{x \in \mathbb{R}^2 \mid x \geq 0, \quad x_1 - \sqrt{2}x_2 \geq 0\}$$

illustrated in Figure 4.2. If the corresponding integer hull $P_I = \text{conv}(P \cap \mathbb{Z}^2)$ were a polyhedron in this case, then the optimization problem

$$\min_x x_1 - \sqrt{2}x_2 \quad \text{s.t.} \quad x \in P_I$$

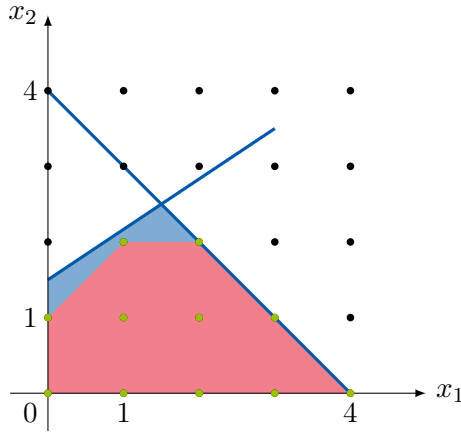


Figure 4.1: Feasible set $P \cap \mathbb{Z}^n$ of the integer problem (4.1) in green, feasible set P of the LP relaxation in blue, and integer hull P_I in red.

would be a linear program with a nonempty feasible set and be bounded below. For a linear program, these two properties guarantee the existence of a solution. Since $P_I = \text{conv}(P \cap \mathbb{Z}^2)$, the existence of a solution on P_I also implies the existence of a solution x^* on $P \cap \mathbb{Z}^2$. This solution x^* would then satisfy $\frac{x_1^*}{x_2^*} \geq \sqrt{2}$ and by construction describe the best rational number with this property. But we know that there does not exist a best rational number $\frac{n}{m} \geq \sqrt{2}$. Instead we can find rational numbers arbitrarily close to the irrational number $\sqrt{2}$.

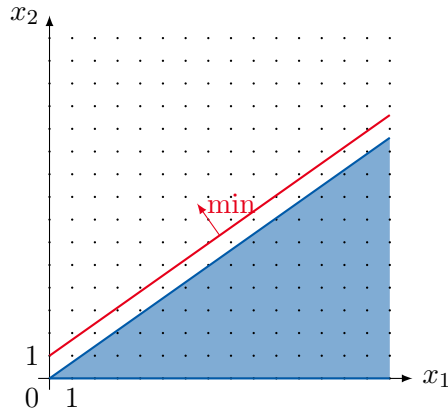


Figure 4.2: Feasible set of Example 4.6

However, if we restrict our attention to rational polyhedra (or bounded polyhedra), then we can ensure that P_I is a polyhedron, too.

Lemma 4.7. Let $P \subseteq \mathbb{R}^n$ be a polyhedron and $P_I = \text{conv}(P \cap \mathbb{Z}^n)$. Then the following holds:

- (a) $P_I \subseteq P$ and $P \cap \mathbb{Z}^n = P_I \cap \mathbb{Z}^n$
- (b) If P is a polytope, then P_I is a rational polytope.
- (c) If P is a rational cone, then $P = P_I$.
- (d) If P is a rational polyhedron, then P_I is a rational polyhedron.

Proof. (a) Due to $P_I = \text{conv}(P \cap \mathbb{Z}^n)$ and P being a polyhedron and thus convex we immediately obtain $P_I \subseteq P$. Furthermore, we have $P \cap \mathbb{Z}^n \subseteq P_I \subseteq P$, which implies $P \cap \mathbb{Z}^n = P_I \cap \mathbb{Z}^n$.

(b) If P is a polytope, then $P \cap \mathbb{Z}^n$ is finite and thus P_I is the convex hull of finitely many integer points. By Theorem 3.32 the set P_I is thus a rational polytope.

(c) By Theorem 3.32 the rational cone P can be written as $P = \text{cone } D$ with a rational matrix $D \in \mathbb{Q}^{n \times q}$. By multiplying D with sufficiently large positive integers, we can assume w.l.o.g. $D \in \mathbb{Z}^{n \times q}$.

We always have the inclusion $P_I \subseteq P$, thus it suffices to show the opposite inclusion $P \subseteq P_I$. Since $0 \in P_I$ and $kD_{*,j} \in P_I$ for all $j = 1, \dots, q$ and all $k \in \mathbb{N}$, it follows that the complete half line $kD_{*,j}$ with $k \geq 0$ is included in P_I for all $j = 1, \dots, q$. And since P_I is a convex set, this implies $P = \text{cone } D \subseteq P_I$.

(d) By Theorem 3.32 the rational polyhedron P can be written as $P = \text{conv } V + \text{cone } D$ with a rational matrices $V \in \mathbb{Q}^{n \times p}$ and $D \in \mathbb{Q}^{n \times q}$. Again we can assume w.l.o.g. $D \in \mathbb{Z}^{n \times q}$.

Let us define the set

$$B := \{D\mu \mid 0 \leq \mu \leq e\} \subset \mathbb{R}^n.$$

Then B is a polytope and $B \subseteq \text{cone } D$. We claim that

$$P_I = (\text{conv } V + B)_I + \text{cone } D =: M.$$

Since $\text{conv } V$ and B are polytopes, their sum is also a polytope and thus $(\text{conv } V + B)_I$ is a rational polytope. Thus, the set M is a rational polyhedron by Theorem 3.32.

It remains to show that we indeed have $P_I = M$.

We begin by showing $P_I \subseteq M$. Since $P_I = \text{conv}(P \cap \mathbb{Z}^n)$ and the set M is convex, it suffices to show $P \cap \mathbb{Z}^n \subseteq M$. To this end consider an arbitrary point $x = V\lambda + D\mu \in P \cap \mathbb{Z}^n$ with $\lambda \geq 0$, $e^T \lambda = 1$ and $\mu \geq 0$. Then we define $\lfloor \mu \rfloor \in \mathbb{N}_0^n$ by rounding all components of μ down. Then we have

$$x = V\lambda + D(\mu - \lfloor \mu \rfloor) + D\lfloor \mu \rfloor.$$

Since both x and $D\lfloor \mu \rfloor$ are integer vectors, it follows that $V\lambda + D(\mu - \lfloor \mu \rfloor)$ is an integer vector and thus $V\lambda + D(\mu - \lfloor \mu \rfloor) \in (\text{conv } V + B)_I$. This shows $x \in M$ and thus the inclusion $P \cap \mathbb{Z}^n \subseteq M$.

To show the opposite inclusion $M \subseteq P_I$ note that

$$\begin{aligned} M &= (\text{conv } V + B)_I + \text{cone } D \\ &\subseteq (\text{conv } V + \text{cone } D)_I + \text{cone } D \\ &= P_I + \text{cone } D \\ &= P_I + (\text{cone } D)_I \\ &\subseteq (P + \text{cone } D)_I \\ &= P_I \end{aligned}$$

□

We now know that the set P_I is a polyhedron under suitable assumptions, e.g. in case P is a rational polyhedron or $P \cap \mathbb{Z}^n$ is bounded. In this case, the problem

$$\min_x c^T x \quad \text{s.t.} \quad x \in P_I \tag{4.4}$$

is a linear program. It turns out that this linear program has a very close relation to the integer linear program (4.1).

Theorem 4.8. Let $P \subseteq \mathbb{R}^n$ be a polyhedron and $P_I = \text{conv}(P \cap \mathbb{Z}^n)$ be nonempty and a polyhedron. Then the following holds:

- (a) The ILP (4.1) is unbounded if and only problem (4.4) is unbounded.
- (b) If the problem (4.4) is bounded, then it attains a solution. It also has at least one integer solution and all integer solutions of problem (4.4) are solutions of the ILP (4.1).
- (c) If the ILP (4.1) is bounded, it attains a solution and all solutions of the ILP (4.1) are also solutions of problem (4.4).

Proof. First note that $P_I = \text{conv}(P \cap \mathbb{Z}^n)$. Every point $x \in P_I$ can thus be written as $x = \sum_{i=1}^{n+1} \lambda_i x^i$ with $\lambda \geq 0$, $e^T \lambda = 1$ and $x^1, \dots, x^{n+1} \in P \cap \mathbb{Z}^n$. For the objective function this implies

$$c^T x = \sum_{i=1}^{n+1} \lambda_i c^T x^i \geq \inf_{x^i \in P \cap \mathbb{Z}^n} c^T x^i$$

and thus

$$\inf_{x \in P_I} c^T x \geq \inf_{x \in P \cap \mathbb{Z}^n} c^T x.$$

On the other hand, we also have $P \cap \mathbb{Z}^n \subset P_I$ and thus obtain

$$\inf_{x \in P_I} c^T x = \inf_{x \in P \cap \mathbb{Z}^n} c^T x$$

- (a) This follows from $P_I \neq \emptyset$ if and only if $P \cap \mathbb{Z}^n \neq \emptyset$ and the observation above.
- (b) By assumption the linear program (4.4) has feasible points and is bounded, i.e. attains a solution. By the observation above this is only possible if there exists at least one feasible point of the ILP with the same objective function value and all such integer feasible points are then solutions of the ILP.
- (c) If the ILP is bounded, then by the observation above the linear program (4.4) is also bounded and thus has a solution. Part (b) then implies that both problems attain solutions with the same objective function value. Since $P \cap \mathbb{Z}^n \subseteq P_I$, all solutions of the ILP are also solutions of the linear program (4.4).

□

An immediate consequence of this result is the following.

Corollary 4.9. Consider an integer linear program (4.1), where $P_I = \text{conv}(P \cap \mathbb{Z})$ is a polyhedron. Then the integer linear program is either infeasible, unbounded or has a solution.

The close relation between the ILP (4.1) and problem (4.4), which is a linear program under suitable assumptions, motivates to try to solve the ILP by solving problem (4.4) instead.

Unfortunately, it is usually prohibitively expensive to compute the integer hull P_I of a polyhedron P . In the cutting plane method, see Section 4.2, we instead try to iteratively improve an approximation of P_I in that area where we expect solutions of the ILP.

4.1.3 Integral Polyhedra

A notable exception, where we get the integer hull P_I for free, is the case $P = P_I$.

Definition 4.10. A polyhedron $P \subseteq \mathbb{R}^n$ is called *integral* (*ganzzahlig*), if $P = P_I$.

Please note: A polyhedron is called rational, if it can be described with rational data A, b . By scaling this data appropriately, we can always describe a rational polyhedron P with integer data A, b . But this is not what we mean by an integral polyhedron. Instead an integral polyhedron coincides with its own integer hull.

Some alternative characterizations of integral polyhedra are collected below¹.

Lemma 4.11. Let $P \subseteq \mathbb{R}^n$ be a nonempty rational polyhedron. Then the following are equivalent:

- (a) P is an integral polyhedron, i.e. $P = P_I$.
- (b) Every nonempty face of P contains at least one integer point.
- (c) Every nonempty *minimal* face of P contains at least one integer point.
- (d) For all $c \in \mathbb{R}^n$, for which the linear program

$$\min_x c^T x \quad \text{s.t.} \quad x \in P$$

attains a solution, it also attains an integer solution.

Proof. (b) $\implies$ (c) Every nonempty minimal face of P is a nonempty face of P .

(c) $\implies$ (d) Let $c \in \mathbb{R}^n$ be given such that the linear program attains a solution $x^* \in P$. Then $c^T x \geq c^T x^*$ is a valid inequality for the polyhedron P , which induces a nonempty face consisting of all solutions of the linear program. This nonempty face contains a nonempty minimal face. And according to (c) this nonempty minimal face contains at least one integer point, i.e. the linear program also has an integer solution.

(d) $\implies$ (b) Every nonempty face F of P is induced by a valid inequality $a^T x \leq \beta$. The linear program

$$\min_x -a^T x \quad \text{s.t.} \quad x \in P$$

thus has exactly the nonempty face F as solution set. With (d) it follows that the face F contains at least one integer point.

¹Here, we call a nonempty face F minimal, if there does not exist a face G with $\emptyset \neq G \subsetneq F$.

(a) $\implies$ (d) Let the linear program with feasible set $P = P_I$ have a solution. Then by Theorem 4.8 least one of these solutions is also a solution of the integer problem.

(d) $\implies$ (a) Since $P \neq \emptyset$ the linear program has a solution for $c = 0$. With (d) it follows that the linear program then also attains an integer solution and thus $P_I \neq \emptyset$.

Since we assumed that P is a rational polyhedron, we know from Lemma 4.7 that P_I is a polyhedron. And since we know $P_I \neq \emptyset$, we know from Theorem 4.2 that a linear program with feasible set P is unbounded if and only if the corresponding ILP or equivalently the linear program with feasible set P_I is unbounded.

We always have the inclusion $P_I \subseteq P$. To prove the opposite inclusion assume that there exists a point $x^* \in P \setminus P_I$. Then we can apply the strict separation theorem to the nonempty, closed and convex set P_I and the point $x^* \notin P_I$ to find a separating inequality

$$c^T x^* < \beta < c^T x \quad \text{for all } x \in P_I.$$

Then the linear program with objective function vector c and feasible set P_I is bounded below and thus so is the linear program with feasible set P . According to (d) the linear program with feasible set P then also attains an integer solution z^* . But for this integer solution we know $z^* \in P_I$ and thus $c^T z^* > \beta > c^T x^*$. Due to $x^* \in P$ this contradicts the fact that z^* solves the linear program with feasible set P . This shows $P = P_I$. $\square$

We need the assumption that P is a rational polyhedron in order to ensure that (b–d) also implies (a). As a counterexample consider the polyhedron $P = \{x \in \mathbb{R}^2 \mid x_1 = \sqrt{2}x_2\}$. Then $P_I = P \cap \mathbb{Z}^2 = \{(0,0)\}$, i.e. the polyhedron is not integral and thus (a) does not hold. But (b, c) hold, because the only nonempty face of P is P itself, which contains an integer point. And (d) holds, because an LP with feasible set P is either unbounded or the whole set P is the solution set, which contains an integer point.

We have to work with minimal faces instead of vertices in the previous result, because a polyhedron $P = P(A, b) \subseteq \mathbb{R}^n$ has vertices only if $\text{rank } A = n$. In this case the minimal faces of P are exactly its vertices. But in case $\text{rank } A < n$ the minimal faces of P have a higher dimension. As an example consider

$$P = \{x \in \mathbb{R}^2 \mid 0 \leq x_1 \leq 1\}.$$

Here, the minimal faces are $F_0 = \{x \in \mathbb{R}^2 \mid x_1 = 0\}$ and $F_1 = \{x \in \mathbb{R}^2 \mid x_1 = 1\}$.

In contrast, for all polyhedra $P = P(A, b) \subseteq \mathbb{R}_+^n$ we know that we can rewrite them as

$$P = \left\{ x \in \mathbb{R}^n \mid \begin{pmatrix} A \\ -I \end{pmatrix} x \leq \begin{pmatrix} b \\ 0 \end{pmatrix} \right\}.$$

Since the resulting matrix has full column rank, i.e. $\text{rank } n$, we know that for these polyhedra the minimal faces are exactly the vertices, i.e. a polyhedron $P \subseteq \mathbb{R}_+^n$ is integral if and only if all of its vertices are integer.

Based on our previous discussion, integral polyhedra have the useful property that we can solve the LP relaxation (4.2) with the feasible set $P = P_I$ instead of the ILP (4.1). And we then know that the ILP has a solution if and only if the LP relaxation has a solution. And in case the LP relaxation has a solution, it also has an integer solution, which then solves the ILP. In case the polyhedron P has vertices, these vertices are integer. And since the simplex method computes vertex solutions of linear programs, we automatically get an integer solution of the LP relaxation.

Verifying whether a polyhedron is integral can be expensive. However, there exist some conditions on the data A, b , which immediately imply that the corresponding polyhedron is integral. A useful notion in this context are unimodular and totally unimodular matrices.

Definition 4.12. Let $A \in \mathbb{R}^{m \times n}$ be a matrix.

- (a) The matrix A is called *unimodular* (*unimodular*), if
 - it has full row rank m and
 - every full-sized square submatrix $A_{*J} \in \mathbb{R}^{m \times m}$ with $|J| = m$ has determinant 0, +1 or -1.
- (b) The matrix A is called *totally unimodular* (*total unimodular*), if
 - every square submatrix A_{IJ} with $|I| = |J| \leq \min\{m, n\}$ has determinant 0, +1 or -1.

Some remarks are in order: For unimodular matrices we consider only full-sized $m \times m$ -submatrices and additionally demand that the matrix has full row rank. For totally unimodular matrices, we do not have a rank requirement, but we consider all square submatrices. Since this includes 1×1 -submatrices, we immediately know that all entries of a totally unimodular matrix have to be 0, +1 or -1.

The importance of unimodular matrices for integer linear programs follows from the subsequent result. Here, we only consider integer data $A \in \mathbb{Z}^{m \times n}$ and $b \in \mathbb{Z}^m$. Since we already restricted our attention to rational data to ensure that P_I is a polyhedron, this is not really a further restriction.

Theorem 4.13. Let $A \in \mathbb{Z}^{m \times n}$ have full row rank m . Then the following are equivalent:

- (a) A is unimodular.

- (b) For all right-hand sides $b \in \mathbb{Z}^m$ such that the polyhedron $P = P^=(A, b)$ is nonempty, the polyhedron P is integral, i.e. all vertices of P are integer.

Proof. Let A be unimodular, $b \in \mathbb{Z}^m$ arbitrary such that $P = P^=(A, b)$ is nonempty, and x^* be a vertex of P . Since A has full row rank, we then know that there exists a basis B such that $x^* = (x_B^*, x_N^*) = (A_B^{-1}b, 0)$. Here, the nonbasic variables $x_N^* = 0$ are obviously integer. The basic variables x_B^* are solutions of the linear equation $A_B x_B = b$ with the regular basic matrix A_B and can thus be computed componentwise using Cramer's rule

$$x_{B_j}^* = \frac{\det A_j}{\det A_B},$$

where in the matrix A_j the j -th column of A_B is replaced by b . Since the basic matrix A_B is regular, the unimodularity of A implies $\det A_B \in \{-1, +1\}$. And because A and b have only integer components, we know that $\det A_j$ is an integer. Consequently, x_B^* is integer. This proves that all vertices of P are integer.

Conversely, let all vertices x^* of $P^=(A, b)$ be integer for all $b \in \mathbb{Z}^m$ such that the polyhedron is nonempty. Consider an arbitrary square submatrix A_{*J} with $|J| = m$. If A_{*J} is singular, we know $\det A_{*J} = 0$ and are done. If A_{*J} is regular, then we know $\det A_{*J} \in \mathbb{Z}$ and $\det A_{*J}^{-1} = \frac{1}{\det A_{*J}}$. Consequently, if we can show that A_{*J}^{-1} is also an integer matrix and thus $\det A_{*J}^{-1} \in \mathbb{Z}$, we know that both matrices have to have determinant $+1$ or -1 . We can compute the j -th column a_j of A_{*J}^{-1} by solving the linear equation $A_{*J} a_j = e_j$. Now choose an integer vector $y \in \mathbb{Z}^n$ such that $y + a_j \geq 0$ and define $b := A_{*J} y + e_j \in \mathbb{Z}^m$. Then we have

$$A_{*J}^{-1} b = A_{*J}^{-1} (A_{*J} y + e_j) = y + A_{*J}^{-1} e_j = y + a_j \geq 0.$$

Consequently, the vector $(y + a_j, 0)$ is a vertex of the polyhedron $P^=(A, b)$ with $b \in \mathbb{Z}^m$ and thus by assumption integer. And since we chose y as an integer vector, the vector a_j also has to be integer. We can use this argument to show that all columns a_j of A_{*J}^{-1} are integer. $\square$

If A has full row rank and is totally unimodular, it is unimodular and thus it follows that $P^=(A, b)$ is integral for all $b \in \mathbb{Z}^m$.

For polyhedra of different forms we can obtain analogous results.

Corollary 4.14. (Hoffman-Kruskal Theorem) Let $A \in \mathbb{Z}^{m \times n}$ be an integer matrix. Then the following are equivalent:

- (a) A is totally unimodular.

(b) For all right-hand sides $b \in \mathbb{Z}^m$ such that the polyhedron

$$P := \{x \in \mathbb{R}^n \mid Ax \leq b, x \geq 0\}$$

is nonempty, the polyhedron P is integral, i.e. all vertices of P are integer.

Proof. We know from Exercise 4.1 that the matrix A is totally unimodular if and only if the matrix $(A \ I)$ is unimodular. And we know from Exercise 4.2 that the polyhedron P is integral if and only if the polyhedron

$$\tilde{P} := \{z = (x, s) \in \mathbb{R}^{n+m} \mid Ax + s = b, x, s \geq 0\}$$

is integer. Together with Theorem 4.13 the claim follows. $\square$

Analogously one can derive the following results. In all of these cases we consider only polyhedra P , whose minimal faces are vertices, i.e. who are integral if and only if all of their vertices are integer.

Corollary 4.15. Let $A \in \mathbb{Z}^{m \times n}$ be an integer matrix. Then the following are equivalent:

(a) A is totally unimodular.

(b) For all right-hand sides $b \in \mathbb{Z}^m$ and all upper bounds $u \in \mathbb{Z}^n$ such that the polyhedron

$$P := \{x \in \mathbb{R}^n \mid Ax \leq b, 0 \leq x \leq u\}$$

is nonempty, the polyhedron P is integral.

(c) For all vectors $a, b \in \mathbb{Z}^m$ and all bounds $l, u \in \mathbb{Z}^n$ such that the polyhedron

$$P := \{x \in \mathbb{R}^n \mid a \leq Ax \leq b, l \leq x \leq u\}$$

is nonempty, the polyhedron P is integral.

Note that unimodularity of the matrix A guarantees that the corresponding polyhedron $P = P^=(A, b)$ is integral for all right-hand sides $b \in \mathbb{Z}^n$. In general, this is more than we need, because we are only interested in one concrete right-hand side vector b .

Remark 4.16 (Optional). *If we are only interested in the integrality of a polyhedron $P = P(A, b)$ with a specific right-hand side b , we can also have a look at total dual integrality*

(TDI, totale duale Ganzzahligkeit). A rational inequality $Ax \leq b$ with $A \in \mathbb{Q}^{m \times n}$ and $b \in \mathbb{Q}^m$ is called TDI, if for all integer vectors $c \in \mathbb{Z}^n$ such that the (dual) problem

$$\min_y b^T y \quad \text{s.t.} \quad A^T y = c, \quad y \geq 0$$

has a solution, it also has an integer solution $y^* \in \mathbb{Z}^m$. One can show that a rational polyhedron $P = P(A, b)$, where the inequality $Ax \leq b$ is TDI and the right-hand side $b \in \mathbb{Z}^m$ is integer, is an integer polyhedron. But we do not cover this approach in this lecture.

Let us close this section by providing two results, which prove total unimodularity of matrices with a special structure. Examples for matrices with these two structures are the incidence matrix of a directed graph and the incidence matrix of an undirected bipartite graph, see Chapter 5 for definitions of these terms.

Lemma 4.17. Let $A \in \mathbb{Z}^{n \times n}$ be a matrix with the following properties:

- $a_{ij} \in \{0, +1, -1\}$ for all i, j .
- Each column $A_{*,j}$ of A contains at most two nonzero elements.
- For all columns $A_{*,j}$ we have $e^T A_{*,j} = 0$.

Then the matrix A is totally unimodular.

Proof. Consider arbitrary subsets $I \subseteq \{1, \dots, m\}$ and $J \subseteq \{1, \dots, n\}$ with $|I| = |J|$. We have to show that $\det A_{IJ} \in \{0, +1, -1\}$. Here, we can compute $\det A_{IJ}$ using the Laplace expansion² (Laplace-Entwicklung).

- (i) If A_{IJ} has dimension 1×1 , it follows from the first condition that $\det A_{IJ} \in \{0, +1, -1\}$.
- (ii) If A_{IJ} has a column full of zeros, we have $\det A_{IJ} = 0$.

²Let $A \in \mathbb{R}^{n \times n}$ be a square matrix and for $i, j \in \{1, \dots, n\}$ let $A_{ij} \in \mathbb{R}^{(n-1) \times (n-1)}$ denote the submatrix without the i -th row and the j -th column. Then using Laplace expansion we know

$$\begin{aligned} \det A &= \sum_{i=1}^n (-1)^{i+j} a_{ij} \det A_{ij} && \text{(expansion along column } j) \\ &= \sum_{j=1}^n (-1)^{i+j} a_{ij} \det A_{ij} && \text{(expansion along row } i). \end{aligned}$$

- (iii) If A_{IJ} has exactly two nonzero entries in every column, then by the third assumption the rows of A_{IJ} add up to zero, i.e. we have $\det A_{IJ} = 0$.
- (iv) Otherwise by the second assumption A_{IJ} has a column with exactly one nonzero entry and we can use Laplace expansion and reduce the dimension of the matrix by one. The reduced matrix has the same determinant as A_{IJ} except for possibly a sign change.

If we are in case (iv), the reduced matrix again has the same properties as A_{IJ} and we can continue using Laplace expansion until we are in case (i–iii). This shows that A_{IJ} can only have determinant 0, +1 or -1. $\square$

Since a matrix A is totally unimodular if and only if A^T is totally unimodular, see Exercise 4.1, one can also impose the conditions in Lemma 4.17 on the rows of A instead of the columns.

Lemma 4.18. Let $A \in \mathbb{Z}^{n \times n}$ be a matrix with the following properties:

- $a_{ij} \in \{0, +1, -1\}$ for all i, j .
- Each column $A_{*,j}$ of A contains at most two nonzero elements.
- The rows of A can be divided into two disjoint sets L, K such that:
 - If a column of A contains two nonzero elements with the same sign, then the corresponding rows are in different sets L, K .
 - If a column of A contains two nonzero elements with different signs, then the corresponding rows are both in the same set L or K .

Then the matrix A is totally unimodular.

Proof. The proof is analogous to the proof of Lemma 4.17, we only have to change the argument in case (iii). If all columns of the square submatrix A_{IJ} contain exactly two elements, we split the index set I into

$$I_L := I \cap L \quad \text{and} \quad I_K := I \cap K.$$

In case one of the two sets is empty, e.g. $I_K = \emptyset$, we have $I_L = I$ and thus know that the two nonzero entries in each column have different signs, i.e. are +1 and -1. Then it follows that the rows of A_{IJ} add up to zero and thus $\det A_{IJ} = 0$.

In case both sets are nonempty, we know that

$$\sum_{i \in I_L} A_{i,J} = \sum_{i \in I_K} A_{i,J},$$

which implies that the rows of A_{IJ} are linearly dependent. This shows $\det A_{IJ} = 0$. $\square$

4.2 Cutting Planes

If the polyhedron P occurring in the integer linear program (4.1) is not integral, i.e. $P_I \subsetneq P$, then we have to solve the following problem: How can we compute a sufficiently good approximation $\hat{P}$ of the integer hull P_I , such that we can solve the linear program

$$\min_x c^T x \quad \text{s.t.} \quad x \in \hat{P}$$

instead of the integer linear program?

We could try to use the polyhedron P , i.e. solve the LP relaxation instead of the ILP. But we do already know that this can result in noninteger solutions and in better objective function values than those available on the integer feasible set.

So we are instead looking for a polyhedron $\hat{P}$ with $P_I \subseteq \hat{P} \subseteq P$ such that

$$\inf_{x \in P_I} c^T x = \inf_{x \in \hat{P}} c^T x$$

and such that the minimum on $\hat{P}$ is also attained in integer points.

In order to obtain such a set $\hat{P}$, we can start with the polyhedron P and add additional constraints, which are feasible inequalities for the polyhedron P_I but not feasible inequalities for P . Such constraints then cut parts of P away, which do not contain any integer feasible points and thus are not needed for the ILP. If you think back to the end of Section 3.2, feasible inequalities inducing facets of P_I or at least high-dimensional faces of P_I would be preferable. For some applications, e.g. the traveling salesperson problem, it is known how to construct facets of P_I .

Example 4.19. *Consider again the polyhedron*

$$P = \{x \in \mathbb{R}^2 \mid -4x_1 + 6x_2 \leq 9, x_1 + x_2 \leq 4, x \geq 0\}$$

from Example 4.5, see also Figure 4.3. Then the constraints

$$x_2 \leq 2 \quad \text{and} \quad -x_1 + x_2 \leq 1$$

are not feasible inequalities for the polyhedron P . But they are feasible inequalities for the integer hull P_I and even induce facets of P_I .

The question remains how we can find these feasible inequalities and how many we need. Fortunately, in order to achieve $\inf_{x \in P_I} c^T x = \inf_{x \in \hat{P}} c^T x$ we do not really need $\hat{P} = P_I$. We only have to find a good description of the area of P_I , where the solution are attained.

If you consider Example 4.19 again with the objective function $\max_x x_1$, then we are actually totally fine working with $\hat{P} = P$. If the objective function is instead $\min_x x_1$, then we can work with $\hat{P} = \{x \in P \mid -x_1 + x_2 \leq 1\}$, i.e. we need only one of the two additional constraints.

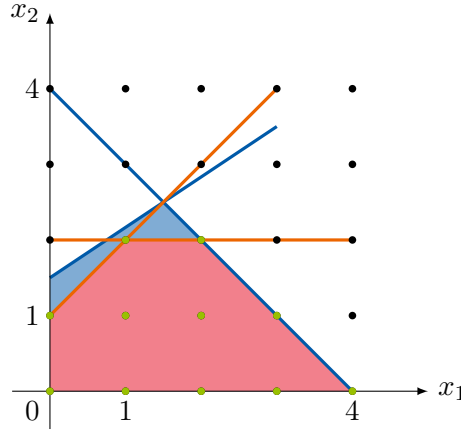


Figure 4.3: Feasible set $P \cap \mathbb{Z}^n$ of the integer problem (4.1) in green with two facet inducing inequalities in orange of the integer hull P_I in red.

4.2.1 Some Ideas for Generating Cuts

In this section we want to collect some ideas to generate cuts, i.e. to find constraints, which we can add to the polyhedron P in order to determine a set $\hat{P}$ closer to the integer hull P_I . Sometimes, we do not have to add new constraints but instead can tighten constraints, which are already present in the description of P .

Tightening existing constraints by rounding Let us first consider a polyhedron of the form

$$P = \{x \in \mathbb{R}^n \mid Ax \leq b\}$$

with $A \in \mathbb{Z}^{m \times n}$ and have a closer look at one of the inequality constraints

$$A_{i*}x = \sum_{j=1}^n a_{ij}x_j \leq b_i.$$

All integer feasible points $x \in P \cap \mathbb{Z}^n$ then also satisfy³

$$A_{i*}x \leq \lfloor b_i \rfloor.$$

This rounding of the right hand side has a larger effect for small absolute values of b_i , i.e. one might divide the whole inequality constraint by the greatest common divisor of all coefficients in A_{i*} before rounding.

³Here, $\lfloor y \rfloor$ is the *floor function* and describes the largest integer $z \in \mathbb{Z}$ with $z \leq y$. For example $\lfloor 1.5 \rfloor = 1$, $\lfloor 1 \rfloor = 1$, $\lfloor 0.1 \rfloor = 0$, and $\lfloor -1.5 \rfloor = -2$.

If we consider a polyhedron of the form

$$P = \{x \in \mathbb{R}^n \mid Ax \leq b, x \geq 0\}$$

with $A \in \mathbb{R}^{m \times n}$, we can use a similar technique. For each inequality constraint we know for all $x \in P \cap \mathbb{Z}^n$

$$\begin{aligned} A_{i*}x &= \sum_{j=1}^n a_{ij}x_j \leq b_i \\ \implies \sum_{j=1}^n \lfloor a_{ij} \rfloor x_j &\leq b_i \quad (\text{because } x \geq 0) \\ \implies \sum_{j=1}^n \lfloor a_{ij} \rfloor x_j &\leq \lfloor b_i \rfloor \quad (\text{because } x \in \mathbb{Z}^n) \end{aligned}$$

Again, the rounding effect is more pronounced if the coefficients have small absolute values, so one might scale the linear equation before rounding. This type of cut is called *Chvatal-Gomory inequality*.

Finally, let us consider a polyhedron of the form

$$P = \{x \in \mathbb{R}^n \mid Ax = b, x \geq 0\}$$

with $A \in \mathbb{R}^{m \times n}$. Then all integer feasible points $x \in P \cap \mathbb{Z}^n$ satisfy for all equality constraints

$$A_{i*}x = b_i \implies A_{i*}x \leq b_i \implies \sum_{j=1}^n \lfloor a_{ij} \rfloor x_j \leq \lfloor b_i \rfloor.$$

Subtracting this new inequality constraint from the original equation yields the constraint

$$\sum_{j=1}^n (a_{ij} - \lfloor a_{ij} \rfloor) x_j \geq b_i - \lfloor b_i \rfloor.$$

This type of cut is called a *Gomory cutting plane*.

Note that all of these rounding techniques require the left side of the inequality to be integer in order to be able to round the right side down to an integer. If we consider mixed integer linear problems instead of integer linear problems and thus noninteger variables can appear on the left side, this argument does not work anymore and we need a modified approach. For an example, see Remark 4.25.

Example 4.20. Consider again Example 4.5 with the polyhedron

$$P = \{x \in \mathbb{R}^2 \mid -4x_1 + 6x_2 \leq 9, x_1 + x_2 \leq 4, x \geq 0\}.$$

(a) Then we can divide the first inequality constraint by $2 > 0$ and obtain

$$-2x_1 + 3x_2 \leq 4.5.$$

All integer feasible points then satisfy the tightened constraint

$$-2x_1 + 3x_2 \leq 4,$$

see Figure 4.4 for an illustration (in orange).

(b) We could also divide the first inequality constraint by $6 > 0$ and obtain

$$-\frac{2}{3}x_1 + x_2 \leq 1.5.$$

Due to $x \geq 0$, all integer feasible points then satisfy the tightened constraint

$$-x_1 + x_2 \leq 1,$$

see Figure 4.4 for an illustration (in yellow). This is in fact a facet inducing inequality.

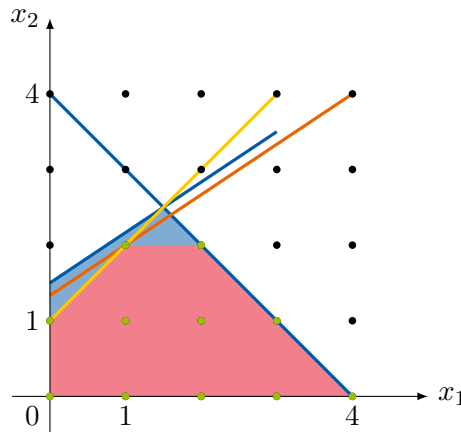


Figure 4.4: Feasible set $P \cap \mathbb{Z}^n$ of the integer problem (4.1) in green with a cut generated by rounding the right-hand side in orange and a cut generated by rounding both sides in yellow.

Combining existing constraints to generate new ones So far, we have only considered ideas how to tighten an existing constraint. However, if we take a look at Figure 4.4 for illustration, in order to describe the integer hull P_I (in red) we sometimes need more or

different constraints than for the polyhedron P (in blue). In the figure, in particular, we need a new constraint to get rid of the top part of the polyhedron. Below, we have a look at two possible approaches to generate new constraints based on existing ones.

Consider a polyhedron $P \subseteq \mathbb{R}^n$. If $A_{i,*}x \leq b_i$ and $A_{j,*}x \leq b_j$ are two feasible inequalities for the polyhedron P , then their sum

$$(c_i A_{i,*} + c_j A_{j,*})x \leq (c_i b_i + c_j b_j)$$

with coefficients $c_i, c_j > 0$ is also a feasible inequality for P . Here, we can choose the coefficients e.g. such that the new constraint depends on fewer variables than the two original ones. And if we then apply the previously discussed rounding techniques to the new constraint, we may obtain a useful new constraint.

Next, consider a polyhedron $P \subseteq \mathbb{R}_+^n$. If $a_1^T x \leq \beta_1$ is a feasible inequality for a subset $Z_1 \subseteq P$ and $a_2^T x \leq \beta_2$ is a feasible inequality for a subset $Z_2 \subseteq P$, then

$$\sum_{j=1}^n \min\{a_{1j}, a_{2j}\} x_j \leq \max\{\beta_1, \beta_2\}$$

is a valid inequality for the set $\text{conv}(Z_1 \cup Z_2)$. Note that we need $x \geq 0$ here in order to be able to compute the minimum on the left side of the inequality coefficient-wise. This trick can be used, when certain estimates only work for a part of $P \cap \mathbb{Z}^n$ and for the remaining integer feasible points we have to use a different estimate. Then we can recombine the two resulting inequalities to a feasible inequality for the whole set $P \cap \mathbb{Z}^n$ as above.

Remark 4.21 (Optional: Special case of the “min-max” argument). *The previous trick can also be used to combine two existing inequalities into one new inequality, if they can be rewritten in a special form. Let the following two inequalities*

$$a^T x - \alpha(x_j - z) \leq \beta \quad \text{and} \quad a^T x + \gamma(x_j - z - 1) \leq \beta$$

both be feasible for P , where $\alpha, \gamma \geq 0$, $\beta \in \mathbb{R}$, and $z \in \mathbb{Z}$ and x_j is one component of x . Then the inequality

$$a^T x \leq \beta$$

is a feasible inequality for all

$$x \in Z_1 := \{x \in P \cap \mathbb{Z}^n \mid x_j \leq z\}$$

due to the first inequality. And the inequality

$$a^T x \leq \beta$$

is also a feasible inequality for all

$$x \in Z_2 := \{x \in P \cap \mathbb{Z}^n \mid x_j \geq z + 1\}$$

due to the second inequality. Since there do not exist any integer feasible points with $x_j \in (z, z + 1)$, this shows that

$$a^T x \leq \beta$$

is a feasible inequality for $P \cap \mathbb{Z}^n$. For this argument to work it is important that z is an integer such that there are no integer points in the interval $(z, z + 1)$.

We use this trick in Remark 4.25 to generate cuts for mixed-integer linear programs.

Example 4.22. Consider again Example 4.5 with the polyhedron

$$P = \{x \in \mathbb{R}^2 \mid -4x_1 + 6x_2 \leq 9, x_1 + x_2 \leq 4, x \geq 0\}.$$

- (a) If we consider the two inequalities $-4x_1 + 6x_2 \leq 9$ and $x_1 + x_2 \leq 4$, adding the second inequality four times to the first results in the new inequality

$$10x_2 \leq 25 \quad \Longleftrightarrow \quad x_2 \leq \frac{5}{2}.$$

All integer feasible points thus have to satisfy

$$x_2 \leq 2,$$

see Figure 4.5 for an illustration (in orange). This is the missing facet inducing inequality.

- (b) For all feasible points with $x_1 \leq 1$ we know

$$-4x_1 + 6x_2 \leq 9 \quad \Longrightarrow \quad 6x_2 \leq 9 + 4x_1 \leq 13.$$

Integer feasible points with $x_1 \leq 1$ thus have to satisfy $x_2 \leq \lfloor 2.5 \rfloor = 2$.

For all feasible points with $x_2 \geq 2$ we know

$$x_1 + x_2 \leq 4 \quad \Longrightarrow \quad x_2 \leq 4 - x_1 \leq 2.$$

Since there exist no integer feasible points with $x_1 \in (1, 2)$, we have thus shown (again) that $x_2 \leq 2$ is a feasible inequality for P_I . Here, we have skipped the “min-max” step to combine the two inequalities $x_2 \leq 2$ and $x_2 \leq 2$ into one, which is feasible on the whole integer hull, because it is the same inequality on both parts of P_I .

(c) We have already seen that we can tighten the first inequality to

$$-x_1 + x_2 \leq 1,$$

see Figure 4.5 for an illustration (in yellow). And we can rewrite this constraint and the second inequality as

$$0x_1 + x_2 - 1(x_1 - 1) \leq 2,$$

$$0x_1 + x_2 + 1(x_1 - 2) \leq 2.$$

Then using the observation above with $z = 1 \in \mathbb{Z}$ we (once more) obtain the constraint

$$x_2 \leq 2.$$

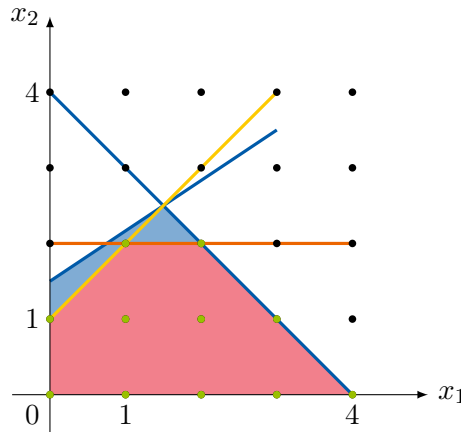


Figure 4.5: Feasible set $P \cap \mathbb{Z}^n$ of the integer problem (4.1) in green with a cut generated by rounding in yellow and a cut generated by adding two constraints orange.

4.2.2 Cutting Plane Method

The cuts discussed in the previous section can be added to the polyhedron P at any time in order to obtain a tighter LP relaxation. If we want to add cuts in a systematic way, we can use a *cutting plane methods* (*Schnittebenen-Verfahren*):

Consider an integer linear program (4.1) with polyhedron P and integer hull $P_I = \text{conv}(P \cap \mathbb{Z}^n)$. Now consider a sequence of polyhedra $(P_k)_k$ with the following properties:

- $P_0 = P$
- $P_{k+1} \subsetneq P_k$ for all $k = 0, 1, \dots$

- $P_I \subseteq P_k$ for all $k = 0, 1, \dots$

Then for all $k = 0, 1, 2, \dots$ we have

$$P_I \subseteq (P_k)_I = \text{conv}(P_k \cap \mathbb{Z}^n) \subseteq \text{conv}(P \cap \mathbb{Z}^n) = P_I,$$

meaning that the integer linear programs

$$\min_x c^T x \quad \text{s.t.} \quad x \in P_k, x \in \mathbb{Z}^n$$

all have exactly the same feasible set as the original ILP (4.1). But the feasible set of the corresponding LP relaxation

$$\min_x c^T x \quad \text{s.t.} \quad x \in P_k$$

becomes smaller from iteration to iteration, i.e. the relaxation become tighter and thus better in every iteration.

In order to generate a new polyhedron $P_{k+1} \subsetneq P_k$, we can use the solution x^k of the LP relaxation corresponding to P_k . If this solution is integer, we have found a solution of the ILP and can terminate the algorithm. And if the solution is not integer, we know from the strict separation theorem, that there exists a hyperplane with

$$a^T x \leq \beta < x^k \quad \forall x \in P_I.$$

Such a constraint is called a *cut* (*Schnitt*), because it cuts x^k away from P_I . We can thus define

$$P_{k+1} := \{x \in P_k \mid a^T x \leq \beta\}.$$

This procedure is formalized in Algorithm 4.23.

Algorithm 4.23. (General Cutting Plane Method)

Input: An integer linear program (4.1).

Set $P_0 := P$ and $k = 0$.

Compute a solution x^0 of the LP relaxation

$$\min_x c^T x \quad \text{s.t.} \quad x \in P_0.$$

while $x^k \notin \mathbb{Z}^n$ **do**

 Determine a cut, i.e. a feasible inequality $a^T x \leq \beta$ for P_I , that is not satisfied by x^k .

Define the new polyhedron P_{k+1} as

$$P_{k+1} := \{x \in P_k \mid a^T x \leq \beta\}.$$

Compute a solution x^{k+1} of the LP relaxation

$$\min_x c^T x \quad \text{s.t.} \quad x \in P_{k+1}.$$

Set $k \leftarrow k + 1$.

end while

return $x^* = x^k$ is a solution of (4.1).

Some remarks on this algorithm:

- Since we ensure $P_I \subseteq P_k$ in all iterations, the LP relaxations can only become infeasible, if the ILP is infeasible.
- If P is rational and the ILP has a solution, we know that the initial LP relaxation is feasible and bounded, i.e. has a solution. And due to $P_I \subseteq P_k \subseteq P$ all other LP relaxations solved in Algorithm 4.23 then also have solutions.
- Once the solution x^k of an LP relaxation is integer, the relation $P_I \subseteq P_k$ implies that x^k is a solution of the ILP and we can terminate the algorithm.
- But we have not yet answered the questions how to compute a cut and how many cuts/iterations we need.

In order to find suitable cuts, we can use an idea, which goes back back to Gomory and uses information from the simplex method. To see this, let the LP relaxation of the ILP (4.1) have the form

$$\min_x c^T x \quad \text{s.t.} \quad Ax = b, x \geq 0$$

and let x^* be a solution generated by the simplex method with basis B , i.e. $x^* = (x_B^*, x_N^*) = (A_B^{-1}b, 0)$. We know that all feasible points of the linear program satisfy

$$A_B x_B + A_N x_N = b \iff x_B + A_B^{-1} A_N x_N = A_B^{-1} b.$$

Introducing the abbreviations $\bar{A} := A_B^{-1} A_N \in \mathbb{R}^{m \times (n-m)}$ and $\bar{b} := A_B^{-1} b \in \mathbb{R}^m$, we can rewrite this equation as

$$x_B + \bar{A} x_N = \bar{b}.$$

If the solution x^* is not integer, then due to $x_N^* = 0$ this has to be due to a noninteger basic variable $(x_B^*)_i \notin \mathbb{Z}$. Then we can derive a *Chvatal-Gomory inequality* for the corresponding equation. Due to $x \geq 0$ all feasible points of the linear program satisfy

$$(x_B)_i + \sum_{j=1}^{n-m} \lfloor \bar{a}_{ij} \rfloor (x_N)_j \leq (x_B)_i + \sum_{j=1}^{n-m} \bar{a}_{ij} (x_N)_j = \bar{b}_i.$$

And integer feasible points of the linear program even satisfy

$$(x_B)_i + \sum_{j=1}^{n-m} \lfloor \bar{a}_{ij} \rfloor (x_N)_j \leq \lfloor \bar{b}_i \rfloor.$$

So we can add this cut to the linear program without loosing any integer feasible points. However, the noninteger solution x^* with $(x_B^*)_i \notin \mathbb{Z}$ does not satisfy this constraint, because due to $x_N^* = 0$ we have

$$(x_B^*)_i + \sum_{j=1}^{n-m} \lfloor \bar{a}_{ij} \rfloor (x_N^*)_j = (x_B^*)_i = \bar{b}_i > \lfloor \bar{b}_i \rfloor.$$

Thus, if we add this cut to the linear program and solve it again, we obtain a different solution in the next iteration.

An alternative formulation of this cut, the *Gomory cutting plane*, can again be obtained by from subtracting the Chvatal-Gomory inequality

$$(x_B)_i + \sum_{j=1}^{n-m} \lfloor \bar{a}_{ij} \rfloor (x_N)_j \leq \lfloor \bar{b}_i \rfloor$$

from the original equation

$$(x_B)_i + \sum_{j=1}^{n-m} \bar{a}_{ij} (x_N)_j = \bar{b}_i,$$

which results in the inequality constraint

$$\sum_{j=1}^{n-m} (\bar{a}_{ij} - \lfloor \bar{a}_{ij} \rfloor) (x_N)_j \geq \bar{b}_i - \lfloor \bar{b}_i \rfloor.$$

Since both the original inequality constraint and the equation are satisfied for all $x \in P_I$, the new inequality constraint is also satisfied for all $x \in P_I$. But it follows from the same arguments used above that the point x^* with $(x_B^*)_i \notin \mathbb{Z}$ does not satisfy this constraint. An advantage of this reformulation of the Gomory cut is that it depends on less variables, i.e. only the nonbasic variables x_N and not the basic variable $(x_B)_i$.

Example 4.24. Consider the integer linear program

$$\begin{aligned} \min_x \quad & x_1 - 2x_2 \quad s.t. \quad -4x_1 + 6x_2 \leq 9, \\ & x_1 + x_2 \leq 4, \\ & x \geq 0, \quad x \in \mathbb{Z}^2. \end{aligned}$$

In standard form, the LP relaxation with the variables $x \in \mathbb{R}^2$ and slack variables $s \in \mathbb{R}^2$ looks as follows

$$\min_{x,s} \begin{pmatrix} 1 \\ -2 \\ 0 \\ 0 \end{pmatrix}^T \cdot \begin{pmatrix} x_1 \\ x_2 \\ s_1 \\ s_2 \end{pmatrix} \quad s.t. \quad \begin{pmatrix} -4 & 6 & 1 & 0 \\ 1 & 1 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ s \end{pmatrix} = \begin{pmatrix} 9 \\ 4 \end{pmatrix}, \quad x, s \geq 0$$

and has the solution $(x^*, s^*) = (1.5, 2.5, 0, 0)$ with basic matrix

$$\begin{aligned} A_B &= \begin{pmatrix} -4 & 6 \\ 1 & 1 \end{pmatrix}, \quad A_B^{-1} = \frac{1}{10} \begin{pmatrix} -1 & 6 \\ 1 & 4 \end{pmatrix} \\ \implies \quad \bar{A} &= A_B^{-1} A_N = A_B^{-1}, \quad \bar{b} = A_B^{-1} b = \frac{1}{10} \begin{pmatrix} 15 \\ 25 \end{pmatrix}. \end{aligned}$$

Since both components of x^* are not integer, we can choose either variable to generate a cut. We choose x_2 and obtain the cut

$$x_2 + \frac{1}{10}s_1 + \frac{4}{10}s_2 = \frac{25}{10} \implies x_2 + 0s_1 + 0s_2 \leq 2.$$

This results in the new linear program in standard form

$$\min_{x,s} \begin{pmatrix} 1 \\ -2 \\ 0 \\ 0 \\ 0 \end{pmatrix}^T \cdot \begin{pmatrix} x_1 \\ x_2 \\ s_1 \\ s_2 \\ s_3 \end{pmatrix} \quad s.t. \quad \begin{pmatrix} -4 & 6 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 0 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ s \end{pmatrix} = \begin{pmatrix} 9 \\ 4 \\ 2 \end{pmatrix}, \quad x, s \geq 0$$

with the solution $(x^*, s^*) = (0.75, 2, 0, 1.25, 0)$. The corresponding basic matrix is

$$\begin{aligned} A_B &= \begin{pmatrix} -4 & 6 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad A_B^{-1} = \frac{1}{4} \begin{pmatrix} -1 & 0 & 6 \\ 0 & 0 & 4 \\ 1 & 4 & -10 \end{pmatrix} \\ \implies \quad \bar{A} &= \frac{1}{4} \begin{pmatrix} -1 & 6 \\ 0 & 4 \\ 1 & -10 \end{pmatrix}, \quad \bar{b} = \frac{1}{4} \begin{pmatrix} 3 \\ 8 \\ 5 \end{pmatrix}. \end{aligned}$$

Since we want x to be integer, we choose x_1 and obtain the cut

$$x_1 + \frac{-1}{4}s_1 + \frac{6}{4}s_3 = \frac{3}{4} \implies x_1 - 1s_1 + 1s_3 \leq 0.$$

This results in the new linear program in standard form

$$\min_{x,s} \begin{pmatrix} 1 \\ -2 \\ 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}^T \cdot \begin{pmatrix} x_1 \\ x_2 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \end{pmatrix} \quad s.t. \quad \begin{pmatrix} -4 & 6 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 \\ 1 & 0 & -1 & 0 & 1 & 1 \end{pmatrix} \cdot \begin{pmatrix} x \\ s \end{pmatrix} = \begin{pmatrix} 9 \\ 4 \\ 2 \\ 0 \end{pmatrix}, \quad x, s \geq 0$$

with the solution $(x^*, s^*) = (1, 2, 1, 1, 0, 0)$. This solution is integer and thus $x^* = (1, 2)$ is the solution of the integer linear program. Figure 4.6 provides an illustration of all occurring LP relaxations, their solutions and the new cuts.

It can be shown that the number of Gomory cuts needed to obtain an integer solution is finite, if

- the feasible set is bounded
- or the cuts are selected according to a suitable rule (e.g. always the noninteger basis variable with the smallest index).

Unfortunately, the number of Gomory cuts needed to obtain an integer solution and consequently the number of linear programs, that need to be solved, can be very large. Some ideas to improve this situation are given below:

- Add not only one cut per iteration, but several, i.e. one cut for each $(x_B^*)_i \notin \mathbb{Z}$.
- Use specialized cuts based on the specific problem structure.
- Combine the cutting plane method with the branch&bound method from Section 4.3 to obtain the branch&cut method from Section 4.4. In this case, the cutting planes are only used to improve the bounds needed for the branch&bound method and thus only a few cuts are added per iteration.

To close this section, it should be mentioned that it is not possible to use Gomory cuts exactly as described above in the mixed-integer case, i.e. when only some but not all variables have to be integer. But it is possible to extend the idea of Gomory cuts to the mixed-integer case.

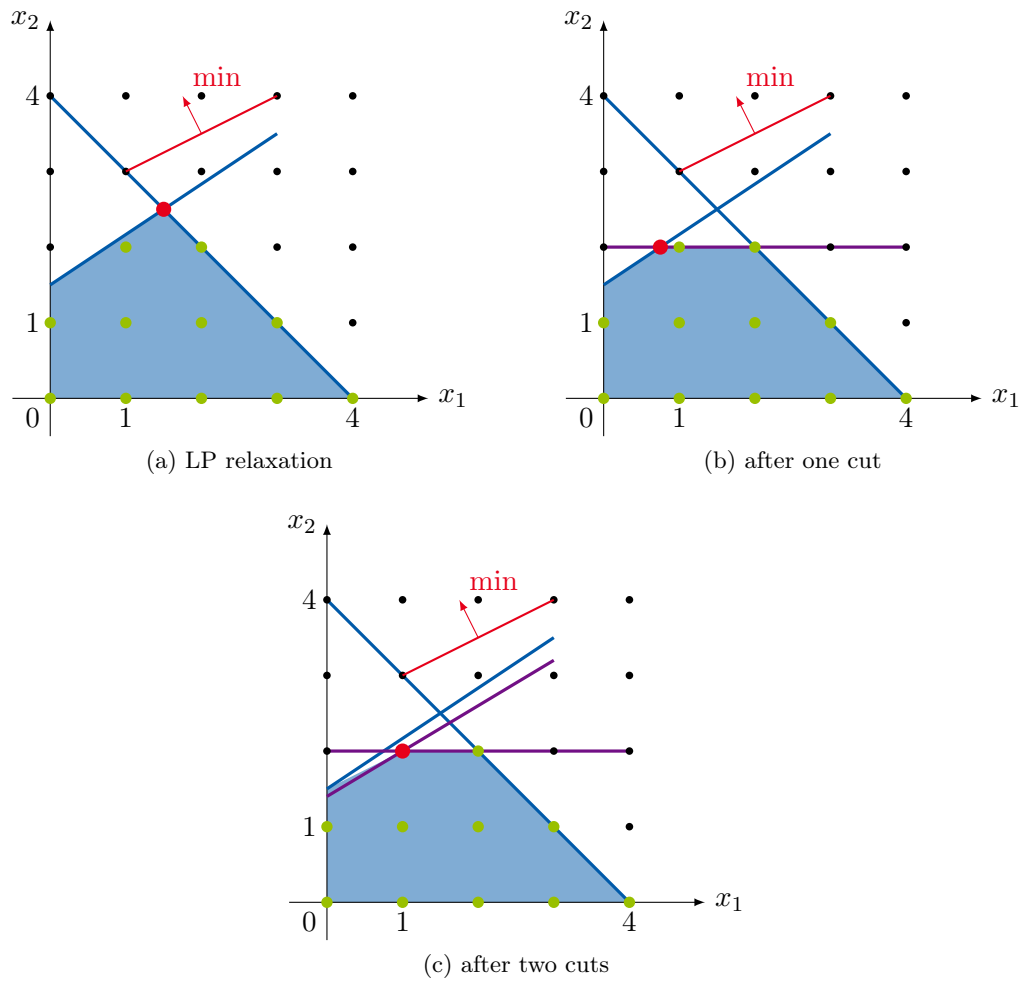


Figure 4.6: Illustration of the Cutting plane method in Example 4.24

Remark 4.25 (Optional: Gomoroy cuts for mixed-integer LPs). Consider a mixed-integer linear program, where for some subset of $\{1, \dots, n\}$ the corresponding variables x_i have to be integer. Let $x^* = (x_B^*, x_N^*)$ be a solution of the corresponding LP relaxation and $(x_B^*)_i \notin \mathbb{Z}$ a variable, which is supposed to be integer but is not yet integer. Then we again consider the equation

$$(x_B)_i + \sum_{j=1}^{n-m} \bar{a}_{ij}(x_N)_j = \bar{b}_i$$

with $\bar{A} = A_B^{-1}A_N$ and $\bar{b} = A_B^{-1}b$. Recall that by construction we have $\bar{b}_i = (x_B^*)_i \notin \mathbb{Z}$.

If we round down on the left-hand side, due to $x \geq 0$ the following inequality is still satisfied for all $x \in P$:

$$(x_B)_i + \sum_{j=1}^{n-m} \lfloor \bar{a}_{ij} \rfloor (x_N)_j \leq \bar{b}_i.$$

But in contrast to before, we cannot just round down the right-hand side anymore, because the nonbasic variables x_N might be allowed to take noninteger values. Fortunately, it is possible to fix this problem by changing the way the cut is generated slightly.

Let us define the following two sets

$$\begin{aligned} J^C &:= \{j = 1, \dots, n-m \mid (x_N)_j \text{ does not have to be integer}\}, \\ J^I &:= \{j = 1, \dots, n-m \mid (x_N)_j \text{ has to be integer}\}. \end{aligned}$$

For all feasible points x of the mixed-integer problem we then obtain

$$\begin{aligned} &\sum_{j \in J^C} \bar{a}_{ij}(x_N)_j + \sum_{j \in J^I} (\bar{a}_{ij} - \lfloor \bar{a}_{ij} \rfloor)(x_N)_j - (\bar{b}_i - \lfloor \bar{b}_i \rfloor) \\ &= \lfloor \bar{b}_i \rfloor - (x_B)_i - \sum_{j \in J^I} \lfloor \bar{a}_{ij} \rfloor (x_N)_j =: z \in \mathbb{Z}. \end{aligned} \tag{4.5}$$

In case $z \geq 0$, it follows that

$$\sum_{j \in J^C} \bar{a}_{ij}(x_N)_j + \sum_{j \in J^I} (\bar{a}_{ij} - \lfloor \bar{a}_{ij} \rfloor)(x_N)_j \geq \bar{b}_i - \lfloor \bar{b}_i \rfloor > 0$$

and in case $z \leq -1$ we obtain

$$\sum_{j \in J^C} \bar{a}_{ij}(x_N)_j + \sum_{j \in J^I} (\bar{a}_{ij} - \lfloor \bar{a}_{ij} \rfloor)(x_N)_j \leq -1 + (\bar{b}_i - \lfloor \bar{b}_i \rfloor) = \bar{b}_i - \lceil \bar{b}_i \rceil < 0.$$

For all mixed-integer feasible points we are thus in one of the following two cases:

$$z \geq 0 \implies \frac{1}{\bar{b}_i - \lfloor \bar{b}_i \rfloor} \left[\sum_{j \in J^C} \bar{a}_{ij}(x_N)_j + \sum_{j \in J^I} (\bar{a}_{ij} - \lfloor \bar{a}_{ij} \rfloor)(x_N)_j \right] \geq 1,$$

$$z \leq -1 \implies \frac{1}{\bar{b}_i - \lceil \bar{b}_i \rceil} \left[\sum_{j \in J^C} \bar{a}_{ij}(x_N)_j + \sum_{j \in J^I} (\bar{a}_{ij} - \lfloor \bar{a}_{ij} \rfloor)(x_N)_j \right] \geq 1.$$

Using the idea of the disjunctive cuts, it follows that all mixed-integer feasible points satisfy the inequality constraint, where we consider the maximum of both possible coefficients on the left hand side, i.e.

$$\sum_{j \in J^C, \bar{a}_{ij} > 0} \frac{\bar{a}_{ij}}{\bar{b}_i - \lceil \bar{b}_i \rceil} (x_N)_j + \sum_{j \in J^C, \bar{a}_{ij} < 0} \frac{\bar{a}_{ij}}{\bar{b}_i - \lceil \bar{b}_i \rceil} (x_N)_j + \sum_{j \in J^I} \frac{\bar{a}_{ij} - \lfloor \bar{a}_{ij} \rfloor}{\bar{b}_i - \lceil \bar{b}_i \rceil} (x_N)_j \geq 1.$$

But the current solution x^* does not satisfy this constraint due to $x_N^* = 0$.

For the components $i \in J^I$ with

$$\frac{\bar{a}_{ij} - \lfloor \bar{a}_{ij} \rfloor}{\bar{b}_i - \lceil \bar{b}_i \rceil} > 1,$$

one can obtain better coefficients by working with $\bar{a}_{ij} - \lceil \bar{a}_{ij} \rceil$ in (4.5) instead of $\bar{a}_{ij} - \lfloor \bar{a}_{ij} \rfloor$, which then results in the coefficients

$$\frac{\bar{a}_{ij} - \lceil \bar{a}_{ij} \rceil}{\bar{b}_i - \lceil \bar{b}_i \rceil} \in (0, 1)$$

using the same computations as above.

4.3 Branch&Bound Method

In its general form, the **branch&bound method** is a solution method for general discrete optimization problems of the form

$$\min_x f(x) \quad \text{s.t.} \quad x \in D \tag{4.6}$$

with a discrete feasible set $D \subset \mathbb{R}^n$. The goal of the algorithm is to determine a global solution of this optimization problem by dividing the feasible sets into suitable subsets (“branching”). In the worst case, this subdivision process is repeated, until the subsets consist of only one point each and thus the optimal function value on this subset can exactly be determined. But preferably one can eliminate whole subsets of the feasible set from the search of a solution by showing that they cannot contain a better point than the the best known feasible point at the moment (“bounding”).

This method can both be formulated in a rather general way and can be tailored to the solution of mixed-integer linear programs.

4.3.1 General Branch&Bound Method

We begin by providing a quite general version of the branch&bound method in Algorithm 4.26.

Algorithm 4.26. (General Branch&Bound Method)

Input: A discrete optimization problem (4.6).

Compute an arbitrary $x \in D$ and initialize the upper bound on the optimal function value of the optimization problem as $U = f(x)$ and the best known point so far as $x^* := x$.

Initialize the set of *active components of the feasible set* as $S := \{D\}$.

while $S \neq \emptyset$ **do**

Branching Choose an active component $X \in S$ and partition it into $r \geq 2$ parts $X_1, \dots, X_r \neq \emptyset$ such that

$$\bigcup_{i=1}^r X_i = X \quad \text{and} \quad X_i \cap X_j = \emptyset \quad \text{for } i \neq j.$$

Bounding For all sets X_i , $i = 1, \dots, r$ determine a lower bound

$$L(X_i) \begin{cases} = \inf_{x \in X_i} f(x) & \text{if } |X_i| = 1, \\ \leq \inf_{x \in X_i} f(x) & \text{if } |X_i| > 1. \end{cases}$$

Updating the upper bound For all sets X_i , $i = 1, \dots, r$ determine a feasible point $x^i \in X_i$ and update the upper bound as

$$U \leftarrow \min\{U, f(x^1), \dots, f(x^r)\}$$

and the best known point as $x^* \in \{x^*, x^1, \dots, x^r\}$ with $f(x^*) = U$.

Updating the active components Update the set of active components to

$$S \leftarrow (S \setminus \{X\}) \cup \{X_1, \dots, X_r\}.$$

Eliminating nonoptimal active components Eliminate all $Y \in S$ with $L(Y) \geq U$ from S .

end while

return x^* is a solution of (4.6).

Some remarks on this algorithm:

- When we compute the lower bounds $L(X_i)$, we demand that these are tight in case $|X_i| = 1$. For all subsets X_i with $|X_i| = 1$ this implies that the new lower bound U satisfies $U \leq f(x^i) = L(X_i)$ and thus that these sets are eliminated from S at the end of an iteration. Consequently, at the beginning of an iteration we have $|X| \geq 2$ and thus can split X into at least two nonempty and disjoint sets $X_1, \dots, X_r$.
- If one of the subproblems with feasible set X_i does not admit a finite lower bound, then the whole discrete optimization problem is unbounded below.
- If D contains only finitely many feasible points, i.e. $2 \leq |D| < \infty$, then the algorithm terminates after a finite number of iterations and returns a solution (= global minimum) of the discrete optimization problem.
- The algorithm returns only one solution of the discrete optimization problem, even if there exists more than one. The reason is that we eliminate all subsets Y with $L(Y) \geq U$ from the set S of active components.

At the moment, the algorithm leaves many important steps unspecified:

- How should we decide which active component $X \in S$ to consider next? Since we use this partition to update the upper bound U and hopefully eliminate many active components Y from S , this choice is important for the speed of the algorithm.

We will discuss some general ideas for this choice below.

- How should we partition X into subsets $X_1, \dots, X_r$?

We will see in the next section that this question has a rather intuitive answer for mixed-integer linear programs.

- How can we compute the lower bounds $L(X_i)$? Each of the problems

$$\min_x f(x) \quad \text{s.t.} \quad x \in X_i$$

is again a discrete optimization problem, so we need some kind of heuristic to compute these lower bounds efficiently. The tighter these bounds are, the faster we can eliminate nonoptimal subsets. Thus, these bounds are critical for a fast algorithm.

Again, we will see that we can compute such bounds reasonably well for mixed-integer linear programs. A similar approach is possible for convex optimization problems with an additional discrete constraint. We also describe a more general approach below.

- How can we compute the integer feasible points $x^i \in X_i$ and how should we choose them? Finding a feasible point of a discrete optimization problem can already be

challenging. Additionally, it would be helpful not to use just any feasible point $x^i \in X_i$ but one with a small objective function value, because a smaller upper bound U allows to eliminate more nonoptimal active components Y from S .

We will see in the next section that for mixed-integer linear programs one can slightly modify the algorithm to only use integer feasible points that occur “naturally” over the course of the algorithm. If we do this, we do not determine an integer feasible point for each subproblem.

Alternatively, one can also try to use heuristics to determine an integer feasible point of each subproblem.

Subproblem Selection Strategies We can associate the subproblems generated in the branch&bound method with a rooted tree, called *branching tree*, where the root node is the subproblem with the original feasible set $X = D$ and each node with feasible set X has r child nodes with feasible sets $X_1, \dots, X_r$, which result from the partition of X .

In this tree, each subproblem with feasible set $X \in S$ (except for the root problem) has exactly one parent node or predecessor. By following these predecessors backwards to the root node, we can determine the distance $d(X)$ of a subproblem from the root problem. Then $d(D) = 0$, each child with feasible set X_i of the root node has $d(X_i) = 1$ and so on. We can use these distances to determine which subproblem to solve next.

Additionally, at the end of every iteration we can define a global lower bound as

$$L := \min_{Y \in S} L(Y).$$

By construction we then know that the optimal function value of the discrete problem is an element of the interval $[L, U]$. Since U is the best known function value $f(x^*)$ so far, the difference

$$U - L,$$

which is called the *optimality gap* (*Optimalitätslücke*), provides a worst case estimate for how far away from the optimal function value we might be with our current best point x^* .

- *Best bound search* (*Beste Schranke Suche*) Out of all active components $X \in S$ choose the one with the smallest lower bound $L(X)$, i.e. with $L = L(X)$. This approach strives to close the optimality gap $U - L$ fast by focusing on increasing the lower bound L .
- *Depth first search* (*Tiefensuche, Last in – first out*) Out of all active components $X \in S$ choose the one with the largest distance $d(X)$ to the root problem. If this choice is not unique, then choose the one with the smallest the lower bound $L(X)$ out of all candidates. This approach focuses on going deep in the branching tree

and has certain computational advantages. We will come back to this point when we adapt the branch&bound method to mixed-integer linear programs. A downside of this approach can be a slow improvement of the lower bound L , because we might be searching for the global solution in the *wrong part* of the feasible set.

- *Breadth first search (Breitensuche, Last in – last out)* Out of all active components $X \in S$ choose the one with the smallest distance $d(X)$ to the root problem. If this choice is not unique, then choose the one with the smallest the lower bound $L(X)$ out of all candidates. This approach spreads the attention of the algorithm over the whole feasible set. Since subproblems with a small distance to the root problem typically have larger feasible sets and smaller lower bounds, the effects of this approach are somewhat similar to the best bound search.

Branch&bound methods often use a combination of best bound steps for a global improvement of the lower bound with depth first steps for a good local resolution of the problem.

Bounding Strategies Instead of solving the discrete subproblem

$$\min_x f(x) \quad \text{s.t.} \quad x \in X_i$$

one typically chooses a simpler feasible set $\tilde{X}_i \supseteq X_i$ and possibly also a simpler objective function $\tilde{f} : \tilde{X}_i \rightarrow \mathbb{R}$ with

$$\tilde{f}(x) \leq f(x) \quad \text{for all } x \in X_i.$$

Then solving the relaxed problem

$$\min_x \tilde{f}(x) \quad \text{s.t.} \quad x \in \tilde{X}_i$$

provides a lower bound $L(X_i)$. But again, it is important to balance the quality of the resulting lower bounds and the computational effort for the solution of the relaxed problem.

4.3.2 Branch&Bound Method for (Mixed-)Integer Linear Programs

Let us now consider mixed-integer linear programs of the form

$$\min_x c^T x \quad \text{s.t.} \quad x \in P, \quad x_i \in \mathbb{Z} \quad (i \in I) \tag{4.7}$$

with $I \subseteq \{1, \dots, n\}$.

Then we start by solving the corresponding LP relaxation

$$\min_x c^T x \quad \text{s.t.} \quad x \in P.$$

If the solution x^* of this LP relaxation satisfies $x_i^* \in \mathbb{Z}$ for all $i \in I$, we have found a solution of the mixed-integer linear program and are done.

Otherwise, we choose one of the noninteger components $x_j^* \notin \mathbb{Z}$ with $j \in I$ and define the two new sets

$$\begin{aligned} X_1 &:= \{x \in P \mid x_i \in \mathbb{Z}(i \in I), x_j \leq \lfloor x_j^* \rfloor\}, \\ X_2 &:= \{x \in P \mid x_i \in \mathbb{Z}(i \in I), x_j \geq \lceil x_j^* \rceil\}. \end{aligned}$$

Then we have $X_1 \cap X_2 = \emptyset$ and every feasible point x of the mixed-integer linear program (4.7) satisfies $x \in X_1$ or $x \in X_2$. We have thus successfully divided the integer feasible set into $r = 2$ subsets. For each of the resulting child problems, we can again solve the LP relaxation and use a noninteger solution to branch again. Note that the solution x^* of the parent problem is by construction not feasible for both child problems.

The resulting algorithm is formalized in Algorithm 4.27.

Algorithm 4.27. (Branch&Bound Method for Mixed-Integer Linear Programs)

Input: A mixed-integer linear program (4.7).

Output: Solutions of the mixed-integer problem or $U = +\infty$, if the mixed-integer problem is infeasible.

Initialize the branching tree with the mixed-integer linear program as active root problem. Initialize the upper bound on the optimal function value as $U = +\infty$ and the list of solution candidates as empty.

while there are active problems in the branching tree **do**

 Choose an active problem with feasible set X and try to compute a solution x^* of the corresponding LP relaxation.

if a solution x^* exists and is mixed-integer feasible **then**

if $c^T x^* < U$ **then**

x^* is the best mixed-integer feasible point found so far. Update the upper bound $U \leftarrow c^T x^*$ and replace the current set of solution candidates with x^* .

else if $c^T x^* = U$ **then**

x^* is as good as the best feasible point found so far. Add x^* to the list of solution candidates.

else

x^* is worse than the best feasible point found so far and thus can be ignored.

end if

else if a solution x^* exists, but is not mixed-integer feasible **then**

if $c^T x^* \leq U$ **then**

 Choose a noninteger component $x_j^* \notin \mathbb{Z}$ with $j \in I$ and define the two new

subproblems with feasible sets

$$\begin{aligned} X_1 &:= \{x \in X \mid x_j \leq \lfloor x_j^* \rfloor\}, \\ X_2 &:= \{x \in X \mid x_j \geq \lceil x_j^* \rceil\}. \end{aligned}$$

Add these two new subproblems to the branching tree as active nodes.

else

The solution of the relaxation is worse than the best mixed-integer feasible point found so far and thus we do not have to consider this problem any more.

end if

else

If the current problem does not have a solution, because it is infeasible, we do not have to consider this problem anymore.

end if

Set the currently considered node to inactive in the branching tree.

end while

return the final set of solution candidates and the upper bound U

It differs in a few details from the more general Algorithm 4.26:

- In contrast to the previous section, it is now possible that a subset X_i or the feasible set of the corresponding LP relaxation are empty. Once this is recognized, we can just set the corresponding subproblem inactive, i.e. eliminate the corresponding subproblem from the set of active components S .
- We use the LP relaxations of the subproblems with feasible set X_i to compute the lower bounds $L(X_i)$.
- We do not compute a mixed-integer feasible point x^i in each subset X_i . Instead, we only use the mixed-integer feasible solutions of LP relaxations to improve the upper bound U . However, for a faster increase of the upper bound, we could add some kind of heuristic to try to cheaply compute mixed-integer feasible points $x^i \in X_i$.

Example 4.28. Consider again the integer linear program

$$\begin{aligned} \min_x \quad & x_1 - 2x_2 \quad \text{s.t.} \quad -4x_1 + 6x_2 \leq 9, \\ & x_1 + x_2 \leq 4, \\ & x \geq 0, \quad x \in \mathbb{Z}^2. \end{aligned}$$

Solving the LP relaxation (P_0) of this problem leads to $x^* = (1.5, 2.5)$, where both components are noninteger. We could thus branch for either variable. We choose x_2 , which

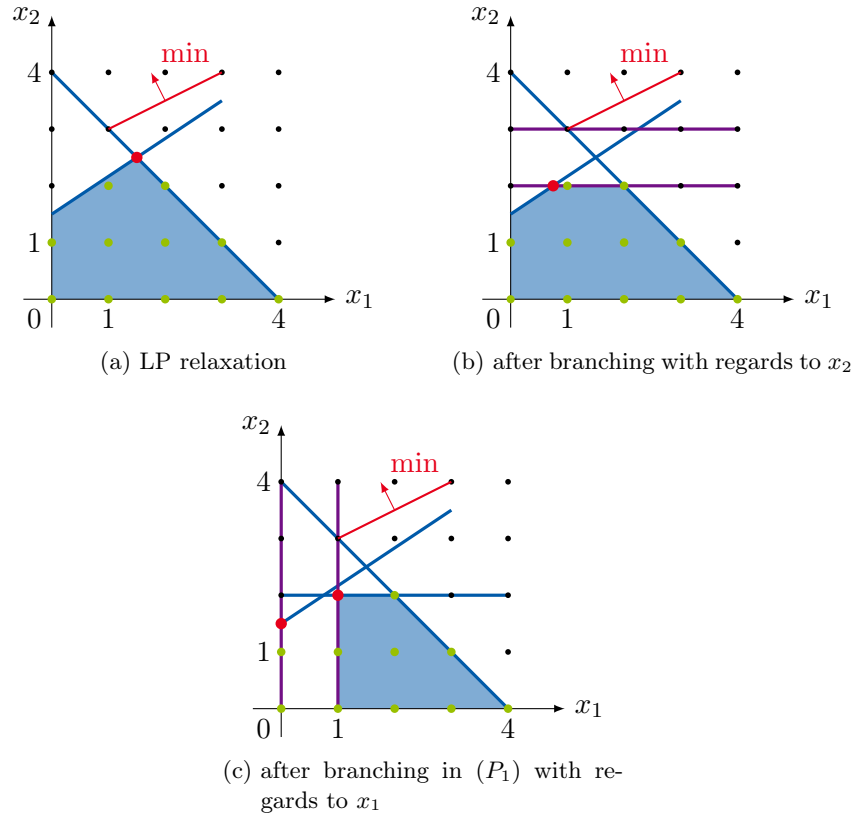


Figure 4.7: Branch&bound method for Example 4.28

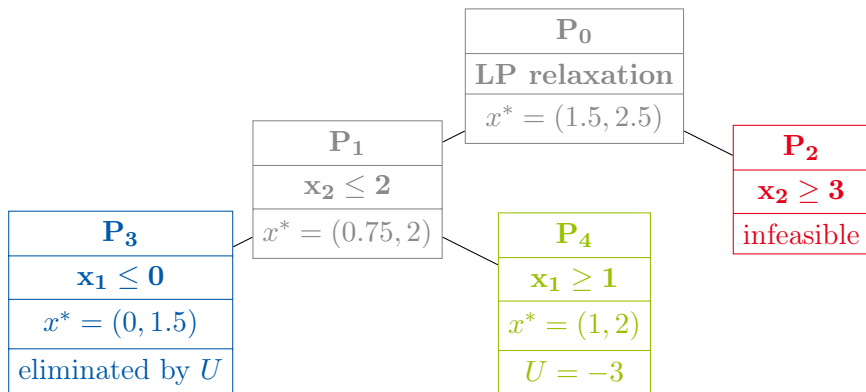


Figure 4.8: Branching tree for Example 4.28

results in the two new linear problems

$$\min_x x_1 - 2x_2 \quad \text{s.t.} \quad -4x_1 + 6x_2 \leq 9, \quad x_1 + x_2 \leq 4, \quad x_2 \leq 2, \quad x \geq 0 \quad (P_1),$$

$$\min_x x_1 - 2x_2 \quad \text{s.t.} \quad -4x_1 + 6x_2 \leq 9, \quad x_1 + x_2 \leq 4, \quad x_2 \geq 3, \quad x \geq 0 \quad (P_2).$$

Here, problem (P_2) is infeasible and problem (P_1) has the solution $x^* = (0.75, 2)$. So we can branch in (P_1) with regards to x_1 and obtain the new problems

$$\min_x x_1 - 2x_2 \quad \text{s.t.} \quad -4x_1 + 6x_2 \leq 9, \quad x_1 + x_2 \leq 4, \quad x_2 \leq 2, \quad x_1 \leq 0, \quad x \geq 0 \quad (P_3),$$

$$\min_x x_1 - 2x_2 \quad \text{s.t.} \quad -4x_1 + 6x_2 \leq 9, \quad x_1 + x_2 \leq 4, \quad x_2 \leq 2, \quad x_1 \geq 1, \quad x \geq 0 \quad (P_4).$$

Here, problem (P_4) has the integer solution $x^* = (1, 2)$ with objective function value -3 . Thus, (P_4) is solved completely and we have found a solution candidate $x^* = (1, 2)$ with the corresponding upper bound $U = -3$.

Problem (P_3) has the solution $x^* = (0, 1.5)$ with objective function value -3 . Since this is not better (smaller) than our current upper bound U , we know that integer points feasible for (P_3) cannot be better than our current solution candidate. Thus, if we are only interested in one integer solution, we do not have to consider (P_3) anymore.

If instead we want to find all integer solutions, we would have to branch in (P_3) one more time for x_2 . This would result in one infeasible problem and one problem with the solution $x^* = (0, 1)$ and objective function value -2 . Now we definitely know that $(1, 2)$ is the unique solution of the integer problem.

This example illustrates two interesting points: It can be necessary to branch on a variable more than one time before one obtains an integer solution. After solving the LP relaxation (P_0) , we branched with regards to x_2 . And in order to obtain an integer solution of (P_3) , we have to branch with regards to x_2 a second time. Additionally, whether we have to solve (P_3) or can eliminate it using the bound U depends on the order, in which we solve the open subproblems⁴.

Within Algorithm 4.27 we have specified that we compute the lower bounds using LP relaxations. In order to select the next subproblem to be solved, we can use the strategies discussed in the previous section. Note that depth first search can be executed particularly efficiently, because two subsequent subproblems typically only differ in one new constraint. Thus, we can use the optimal basis and dual solution of the parent node to start the dual simplex method for the child node.

Branching Strategies We have specified that we generate the new subproblems X_1, X_2 by branching on a noninteger variable x_j^* with $j \in I$. In case there is more than one such noninteger variable, we have to decide which one to use for branching.

Two possible simple branching strategies are given below:

⁴Fun fact: Sometimes even the order, in which the variables of the problem are numbered, can affect the size of the resulting branching tree.

- *maximum infeasibility*: Choose the variable x_j^* with $j \in I$ as branching variable, where x_j^* is furthest away from being integer, i.e. where $x_j^* - \lfloor x_j^* \rfloor$ is closest to 0.5. The idea is that this leads to a large change in the solutions of the two new subproblems compared to the current solution x^* .
- *minimum infeasibility*: Choose the variable x_j^* with $j \in I$ as branching variable, where x_j^* is closest to being integer, i.e. where $x_j^* - \lfloor x_j^* \rfloor$ is closest to 0 or 1. The idea here is that this variable is almost integer already and thus hopefully becomes integer in the subproblem.

While these two rules sound intuitive, they in practice perform approximately as good as a random choice of the branching variable. Two alternative branching strategies, which are computationally more expensive but can lead to a significant decrease in the size of the branching tree, are given below.

Remark 4.29 (Optional: more advanced branching strategies). *For every noninteger variable x_j^* with $j \in I$ we introduce the following notation*

$$r_i^- := x_i^* - \lfloor x_i^* \rfloor \quad \text{and} \quad r_i^+ := \lceil x_i^* \rceil - x_i^*.$$

If $x^{-,i}$ and $x^{+,i}$ are the solutions of the LP relaxations of the two child problems

$$\begin{aligned} \min_x c^T x \quad \text{s.t.} \quad x \in \tilde{P}, \quad x_i \leq \lfloor x_i^* \rfloor, \quad (P^-) \\ \min_x c^T x \quad \text{s.t.} \quad x \in \tilde{P}, \quad x_i \geq \lceil x_i^* \rceil, \quad (P^+) \end{aligned}$$

then we denote the change in the objective function as

$$\Delta_i^- := c^T x^{-,i} - c^T x^* \geq 0 \quad \text{and} \quad \Delta_i^+ := c^T x^{+,i} - c^T x^* \geq 0.$$

To choose a branching variable, we can compute a score s_i for each possible branching variable x_i and then choose the one with the highest score. This score can be computed e.g. as

$$s(d_i^-, d_i^+) = (1 - \mu) \min\{d_i^-, d_i^+\} + \mu \max\{d_i^-, d_i^+\},$$

with a weighting parameter $\mu \in [0, 1]$ and variables d_i^-, d_i^+ , that somehow reflect the change in the objective function value when branching on variable i , e.g. $d_i^\pm = \Delta_i^\pm$.

Two examples of branching strategies based on such a score are:

- *pseudo cost (Pseudo-Kosten)*: Use averages δ_i^-, δ_i^+ of historic data on the relative changes $\frac{\Delta_i^-}{r_i^-}$ and $\frac{\Delta_i^+}{r_i^+}$ of the objective function, when the branching variable i was used in previous subproblems, and choose the branching variable with the highest score based on the expected changes $d_i^- := \delta_i^- r_i^-$, $d_i^+ := \delta_i^+ r_i^+$ for the current subproblem.

- strong branching: For a candidate set of branching variables (e.g. all currently noninteger variables) temporarily generate the two child problems and compute (or approximate) the resulting changes $d_i^- := \Delta_i^-$, $d_i^+ := \Delta_i^+$. Based on this information, pick the variable with the highest score and branch on that variable.

In [1] a combination of these two, i.e. of strong branching (which can become very expensive) and pseudo cost branching (which requires sufficient and reliable historical data) is suggested.

4.4 Branch&Cut Method

In the branch&bound method we have to compute lower bounds $L(X_i)$ for the subproblems

$$\min_x f(x) \quad \text{s.t.} \quad x \in X_i$$

and we typically do this by solving a simpler relaxed problem

$$\min_x f(x) \quad \text{s.t.} \quad x \in \tilde{X}_i$$

with $X_i \subseteq \tilde{X}_i$. Since these lower bounds $L(X_i)$ are used to eliminate active components with $L(X_i) \geq U$, a tighter relaxation $\tilde{X}_i$ of X_i and thus a tighter lower bound $L(X_i)$ helps to eliminate active components earlier and thus speeds up the branch&bound method.

In the branch&bound method for mixed-integer linear programs, we used the LP relaxations of the subproblems to compute the lower bounds. And we know from Section 4.2 that we can use cuts to tighten the LP relaxation and thus obtain better lower bounds. This motivates to augment the branch&bound method from Section 4.3 for mixed-integer linear programs

$$\min_x f(x) := c^T x \quad \text{s.t.} \quad x \in D = \{x \in P \mid x_i \in \mathbb{Z} \ (i \in I)\}$$

with cuts. The resulting method is called *branch&cut method*, see Figure 4.9 for a formalized description.

Some remarks on this algorithm:

- In order to generate cuts, we can use the techniques described in Section 4.2. Since y^i is the solution of the LP relaxation, we can use e.g. a (mixed-integer) Gomory cut. Besides such general purpose cuts it is sometimes also possible to use the special structure of the optimization problem to generate problem specific cuts.
- If the solution y^i of the LP relaxation is not integer and thus the lower bound probably not tight, we do not immediately branch, but instead first try to improve the solution and the resulting lower bound by adding one or more cuts.

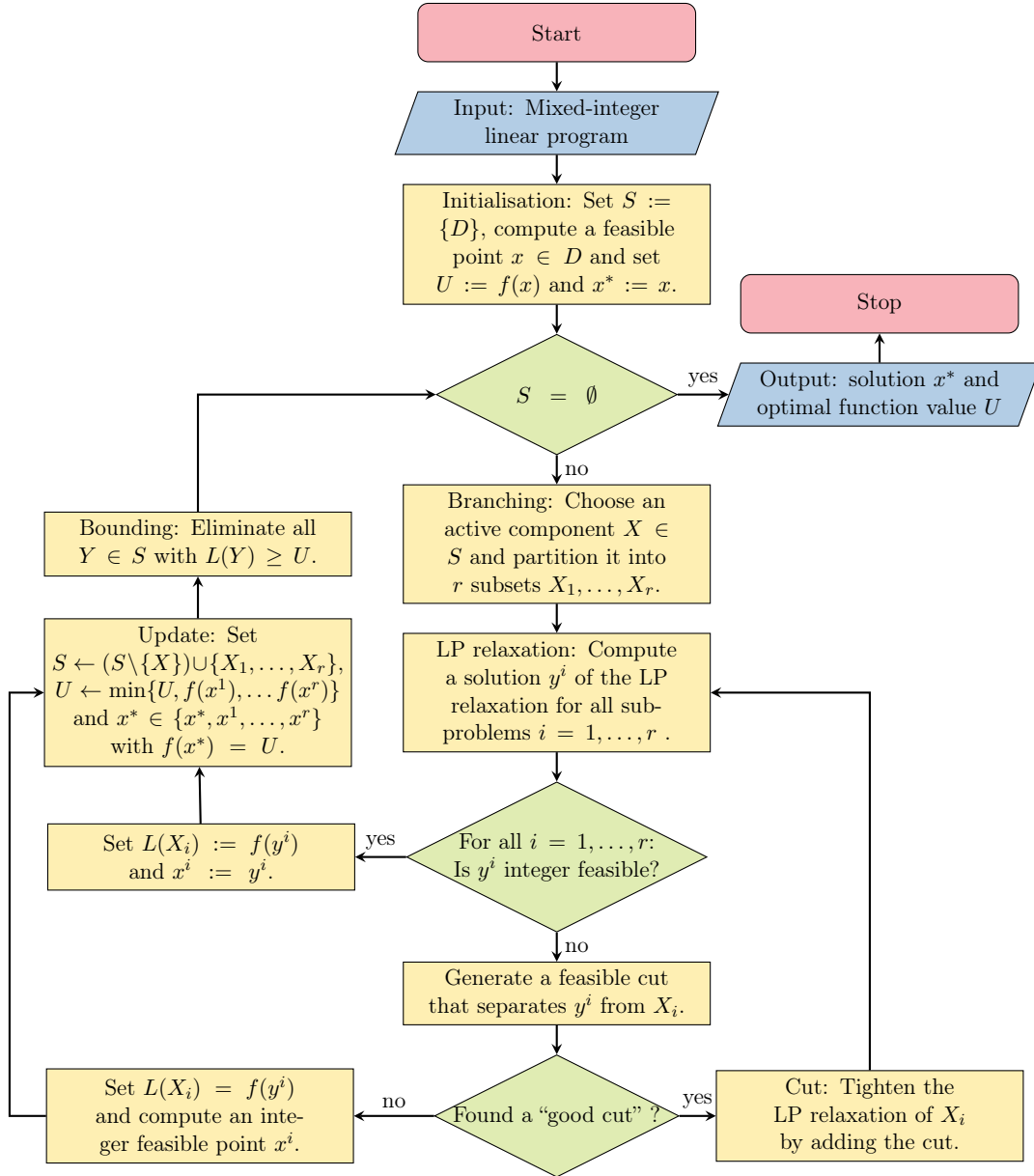


Figure 4.9: Flow Chart for the Branch&Cut Method

However, the idea here is to add only a few cuts in each subproblem. If this does not help, e.g. because the resulting lower bound, i.e. $f(y^i)$, does not increase, then we stop adding cuts and instead branch this subproblem again in a later iteration.

- The number of cuts/constraints added to a subproblem can become quite large over the iterations. To combat this, one can e.g. limit the number of cuts added to a subproblem and drop older cuts, when new ones are added.
- In the first iteration, when we look at the whole feasible set $X = D$, it is often helpful to skip the branching step (i.e. use $r = 1$) and instead to directly add some cuts in order to tighten the LP relaxation. When we then start the branching process based on this tighter LP relaxation, the resulting branching tree is often much smaller.
- The branch&cut method contains both extreme cases: The branch&bound method, where we do not add any cuts, and the cutting plane method, where we always choose $r = 1$ and thus do not branch. However, the hybrid method, where we use both cuts and branching, tends to work better in practice.

4.5 Application to the Traveling Salesperson Problem

Recall the traveling salesperson problem from Section 1.1, where someone wants to visit the $n \geq 2$ cities $S_1, S_2, \dots, S_n$ starting in S_1 . They want to visit each city exactly once and return to S_1 at the end of their trip. To travel, they can use all connections in

$$E = \{(i, j) \mid \text{there exists a direct connection from } S_i \text{ to } S_j\},$$

where for each such connection $(i, j) \in E$ the corresponding travel time or cost is given by $c_{ij} > 0$. The roundtrip should be chosen such that the overall travel time/cost is minimal.

We can represent this problem as a graph $G = (V, E)$, where the nodes $V = \{1, 2, \dots, n\}$ correspond to the cities $S_1, S_2, \dots, S_n$ and the edges $e = (i, j) \in E$ are given as above with edge weights $w(e) = w((i, j)) = c_{ij}$.

To simplify the subsequent discussion, we assume that this graph does not contain any loops, i.e. edges of the form $e = (i, i)$. This assumption can be made without loss of generality, because such loops would not be used on a roundtrip anyway. Additionally, we assume that the graph does not contain any parallel edges. If the graph does contain parallel edges, it suffices to keep the one with the lowest cost, because only that one might be used in an optimal round tour. Note that edges of the form (i, j) and (j, i) are inverted and not parallel, i.e. both of those can occur at the same time.

Furthermore, we can assume without loss of generality that the graph G is fully connected, by inserting all missing edges $e = (i, j) \notin E$ with weights $c_{ij} = +\infty$.

The problem can then be completely characterized by the *cost matrix* $C \in (\mathbb{R} \cup \{\infty\})^{n \times n}$ with entries c_{ij} . In terms of graph theory, this cost matrix is the adjacency matrix of the weighted directed graph G .

Recall that we can use the binary decision variables

$$x_{ij} = \begin{cases} 1 & \text{connection } (i, j) \text{ is used on the tour,} \\ 0 & \text{else,} \end{cases}$$

in order to model round tours. The resulting optimization problem $\text{TSP}(C)$ including subtour elimination constraints was then of the form

$$\begin{aligned} \min_{x_{ij}} \quad & \sum_{i,j=1}^n c_{ij} x_{ij} \quad (\text{total travel time}) \\ \text{s.t.} \quad & \sum_{i=1}^n x_{ij} = 1 \quad \forall j = 1, \dots, n \quad (\text{each city is reached exactly once}), \\ & \sum_{j=1}^n x_{ij} = 1 \quad \forall i = 1, \dots, n \quad (\text{each city is left exactly once}), \\ & \sum_{i \in U, j \in V \setminus U} x_{ij} \geq 1 \quad \forall U \subset V : 2 \leq |U| \leq n-2 \quad (\text{subtour elimination}). \end{aligned}$$

If we use an edge $e = (i, j) \notin E$ on a roundtrip, then the corresponding costs are $c_{ij} = +\infty$. Here, we use the typical conventions

$$\infty + \infty = \infty, \quad \infty \cdot 0 = 0, \quad \infty + c = \infty \quad \text{for all } c \in \mathbb{R}.$$

4.5.1 Reduced Costs

Instead of working directly with the cost matrix C , we first transform this matrix into the *reduced cost matrix* $C^* \in \mathbb{R}^{n \times n}$, which is computed as follows:

- **Step 1:** For each row $i = 1, \dots, n$ compute

$$\alpha_i := \min_{j=1, \dots, n} c_{ij}.$$

Subtract this minimum from the corresponding row to obtain the matrix $\tilde{C} \in \mathbb{R}^{n \times n}$ with the entries

$$\tilde{c}_{ij} = c_{ij} - \alpha_i.$$

- **Step 2:** For each column $j = 1, \dots, n$ compute

$$\beta_j := \min_{i=1, \dots, n} \tilde{c}_{ij}.$$

Subtract this minimum from the corresponding column to obtain the matrix $C^* \in \mathbb{R}^{n \times n}$ with the entries

$$c_{ij}^* = \tilde{c}_{ij} - \beta_j.$$

The reduced cost matrix still satisfies $C^* \geq 0$ and $c_{ij}^* = \infty$ if and only if $c_{ij} = \infty$. But in contrast to the original cost matrix C ; the reduced cost matrix contains a zero entry in each row and in each column.

In every round tour x we arrive at each city S_j exactly once and we leave each city S_i exactly once. We thus obtain the relation

$$\sum_{i,j=1}^n c_{ij}^* x_{ij} = \sum_{i,j=1}^n \tilde{c}_{ij} x_{ij} - \sum_{j=1}^n \beta_j = \sum_{i,j=1}^n c_{ij} x_{ij} - \underbrace{\left(\sum_{i=1}^n \alpha_i + \sum_{j=1}^n \beta_j \right)}_{=:\gamma(C)}.$$

This shows that all roundtours cost exactly $\gamma(C)$ more in $\text{TSP}(C)$ than in $\text{TSP}(C^*)$.

Due to the previous observation, both problems have the same solutions. If $z(C)$ and $z(C^*)$ are the minimal travel costs for $\text{TSP}(C)$ and $\text{TSP}(C^*)$, then we know

$$z(C^*) \geq 0 \quad \text{and} \quad z(C) = z(C^*) + \gamma(C) \geq \gamma(C).$$

Consequently, $\gamma(C)$ provides a lower bound for the minimal travel costs in $\text{TSP}(C)$.

4.5.2 Tailoring the Branch&Bound Method

Now we want to tailor the branch&bound method from Section 4.3 to the traveling salesperson problem. To this end, let D denote the set of all roundtrips x and $f(x) = \sum_{i,j=1}^n c_{ij} x_{ij}$ the objective function.

In the set S we again collect the set of all active components X of the feasible set D . For each such component X we have to consider the subproblem TSP_X

$$\min_x \sum_{i,j=1}^n c_{ij} x_{ij} \quad \text{s.t.} \quad x \in X.$$

For each such subproblem we have to do the following:

- Compute a lower bound $L(X)$.
- Compute a feasible point $x \in X$. Although we can also mostly skip this step as we did in Section 4.3.2.
- Partition X into at least two nonempty and disjoint subsets.

In the initial iteration we consider $X = D$. In this case a known lower bound is $L(X) = \gamma(C)$. In order to compute a feasible point $x \in X$, we can use some kind of heuristic, e.g. the nearest neighbor heuristic. So it only remains to decide how to partition the set $X = D$.

The reduced cost matrix C^* contains a zero entry in each row and each column. We can thus pick a pair of cities S_k, S_l such that $c_{kl}^* = 0$. Then we divide $X = D$ into

$$X_1 = \{x \in X \mid x_{kl} = 1\} \quad \text{and} \quad X_2 = \{x \in X \mid x_{kl} = 0\}.$$

These two sets are disjoint and their union is X . Formally, both sets are also nonempty, because we modified the graph G such that it is fully connected. But it is possible that one of the two components only contains roundtours with cost $+\infty$.

Both resulting subproblems are again traveling salesperson problems with the modified cost matrices $C(X_1)$ and $C(X_2)$ with entries

$$\begin{aligned} c(X_1)_{ij} &= \begin{cases} +\infty & \text{if } i = k \text{ and } j \neq l, \\ +\infty & \text{if } j = l \text{ and } i \neq k, \\ c_{ij}^* & \text{else,} \end{cases} \\ c(X_2)_{ij} &= \begin{cases} +\infty & \text{if } (i, j) = (k, l), \\ c_{ij}^* & \text{else.} \end{cases} \end{aligned}$$

For these two matrices we can again compute the corresponding reduced cost matrixes $C^*(X_1)$ and $C^*(X_2)$, which corresponds to reducing the costs of roundtrips by $\gamma(C(X_1))$ and $\gamma(C(X_2))$ respectively.

Consequently, lower bounds for the two subproblems with feasible sets X_1 and X_2 can cheaply be computed as

$$L(X_1) = \gamma(C(X_1)) + \gamma(C) \quad \text{and} \quad L(X_2) = \gamma(C(X_2)) + \gamma(C).$$

And since both resulting subproblems are traveling salesperson problems with cost matrices $C(X_i)$ and reduced cost matrices $C^*(X_i)$, we can treat them the same as the initial problem with $X = D$. Some details to keep in mind here:

- We reduce the cost matrices again in every iteration. To compute the lower bound $L(X)$ for a current subproblem, we thus have to add up the term $\gamma(C(X))$ resulting from reducing the cost matrix $C(X)$ of the current subproblem and the respective terms for all parent problems up to the root problem.
- Since we fix a connection x_{kl} to 1 or 0 each time we branch a subproblem, the feasible sets X of the subproblems can at some point become empty. This can be very noticeable, when the corresponding cost matrix $C(X)$ contains only infinite entries in some row or column and thus the reduced cost matrix cannot be computed anymore. If we notice this problem, we can just discard the respective (infeasible) subproblem immediately.

- Computing the reduced cost matrices $C^*(X_1)$ and $C^*(X_2)$ is relatively cheap.
In $C(X_2)$ each row $i \neq k$ and each column $j \neq l$ already contains a zero entry. Thus we only have to compute the minimum in the k -th row and then in the l -th column.
In the matrix $C(X_1)$ we can eliminate the k -th row and the l -th column completely, because in this subproblem we have decided to travel from S_k to S_l and thus do not need any other connections exiting S_k and arriving at S_l . For the resulting smaller matrix we then have to compute the corresponding reduced matrix.
- When we follow a branch of the branching tree from the root node downwards, each connection x_{kl} should be chosen as branching variable at most once.
In the “right” subproblem with feasible set X_2 this is not a problem, because we set $c(X_2)_{kl} = \infty$ and this cost remains infinite from then on.
But in the “left” subproblem with feasible set X_1 we still have $c(X_1)_{kl} = 0 = c^*(X_1)_{kl}$ after branching and reducing the cost matrix and thus could choose the same connection x_{kl} as branching variable again in a later iteration. This can be avoided by eliminating the k -th row and the l -th column as described above.
- Instead of using α_i and β_j as defined in Section 4.5.1 to compute the reduced cost matrices, it is also possible to work with other numbers. However, it is important that the reduced cost matrix remains nonnegative and that it is possible to obtain tight lower bounds. One possible simple modification is to just choose a different, possibly mixed order, in which one computes and subtracts the row minima α_i and the column minima β_j .

Example 4.30. We consider the TSP problem with $n = 4$ cities and the cost matrix

$$C = \begin{pmatrix} \infty & 1 & 3 & 8 \\ 6 & \infty & 2 & 7 \\ 9 & 13 & \infty & 4 \\ 11 & 1 & 7 & \infty \end{pmatrix}.$$

In this cost matrix we have infinite costs on the diagonal because we do not consider loops. Since all other costs are finite, the underlying graph $G = (V, E)$ is fully connected, see Figure 4.10. If we want compute the reduced cost matrix, we obtain the coefficients

$$\alpha = (1, 2, 4, 1),$$

the intermediary matrix

$$\tilde{C} = \begin{pmatrix} \infty & 0 & 2 & 7 \\ 4 & \infty & 0 & 5 \\ 5 & 9 & \infty & 0 \\ 10 & 0 & 6 & \infty \end{pmatrix},$$

the coefficients

$$\beta = (4, 0, 0, 0),$$

and finally the reduced cost matrix

$$C^* = \begin{pmatrix} \infty & 0 & 2 & 7 \\ 0 & \infty & 0 & 5 \\ 1 & 9 & \infty & 0 \\ 6 & 0 & 6 & \infty \end{pmatrix}$$

and the initial lower bound

$$L(D) = \gamma(C) = 1 + 2 + 4 + 1 + 4 + 0 + 0 + 0 = 12.$$

Iteration 1: If we choose our initial branching variable as x_{12} with $c_{12}^* = 0$, we obtain the two subproblems with cost matrices

$$C(X_1) = \begin{pmatrix} \infty & 0 & \infty & \infty \\ 0 & \infty & 0 & 5 \\ 1 & \infty & \infty & 0 \\ 6 & \infty & 6 & \infty \end{pmatrix} \quad \text{and} \quad C(X_2) = \begin{pmatrix} \infty & \infty & 2 & 7 \\ 0 & \infty & 0 & 5 \\ 1 & 9 & \infty & 0 \\ 6 & 0 & 6 & \infty \end{pmatrix}.$$

Reducing these cost matrices results in

$$C^*(X_1) = \begin{pmatrix} \infty & 0 & \infty & \infty \\ 0 & \infty & 0 & 5 \\ 1 & \infty & \infty & 0 \\ 0 & \infty & 0 & \infty \end{pmatrix} \quad \text{and} \quad C^*(X_2) = \begin{pmatrix} \infty & \infty & 0 & 5 \\ 0 & \infty & 0 & 5 \\ 1 & 9 & \infty & 0 \\ 6 & 0 & 6 & \infty \end{pmatrix}$$

with the corresponding lower bounds

$$\begin{aligned} L(X_1) &= L(D) + \gamma(C(X_1)) = 12 + 6 = 18, \\ L(X_2) &= L(D) + \gamma(C(X_2)) = 12 + 2 = 14. \end{aligned}$$

Iteration 2: If we want to use the best bound search, we should consider the subproblem with feasible set X_2 next, because it has the smaller lower bound $L(X_2) = 14$. In this subproblem, we can use x_{13} as the next branching variable with $c^*(X_2)_{13} = 0$. If we do this, we obtain two new subproblems with cost matrices

$$C(X_{2,1}) = \begin{pmatrix} \infty & \infty & 0 & \infty \\ 0 & \infty & \infty & 5 \\ 1 & 9 & \infty & 0 \\ 6 & 0 & \infty & \infty \end{pmatrix} \quad \text{and} \quad C(X_{2,2}) = \begin{pmatrix} \infty & \infty & \infty & 5 \\ 0 & \infty & 0 & 5 \\ 1 & 9 & \infty & 0 \\ 6 & 0 & 6 & \infty \end{pmatrix}.$$

Reducing these cost matrices again results in

$$C^*(X_{2,1}) = \begin{pmatrix} \infty & \infty & 0 & \infty \\ 0 & \infty & \infty & 5 \\ 1 & 9 & \infty & 0 \\ 6 & 0 & \infty & \infty \end{pmatrix} \quad \text{and} \quad C(X_{2,2}) = \begin{pmatrix} \infty & \infty & \infty & 0 \\ 0 & \infty & 0 & 5 \\ 1 & 9 & \infty & 0 \\ 6 & 0 & 6 & \infty \end{pmatrix}$$

with the corresponding lower bounds

$$\begin{aligned} L(X_{2,1}) &= L(X_2) + \gamma(C(X_{2,1})) = 14 + 0 = 14, \\ L(X_{2,2}) &= L(X_2) + \gamma(C(X_{2,2})) = 14 + 5 = 19. \end{aligned}$$

Iteration 3: Due to best bound search, we consider the subproblem with feasible set $X_{2,1}$ next and can branch e.g. on $x_{2,1}$ with $c^*(X_{2,1})_{2,1} = 0$. Note that we cannot branch on $x_{1,2}$ again, because we already did before. Then we obtain the two new subproblem with cost matrices

$$C(X_{2,1,1}) = \begin{pmatrix} \infty & \infty & 0 & \infty \\ 0 & \infty & \infty & \infty \\ \infty & 9 & \infty & 0 \\ \infty & 0 & \infty & \infty \end{pmatrix} \quad \text{and} \quad C(X_{2,1,2}) = \begin{pmatrix} \infty & \infty & 0 & \infty \\ \infty & \infty & \infty & 5 \\ 1 & 9 & \infty & 0 \\ 6 & 0 & \infty & \infty \end{pmatrix}.$$

The corresponding reduced cost matrices are

$$C^*(X_{2,1,1}) = \begin{pmatrix} \infty & \infty & 0 & \infty \\ 0 & \infty & \infty & \infty \\ \infty & 9 & \infty & 0 \\ \infty & 0 & \infty & \infty \end{pmatrix} \quad \text{and} \quad C^*(X_{2,1,2}) = \begin{pmatrix} \infty & \infty & 0 & \infty \\ \infty & \infty & \infty & 0 \\ 0 & 9 & \infty & 0 \\ 5 & 0 & \infty & \infty \end{pmatrix}$$

with the corresponding lower bounds

$$\begin{aligned} L(X_{2,1,1}) &= L(X_{2,1}) + \gamma(C(X_{2,1,1})) = 14 + 0 = 14, \\ L(X_{2,1,2}) &= L(X_{2,1}) + \gamma(C(X_{2,1,2})) = 14 + 5 + 1 = 20. \end{aligned}$$

Here, the feasible set $X_{2,1,1}$ contains only one roundtour, which is

$$x^* = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 1 & 0 & 0 \end{pmatrix}$$

and corresponds to

$$S_1 \rightarrow S_3 \rightarrow S_4 \rightarrow S_2 \rightarrow S_1.$$

Since this roundtour only uses connections with reduced costs equal to zero, the traveling cost is exactly $f(x^*) = L(X_{2,1,1}) = 14$. Now that we have found a feasible point x^* of the TSP, we can set the upper bound to $U = f(x^*) = 14$. And since all other active components of the feasible set $Y \in S$ have lower bounds greater than zero, we can discard them and know that x^* is the unique solution of this TSP. An illustration of the branching tree can be found in Figure 4.11.

4.6 Exercises

Exercise 4.1. Let $A \in \mathbb{R}^{m \times n}$ be given and I the identity matrix of suitable dimension. Prove the following:

- (a) A is totally unimodular if and only if the matrix $\begin{pmatrix} A & I \end{pmatrix}$ is unimodular.
- (b) A is totally unimodular if and only if the following matrix is totally unimodular:

$$\begin{pmatrix} A \\ -A \\ I \\ -I \end{pmatrix}$$

- (c) A is totally unimodular if and only if the matrix A^T is totally unimodular.

Exercise 4.2. Let $A \in \mathbb{Z}^{m \times n}$ and $b \in \mathbb{Z}^m$ be given and consider the feasible set

$$P := \{x \in \mathbb{R}^n \mid Ax \leq b, x \geq 0\}.$$

Using slack variables s , we can rewrite this feasible set as

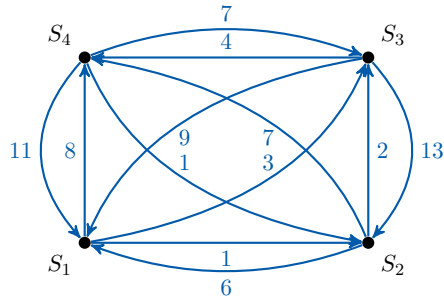
$$\tilde{P} := \{z = (x, s) \in \mathbb{R}^{n+m} \mid Ax + s = b, x, s \geq 0\}.$$

- (a) Prove that $x \in P$ is integer if and only if $z = (x, s) \in \tilde{P}$ is integer.
- (b) Prove that P is integral if and only if $\tilde{P}$ is integral.

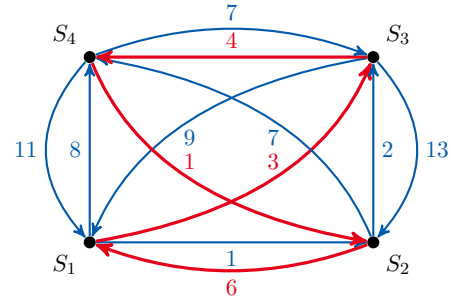
Exercise 4.3. Consider a directed graph $G = (V, E)$ without loops. Then every edge $e \in E$ can be written as $e = (u, v)$ with head $v \in V$ and tail $u \in V$ and $u \neq v$. Such a graph can be represented using the *incidence matrix* $M \in \mathbb{R}^{|V| \times |E|}$ with

$$m_{v,e} := \begin{cases} +1 & \text{if } e = (v, *), \\ -1 & \text{if } e = (*, v), \\ 0 & \text{else.} \end{cases}$$

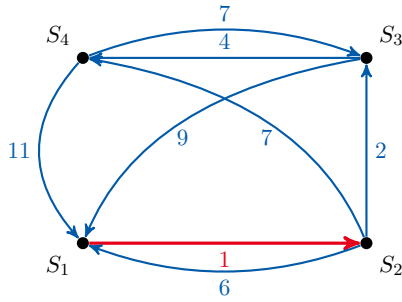
Prove that this incidence matrix M is totally unimodular.



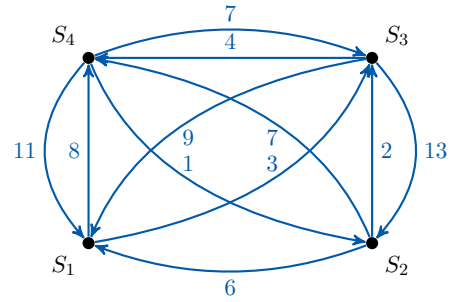
(a) initial graph corresponding to $X = D$



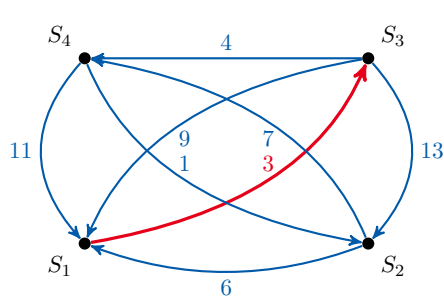
(b) optimal roundtour



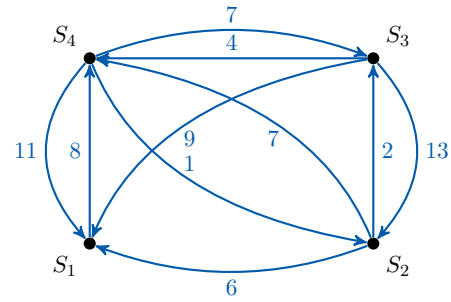
(c) graph corresponding to X_1



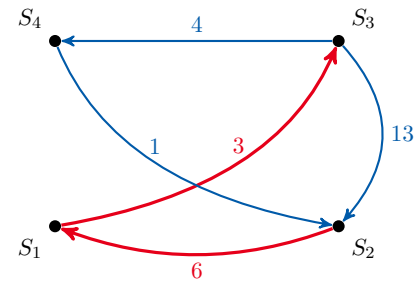
(d) graph corresponding to X_2



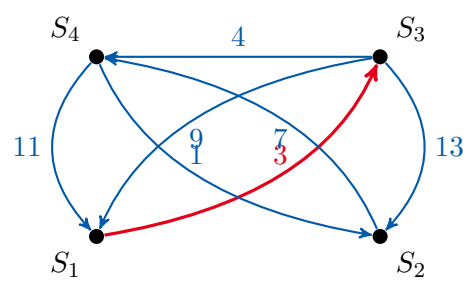
(e) graph corresponding to $X_{2,1}$



(f) graph corresponding to $X_{2,2}$



(g) graph corresponding to $X_{2,1,1}$



(h) graph corresponding to $X_{2,1,2}$

Figure 4.10: Graphs corresponding to Example 4.30

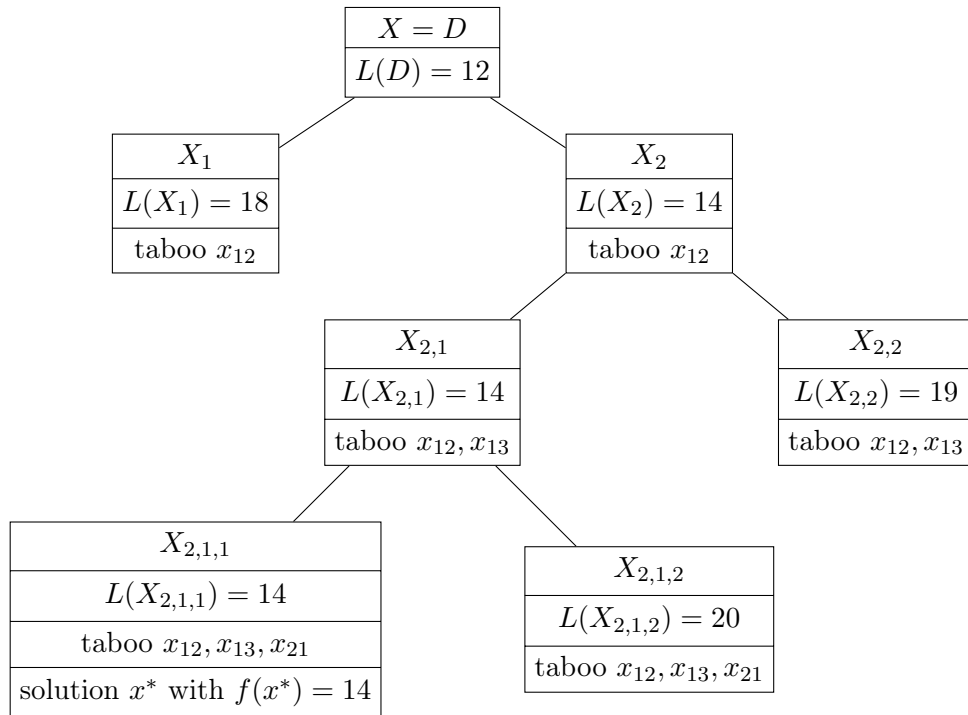


Figure 4.11: Branching tree corresponding to Example 4.30

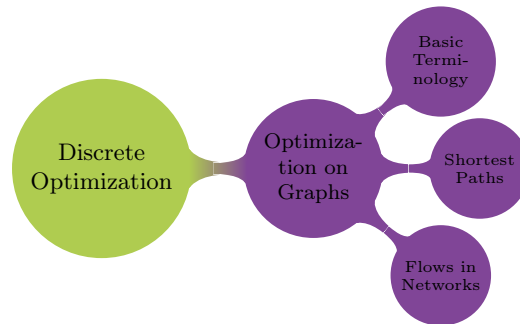
Exercise 4.4. Consider a undirected graph $G = (V, E)$ without loops. Then every edge $e \in E$ can be written as $e = \{u, v\}$ with two endnodes $u, v \in V$. Such a graph can be represented using the *incidence matrix* $M \in \mathbb{R}^{|V| \times |E|}$ with

$$m_{v,e} := \begin{cases} +1 & \text{if } e = \{v, *\}, \\ 0 & \text{else.} \end{cases}$$

Prove that this incidence matrix M is totally unimodular in case the graph G is *bipartite*, i.e. if the vertices V can be divided into two disjoint sets S and D such that all edges $e \in E$ have one endnote in S and one endnode in D .

Remark: One can actually show that G is bipartite if and only if the incidence matrix M is totally unimodular. The proof is based on the fact that A matrix satisfying the first two conditions in Lemma 4.18 is totally unimodular if and only if it also satisfies the third condition.

5 Optimization in Graphs



In this chapter, we want to consider several optimization problems in graphs. We begin by recalling some basic terminology and ideas in Section 5.1 and then discuss several methods to compute shortest paths in Section 5.2. To close this chapter, we consider networks in Section 5.3 and develop two algorithms to compute maximum flows. Although for example the maximum flow problem can also be formulated as a continuous linear program, it is possible to obtain more efficient solution algorithms by explicitly using the discrete structure of the graph.

Other optimization problems in graphs, which we consider in this lecture, are the traveling salesperson problem, see Section 4.5 and the minimum spanning tree problem, see Section 6.1.

5.1 Repetition: Basic Terminology

In this section, we want to recall the basic vocabulary used to describe graphs as well as some useful properties of graphs.

Definition 5.1.

- (a) A *graph* (*Graph*) is a tuple $G = (V, E)$ with a finite set of *vertices* (*Knoten*) $V \neq \emptyset$ and a finite set of *edges* (*Kanten*) E .
- (b) An *undirected edge* (*ungerichtete Kante*) $e \in E$ is of the form $e = \{u, v\}$ with two *endnodes* (*Endknoten*) $u, v \in V$. A *directed edge* (*arrow, arc*) (*gerichtete Kante*)

$e \in E$ is of the form $e = (u, v)$, where the two endnodes are called *tail* (*Fuß*) $u \in V$ and *head* (*Spitze*) $v \in V$.

- (c) A graph with undirected edges is called *undirected* (*ungerichtet*), a graph with directed edges is called *directed* (*gerichtet*) or *digraph*.
- (d) An edge $e \in E$ of the form $e = \{v, v\}$ or $e = (v, v)$ is called a *loop* (*Schleife*). Two edges $e_1, e_2 \in E$ with $e_1 = \{u, v\}$ and $e_2 = \{u, v\}$ (or $e_1 = (u, v)$ and $e_2 = (u, v)$) are called *parallel*.
- (e) Two directed edges $e_1, e_2 \in E$ with $e_1 = (u, v)$ and $e_2 = (v, u)$ are called *inverted* (*invertiert*).
- (f) Two nodes $u, v \in V$ are called *adjacent* (*adjazent*, *benachbart*), if there exists an edge $e \in E$ connecting the two nodes. An edge $e \in E$ is called *incident* (*inzident*) on a node $v \in V$, if v is one of the endnodes of e .

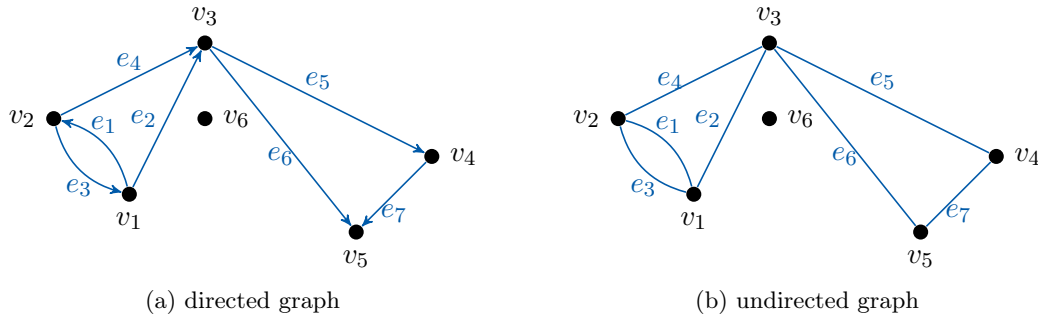


Figure 5.1: Illustration of graphs

If we allow parallel edges, we cannot identify edges purely by stating their endnodes. For this reason we often consider only graphs without parallel edges. However, keep in mind that in directed graphs inverted edges are not considered as parallel, i.e. those we typically allow. So in Figure 5.1 the directed graph does not contain parallels, whereas the undirected one does.

Loops can also cause problems in some algorithms. So we often restrict ourselves to graphs without loops.

At the moment, we describe graphs by a set of nodes V and a set of edges E . The following representations based on matrices are often more compact and easier to handle.

Definition 5.2.

- (a) For a directed graph $G = (V, E)$ without parallel edges and nodes $V = \{v_1, \dots, v_n\}$ the matrix $A \in \mathbb{R}^{n \times n}$ with entries

$$a_{ij} = \begin{cases} 1 & \text{if } e = (v_i, v_j) \in E, \\ 0 & \text{else} \end{cases}$$

is called *adjacency matrix* (*Adjazenzmatrix*) of G .

- (b) For an undirected graph $G = (V, E)$ without parallel edges the adjacency matrix is defined analogously.

For an undirected graph, the adjacency matrix is always symmetric, so it suffices to save one half of it. For a directed graph the adjacency matrix does not have to be symmetric. Loops correspond to entries $a_{ii} = 1$. For graphs with parallel edges, we could define a_{ij} as the number of edges from v_i to v_j .

The adjacency matrices of the graphs in Figure 5.1 are

$$A_{\text{directed}} = \begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix} \quad \text{and} \quad A_{\text{undirected}} = \begin{pmatrix} 0 & 2 & 1 & 0 & 0 & 0 \\ 2 & 0 & 1 & 0 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 1 & 0 \\ 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

Definition 5.3.

- (a) For a directed graph $G = (V, E)$ without loops and nodes $V = \{v_1, \dots, v_n\}$ and edges $E = \{e_1, \dots, e_m\}$ the matrix $I \in \mathbb{R}^{n \times m}$ with entries

$$I_{ij} = \begin{cases} 1 & \text{if } v_i \text{ is the tail of } e_j, \\ -1 & \text{if } v_i \text{ is the head of } e_j \\ 0 & \text{else} \end{cases}$$

is called *incidence matrix* (*Inzidenzmatrix*) of G .

- (b) For an undirected graph $G = (V, E)$ without loops the incidence matrix is defined analogously, but both endnodes of an edge have +1 as entry.

Every column of the incidence matrix corresponds to one edge. In a directed graph without loops, every column has exactly one entry $+1$ and one entry -1 and in an undirected graph without loops every column has exactly two entries $+1$. Parallel edges can be represented by inserting a column for each parallel edge. But loops are problematic to represent in incidence matrices. In the literature, the signs in the incidence matrix of directed graphs are sometimes reversed.

The incidence matrices of the graphs in Figure 5.1 are

$$I_{\text{directed}} = \begin{pmatrix} 1 & 1 & -1 & 0 & 0 & 0 & 0 \\ -1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & -1 & 0 & -1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & -1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & -1 & -1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix},$$

$$I_{\text{undirected}} = \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}.$$

In applications of graphs, we are often looking for paths between two nodes along edges within the graph. In this context, the following concepts are important.

Definition 5.4. Let $G = (V, E)$ be an undirected graph.

(a) Then a finite sequence

$$P = (v_1, e_1, v_2, e_2, \dots, v_{k-1}, e_{k-1}, v_k)$$

of nodes $v_i \in V$ and edges $e_i \in E$ with $k \geq 2$ is called a *path* (*Pfad*), if $e_i = \{v_i, v_{i+1}\}$ for all $i = 1, \dots, k-1$. If $v_1 = v_k$, the path is called a *cycle* (*Kreis*). If every node and every edge appears at most once in P (with $v_1 = v_k$ being the only possible exception), the path is called *simple* (*einfach*).

(b) An undirected graph, that does not contain any simple cycles is called *acyclic* (*kreisfrei*). An undirected graph, where every pair of nodes $u \neq v$ can be connected via a path, is called *connected* (*zusammenhängend*).

Note that a path has to consist of at least two nodes and one edge, i.e. a single node is not considered a path. What we call a path is sometimes referred to as a walk in

the literature. In this terminology, only simple paths would be called paths and one can insert trails between walks and paths, where crossing a node more than once is okay, but edges can be used at most once.

If the graph G does not contain parallel edges, it suffices to provide the sequence of nodes $(v_1, v_1, \dots, v_k)$, because the connecting edges are uniquely defined by their endnodes. Providing only the sequence of edges $(e_1, \dots, e_{k-1})$ is theoretically also possible. However, since undirected edges can be traversed in both directions, this can lead to ambiguity e.g. in case $(\{u, v\}, \{u, v\})$.

For directed graphs, the terminology is a bit more expansive, because we need to consider both directed and undirected paths.

Definition 5.5. Let $G = (V, E)$ be a directed graph.

(a) Then a finite sequence

$$P = (v_1, e_1, v_2, e_2, \dots, v_{k-1}, e_{k-1}, v_k)$$

of nodes $v_i \in V$ and edges $e_i \in E$ with $k \geq 2$ is called an *(undirected) path (Pfad)*, if $e_i = (v_i, v_{i+1})$ or $e_i = (v_{i+1}, v_i) \in E$ for all $i = 1, \dots, k-1$. If $v_1 = v_k$, the path is called a *cycle (Kreis)*. If every node and every edge appears at most once in P (with $v_1 = v_k$ being the only possible exception), the path is called *simple (einfach)*.

(b) A sequence

$$P = (v_1, e_1, v_2, e_2, \dots, v_{k-1}, e_{k-1}, v_k)$$

of nodes $v_i \in V$ and edges $e_i \in E$ with $k \geq 2$ is called a *directed path (gerichteter Pfad)*, if $e_i = (v_i, v_{i+1})$ for all $i = 1, \dots, k-1$.

(c) A directed graph, that does not contain any simple cycles, is called *acyclic (kreisfrei)*. If it does not contain any directed simple cycles, it is called *directed acyclic (gerichtet kreisfrei)*.

(d) A directed graph, where every pair of nodes $u \neq v$ can be connected via a path, is called *weakly connected (schwach zusammenhängend)*. If every pair of nodes can be connected via a directed path, is called *strongly connected (stark zusammenhängend)*.

The terminology used to describe connectivity of directed graphs does vary a bit in literature. In our terminology, a directed graph is weakly connected, if the corresponding undirected graph is connected.

So far, we allow for arbitrary paths in the definition of connectivity, not necessarily simple paths. However, as the following result shows, whenever two different nodes are connected via a path, they can also be connected via a simple path.

Lemma 5.6. A graph $G = (V, E)$ contains a path between two nodes $u \neq v$ if and only if it contains a simple path between the two nodes.

We do not state a formal proof here, but instead describe the idea: Start at the initial point u and follow the path. If you return to a node $\tilde{v}$, where you have been before, stop and cut out the part of the path between the last time the node $\tilde{v}$ occurred and now. The part of the path, that has been cut out, is a cycle. Then continue along the path and keep cutting out cycles until you have reached the end. What is left of the path after this procedure is a simple path between the nodes u and v .

The reason why we do not consider cycles in the previous result, i.e. why we demand $u \neq v$, are cycles of the form $P = (u, \{u, v\}, v, \{u, v\}, u)$, where we use the same edge $\{u, v\}$ twice. This cycle is not a simple cycle and cannot be reduced to a simple cycle. Fortunately, cycles of this form are rather degenerate and typically do not pose a problem (except for proper terminology). And if a graph contains a cycle consisting of at least three different nodes, it is again possible to reduce this cycle to a simple cycle.

5.1.1 Trees and Spanning Trees

For some applications, we need graphs to be connected, for some we need them to be acyclic. Graphs, that have both properties, i.e. are connected and acyclic, are called trees.

Definition 5.7. An undirected graph is called a *tree*, if it is acyclic and connected.

If we consider a connected but not acyclic graph, we may be able to find a subgraph, that is a tree.

Definition 5.8. Let $G = (V, E)$ be an undirected graph.

- (a) A graph $\tilde{G} = (\tilde{V}, \tilde{E})$ with $\tilde{V} \subseteq V$ and $\tilde{E} \subseteq E$ is called a *subgraph* (*Teilgraph*) of the graph G .
- (b) A subgraph $\tilde{G} = (\tilde{V}, \tilde{E})$ of G with $\tilde{V} = V$ is called a *spanning subgraph* (*erzeugender Teilgraph*).
- (c) A spanning subgraph of G is called a *spanning tree* (*spannender Baum*) of G , if it is a tree.

A tree with n nodes has to have at least $n - 1$ edges to be connected. In fact, we can show that it has exactly $n - 1$ edges and also provide some other useful characterizations of trees.

Theorem 5.9. Let $G = (V, E)$ be an undirected graph with $|V| = n$ nodes. Then the following are equivalent:

- (a) G is a tree, i.e. acyclic and connected.
- (b) G is acyclic and adding a new edge generates a unique simple cycle.
- (c) Every two points $u \neq v$ in G are connected via a unique simple path.
- (d) G is connected and eliminating an existing edge causes it to become disconnected.
- (e) G is connected and has $n - 1$ edges.
- (f) G is acyclic and has $n - 1$ edges.

So far, we only considered trees in undirected graphs. The reason is that both being acyclic and being connected is based on paths. And in directed graphs we have to differentiate between undirected and directed paths. If we tried to define trees in directed graphs as strongly connected and (directed) acyclic, we would run into a problem, because a strongly connected graph with more than one node automatically contains directed (and thus also undirected) cycles. For this reason, trees in directed graphs are defined as follows:

Definition 5.10. Let $G = (V, E)$ be a directed graph.

- (a) G is called a *tree*, if it is acyclic and weakly connected, i.e. if the corresponding undirected graph is a tree.
- (b) A node $r \in V$ is called a *root* (*Wurzel*), if G contains a directed path from r to every other node $v \in V \setminus \{r\}$.
- (b) G is called a *rooted tree* (*Wurzelbaum*), if it is a tree and contains a root r .

Note that a rooted tree contains exactly one root r . If it contained two different roots $r_1 \neq r_2$, then it would also contain a directed cycle through these two roots, a contradiction to G being acyclic.

5.1.2 Breadth-First Search and Depth-First-Search

Now consider an undirected connected graph $G = (V, E)$ and an arbitrary node $r \in V$ or a directed graph $G = (V, E)$ with root r . Then we know that there exists a path from r to every other node in the graph. A common task is to actually find such a path from the start node r to every other node, sometimes a path with as few edges as possible. We will later generalize this in weighted graphs, where each edge is associated with a weight, e.g. a distance or costs. But for now let us call the number of edges in a path its *length* (*Länge*).

For all nodes connected directly to r we have found a path. All the neighbors of nodes connected directly to r can be reached via these nodes. But maybe we have already reached some of them directly. So we have to keep track of all nodes we have already reached and the path there. In fact, it suffices to remember only the previous node on that path, because we can then reconstruct the path going backwards from a node to the start node r .

If we want to implement such a path-finding method, we have to decide in which order we want have a look at the neighbors of newly reached nodes. For this reason, we work with a *queue* (*Warteschlange*), which is an ordered list of objects, in our case of nodes. Using the well-known FIFO (first in – first out) or LIFO (last in – first out) strategies then results in either the so-called *breadth-first search* (*Breitensuche*, *BFS*) or the *depth-first search* (*Tiefensuche*, *DFS*).

Algorithm 5.11. (Breadth-First Search)

Input: A connected undirected graph $G = (V, E)$ and start node r or a directed graph $G = (V, E)$ with a root r .

- 1 Initialize the queue L as $L = (r)$, the distances $d(r) = 0$, $d(v) = \infty$ for all $v \in V \setminus \{r\}$ and the predecessors as $\pi(v) = 0$ for all $v \in V$.
- 2 **while** $L \neq \emptyset$ **do**
- 3 Take the first element v out of the queue L
- 4 **for all** $u \in V$ with $\{v, u\} \in E$ or $(v, u) \in E$ **do**
- 5 **if** $d(u) = \infty$ **then**
- 6 Set $d(u) = d(v) + 1$ and $\pi(u) = v$.
- 7 Attach u to the end of the queue L .
- 8 **end if**
- 9 **end for**
- 10 **end while**

Output: The predecessors $\pi(v)$ and the distances $d(v)$ for all $v \in V$.

Note that in breadth-first search the first batch of nodes entered into the queue L all have distance $d(u) = 1$. Since we always take out the first node from the queue L , the

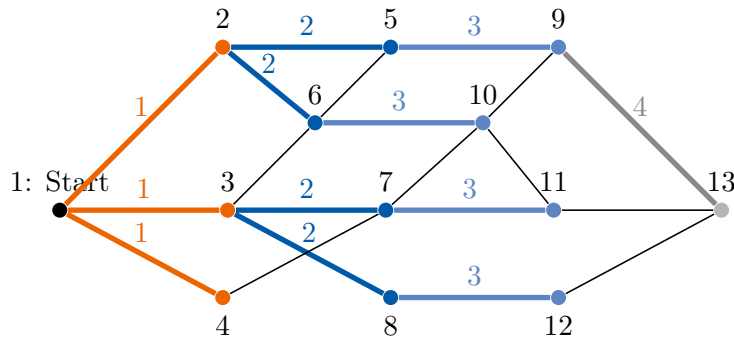


Figure 5.2: Breadth-first search starting in node 1

next nodes we enter all have $d(u) = 2$. Once all nodes with $d(v) = 1$ have been taken out of the queue, we start taking out the nodes with $d(v) = 2$ and begin entering nodes with $d(u) = 3$. Thus, nodes with smaller distances $d(v)$ enter and leave the queue before nodes with larger distances.

Theorem 5.12. Let $G = (V, E)$ be a connected undirected graph with arbitrary start node r or a directed graph with root r . Then the breadth-first search computes paths to all other nodes, in fact shortest paths.

The predecessors $\pi(v)$ allow us to define the graph $G_\pi = (V, E_\pi)$ with

$$E_\pi = \{(\pi(v), v) \mid v \in V \setminus \{r\}\},$$

which is a spanning tree of G (with root r) and contains shortest paths from r to all other nodes v .

If BFS is applied to a graph, where not all nodes can be reached from r via a (directed) path, then it generates shortest paths to all reachable nodes. Thus, if at the end of BFS there remain nodes with $d(v) = \infty$, those were not reachable.

The number of iterations in BFS is $|V|$ and in each iteration we have to take the current node v out of the queue and consider all adjacent edges. In undirected graphs, we consider all edges $e \in E$ twice over the course of the algorithm, in directed graphs only once. This results in a total complexity of $O(|V| + |E|)$.

If we are only interested in paths from r to all other nodes, but not necessarily shortest paths, we can also use depth-first search. This is conceptually the same as the right-hand rule (or left-hand rule) one can use to find a path through a maze (Irrgarten). But again, while this method does construct paths from r to all other nodes, it does usually not compute shortest paths.

graph $G_\pi = (V, E_\pi)$ with

$$E_\pi = \{(\pi(v), v) \mid v \in V \setminus \{r\}\},$$

which is a spanning tree of G (with root r).

The complexity of DFS is the same as of BFS, i.e. $O(|V| + |E|)$, because we only changed the order, in which we take the nodes out of the queue.

The central object in both BFS and DFS is the *queue* (*Warteschlange*) L , in which we collect the newly discovered nodes. The difference between the two algorithms is the *service discipline* (*Warteschlangendisziplin*). In BFS, we have a FIFO queue and in DFS we have a LIFO queue.

To implement these two types of queues, one uses two different data structures: The data structure used to represent a FIFO queue is also called a *queue* and consists of a sorted list and three operations “determine, if the list is empty”, “attach a new element to the end of the list” and “take the first element out of the list”. The data structure used to represent a LIFO queue is called a *stack* and consists of a sorted list and three operations “determine, if the list is empty”, “attach a new element to the end of the list” and “take the last element out of the list”.

5.2 Shortest Paths

Recall that breadth-first search did not only find paths from one start node to all other nodes in a (weakly) connected graph, but that it actually found shortest paths in terms of the number of edges on the path.

In many applications, the edges in graphs are associated with a weight, that can represent costs, distances or capacities. These weights can of course be different for different edges.

Definition 5.15. Let $G = (V, E)$ be a directed or undirected graph. G is called a *weighted* (*gewichteter*) graph, if each edge $e \in E$ is associated with a *weight* (*Gewicht*) $w(e) \in \mathbb{R}$.

One can also attach weights to nodes, but we only consider weighted edges here. Such edge weights can be incorporated in the incidence matrix of G by saving $+w(e)$ and $-w(e)$ instead of $+1$ and -1 . For simple graphs, one can also save the weights in the adjacency matrix.

If these weights represent e.g. transport costs or travel time, it makes sense to ask for shortest paths, where the length of a path P is now defined as follows.

Definition 5.16. Let $G = (V, E)$ be a weighted (directed) graph with edge weights $w(e)$ for all edges $e \in E$.

- (a) The *length* (*Länge*) of a path P in G is then defined as

$$w(P) = \sum_{e \in P} w(e).$$

- (b) A (directed) path P between two nodes $u, v \in V$ is called a *shortest path* (*kürzester Weg*) between those two nodes, if its length is minimal among all (directed) paths between those two nodes.
- (c) If a shortest path P exists between two nodes u, v , we define the *distance* (*Distanz*) of these two nodes as $\text{dist}(u, v) := w(P)$. If no path exists between u and v , we set $\text{dist}(u, v) := +\infty$. If paths exist between u and v but no shortest path, we set $\text{dist}(u, v) := -\infty$.

In case all weights $w(e)$ are equal, we can compute such shortest paths with BFS. But if the weights have different values (which is typically the case), a path with more edges can be shorter than a path with fewer edges.

A complication to keep in mind here: If all weights $w(e)$ are nonnegative, then shortest paths exist between all points, which are connected via some (directed) path. But if we allow negative weights, then (directed) cycles with negative length are possible. In this case, no shortest paths exist for all points that can be connected via a path including such a cycle with negative length, because every additional tour through that cycle decreases the length of the path.

A property of paths, that we will use frequently, is the following: If P is a shortest path between two nodes, then for all pairs of nodes u, v on that path P the respective part of P is a shortest path between u and v . Otherwise, we could switch out this part of P for some shorter path between u and v and in doing so decrease the length of P .

To simplify the subsequent discussion, from now on we only consider directed graphs and assume that they are simple, i.e. do not contain loops or parallel edges. Loops with nonnegative weights would not be used on a shortest path and loops with negative edge weights only cause problems. In case there exist parallel edges between two nodes, we only use the one with the lowest weight on shortest paths and can eliminate the others. But recall that inverted edges are not parallel.

5.2.1 Repetition: Dijkstra's Algorithm

In Dijkstra's algorithm, see Algorithm 5.17, we use an idea analogous to BFS, but with a modified queue S . At the beginning, this queue S contains all nodes and we always choose the one with the lowest value $d(v)$ next. In case all weights $w(e)$ are the same, this degenerates to BFS.

Algorithm 5.17. (Dijkstra's Algorithm)

Input: A directed simple graph $G = (V, E)$ with root r , $n = |V|$ nodes and nonnegative edge weights $w(e) \geq 0$ for all $e \in E$.

- 1 Initialize the queue as $S = V$, the distances as $d(r) = 0$, $d(v) = \infty$ for all $v \in V \setminus \{r\}$ and the predecessors as $\pi(v) = 0$ for all $v \in V$.
- 2 **for** $k = 1$ **to** n **do**
- 3 Remove an element v with $d(v) = \min\{d(u) \mid u \in S\}$ from the queue S .
- 4 **for all** $u \in S$ with $e = (v, u) \in E$ **do**
- 5 **if** $d(u) > d(v) + w(e)$ **then**
- 6 Update $d(u) = d(v) + w(e)$ and $\pi(u) = v$.
- 7 **end if**
- 8 **end for**
- 9 **end for**

Output: The predecessors $\pi(v)$ and distances $d(v)$ for all $v \in V$.

Note that in the description of the algorithm, we now denote the queue as S and not L anymore. The reason is that the queue can now be an unsorted set, out of which we always pick the node with the smallest value $d(v)$. In contrast, in BFS and DFS, we needed the queue to be a sorted list, because we chose the next node based on the order in which the nodes were added to the list.

Figure 5.4 and Table 5.1 illustrate the behavior of Dijkstra's Algorithm starting in the root 1.

k	v	S	d	π
0	-	$\{1, 2, 3, 4, 5\}$	$(0, \infty, \infty, \infty, \infty)$	$(0, 0, 0, 0, 0)$
1	1	$\{2, 3, 4, 5\}$	$(0, 1, 3, 2, \infty)$	$(0, 1, 1, 1, 0)$
2	2	$\{3, 4, 5\}$	$(0, 1, 2, 2, 4)$	$(0, 1, 2, 1, 2)$
3	4	$\{3, 5\}$	$(0, 1, 2, 2, 3)$	$(0, 1, 2, 1, 4)$
4	3	$\{5\}$	$(0, 1, 2, 2, 3)$	$(0, 1, 2, 1, 4)$
5	5	$\emptyset$	$(0, 1, 2, 2, 3)$	$(0, 1, 2, 1, 4)$

Table 5.1: Illustration of Dijkstra's Algorithm

Some remarks on Dijkstra's algorithm:

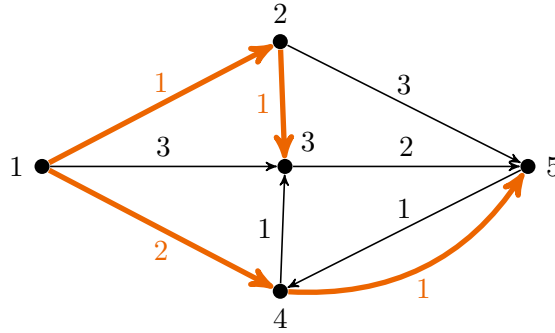


Figure 5.4: Illustration of Dijkstra's Algorithm

- Dijkstra's algorithm correctly determines shortest paths from the initial node r to all other nodes, as long as all edge weights are nonnegative, see Theorem 5.18.

If we allow for negative edge weights, Dijkstra's algorithm can produce wrong results even if no cycles with negative length exists.

- If only a shortest path from r to some other node v^* needs to be computed, we can terminate Dijkstra's algorithm once v^* has been eliminated from the queue S .

By starting Dijkstra's algorithm in all possible initial nodes r , it is also possible to compute shortest paths from all startnodes r to all endnodes v , which can be saved in a distance matrix $D \in \mathbb{R}^{|V| \times |V|}$ and a predecessor matrix $P \in \mathbb{R}^{|V| \times |V|}$. However, this distance matrix is typically dense and thus can only be computed and saved for small problem instances.

- In Dijkstra's algorithm, we have to execute $|V|$ iterations and consider all nodes still contained in the queue S , for which $|V|$ is also an upper bound. This results in a complexity of $O(|V|^2)$ operations. Using efficient data structures, called *heaps*, it is possible to improve this complexity. Details can be found in books on graph theory, e.g. [2].

For computing the full distance matrix, we have to run Dijkstra's Algorithm $|V|$ -times, which results in a complexity of $O(|V|^3)$ operations.

Theorem 5.18. Let $G = (V, E)$ be a simple directed graph with root r , $n = |V|$ nodes and nonnegative edge weights $w(e) \geq 0$ for all $e \in E$. Then Algorithm 5.17 is well defined and at the end we have $d(v) = \text{dist}(r, v)$ for all nodes $v \in V$.

If we allow negative edge weights, Dijkstra's algorithm does not work correctly anymore, see Example 5.19. But there exist applications, where negative edge weights occur naturally.

Example 5.19. Consider the graph in Figure 5.5, where Dijkstra's algorithm with the root $r = 1$ computes the wrong values $d = (0, 2, 1)$. The correct distances are $d = (0, 2, 0)$. The reason we get the wrong distances here is that we remove the nodes from S in the order 1, 3, 2. And since we removed 3 before 2, we do not update $d(3)$ anymore, when 2 is removed from S .

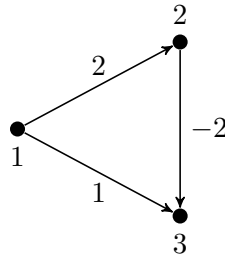


Figure 5.5: Example for Algorithm 5.17 failing due to negative edge weights

5.2.2 Repetition: Algorithm of Bellman and Ford

In contrast to Dijkstra's algorithm, the algorithm of Bellman-Ford, see Algorithm 5.20, can not only handle negative edge weights correctly, but also detects automatically, if a directed cycle with negative length exists.

Algorithm 5.20. (Algorithm of Bellman-Ford)

Input: A directed simple graph $G = (V, E)$ with root r , $n = |V|$ nodes and edge weights $w(e)$ for all $e \in E$.

- 1 Initialize the distances as $d(r) = 0$, $d(v) = \infty$ for all $v \in V \setminus \{r\}$ and the predecessors as $\pi(v) = 0$ for all $v \in V$.
- 2 **for** $k = 1$ **to** $n - 1$ **do**
- 3 **for all** $e = (u, v) \in E$ **do**
- 4 **if** $d(v) > d(u) + w(e)$ **then**
- 5 Update $d(v) = d(u) + w(e)$ and $\pi(v) = u$.
- 6 **end if**
- 7 **end for**
- 8 **end for**
- 9 **for all** $e = (u, v) \in E$ **do**
- 10 **if** $d(v) > d(u) + w(e)$ **then**
- 11 **print** ERROR: There exists a directed cycle with negative length.
- 12 **end if**
- 13 **end for**

Output: The predecessors $\pi(v)$ and distances $d(v)$ for all $v \in V$.

So contrary to Dijkstra's algorithm, we now do not have any kind of queue anymore, but instead cycle through *all edges* $(n - 1)$ -times and update as before. Formally, we would also have to define in which order we consider the edges within each of the $n - 1$ iterations, but this is not important for the theoretical properties of the method. If after these $n - 1$ iterations there still exists an edge, that would improve a distance, this has to be caused by a cycle with negative length.

Theorem 5.21. Let $G = (V, E)$ be a simple directed graph with root r , $n = |V|$ nodes and edge weights w_e for all $e \in E$.

- (a) Then Algorithm 5.20 is well defined.
- (b) If G does not contain a directed cycle with negative length, the final distances satisfy $d(v) = \text{dist}(r, v)$ for all $v \in V$.
- (c) If G contains a directed cycle with negative length, the algorithm identifies that correctly.

Note that this algorithm needs to be run completely and only at the end of the algorithm the final distances are correct (if no directed cycle with negative length). This is an important difference to the Dijkstra's Algorithm, where the distance to a node was correct once the node had been removed from the queue. For this reason, the Algorithm of Bellman and Ford is sometimes called a *label correcting algorithm*, whereas Dijkstra's Algorithm is called a *label setting algorithm*.

This algorithm considers each edge n times and thus has a complexity of $O(|V| \cdot |E|)$ operations. If it is run once for all startnodes r , it can be used to compute shortest paths between all pairs of nodes and has a complexity of $O(|V|^2 \cdot |E|)$.

Let us apply this algorithm to the counterexample for Dijkstra's algorithm, see Figure 5.6 and Table 5.2. Since the example has 3 nodes, we have to cycle through all edges twice. Here, we used the order e_1, e_2, e_3 for both iterations. We see that the algorithm of Bellman and Ford determines the correct distances.

Now let us modify this example such that it contains a cycle with negative length, see Figure 5.7 and Table 5.3. If we use the same order e_1, e_2, e_3 and cycle through all edges twice, then after the end of the second iteration the distances are still not optimal and thus the algorithm of Bellman and Ford detects the presence of a cycle with negative length.

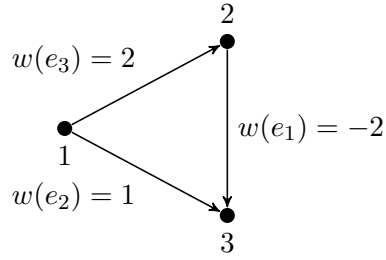


Figure 5.6: Illustration of Algorithm 5.20 with root 1 and negative edge weights

k	d	π
0	$(0, \infty, \infty)$	$(0, 0, 0)$
1	$(0, 2, 1)$	$(0, 1, 1)$
2	$(0, 2, 0)$	$(0, 1, 2)$

Table 5.2: Illustration of Algorithm 5.20 with root 1 and negative edge weights

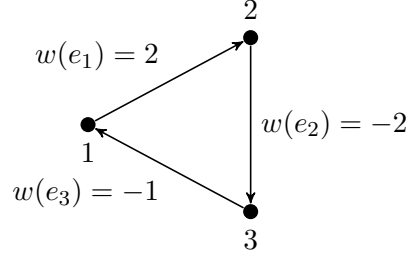


Figure 5.7: Illustration of Algorithm 5.20 with root 1 and a cycle with negative length

k	d	π
0	$(0, \infty, \infty)$	$(0, 0, 0)$
1	$(-1, 2, 0)$	$(3, 1, 2)$
2	$(-2, 1, -1)$	$(3, 1, 2)$

Table 5.3: Illustration of Algorithm 5.20 with root 1 and a cycle with negative length

5.2.3 Algorithm of Floyd and Warshall

Dijkstra's algorithm fails in the presence of negative edge weights. The algorithm of Bellman and Ford is an improvement over Dijkstra's algorithm in the sense that it can handle negative edge weights and that it detects the existence of cycle with negative length.

Let us now change our objective slightly: Instead of searching for a shortest path from u to v let us now search for a shortest *simple* path from u to v . Since we only consider finite graphs G , there exist only finitely many simple paths from u to v and thus also a shortest simple path. Thus, shortest simple paths exists even if the graph G contains directed cycles with negative length. And if G does not contain directed cycles with negative length, then all shortest paths can be chosen as simple paths (because cycles have nonnegative length and thus cannot be used to decrease the length of a path).

Since we only consider simple graphs $G = (V, E)$ with nodes $V = (v_1, \dots, v_n)$, we can save the edge weights in a weight matrix $W \in (\mathbb{R} \cup \{+\infty\})^{|V| \times |V|}$ with entries

$$w_{ij} = \begin{cases} w(e) & \text{if } e = (v_i, v_j) \in E, \\ +\infty & \text{else.} \end{cases}$$

This weight matrix is basically the weighted version of the adjacency matrix, but we use $+\infty$ instead of 0 for nonexistent edges.

Algorithm 5.22. (Algorithm of Floyd and Warshall)

Input: A directed simple graph $G = (V, E)$ with $n = |V|$ nodes and weight matrix W for all $e \in E$.

```

1 Initialize the distances as  $d_{ij} = w_{ij}$  and the predecessors as  $p_{ij} = i$  for all pairs of
  nodes  $v_i, v_j \in V$ .
2 for  $k = 1, \dots, n$  do
3   for all  $i = 1, \dots, n$  do
4     for all  $j = 1, \dots, n$  do
5       if  $d_{ij} > d_{ik} + d_{kj}$  then
6         Update  $d_{ij} = d_{ik} + d_{kj}$  and  $p_{ij} = p_{kj}$ .
7         if  $i = j$  and  $d_{ii} < 0$  then
8           print ERROR:  $G$  contains a directed cycle with negative
             length.
9         end if
10      end if
11    end for
12  end for
13 end for

```

Output: The predecessor matrix P and distance matrix D or the information that a cycle with negative length exists.

The algorithm of Floyd and Warshall, see Algorithm 5.22, tries to compute shortest simple paths between all pairs of nodes v_i and v_j . The corresponding information can be saved in a distance matrix $D \in (\mathbb{R} \cup \{+\infty\})^{|V| \times |V|}$ and a predecessor matrix $P \in \mathbb{R}^{|V| \times |V|}$ with the entries

$$\begin{aligned} d_{ij} &= \text{length of the shortest path from } v_i \text{ to } v_j, \\ p_{ij} &= \text{predecessor of node } v_j \text{ on a shortest path from } v_i \text{ to } v_j. \end{aligned}$$

For comparison, in Dijkstra's algorithm and in the algorithm of Bellman and Ford, we considered only one fixed initial point r and thus could save the information in two vectors d and π . These vectors are exactly the rows of D and P corresponding to the node r .

In the algorithm of Floyd and Warshall we initialize D and P by using the direct connection from v_i to v_j . We can do this, because in the weight matrix we basically added all missing edges with infinite edge weights. Note that we only consider simple graphs and thus have $d_{ii} = w_{ii} = +\infty$ for all nodes $v_i \in V$.

Then we iterate over all nodes v_k once and check for all pairs of nodes v_i, v_j , if the current path from v_i to v_j is shorter than taking the current shortest path from v_i to v_k and then from there the current shortest path from v_k to v_j . If yes, then we have found a shorter path from v_i to v_j and update the distance d_{ij} and the predecessor p_{ij} accordingly. This is called a *triple operation* and is the reason why the algorithm of Floyd and Warshall is sometimes also called the *triple algorithm*.

If at some point we observe $d_{ii} < 0$, then there exists a directed path from v_i to v_i with negative length, i.e. a directed cycle with negative length. In this case, we terminate the algorithm immediately. We could also wait and perform this test only once at the end of the algorithm.

In order to prove correctness of this algorithm, we need to have a closer look at simple cycles in directed graphs first. Recall that a path in a directed graph is a cycle, if its startnode v_1 and endnode v_k coincide. If no edge and no node on that path is used more than once (except for $v_1 = v_k$), then the cycle is called a simple cycle. If we restrict our attention to directed simple cycles, it suffices to demand that no node (except for $v_1 = v_k$) is used more than once on the path, because reusing an edge would also require reusing both of its endnodes.

Lemma 5.23. Let $G = (V, E)$ be a simple directed graph with weight matrix $W \in (\mathbb{R} \cup \{+\infty\})^{|V| \times |V|}$. If G contains a directed cycle with negative length, it also contains a simple directed cycle with negative length.

Proof. Let C be a directed cycle with negative length and out of all directed cycles with negative length one with minimal number of edges. If C is not a directed simple cycle, it contains a directed simple cycle $\tilde{C}$. Either this directed cycle $\tilde{C}$ also has negative length, in which its existence contradicts the choice of C with minimal number of edges. Or $\tilde{C}$ has positive length, in which case we can eliminate it from C to again construct a directed cycle with negative length and fewer edges than C . $\square$

Another observation, which we use frequently, is that in directed graphs, which do not contain directed cycles with negative length, we can always choose shortest directed paths as simple paths. Furthermore, subsets of such shortest paths are again shortest paths connecting their endnodes.

With this preparation we can now prove that the algorithm of Floyd and Warshall works correctly.

Theorem 5.24. Let $G = (V, E)$ be a simple directed graph with weight matrix $W \in (\mathbb{R} \cup \{+\infty\})^{|V| \times |V|}$.

- (a) Then Algorithm 5.22 is well defined.
- (b) If in iteration k we have $d_{ii} \geq 0$ for all nodes $v_i \in V$, then the distances d_{ij} correspond to a shortest simple path from v_i to v_j using only intermediary nodes in $v_1, \dots, v_k$.
- (c) G contains a directed cycle with negative length if and only if there exists a node v_i with $d_{ii} < 0$ in some iteration k .
- (d) If G does not contain a directed cycle with negative length, then the algorithm correctly computes the distance matrix D and predecessor matrix P .

Proof. To simplify this proof a little, we assume that, instead of incrementally updating the distances over the course of one iteration, we use the distances from the start of the iteration to update all distances at the same time at the end of the iteration. One can show that this does not make a difference in the end.

- (a) If we use the typical conventions for computations with ∞ , then the algorithm is well defined and terminates after executing all three nested loops.

- (b) For all $k = 0, 1, \dots, n$ let $D^{*,k} \in (\mathbb{R} \cup \{+\infty\})^{|V| \times |V|}$ denote the lengths of shortest directed simple paths using only intermediary nodes in $v_1, \dots, v_k$. In case $k = 0$ this means that only direct connections are allowed. Let us now prove the claim with induction over k .

For $k = 0$ we consider only direct connections from v_i to v_j , which is exactly how we initialized the matrices D and P , i.e. we have $D^{*,0} = D$.

Let the claim now be true for iterations $0, 1, \dots, k-1$ and consider iteration k . Then by construction and the induction assumption at the end of this iteration all d_{ij} correspond to the length of a directed path from v_i to v_j using only intermediary nodes in $v_1, \dots, v_k$, which is described by the predecessor nodes. Now consider an arbitrary pair of nodes v_i, v_j with $d_{ij}^{*,k} < d_{ij}$ and let P_{ij}^k be a corresponding directed shortest simple path from v_i to v_j using only intermediary nodes in $v_1, \dots, v_k$ with length $w(P_{ij}^k) = d_{ij}^{*,k}$.

Case 1: “ v_k is used in P_{ij}^k ” Then we can split the simple path P_{ij}^k into the two simple paths $P_{ik} = (v_1, \dots, v_k)$ and $P_{kj} = (v_k, \dots, v_j)$. Since P_{ij}^k is a simple path using only intermediary nodes in $v_1, \dots, v_k$, we know that P_{ik} and P_{kj} use only intermediary nodes in $v_1, \dots, v_{k-1}$. And since P_{ij}^k is a shortest path from v_i to v_j using only the intermediary nodes $v_1, \dots, v_k$, we know that P_{ik} and P_{kj} are shortest paths using only intermediary nodes in $v_1, \dots, v_{k-1}$. It follows using the induction assumption that

$$w(P_{ij}^k) = w(P_{ik}) + w(P_{kj}) = d_{ik}^{*,k-1} + d_{kj}^{*,k-1} = d_{ik} + d_{kj}.$$

Using this together with

$$w(P_{ij}^k) = d_{ij}^{*,k} \leq d_{ij}^{*,k-1} = d_{ij}$$

shows

$$d_{ij} \geq d_{ik} + d_{kj}.$$

Thus, in the k -th iteration we update d_{ij} to $d_{ik} + d_{kj} = d_{ij}^{*,k}$.

Case 2: “ v_k is not used in P_{ij}^k ” Let P_{ik} and P_{kj} be shortest simple paths from v_i to v_k and from v_k to v_j using only intermediary nodes in $v_1, \dots, v_{k-1}$ and consider the combined path

$$\tilde{P}_{ij} = P_{ik}^{k-1} + P_{kj}^{k-1} = (v_i, \dots, v_k, \dots, v_j).$$

Then $\tilde{P}_{ij}$ is path from v_i to v_j using only intermediary nodes in $v_1, \dots, v_k$, but not necessarily a simple path. By the induction assumption we have

$$w(\tilde{P}_{ij}) = w(P_{ik}) + w(P_{kj}) = d_{ik}^{*,k-1} + d_{kj}^{*,k-1} = d_{ik} + d_{kj}.$$

In case this combined path $\tilde{P}_{ij}$ does not contain a cycle with negative length, we know $w(\tilde{P}_{ij}) \geq w(P_{ij}^k) = d_{ij}^{*,k}$ and thus we do not update d_{ij} in the k -th iteration but instead leave it at the correct value $d_{ij} = d_{ij}^{*,k} = d_{ij}^{*,k-1}$.

In case $\tilde{P}_{ij}$ contains a cycle with negative length, it also contains a simple cycle C with negative length. Since both P_{ik} and P_{kj} are simple paths, this simple cycle C has to contain the node v_k . And since both P_{ik} and P_{kj} contain only intermediary nodes in $v_1, \dots, v_{k-1}$, the simple cycle $C = (v_k, \dots, v_k)$ either also only contains intermediary nodes in $v_1, \dots, v_{k-1}$, or we have $v_i = v_j$ and can rewrite the cycle as $C = (v_i = v_j, \dots, v_k, \dots, v_i = v_j)$.

In the first case, we use that $C = (v_k, \dots, v_k)$ is a simple cycle with negative length containing only intermediary nodes in $v_1, \dots, v_{k-1}$ together the induction assumption and obtain that at the end of the k -th iteration we have

$$d_{kk} \leftarrow \min\{d_{kk}, d_{kk} + d_{kk}\} \leq d_{kk} = d_{kk}^{*,k-1} \leq w(C) < 0.$$

But this contradicts the requirement that all d_{ii} remain nonnegative.

In the second case, we can use that $C = (v_i = v_j, \dots, v_k, \dots, v_i = v_j)$ consist of two simple paths from v_i to v_k and v_k to v_i , each using only intermediary nodes in $v_1, \dots, v_{k-1}$ and thus at the end of the k -th iteration we have

$$d_{ii} \leftarrow \min\{d_{ii}, d_{ik} + d_{ki}\} \leq d_{ik} + d_{ki} = d_{ik}^{*,k-1} + d_{ki}^{*,k-1} \leq w(C) < 0.$$

Again, this contradicts the requirement that all d_{ii} remain nonnegative.

- (c) Let k be the first iteration in which some d_{ii} becomes negative. Since at the start of the algorithm we have $d_{ii} = +\infty > 0$, we know that there exists a prior iteration $k-1$, in which we had $d_{ii} > 0$. In the k -th iteration we then updated

$$d_{ii} \leftarrow \min\{d_{ii}, d_{ik} + d_{ki}\} = d_{ik} + d_{ki} < 0.$$

According to (b) both d_{ik} and d_{ki} are the length of shortest paths P_{ik} and P_{ki} . Combining both paths then results in a directed cycle including the node v_k with negative length.

To show the opposite direction, let C be a w.l.o.g. simple directed cycle with negative length and v_k the node with highest index k on this cycle. Assume that all d_{ii} remained nonnegative over the course of the algorithm. Then we can write C as $C = (v_k, \dots, v_k)$ and by construction all intermediary nodes are in $v_1, \dots, v_{k-1}$. This implies

$$d_{kk}^{*,k-1} \leq w(C).$$

Using (b) this implies that in the k -th iteration we update

$$d_{kk} \leftarrow \min\{d_{kk}, d_{kk} + d_{kk}\} \leq d_{kk} = d_{kk}^{*,k-1} \leq w(C) < 0.$$

But this contradicts the assumption that all d_{ii} remain nonnegative.

- (d) If G does not contain directed cycles with negative length, then by (c) all d_{ii} remain nonnegative and thus by (b) at the end of the n -th iteration we have computed shortest simple paths. Since the graph does not contain cycles with negative length, shortest simple paths are also shortest paths.

□

Since the algorithm consists of three nested loops iterating over all nodes $v \in V$, the algorithm has a complexity of $O(|V|^3)$ operations. It can be used to determine all shortest paths in dense graphs. In graphs with fewer edges and nonnegative edge weights, it can be faster to just run Dijkstra's algorithm once for each startnode.

To close this chapter on shortest paths, we apply this algorithm to the counterexample for Dijkstra's algorithm, see Figure 5.8 and Table 5.4. We see that the algorithm of Floyd and Warshall determines the correct distances.

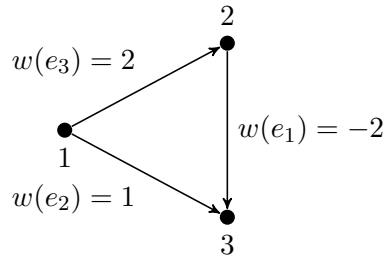


Figure 5.8: Illustration of Algorithm 5.22 with negative edge weights

k	D	P	cycle detected?
0	$\begin{pmatrix} \infty & 2 & 1 \\ \infty & \infty & -2 \\ \infty & \infty & \infty \end{pmatrix}$	$\begin{pmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{pmatrix}$	
1	$\begin{pmatrix} \infty & 2 & 1 \\ \infty & \infty & -2 \\ \infty & \infty & \infty \end{pmatrix}$	$\begin{pmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{pmatrix}$	no
2	$\begin{pmatrix} \infty & 2 & 0 \\ \infty & \infty & -2 \\ \infty & \infty & \infty \end{pmatrix}$	$\begin{pmatrix} 1 & 1 & 2 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{pmatrix}$	no
3	$\begin{pmatrix} \infty & 2 & 0 \\ \infty & \infty & -2 \\ \infty & \infty & \infty \end{pmatrix}$	$\begin{pmatrix} 1 & 1 & 2 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{pmatrix}$	no

Table 5.4: Illustration of Algorithm 5.22 with negative edge weights

Now let us modify this example again such that it contains a directed cycle with negative length, see Figure 5.9 and Table 5.5. In this case, the algorithm terminates with $k = 3$, $i = 1$ and $j = 1$, because it detects $d_{11} = -1 < 0$. The last iteration is thus not completed. We can also find the directed cycle with negative length in the first row of the predecessor matrix, which says that the current shortest way from 1 to 1 is of the form

$$1 \leftarrow 3 \leftarrow 2 \leftarrow 1.$$

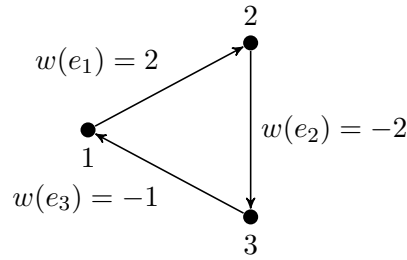


Figure 5.9: Illustration of Algorithm 5.20 with a directed cycle with negative length

k	D	P	cycle detected?
0	$\begin{pmatrix} \infty & 2 & \infty \\ \infty & \infty & -2 \\ -1 & \infty & \infty \end{pmatrix}$	$\begin{pmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 3 & 3 \end{pmatrix}$	
1	$\begin{pmatrix} \infty & 2 & \infty \\ \infty & \infty & -2 \\ -1 & 1 & \infty \end{pmatrix}$	$\begin{pmatrix} 1 & 1 & 1 \\ 2 & 2 & 2 \\ 3 & 1 & 3 \end{pmatrix}$	no
2	$\begin{pmatrix} \infty & 2 & 0 \\ \infty & \infty & -2 \\ -1 & 1 & -1 \end{pmatrix}$	$\begin{pmatrix} 1 & 1 & 2 \\ 2 & 2 & 2 \\ 3 & 1 & 2 \end{pmatrix}$	yes

Table 5.5: Illustration of Algorithm 5.20 with a directed cycle with negative length

5.3 Flow Problems in Networks

So far, we have interpreted edge weights in graphs as costs or distances and searched for shortest paths or minimum spanning trees. An alternative interpretation of edge weights is to see them as capacities of the corresponding edges.

Example 5.25. Consider a network for the transportation of gas, electricity, water, products, data, We can describe such a network as a directed graph $G = (V, E)$ with

maximum capacities $u_e \geq 0$ for all edges $e \in E$. Consider additionally two nodes $s, t \in V$ with $s \neq t$. Then we can ask much can be transported within the network capacities from node s to node t ? A related question is where the bottleneck in the network is located, i.e. the tightest spot that prevents us from transporting more from s to t ?

In order to be able to model and answer these question, we extend our definition of graphs slightly.

Definition 5.26. A *network* (*Netzwerk*) is a directed graph $G = (V, E)$ without loops with the following additional information:

- a *source* (*Quelle*) $s \in V$ and a *sink* (*Senke*) $t \in V$ with $s \neq t$,
- *capacities* (*Kapazitäten*) $u_e \in [0, \infty]$ for all edges $e \in E$.

If all capacities are $u_e = +\infty$, the network is called *uncapacitated* (*unkapazitiert*), otherwise it is called *capacitated* (*kapazitiert*).

At first glance, networks might seem to be the same as weighted graphs with two special nodes s, t . But edge capacities have a different effect than edge weights: When we consider e.g. shortest paths or minimum spanning trees in weighted graphs, then we have to “pay” the edge weight $w(e)$ for each edge e , which we use. In contrast, in a network, we have capacities $u_e \in [0, \infty]$ for each edge $e \in E$, meaning that we can transport only amounts $x_e \in [0, u_e]$ across each edge. (Sometimes, we additionally attach unit transportation costs c_e to the edges e , meaning that transporting x_e along the edge e costs $c_e \cdot x_e$.)

In a network, we can then have a look at a flow x from the source s to the sink t . To describe such a flow, for each node $v \in V$, we define the set of all *outgoing edges* (*ausgehende Kanten*) $E^+(v)$ and the set of all *incoming edges* (*reingehende Kanten*) $E^-(v)$ as

$$\begin{aligned} E^+(v) &:= \{e \in E \mid e = (v, u)\}, \\ E^-(v) &:= \{e \in E \mid e = (u, v)\}. \end{aligned}$$

Recall that we used the same signs in the definition of the incidence matrix I_G of a directed graph, see Definition 5.3, which has the entries

$$(I_G)_{ij} = \begin{cases} +1 & \text{if } e_j = (v_i, *), \\ -1 & \text{if } e_j = (*, v_i), \\ 0 & \text{else.} \end{cases}$$

With these preparations, we can now define a flow.

Definition 5.27. Let $G = (V, E)$ be a network with capacities $u_e \geq 0$ for all edges $e \in E$ and source and sink $s, t \in V$. Then a *flow* (*Fluss*) from s to t is a vector $x \in \mathbb{R}^{|E|}$ satisfying the *capacity constraints* (*Kapazitätsschranken*)

$$0 \leq x \leq u$$

and *flow conservation* (*Flusserhaltung*) or *Kirchoff's law* (*Kirchhoffsche Gesetze*)

$$\sum_{e \in E^+(v)} x_e - \sum_{e \in E^-(v)} x_e = (I_G \cdot x)_v = 0 \quad \text{for all } v \in V \setminus \{s, t\},$$

i.e. outflow minus inflow has to be zero in all nodes except for the source and the sink. The *value* (*Wert*) of the flow is then given by the (excess) outflow in the source s or the (excess) inflow in the sink t , i.e.

$$|x| := \sum_{e \in E^+(s)} x_e - \sum_{e \in E^-(s)} x_e = (I_G \cdot x)_s = -(I_G \cdot x)_t = \sum_{e \in E^-(t)} x_e - \sum_{e \in E^+(t)} x_e.$$

A vector x satisfying the flow conservation constraints, but not necessarily the capacity constraints, is sometimes called a *stream* (*Strom*).

In order to describe bottlenecks in networks, we need the concept of a cut.

Definition 5.28. Let $G = (V, E)$ be a network with capacities $u_e \geq 0$ for all edges $e \in E$ and source and sink $s, t \in V$. A *cut* (*Schnitt*) is a partition of the nodes into two disjoint sets V_s and V_t such that

$$V_s \cap V_t = \emptyset, \quad V_s \cup V_t = V, \quad s \in V_s, \quad t \in V_t.$$

The *capacity* (*Kapazität*) of a cut is defined as

$$U(V_s, V_t) := \sum_{\substack{e=(v,w) \in E: \\ v \in V_s, w \in V_t}} u_e.$$

Each cut defines a subset of edges, through which the whole flow from the source s to the sink t has to flow. The capacity of each cut is thus an upper bound on the value of each flow. Consequently, the cuts with the smallest capacities can be used to describe bottlenecks in the network and provide the tightest upper bounds on the flow values.

With these preparations, we can now define the max-flow and the min-cut problem.

Definition 5.29. Let $G = (V, E)$ be a network with capacities $u_e \geq 0$ for all edges $e \in E$ and source and sink $s, t \in V$.

(a) Then the *max-flow problem* (*Problem des maximalen Flusses*) is defined as

$$\begin{aligned} \max_{x \in \mathbb{R}^{|E|}} \quad & |x| \quad (\text{value of the flow}), \\ \text{s.t.} \quad & 0 \leq x \leq u \quad (\text{capacity constraints}), \\ & (I_G \cdot x)_v = 0 \quad \text{for all } v \in V \setminus \{s, t\} \quad (\text{flow conservation}). \end{aligned}$$

(b) And the *min-cut problem* (*Problem des minimalen Schnitts*) is defined as

$$\min_{V_s, V_t \subseteq V} U(V_s, V_t) \quad \text{s.t.} \quad V_s \cap V_t = \emptyset, \quad V_s \cup V_t = V, \quad s \in V_s, \quad t \in V_t.$$

Some remarks and simplifying assumptions, that we use from now on:

- Since we want to represent networks with their incidence matrix, we assume that the graph G does not contain any loops. This is not a restricting assumption since loops typically do not exist in transport networks anyway.
- As long as we do not consider transport costs¹ c_e for the edges e , we can assume w.l.o.g. that the graph G does not contain parallel edges. If two edges e_1, e_2 are parallel, we can combine them into one new edge e^* with capacity $u_{e^*} = u_{e_1} + u_{e_2}$. Alternatively, we can also get rid of parallel edges e_1, e_2 by inserting an additional node into e.g. e_1 and thus splitting it into two parts e_{1a}, e_{1b} , which both have the original capacity² u_{e_1} .
- The graph G should be weakly connected. If G is not weakly connected, then it has at least two connected components and no flow is possible from one connected component to another. Thus, we can consider all connected components individually.

¹If we do consider transport costs and we have $c_{e_1} = c_{e_2}$, we can use the same technique and define $c_{e^*} = c_{e_1}$. But if the transport costs for both parallel edges are different, combining the two edges into one is more complicated, because one would first use the cheaper edge up to its capacity and then begin to use the more expensive edge. This results in transport costs with a kink, when the capacity of the cheaper edge is reached.

²If we consider transport costs, we cannot assign both edges e_{1a}, e_{1b} the original transport costs c_{e_1} , because then we would pay double for using the edge e_1 . But we could for example assign both edges e_{1a}, e_{1b} the transport costs $c_{e_1}/2$.

- There should exist at least one directed path from source s to sink t . Otherwise, no flows with positive value exist.
- The source and sink should be two different nodes, i.e. $s \neq t$. Furthermore, we can assume that no edges end in the source and no edges start in the sink, i.e. $E^-(s) = \emptyset$ and $E^+(t) = \emptyset$. If such edges exist, they can be deleted from the network without changing a maximum flow, i.e. a flow with maximum $|x|$. This simplifies the formula for the value of a flow x to

$$|x| = \sum_{e \in E^+(s)} x_e = \sum_{e \in E^-(t)} x_e.$$

Deleting such edges also does not change the capacity of a cut, because we always have $s \in V_s$, $t \in V_t$ and only consider edges from V_s to V_t in the capacity of a cut.

- We only consider networks with one source and one sink. In case there exist sets $S \subset V$ of source nodes and $T \subset V$ of sink nodes with $S \cap T = \emptyset$, then we can ask about the maximum total amount, that can be pumped into the network in all source nodes $s \in S$ combined and, respectively, taken out of the network in all sink nodes $t \in T$ combined. However, you can always reformulate this problem by introducing a super source s^* , which is connected to all existing source nodes $s \in S$ via an edge $e = (s^*, s)$ with infinite³ capacity, and a super sink t^* , which is connected to all existing sink nodes $t \in T$ via an edge $e = (t, t^*)$ with infinite capacity.
- We assume that all capacities u_e satisfy $u_e \in (0, \infty)$, i.e. are positive and finite. Edges with zero capacity can just be removed from the network. Edges with infinite capacity do not appear in real networks, but can be caused e.g. by inserting a supersink. However, for such artificial edges, it is usually also possible to choose a finite, but sufficiently large capacity.
- To simplify the description of the network, we sometimes insert the *backflow edge* (*Rückstromkante*) $e^* = (t, s)$ with infinite (or sufficiently large) capacity into the network. Then we can rewrite the flow conservation as

$$I_G \cdot x = 0$$

and the flow x_{e^*} along the new backflow edge $e^* = (t, s)$ is exactly the value of the flow in the network with source s and sink t .

³If we want to avoid infinite capacities here, we can also choose the capacities of these edges finite but sufficiently large, e.g. as

$$u_{(s^*, s)} > \sum_{e \in E^+(s)} u_e.$$

With this backflow edge we can then rewrite the max-flow problem as

$$\begin{array}{lll} \max_{x \in \mathbb{R}^{|E|}} & x_{(t,s)} & \text{(value of the flow),} \\ \text{s.t.} & 0 \leq x \leq u & \text{(capacity constraints),} \\ & I_G \cdot x = 0 & \text{(flow conservation).} \end{array}$$

This shows that the max-flow problem can be formulated as a linear program. To be precise, it is a special case of a transport problem, which we consider in the lecture *Operations Research*, and can thus be solved using either a standard simplex method or the network simplex method. Additionally, we know that the incidence matrix I_G of a directed graph is totally unimodular. Thus, if all capacities u_e are integer, we know that the polyhedron describing the feasible set of this LP is integral. Consequently, even if we have integrality conditions on the flow x , we can still use the simplex method to solve the max flow problem, because all vertex solutions are automatically integer. In the subsequent sections, we will instead develop alternative solution algorithms, which better utilize the specific structure of the max-flow problem.

5.3.1 Maximum Flows and Minimum Cuts

We have already seen that the capacity of each cut provides an upper bound for the value of each flow. This hints at the fact that the max-flow problem might be in some sense dual to the min-cut problem. In this section, we want to explore this connection.

To this end, consider an arbitrary flow x in the network $G = (V, E)$ with finite capacities $u_e > 0$ for all edges $e \in E$ and source and sink $s, t \in V$. This flow should satisfy $0 \leq x \leq u$. Using the incidence matrix I_G of the graph G , we know that $(I_G \cdot x)_v$ is the outflow minus inflow in node v . For a maximum flow, $(I_G \cdot x)_s$ should be as large as possible. And due to flow conservation, we know $(I_G \cdot x)_v = 0$ for all $v \neq s, t$ and $(I_G \cdot x)_t = -(I_G \cdot x)_s$. Using the vector $d \in \mathbb{R}^{|V|}$ with

$$d_v = \begin{cases} -1 & \text{if } v = s, \\ +1 & \text{if } v = t, \\ 0 & \text{else} \end{cases}$$

we can thus measure the total flow from s to t with an additional variable $x_0 \in \mathbb{R}$ defined by the equation

$$I_G x + d x_0 = 0.$$

Note that d is exactly the column we would obtain in the incidence matrix for backflow edge $e^* = (t, s)$ and x_0 describes the flow over this edge. But we want to keep this vector d separate for now. The maximum flow problem is then given by the linear program

$$\max_{x, x_0} x_0 \quad \text{s.t.} \quad I_G x + d x_0 = 0, \quad 0 \leq x \leq u. \quad (5.1)$$

Since this is a linear program, we can also have a look at the corresponding dual problem, which is given by

$$\min_{y \in \mathbb{R}^{|V|}, z \in \mathbb{R}^{|E|}} 0^T y + u^T z \quad \text{s.t.} \quad I_G^T y + z \geq 0, \quad d^T y = 1, \quad z \geq 0. \quad (5.2)$$

It turns out that this dual problem is closely related to the min-cut problem.

Lemma 5.30. Consider a network $G = (V, E)$ with finite capacities $u_e > 0$ for all edges $e \in E$ and source and sink $s, t \in V$. Then for each cut V_s, V_t the vectors $y \in \mathbb{R}^{|V|}, z^* \in \mathbb{R}^{|E|}$ with

$$\begin{aligned} y_v &= \begin{cases} 0 & \text{if } v \in V_s, \\ 1 & \text{if } v \in V_t, \end{cases} \\ z_e^* &= \begin{cases} 1 & \text{if } e = (v, w) \text{ with } v \in V_s, w \in V_t, \\ 0 & \text{else} \end{cases} \end{aligned}$$

is feasible for the dual problem (5.2). Out of all vectors z such that (y, z) is feasible for (5.2), the vector z^* generates the smallest objective function value, which is exactly the capacity of the cut V_s, V_t .

Proof. To verify feasibility of (y, z^*) for the dual problem note that we have $z^* \geq 0$ and

$$d^T y = -y_s + y_t = -0 + 1 = 1.$$

Finally, for all $e = (v, w) \in E$ we have

$$(I_G^T y + z^*)_e = y_v - y_w + z_e^*.$$

In case $v \in V_s$, we have

$$(I_G^T y + z)_e = 0 - y_w + z_e^* = \begin{cases} 0 - 0 + 0 = 0 & \text{if } w \in V_s, \\ 0 - 1 + 1 = 0 & \text{if } w \in V_t. \end{cases}$$

In case $v \in V_t$, we have

$$(I_G^T y + z)_e = 1 - y_w + z_e^* = \begin{cases} 1 - 0 + 0 = 1 & \text{if } w \in V_s, \\ 1 - 1 + 0 = 0 & \text{if } w \in V_t. \end{cases}$$

This shows the feasibility of (y, z^*) for the dual problem.

The claim that $U(V_s, V_t) = u^T z^*$ follows directly from the definition of z^* .

To see that z^* generates the smallest objective function value $u^T z$ out of all (y, z) feasible for the dual problem, note that $u \geq 0$ and thus the objective function is monotonically increasing in z . To minimize the objective function, we thus have to choose $z \geq 0$ as small as possible. And if you consider the computations above, you see that z^* is chosen as small as possible while still satisfying $I_G^T y + z \geq 0$. $\square$

With this preparation, we can now prove the max-flow/min-cut theorem.

Theorem 5.31. (Max-Flow/Min-Cut Theorem) Let $G = (V, E)$ be a network with finite capacities $u_e > 0$ for all edges $e \in E$ and source and sink $s, t \in V$. Then the following holds:

- (a) If $x \in \mathbb{R}^{|E|}$ describes a flow from s to t and $x_0 \in \mathbb{R}$ is the value of this flow, then for all cuts (V_s, V_t) we have $x_0 \leq U(V_s, V_t)$.
- (b) If $x^* \in \mathbb{R}^{|E|}$ describes a maximum flow from s to t and $x_0^* \in \mathbb{R}$ the value of this flow, then there exists a cut (V_s^*, V_t^*) with $x_0^* = U(V_s^*, V_t^*)$. This cut is then a solution of the minimum cut problem.
- (c) If (V_s^*, V_t^*) is a solution of the minimum cut problem (which always exists), then there exists a feasible flow $x^* \in \mathbb{R}^{|E|}$ from s to t with value $x_0^* \in \mathbb{R}$ such that $x_0^* = U(V_s^*, V_t^*)$. This flow is then a solution of the maximum flow problem.
- (d) A flow $x \in \mathbb{R}^{|E|}$ with value $x_0 \in \mathbb{R}$ and a cut (V_s, V_t) are solutions of the respective problems if and only if

$$x_e = \begin{cases} u_e & \text{if } e = (u, v) \text{ with } u \in V_s, v \in V_t, \\ 0 & \text{if } e = (u, v) \text{ with } u \in V_t, v \in V_s. \end{cases}$$

Proof. (a) This follows from Lemma 5.30 together with weak duality.

- (b) Let (x^*, x_0^*) be a solution of the max flow problem (5.1) and (y^*, z^*) a solution of the corresponding dual problem (5.2). Then we know

$$d^T y^* = -y_s^* + y_t^* = 1 \quad \implies \quad y_t^* = y_s^* + 1.$$

Define

$$V_s^* := \{v \in V \mid y_v^* < y_t^*\} \quad \text{and} \quad V_t^* := \{v \in V \mid y_v^* \geq y_t^*\}.$$

Then (V_s^*, V_t^*) form a cut. To show that this cut has the capacity x_0^* and thus is a solution of the minimum cut problem by (a), we use the complementarity

conditions for the primal and dual problem

$$z_e^*(u_e - x_e^*) = 0, \quad x_e^*(y_v^* - y_w^* + z_e^*) = 0 \quad \forall e = (v, w) \in E$$

For all edges $e = (v, w)$ with $v \in V_s^*$ and $w \in V_t^*$ we have due to the definition of V_s^*, V_t^*

$$z_e^* \geq -y_v^* + y_w^* > 0$$

and thus $x_e^* = u_e$. For all edges $e = (v, w)$ with $v \in V_t^*$ and $w \in V_s^*$ we have

$$y_v^* - y_w^* + z_e^* \geq y_v^* - y_w^* > 0$$

and thus $x_e^* = 0$. Then it follows that the value of the flow is given by

$$\begin{aligned} x_0^* &= \sum_{v \in V_s^*} \left(\sum_{w \in V_t^*, (v, w) \in E} x_{(v, w)}^* - \sum_{w \in V_t^*, (w, v) \in E} x_{(w, v)}^* \right) \\ &= \sum_{v \in V_s^*} \left(\sum_{w \in V_t^*, (v, w) \in E} u_{(v, w)} - \sum_{w \in V_t^*, (w, v) \in E} 0 \right) = U(V_s^*, V_t^*) \end{aligned}$$

- (c) Let (V_s^*, V_t^*) be a solution of the minimum cut problem, which always exists because there are only finitely many possible cuts. Then by (a) all feasible flows satisfy $x_0 \leq U(V_s^*, V_t^*)$ and thus the maximum flow problem also has a solution x^* with value x_0^* . By (b) there then exists a solution of the minimum cut problem (V_s, V_t) with $x_0^* = U(V_s, V_t)$. Because (V_s^*, V_t^*) also is a solution of the minimum cut problem, it follows that $x_0^* = U(V_s, V_t) = U(V_s^*, V_t^*)$.
- (d) Let x be a feasible flow with value x_0 and (V_s, V_t) be a cut with the given property. Then we know that the value satisfies

$$\begin{aligned} x_0 &= \sum_{v \in V_s} \left(\sum_{w \in V_t, (v, w) \in E} x_{(v, w)} - \sum_{w \in V_t, (w, v) \in E} x_{(w, v)} \right) \\ &= \sum_{v \in V_s} \left(\sum_{w \in V_t, (v, w) \in E} u_{(v, w)} - \sum_{w \in V_t, (w, v) \in E} 0 \right) = U(V_s, V_t) \end{aligned}$$

and thus we have a maximum flow/minimum cut.

Conversely, let x^* be a maximum flow with value x_0^* and (V_s^*, V_t^*) be a minimum cut. Then we know by (c) that

$$\sum_{v \in V_s^*} \left(\sum_{w \in V_t^*, (v, w) \in E} x_{(v, w)}^* - \sum_{w \in V_t^*, (w, v) \in E} x_{(w, v)}^* \right)$$

$$\begin{aligned}
 &= x_0^* = U(V_s^*, V_t^*) \\
 &= \sum_{v \in V_s^*} \left(\sum_{w \in V_t^*, (v,w) \in E} u_{(v,w)}^* - \sum_{w \in V_t^*, (v,w) \in E} 0 \right).
 \end{aligned}$$

This is only possible, if the given property is satisfied. $\square$

This result shows that in case of finite capacities the max-flow problem and the min-cut problem are indeed dual to each other. Both problems always have a solution and their optimal function values coincide.

Part (d) of the previous result states that a maximum flow uses all edges of a minimum cut in forward direction to their full capacity and all edges in backward direction are unused.

5.3.2 Algorithm of Ford and Fulkerson

To compute a maximum flow, we want to develop the algorithm of Ford and Fulkerson. The basic idea here is to start with a feasible flow x , e.g. the zero flow, and in every iteration change the flow along a path to increase the total flow $|x|$.

Definition 5.32. Let $G = (V, E)$ be a network with finite capacities $u_e > 0$ for all edges $e \in E$ and source and sink $s, t \in V$. Let $x \in \mathbb{R}^{|E|}$ be a flow from s to t . A (possibly undirected) path P from s to t is called an *x -augmenting path* (*x -vermehrender Pfad*), if

$$x_e \begin{cases} < u_e & \text{for all forward edges } e \in P, \\ > 0 & \text{for all backwards edges } e \in P. \end{cases}$$

Here, we call an edge $e \in P$ a *forward edge* (*Vorwärtspfeil*), if the path P from s to t traverses it in the correct orientation, and a *backwards edge* (*Rückwärtspfeil*) otherwise.

The reason these paths are called x -augmenting paths becomes clear with the next result.

Lemma 5.33. Let $G = (V, E)$ be a network with capacities $u_e > 0$ for all edges $e \in E$ and source and sink $s, t \in V$. Let $x \in \mathbb{R}^{|E|}$ be a feasible flow from s to t and P an x -augmenting path. Then there exists a new flow $\hat{x}$ with a larger value $|\hat{x}| > |x|$.

Proof. Since P is an x -augmenting path, there exist an $\varepsilon > 0$ such that

$$\hat{x}_e = \begin{cases} x_e + \varepsilon & \text{for all forward edges } e \in P, \\ x_e - \varepsilon & \text{for all backwards edges } e \in P, \\ x_e & \text{else} \end{cases}$$

still satisfies $0 \leq \hat{x} \leq u$. Furthermore, $\hat{x}$ is still a flow from s to t , because in all nodes $v \neq s, t$ the changes add up to zero. But the value of the flow has increased to $|\hat{x}| = |x| + \varepsilon > x_0$. $\square$

As long as a x -augmenting path exists, the previous results shows that x is not yet a maximum flow. However, the opposite is also true as the following result shows.

Theorem 5.34. Let $G = (V, E)$ be a network with finite capacities $u_e > 0$ for all edges $e \in E$ and source and sink $s, t \in V$. Then a flow x from s to t is a maximum flow if and only if there does not exist an x -augmenting path.

Proof. We have already shown that x is not a maximum flow, if an x -augmenting path exists. It remains to prove that the nonexistence of an x -augmenting path implies that x is a maximum flow. To this end, let us define the cut

$$V_s := \{v \in V \mid \text{there exists an } x\text{-augmenting path from } s \text{ to } v\} \cup \{s\}, \quad V_t = V \setminus V_s.$$

This is a cut, because by assumption we have $t \in V_t$. For all backwards edges $e = (v, w)$ with $v \in V_t$ and $w \in V_s$ we know that there exists an x -augmenting path from s to w but not from s to v . It follows that $x_e = 0$. For all forward edges $e = (v, w)$ with $v \in V_s$ and $w \in V_t$ we know that there exists an x -augmenting path from s to v but not from s to w . It follows that $x_e = u_e$. This shows that x is a maximum flow (and (V_s, V_t) a minimum cut). $\square$

This result is the core of the algorithm of Ford and Fulkerson, see Algorithm 5.35.

In this algorithm, we have to compute x -augmenting path in every iteration. To do this, let x be a feasible flow. For an x -augmenting path, we can then only use forward edges e with $x_e < u_e$ and backwards edges e with $x_e > 0$. For this reason, we define the *residual graph* (*Restgraph*) corresponding to x as $G_r(x) = (V, E_r(x))$ with

$$E_r(x) = \{e \in E \mid r(e) := u_e - x_e > 0\} \cup \{\overleftarrow{e} \mid e \in E, \quad r(\overleftarrow{e}) := x_e > 0\}.$$

Here, for edges $e \in E$ with $x_e > 0$ we add the corresponding inverted edge $\overleftarrow{e}$ to the residual graph, because these edges e are used in the inverted direction in x -augmenting paths. For edges $e \in E$ with $x_e \in (0, u_e)$ both the edge e itself and the corresponding inverted edge $\overleftarrow{e}$ are added to the residual graph. The values $r(e)$ and $r(\overleftarrow{e})$ defined here

are called the *residual capacities* (*Restkapazitäten*) of the respective edges. Directed paths from s to t in the residual graph are then x -augmenting paths and can be determined e.g. with BFS or DFS.

Algorithm 5.35. (Algorithm of Ford and Fulkerson)

Input: Let $G = (V, E)$ be a network with finite capacities $u_e > 0$ for all edges $e \in E$ and source and sink $s, t \in V$.

- 1 Choose a feasible initial flow x from s to t , e.g. $x = 0$.
- 2 **while** an x -augmenting path P exists **do**
- 3 Determine

$$\begin{aligned}\varepsilon_1 &= \min\{u_e - x_e \mid e \in P \text{ forward edge}\}, \\ \varepsilon_2 &= \min\{x_e \mid e \in P \text{ backwards edge}\}, \\ \varepsilon &= \min\{\varepsilon_1, \varepsilon_2\}.\end{aligned}$$

- 4 Update the flow as

$$x_e \leftarrow \begin{cases} x_e + \varepsilon & \text{for all forward edges } e \in P, \\ x_e - \varepsilon & \text{for all backwards edges } e \in P, \\ x_e & \text{else} \end{cases}$$

- 5 **end while**

Output: A maximum flow x .

Example 5.36. Let us apply the Algorithm of Ford and Fulkerson to the graph illustrated in Figure 5.10. In the first iteration, we increase the value of the flow from 0 to 3 by using all edges along the x -augmenting zigzag path to their full capacity.

This leaves us with two unused edges. If we were only allowed to increase the value of the flow within the algorithm, which would correspond to only using directed x -augmenting paths, we would have to terminate here.

But in the Algorithm of Ford and Fulkerson, x -augmenting paths do not have to be directed and the backwards edges result in a reduction of the flow along these edges. We use this in the second iteration to reduce the flow along the middle edge by 2, which allows us to now also use the two previously unused edges and increase the value of the flow from 3 to 5.

Since the resulting residual path now does not contain a directed path from s to t anymore, we can terminate the algorithm and have found a maximal flow.

We already know from Theorem 5.34 that the Algorithm of Ford and Fulkerson computes a maximum flow, if it terminates after finitely many iterations. Whether or not

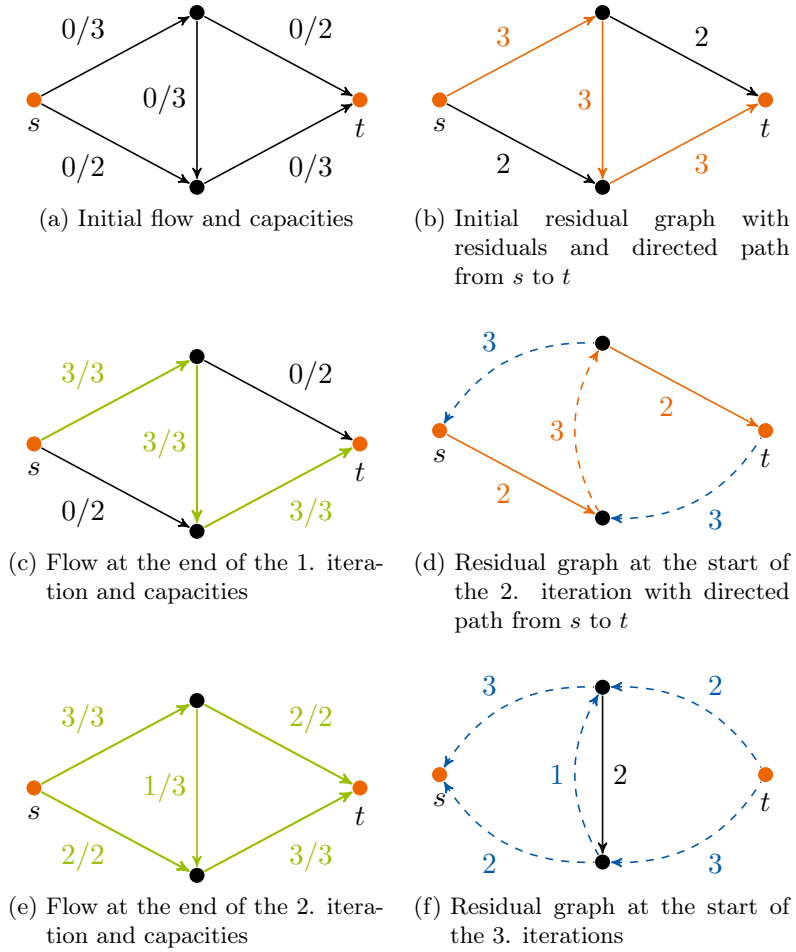


Figure 5.10: Illustration of Example 5.36

the algorithm terminates after finitely many iterations, depends on the capacities u_e . If all capacities are finite and rational, one can show that the algorithm terminates after finitely many iterations. For irrational capacities, Ford and Fulkerson have provided an example, where the algorithm does not terminate and the limit of the generated flows also is not a maximal flow.

For finite integer capacities, the argument is as follows: Let $C \in \mathbb{N}$ be the largest occurring capacity. The sets $V_s := \{s\}$ and $V_t := V \setminus \{s\}$ form a cut and, if we assume that G does not contain parallel edges, this cut contains at most $|V| - 1$ edges. Therefore $C \cdot (|V| - 1)$ is an upper bound for the capacity of the cut (V_s, V_t) . Due to the max-flow min-cut theorem, $C \cdot (|V| - 1)$ is then also an upper bound for the value of every flow. Since all capacities are integer and we start with the integer flow $x = 0$, all residuals are integer and thus ε is a positive integer in every iteration. This means that the value of the flow increases by at least 1 in every iteration and thus the algorithm terminates after at most $C \cdot (|V| - 1)$ iterations. Together with the fact that BFS or DFS have a complexity of $O(|V| + |E|)$, this results in a complexity of $O((|V| + |E|) \cdot |V| \cdot C)$ for finite integer capacities.

It is possible to construct examples with integer capacities, where the algorithm really needs $|x^*|$ iterations to compute the optimal flow x^* , see e.g. Figure 5.11.

A closer look at this example reveals that we always use an x -augmenting path with 3 edges. However, there also exist x -augmenting paths with only 2 edges. If we use those in the Algorithm of Ford and Fulkerson, then we find the maximum flow after 2 iterations.

So a problem in the Algorithm of Ford and Fulkerson is that it allows to use all existing x -augmenting paths. In contrast, in the *Algorithm of Edmonds and Karp* you always use an x -augmenting path with as few edges as possible, which can be determined e.g. with BFS. And in the *Algorithm of Dinic*, the flow is again modified along such “shortest” x -augmenting paths, possibly several at the same time.

If we include this condition that the x -augmenting path is always chosen with as few edges as possible, we can improve the bound on the number of needed iterations.

Theorem 5.37. Consider a network with positive, finite, and integer capacities. If in the algorithm of Ford and Fulkerson the x -augmenting paths in the residual graph are determined with BFS, i.e. have the minimal number of edges, the algorithm terminates after at most $O(|V| \cdot |E|)$ iterations.

Proof (optional). For a flow x and nodes $v, w \in V$ let $\text{dist}_x(v, w)$ be the length of a shortest path (with respect to the number of edges) from v to w in the residual graph $G_r(x)$.

Now let x^+ be the flow generated by updating the flow x . Then we claim $\text{dist}_{x^+}(s, v) \geq \text{dist}_x(s, v)$ for all $v \in V$. If this were wrong, choose $v_0 \in V$ as the node with the smallest

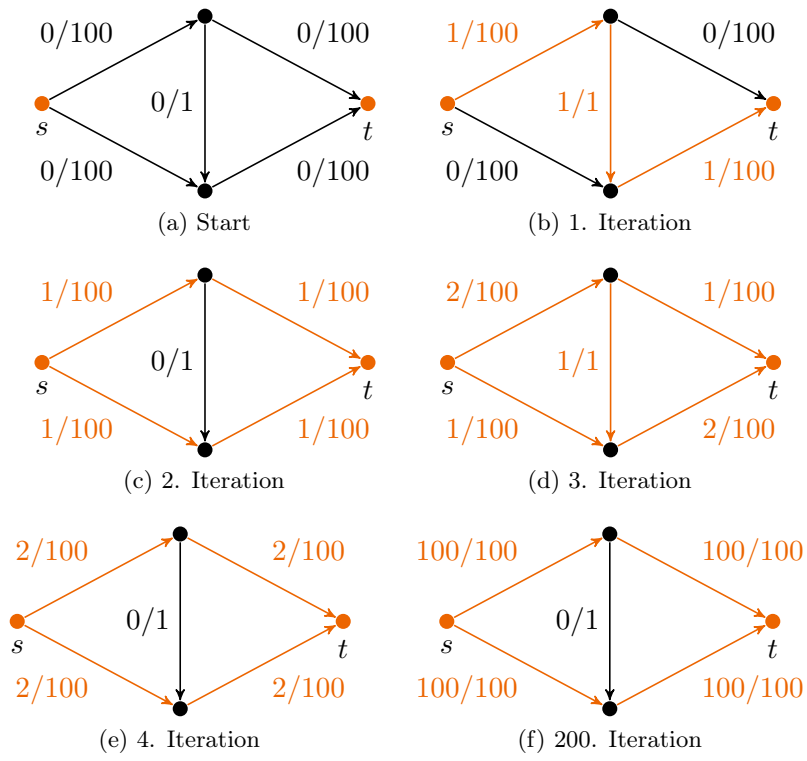


Figure 5.11: Illustration of an Example, where the Algorithm of Ford and Fulkerson needs $|x^*|$ iterations

value $\text{dist}_{x^+}(s, v_0)$ such that

$$\text{dist}_{x^+}(s, v_0) < \text{dist}_x(s, v_0).$$

For all nodes $v \in V$ we then have

$$\text{dist}_{x^+}(s, v) < \text{dist}_{x^+}(s, v_0) \implies \text{dist}_{x^+}(s, v) \geq \text{dist}_x(s, v).$$

Now consider a shortest path P^+ from s to v_0 in the residual graph $G_r(x^+)$ and let v_1 be the predecessor of v_0 on this path. Then we have

$$\text{dist}_{x^+}(s, v_1) = \text{dist}_{x^+}(s, v_0) - 1 < \text{dist}_{x^+}(s, v_0) \implies \text{dist}_{x^+}(s, v_1) \geq \text{dist}_x(s, v_1).$$

Now we have a closer look at the edge $e = (v_1, v_0)$ and the flow x_e along that edge.

- In case $x_e < u_e$, we had $e \in E_r(x)$ and thus know

$$\text{dist}_x(s, v_0) \leq \text{dist}_x(s, v_1) + 1 \leq \text{dist}_{x^+}(s, v_1) + 1 = \text{dist}_{x^+}(s, v_0),$$

a contradiction to the choice of v_0 .

- In case $x_e = u_e > 0$, we had $e \notin E_r(x)$. Since $e = (v_1, v_0) \in P^+$ we know $e \in E_r(x^+)$. The x -augmenting path P in $G_r(x)$ thus included the backwards edge $\overleftarrow{e} = (v_0, v_1)$, which implies

$$\text{dist}_x(s, v_0) \leq \text{dist}_x(s, v_1) - 1 \leq \text{dist}_{x^+}(s, v_1) - 1 < \text{dist}_{x^+}(s, v_0),$$

again a contradiction to the choice of v_0 .

Now consider an arbitrary iteration and let x be the flow at the start of this iteration. Let P be a shortest path from s to t in the residual graph $G_r(x)$ and ε the smallest residual along this path. Each edge $e = (v, w) \in P$ with $r(e) = \varepsilon$ is called *critical*. At the end of the iteration the flow x is modified to x^+ such that in the new residual graph $G_r(x^+)$ all of the critical edges e are not included. For a critical edge $e = (v, w)$ to become critical again in a later iteration, there has to have been an intermediate iteration with a flow x^+ gegeben haben, where the inverse edge $\overleftarrow{e} = (w, v)$ was on the shortest path in $G_r(x^+)$ from s to t . This implies

$$\begin{aligned} \text{dist}_x(s, t) &= \text{dist}_x(s, v) + 1 + \text{dist}_x(w, t), \\ \text{dist}_x(s, v) &= \text{dist}_x(s, w) + 1, \\ \text{dist}_x(v, t) &= \text{dist}_x(w, t) + 1, \\ \text{dist}_{x^+}(s, t) &= \text{dist}_{x^+}(s, w) + 1 + \text{dist}_{x^+}(v, t) \end{aligned}$$

It follows that

$$\text{dist}_{x^+}(s, t) = \text{dist}_{x^+}(s, w) + 1 + \text{dist}_{x^+}(v, t)$$

$$\begin{aligned}
 &\geq \text{dist}_x(s, w) + 1 + \text{dist}_x(v, t) \\
 &= \text{dist}_x(s, v) + 2 + \text{dist}_x(w, t) + 1 \\
 &= \text{dist}_x(s, t) + 2.
 \end{aligned}$$

Since we know that $\text{dist}_x(s, t)$ is monotonically increasing over the iterations, this means that e can only become critical again after the length of the shortest path from s to t has grown by 2. Since the length of the shortest path from s to t is bounded above by $|V| - 1$ this means that each edge can become critical at most $\frac{|V|}{2}$ times. After $O(|V| \cdot |E|)$ iterations we have thus run out of critical edges. And since each path from s to t in $G_r(x)$ has to contain at least one critical edge, the algorithm has to terminate. $\square$

5.3.3 Algorithm of Goldberg-Tarjan

A *flow* x has to satisfy both the capacity constraints and flow preservation. If only the flow preservation was satisfied, we called x a *stream*. So far, we focused mostly on flow preservation and used streams to modify flows, e.g. along x -augmenting paths.

But instead, we can also focus on the capacity constraints.

Definition 5.38. Let $G = (V, E)$ be a network with finite capacities $u_e > 0$ for all edges $e \in E$ and source and sink $s, t \in V$. A vector $x \in \mathbb{R}^{|E|}$ is called a *preflow* (*Präfluss*), if

- it satisfies the capacity constraints $0 \leq x \leq u$ and
-

$$\text{ex}(v) := \sum_{e \in E^-(v)} x_e - \sum_{e \in E^+(v)} x_e \geq 0$$

for all $v \in V \setminus \{s\}$. Here, $\text{ex}(v)$ is the *excess* (*Überschuss*) in node v and nodes with $\text{ex}(v) > 0$ are called *active* (*aktiv*).

So a preflow only has to satisfy the capacity constraint but not flow preservation. In the source s , we can have more outflow than inflow. And in all other nodes $v \neq s$, we can now have more inflow than outflow, i.e. in these nodes flow can be collected in some kind of reservoir.

Since a preflow still satisfies the capacity constraints, we can again work with the residual graph $G_r(x) = (V, E_r(x))$ with

$$E_r(x) = \{e \in E \mid r(e) := u_e - x_e > 0\} \cup \{\overleftarrow{e} \mid e \in E, \quad r(\overleftarrow{e}) := x_e > 0\}.$$

The *Algorithm of Goldberg-Tarjan*, also known as the *Preflow-Push-Algorithm* or *Push-Relabel-Algorithm*, is then based on the following idea: Imagine the network as a system

of water tubes. Each node $v \in V$ is located in a certain height $d(v)$ above the ground, the source s at height $d(s) = |V|$ and the sink t at height $d(t) = 0$. In each node $v \in V \setminus \{s, t\}$ there is a reservoir, which collects the excess water at this node. In each edge $e = (v, w) \in E_r(x)$ of the residual graph, there is a valve, which can temporarily be opened, if the edge has a downward slope of 1, i.e. if $d(v) = d(w) + 1$. When we open the valve, we can push excess water from node v to node w , but only within the residual capacity $r(e)$. Afterwards, the valve closes again automatically.

The algorithm starts with all nodes located at height 0 and the source s at height $|V|$. Additionally, we start having pushed as much water from the source s along the edges $e \in E^+(s)$ to all adjacent nodes of s as the capacities u_e allow. In every iteration we then either open a feasible valve and push as much excess along that edge as feasible and available (*push*) or we raise an active node to a higher height in order find a feasible valve (*relabel*) to open in the next iteration. This results in Algorithm 5.39.

Algorithm 5.39. (Algorithm of Goldberg and Tarjan)

Input: Let $G = (V, E)$ be a network with finite capacities $u_e > 0$ for all edges $e \in E$ and source and sink $s, t \in V$.

1 Initialize the preflow x as

$$\begin{aligned} x_e &= u_e & \forall e \in E^+(s), \\ x_e &= 0 & \text{else.} \end{aligned}$$

2 Initialize the heights as

$$\begin{aligned} d(s) &= |V| \\ d(v) &= 0 & \forall v \in V \setminus \{s\} \end{aligned}$$

3 Initialize the set of active nodes as

$$A = \{v \in V \setminus \{s, t\} \mid \text{ex}(v) > 0\}$$

4 Determine the residual graph $G_r(x) = (V, E_r(x))$ corresponding to x .

5 **while** $A \neq \emptyset$ **do**

6 Choose an active node $v \in A$.

7 Compute the set

$$E(v) = \{e = (v, w) \in E_r(x) \mid d(v) = d(w) + 1\}$$

```

8  if  $E(v) \neq \emptyset$  then
9      Push: Choose an edge  $e = (v, w) \in E_r(x)$  and set
           
$$\delta = \min\{r(e), \text{ex}(v)\}.$$

           In case  $(v, w) \in E$ , set  $x_e \leftarrow x_e + \delta$ . In case  $(w, v) \in E$ , set  $x_e \leftarrow x_e - \delta$ .
10  else
11      Relabel: Set
           
$$d(v) = \min\{d(w) + 1 \mid (v, w) \in E_r(x)\}$$

12  end if
13  Update the residual graph  $G_r(x)$  and the set of active nodes
           
$$A = \{v \in V \setminus \{s, t\} \mid \text{ex}(v) > 0\}$$

14 end while
Output: A maximal flow  $x$ .

```

Example 5.40. We apply the Algorithm of Goldberg and Tarjan to the graph illustrated in Figure 5.12.

The initial preflow is

$$x_{(s,1)} = u_{(s,1)} = 3, \quad x_{(s,2)} = u_{(s,2)} = 2, \quad x_e = 0 \text{ else}$$

and the initial heights are

$$d(s) = 4, \quad d(1) = 0, \quad d(2) = 0, \quad d(t) = 0.$$

This results in the excess

$$\text{ex}(s) = -5, \quad \text{ex}(1) = 3, \quad \text{ex}(2) = 2, \quad \text{ex}(t) = 0.$$

So nodes 1 and 2 are active. But in the residual graph they do not have successor nodes with height -1 . So we cannot push, but instead relabel node 1 as

$$d(1) = \min\{d(2) + 1, d(t) + 1, d(s) + 1\} = \min\{1, 1, 5\} = 1.$$

In the second iteration, we can now push the excess from node 1 to nodes 2 or t . We push 3 units from 1 to 2 and obtain the new preflow

$$x_{(s,1)} = 3, \quad x_{(s,2)} = 2, \quad x_{(1,2)} = 3, \quad x_e = 0 \text{ else.}$$

Now only node 2 is active with the excess

$$\text{ex}(2) = 5$$

and we obtain the 2. residual graph. Since the active node has height $d(2) = 0$ and its successors in the residual graph have the heights $d(s) = 4$, $d(1) = 1$ and $d(t) = 0$, we have to relabel node 2 to

$$d(2) = \min\{d(1) + 1, d(s) + 1, d(t) + 1\} = 1.$$

In the next iteration we can then push 3 units from node 2 to node t and obtain the new preflow

$$x_{(s,1)} = 3, \quad x_{(s,2)} = 2, \quad x_{(1,2)} = 3, \quad x_{(2,t)} = 3, \quad x_e = 0 \text{ else}$$

and the 3. residual graph. Node 2 still has an excess of $\text{ex}(2) = 2$, i.e. is still active. Since its successors in the residual graph have the heights $d(s) = 4$ and $d(1) = 1$, we have to relabel node 2 to $d(2) = 2$. In the next iteration we can then push the complete excess $\text{ex}(2) = 2$ from node 2 back to node 1 and obtain the new preflow

$$x_{(s,1)} = 3, \quad x_{(s,2)} = 2, \quad x_{(1,2)} = 1, \quad x_{(2,t)} = 3, \quad x_e = 0 \text{ else}$$

and the 4. residual graph. The only active nodes 1 with $\text{ex}(1) = 2$ and $d(1) = 1$. Its successors in the residual graph have the heights $d(s) = 4$, $d(2) = 2$ and $d(t) = 0$. We can thus push the whole excess from node 1 to t and obtain the maximal flow

$$x_{(s,1)} = 3, \quad x_{(s,2)} = 2, \quad x_{(1,2)} = 1, \quad x_{(2,t)} = 3, \quad x_{(1,t)} = 2$$

and the corresponding final residual graph.

If x is a preflow, then the corresponding residual graph $G_r(x) = (V, E_r(x))$ has the following property, which we need in the following theorem.

Lemma 5.41. (optional) Let $G = (V, E)$ be a network with finite capacities $u_e > 0$ for all edges $e \in E$ and source and sink $s, t \in V$. If x is a preflow and v^* an active node, i.e. $\text{ex}(v^*) > 0$, then there exists a directed path in G_r from v^* to s .

Proof. Let $v^* \in V$ be an active node and $\tilde{V} \subseteq V$ be the set of all nodes v , for which there exists a directed path from v^* to v in $G_r(x)$, including v^* itself. Assume $s \notin \tilde{V}$.

Since x is a preflow, the source s is the only node, for which $\text{ex}(v) < 0$ is allowed. Since $s \notin \tilde{V}$ and $v^* \in \tilde{V}$ is active, this implies

$$\sum_{v \in \tilde{V}} \text{ex}(v) > 0$$

On the other hand, we know

$$\sum_{v \in \tilde{V}} \text{ex}(v) = \sum_{v \in \tilde{V}} \left[\sum_{e \in E^-(v)} x_e - \sum_{e \in E^+(v)} x_e \right] = \sum_{e \in (V \setminus \tilde{V}) \times \tilde{V}} x_e - \sum_{e \in \tilde{V} \times (V \setminus \tilde{V})} x_e \leq 0,$$

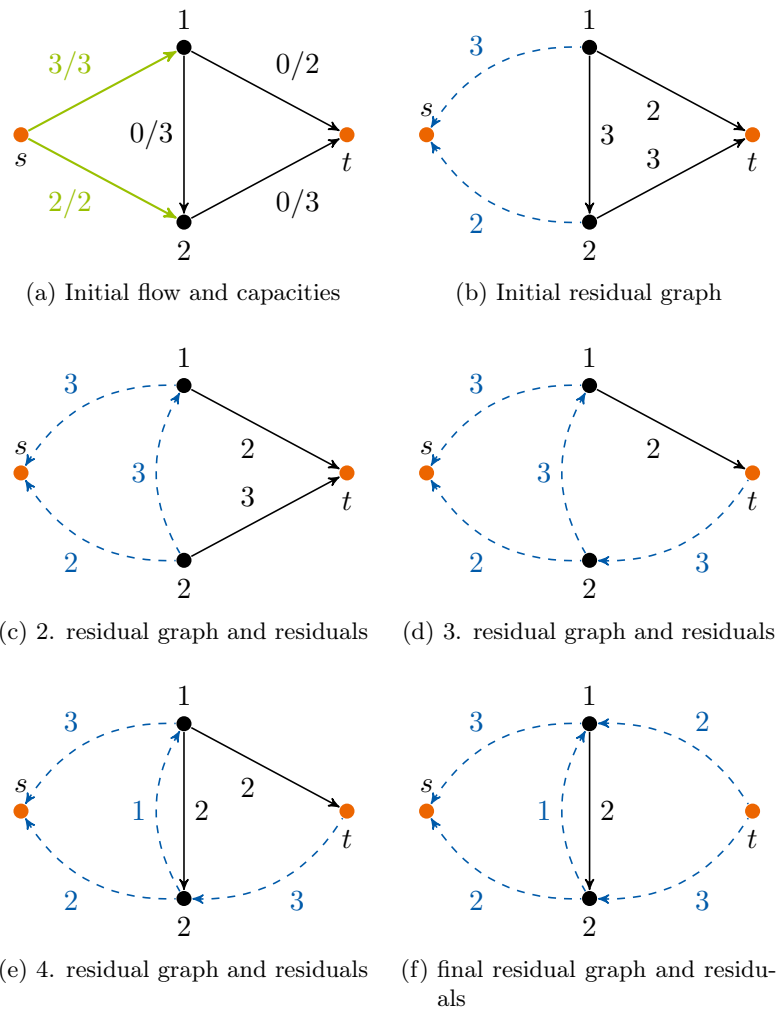


Figure 5.12: Illustration of Example 5.40

because the first sum is equal to zero: For all $e = (v, w) \in (V \setminus \tilde{V}) \times \tilde{V}$ we have $\overleftarrow{e} = (w, v) \notin E_r(x)$ because otherwise $w \in \tilde{V}$. This implies $x_e = 0$.

All in all, we see that the assumption $s \notin \tilde{V}$ leads to a contradiction. $\square$

This algorithm has the following useful properties:

Theorem 5.42. (optional) Let $G = (V, E)$ be a network with capacities $u_e > 0$ for all edges $e \in E$ and source and sink $s, t \in V$. Then the Algorithm of Goldberg-Trajan satisfies:

- (a) x is always a preflow.
- (b) We always have $d(s) = |V|$, $d(t) = 0$ and $d(v) \in \{0, 1, 2, \dots\}$ for all $v \in V \setminus \{s, t\}$.
- (c) $(v, w) \in E_r(x)$ implies $d(v) \leq d(w) + 1$.
- (d) For each occurring x there is no path in the residual graph $G_r(x)$ from s to t .
- (e) If $A = \emptyset$ then x is a flow.
- (f) If $A = \emptyset$, then x is a maximal flow.
- (g) $d(v)$ is monotonically increasing for all $v \in V$ over the iterations.
- (h) We always have $d(v) \leq 2|V| - 1$ for all $v \in V$.
- (i) Every node $v \in V$ is relabeled at most $2|V| - 1$ times. In total, the relabel operation is executed at most $(2|V| - 1)(|V| - 2)$ times.

Proof. (a) This follows from the update rule, because the outflow from a node v along an edge $e = (v, u)$ is increased by at most

$$\min\{u_e - x_e, \text{ex}(v)\}$$

and the inflow into a node v along an edge $e = (u, v)$ is decreased by at most

$$\min\{x_e, \text{ex}(v)\}.$$

This preserves the capacity constraints and the preflow properties.

- (b) For the nodes $v \neq s, t$ there is nothing to show. For s we always have $d(s) = |V|$ and for t we always have $d(t) = 0$, because s, t are never included in the set of active nodes A and we only change $d(v)$ for nodes $v \in A$.

- (c) We prove this with induction. After the initialization we have $E_r(x) = \{(v, w) \in E \mid v \neq s\} \cup \{(v, s) \mid (s, v) \in E\}$ and thus for all $(v, w) \in E_r(x)$

$$d(v) = 0 \leq d(w) + 1.$$

If v^* is relabeled in an iteration, then x and thus $E_r(x)$ remain unchanged. Only $d(v^*)$ is increased but only so far that the condition is still satisfied.

If instead we push along an edge $e = (v^*, w^*) \in E_r(x)$, the set $E_r(x)$ can only change if $e = (v^*, w^*)$ is removed from $E_r(x)$ and instead $\overleftarrow{e} = (w^*, v^*)$ is included. But then the choice of e implies $d(v^*) = d(w^*) + 1$.

- (d) Assume that at some point there exists a (simple) path from s to t in $G_r(x)$. Since such a path can contain at most $|V| - 1$ edges, parts (b) and (c) imply

$$|V| = d(s) \leq d(t) + (|V| - 1) = |V| - 1,$$

a contradiction.

- (e) In case $A = \emptyset$, we know for all $v \in V \setminus \{s, t\}$, that $\text{ex}(v) = 0$. And since we always assume $E^-(s) = \emptyset$ and $E^+(t) = \emptyset$, this flow preservation implies $\text{ex}(t) \geq 0$ and $\text{ex}(s) \leq 0$.
- (f) This follows from part (d): If x were a flow but not maximal, then the residual graph would contain a path from s to t , along which we could augment the flow, see Algorithm of Ford and Fulkerson.
- (g) The value of $d(v)$ is only changed in a relabel operation. Before the relabel operation we have $d(v) \leq d(w) + 1$ for all $(v, w) \in E_r(x)$ according to (c) and thus

$$d(v) \leq \min\{d(w) + 1 \mid (v, w) \in E_r(x)\}.$$

If the relabel operation is executed, we have $E(v) = \emptyset$ and thus

$$d(v) < \min\{d(w) + 1 \mid (v, w) \in E_r(x)\}.$$

This shows that the relabel operation increases $d(v)$.

- (h) We only have to show the claim for the nodes $v \neq s, t$. The value $d(v)$ can only be increased, when $v \in A$ holds, i.e. $\text{ex}(v) > 0$. By Lemma 5.41 the residual graph $G_r(x)$ contains a directed (simple) path from v to s . This path contains at most $|V| - 1$ edges. Together with $d(s) = |V|$ and (c) this implies

$$d(v) \leq d(s) + (|V| - 1) = 2|V| - 1.$$

- (i) Each time the node $v \neq s, t$ is relabeled, $d(v)$ increases by at least $+1$, see proof of part (g). According to (h) each node $v \neq s, t$ can thus be relabeled at most $2|V| - 1$ times. Since the nodes s, t are never relabeled, the relabel operation can be executed at most $(2|V| - 1)(|V| - 2)$ times. $\square$

We now know that the algorithm computes a maximum flow, if it terminates after finitely many iterations. It remains to show that it does terminate. We already know that the relabel operation is executed only finitely many times. Thus, we only have to find a bound on the number of push operations. To this end, we divide the push operations into *saturating* (*saturierende*) pushes with $\delta = r(e)$, i.e. where the edge e has the new residuum $r(e) = 0$ after the push, and *nonsaturating pushes*.

Lemma 5.43. (optional) Let $G = (V, E)$ be a network with capacities $u_e > 0$ for all edges $e \in E$ and source and sink $s, t \in V$. Then the data generated by the Algorithm of Goldberg-Trajan satisfies:

- (a) At most $2|V| \cdot |E|$ saturating push operations are executed.
- (b) At most $4|V|^2 \cdot |E|$ nonsaturating push operations are executed.

Proof. (a) If a saturating push is executed for an edge $e = (v, w)$, then we have $d(v) = d(w) + 1$. We change x_e to u_e to 0 and the edge e is removed from the residual graph $E_r(x)$ and replaced by $\overleftarrow{e}$. Before the edge e can appear in $E_r(x)$ again, we have to have a push along the edge $\overleftarrow{e}$, which requires $d(w) = d(v) + 1$. Thus, if we push along $e = (v, w)$ again, we know

$$d(v) = d(w) + 1 \geq d(v) + 2,$$

i.e. $d(v)$ has to have increased by at least 2. Since we already know $d(v) \leq 2|V| - 1$, we can have at most $|V|$ saturating pushes along the edge e . Since we also have to consider the inverted edges $\overleftarrow{e}$, we can have at most $2|V||E|$ saturating pushes.

- (b) Consider the sum of the heights of all active nodes

$$D(x) := \sum_{v \in V \setminus \{s, t\}, \text{ex}(v) > 0} d(v).$$

We always have $D(x) \geq 0$ and at the start of the algorithm we know $D(x) = 0$.

In a nonsaturating push along an edge $e = (v, w)$ we have $\text{ex}(v) = 0$ after the push. Due to $d(v) = d(w) + 1 \geq 1$ the value of $D(x)$ decreases by at least 1 with each nonsaturating push.

In a saturating push along an edge $e = (v, w)$ the endnode w can become active. This can increase $D(x)$ by at most $2|V| - 1$. Since there are at most $2|V||E|$ saturating pushes, these can increase $D(x)$ by at most $2|V||E|(2|V| - 1)$ in total.

Each relabel operation also increases $D(x)$. Since we only relabel the $|V| - 2$ nodes $v \neq s, t$ and we already know that $d(v)$ is monotonically increasing and bounded above by $2|V| - 1$, it follows that all relabel operations together can increase $D(x)$ by at most $(|V| - 2) \cdot (2|V| - 1)$.

Saturating pushes and relabel operations together increase $D(x)$ over the whole algorithm by at most

$$\begin{aligned} & 2|V||E|(2|V| - 1) + (2|V| - 1)(|V| - 2) = (2|V| - 1)(2|V||E| + |V| - 2) \\ = & \begin{cases} 4|V|^2|E| - 3|V| + 2 \leq 4|V|^2|E| - 3 \cdot 2 + 2 & \text{if } |E| = |V| - 1, \\ 4|V|^2|E| + 2|V|(|V| - 2) - (2|V||E| + |V| - 2) \\ \leq 4|V|^2|E| + 2|V|^2 - 2|V||E| & \text{if } |E| \geq |V|, \end{cases} \\ \leq & 4|V|^2|E|. \end{aligned}$$

Since every nonsaturating push decreases $D(x)$ by at least 1 and we always have $D(x) \geq 0$, it follows that there can be at most $4|V|^2 \cdot |E|$ nonsaturating pushes. $\square$

In total the algorithm needs at most $O(|V|^2 \cdot |E|)$ nonsaturating pushes, $O(|V| \cdot |E|)$ saturating pushes, and $O(|V|^2)$ relabel operations. It is possible to implement the algorithm such that it computes a maximum flow in $O(|V|^2 \cdot |E|)$ iterations. With suitable modifications, it is possible to improve the complexity of the algorithm: If the active nodes are stored in a FIFO queue or if we always pick the active node v with the highest height $d(v)$, we can reduce the complexity to $O(|V|^3)$, see [2, Section 9.7.2]

5.4 Exercises

Exercise 5.1. Consider a gas network described by a directed graph $G = (V, E)$ with maximum capacities $u_e \geq 0$ for all edges $e \in E$. Within this gas network, we have a nonempty set of source nodes $S \subset V$ and a nonempty set of sink nodes $T \subset V$ with $S \cap T = \emptyset$. In each source node $s \in S$ a supply $\sigma_s > 0$ of gas is available and in each sink node $t \in T$ we have a demand $d_t > 0$. Assume that supply equals demand, i.e.

$$\sum_{s \in S} \sigma_s = \sum_{t \in T} d_t.$$

- (a) The first question the *transport system operator (TSO)* of this network has to answer is, whether it is possible to transport the whole supply of gas from the

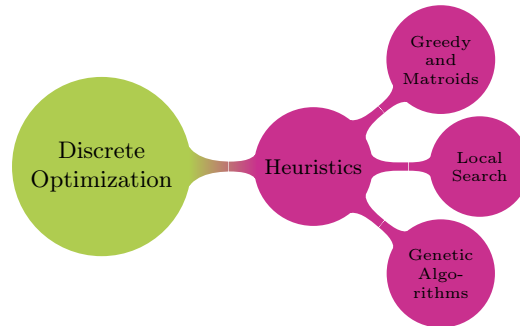
sources to the sinks within the given network capacity? To answer this question, formulate an auxiliary max-flow problem with one supersource and one supersink.

- (b) Now in addition to the capacities u_e we also attach transport costs $c_e \geq 0$ to each edge $e \in E$. The goal of the TSO is then to transport the whole supply of gas from the sources to the sinks with minimal costs. Develop a suitable optimization problem. What type of optimization problem is this?

Exercise 5.2. Consider a directed graph $G = (V, E)$ with edge weights w_e for all $e \in E$. Let s, t be two nodes in V and assume that we are looking for a shortest path from s to t .

- (a) Define a network and a minimum cost transportation problem in this network, whose solution is a shortest path from s to t .
- (b) The resulting problem is a linear program, of which we are looking for an integer solution. Why is that not a problem?

6 Heuristics



In this chapter, we consider mixed-integer optimization problems and want to have a look at some general methods, which try to find a good – but not necessarily optimal – feasible point. To be able to differentiate between different types of solution methods for optimization problems, we use the following terms:

- *solution algorithm*: Terminates in or converges to some special type of feasible point, e.g. a global optimum or stationary point of the considered optimization problem. Examples are the simplex method, the branch&bound method or gradient descent.
- *approximation (Approximation)*: Does not necessarily terminate in or converge to e.g. a global optimum, but you have some kind of quality guarantee such as “the objective function value in the final iterate is at worst twice as big as in the optimum”. An example is the minimum spanning tree heuristic for TSP.
- *heuristic (Heuristik)*: Computes or converges to some feasible point, but there is no quality guarantee, i.e. the final iterate can be arbitrarily worse than the optimal solution. An example is the nearest neighbor heuristic for TSP.

However, please be aware that these terms are not always used consistently in the literature. For example, some people consider approximations to be heuristics with an additional quality guarantee.

Solution algorithms are preferable in the sense that you know what you get, i.e. the computed feasible points satisfy some kind of optimality condition. However, especially for very large optimization problems, it may not be possible to apply such a solution algorithm. In contrast, heuristics have the advantage that they typically find a good

feasible point fast and can often still be applied to large problems. Heuristics can also be combined with solution algorithms, e.g. to determine a good initial point or to decrease the upper bounds in branch&bound or branch&cut faster.

There exists a whole zoo of heuristics and they often have to be tailored to the specific optimization problem in order to perform well. Here, we only discuss some general basic ideas. In Section 6.1 we begin with greedy methods, in Section 6.2 we describe some variants of local search and in Section 6.3 we finally have a look at genetic algorithms.

6.1 Greedy Methods and Matroids

In this section, we consider optimization problems of the form

$$\min_{M \in U} f(M)$$

with the following components:

- G is a finite ground set (Grundmenge) with power set (Potenzmenge) 2^G
- $U \subseteq 2^G$ is the family of feasible subsets of G ,
- $f : 2^G \rightarrow \mathbb{R}$ is a *modular* (*modulare*) objective function, meaning¹

$$f(v) \in \mathbb{R} \text{ for all } v \in G \text{ and}$$

$$f(M) = \sum_{v \in M} f(v) \text{ for all } M \subseteq G.$$

In a *greedy method* (*Greedy-Verfahren*) we try to construct the optimal solution of such a problem iteratively. To do so, we begin with the empty set and in each iteration extend the solution by one additional component, which is chosen *greedy* (*gierig*) with regards to the objective function f . An example for such a greedy method is the nearest neighbor heuristic for TSP, where we always visit the closest new city next. However, we know that the round trips computed with the nearest neighbor heuristic can be arbitrarily worse than the optimal round trip. This shows that greedy methods can fail spectacularly in general. However, we will show that there are classes of optimization problems, where this type of method works.

To this end, we need to assume that the family of feasible subsets U has a specific structure.

Definition 6.1. Consider a finite ground set G .

- A family $U \subseteq 2^G$ is called an *independence system* (*Unabhängigkeitssystem*), if

¹When evaluating the function f in an element $v \in G$, we should write $f(\{v\})$, because the function f is defined for subsets of G . However, so simplify the notation a little bit, we write $f(v)$ instead.

- it contains the empty set $\emptyset$
- and for all sets $M \in U$ and all subsets $N \subseteq M$ we also have $N \in U$.

An independence system is called *nontrivial* (*nichttrivial*), if it contains at least one other element besides $\emptyset$.

- All sets $M \in U$ are called *independent* (*unabhängig*), all other sets $M \in 2^G \setminus U$ are called *dependent* (*abhängig*).
- A maximal (with regards to inclusion) independent set $B \in U$ is called a *basis* (*Basis*) of G .

If the terminology reminds you of linear algebra and vector spaces, you are not wrong.

Example 6.2. (a) Linear independence: Consider a finite set $G = \{v_1, \dots, v_n\} \subseteq \mathbb{R}^m$ of vectors $v_i \in \mathbb{R}^m$ and let U contain all linear independent subsets of these vectors. Then U is an independence system, because $\emptyset \in U$ and every subset of linearly independent vectors is again linearly independent. A basis of G is then a subset of G , which is a basis (in the sense of linear algebra) of the vector space $\text{lin}(v_1, \dots, v_n)$. If we set e.g. $f(v) = -1$ for all $v \in G$, we can search for a maximal independent set, i.e. for a basis.

(b) Binary knapsack problem: Recall the optimization problem

$$\min_x \quad c^T x \quad \text{s.t.} \quad x \in \{0, 1\}^n, \quad w^T x \leq W.$$

Then $U := \{M \subseteq \{1, \dots, n\} \mid \sum_{i \in M} w_i \leq W\}$ is an independence system. If we define the objective function f as $f(i) = c_i$ for all $i = 1, \dots, n$, we can search for an optimal solution of the knapsack problem.

(c) Spanning trees/forests: Consider an undirected graph $G = (V, E)$ and let U be the family of all subsets of $M \subseteq E$ such that the graph $G_M := (V, M)$ is acyclic (*kreisfrei*). Then U is an independence system, because a graph without edges does not contain a cycle and removing edges from an acyclic graph preserves this property. A basis is then a maximal acyclic subgraph, i.e. a spanning forest.

Due to the modular objective function

$$f(M) = \sum_{v \in M} f(v)$$

for all $M \subseteq G$ and the structure of independence systems, Algorithm 6.3 is an intuitive solution approach for optimization problems on independence systems.

Algorithm 6.3. (Greedy Method for optimization problems on independence systems)

Sort the finite ground set $G = \{v_1, \dots, v_n\}$ such that $f(v_1) \leq f(v_2) \leq f(v_n)$.
Initialize $M_{\text{greedy}} = \emptyset$.
for $i = 1, \dots, n$ **do**
 If $M_{\text{greedy}} \cup \{i\} \in U$, update $M_{\text{greedy}} \leftarrow M_{\text{greedy}} \cup \{i\}$.
end for
Output: Return the set M_{greedy} .

Note that this algorithm keeps adding elements to the set M_{greedy} as long as the set remains independent, whether or not this improves the objective function value. It is thus not surprising that Algorithm 6.3 computes a basis of G .

Lemma 6.4. The set M_{greedy} determined by Algorithm 6.3 is a basis of G .

Proof. By construction we have $M_{\text{greedy}} \in U$. Assume that M_{greedy} is not a basis, i.e. that there exists an independent set $M \in U$ with $M_{\text{greedy}} \subsetneq M$. Then there exists a $v_i \in M \setminus M_{\text{greedy}}$. But this element v_i would have been considered in the i -th iteration of the algorithm and would have been added to M_{greedy} , because in this iteration we would have had $M_{\text{greedy}} \cup \{i\} \subseteq M$ and thus $M_{\text{greedy}} \cup \{i\} \in U$. $\square$

Unfortunately, the basis computed by Algorithm 6.3 can be arbitrarily bad with regards to the objective function.

Example 6.5. Consider the binary knapsack problem with n objects, individual costs $c = (-2, -1, \dots, -1)$, weights $w = (n, 1, \dots, 1)$ and weight limit $W = n$. Then the greedy solution is to pick the first object. Since this object uses the whole weight limit, we are then done and have an objective function value of -2 . If instead we skipped the first object, we could pick all other objects, resulting in an objective function value of $-(n-1)$. For $n \rightarrow \infty$ the greedy solution can thus be arbitrarily worse than the optimal solution.

However, for independence system with an additional property we can show that the greedy method actually does work. To this end, we introduce the following concepts.

Definition 6.6. Let G be a finite ground set and $U \subseteq 2^G$ a nontrivial independence system.

- For a set $F \subseteq G$ every (with regards to inclusion) maximal set $B \in U$ with $B \subseteq F$ is called a *basis of F* .

- The independence system U is called a *matroid* (*Matroid*), if for all sets $F \subseteq G$ all bases B of F have the same cardinality $|B|$.
- For a set $F \subseteq G$ we call

$$\begin{aligned} r(F) &:= \max\{|B| \mid B \in U \text{ basis of } F\}, \\ \underline{r}(F) &:= \min\{|B| \mid B \in U \text{ basis of } F\} \end{aligned}$$

the *rank* (*Rang*) of F and the *lower rank* (*unteren Rang*) of F .

- The quotient

$$q(G, U) := \min_{F \subseteq G} \frac{r(F)}{\underline{r}(F)}$$

is called the *rank quotient* (*Rangquotient*) of the independence system U .

By definition we always have $q(G, U) \leq 1$ and $q(G, U) = 1$ if and only if U is a matroid. An alternative characterization of the matroid property is based on an augmentation property similar to the exchange theorem of Steinitz known from linear algebra.

Lemma 6.7. Let G be a finite ground set and $U \subseteq 2^G$ a nontrivial independence system. Then the following are equivalent:

- (a) U is a matroid, i.e. $q(G, U) = 1$.
- (b) For all $M, N \in U$ with $|M| = |N| + 1$ there exists a $v \in M \setminus N$ with $N \cup \{v\} \in U$.
- (c) For all $M, N \in U$ with $|M| > |N|$ there exists a $v \in M \setminus N$ with $N \cup \{v\} \in U$.

Proof. (a) $\implies$ (b) Assume (b) is wrong. Then there exist $M, N \in U$ with $|M| = |N| + 1$, but for all $v \in M \setminus N$ we have $N \cup \{v\} \notin U$. Then we consider the set $F = M \cup N$ with

$$\begin{aligned} r(F) &:= \max\{|B| \mid B \in U \text{ basis of } F = M \cup N\} \geq |M| \\ \underline{r}(F) &:= \min\{|B| \mid B \in U \text{ basis of } F = M \cup N\} \leq |N|. \end{aligned}$$

This implies

$$q(G, U) \leq \frac{r(F)}{\underline{r}(F)} \leq \frac{|N|}{|M|} < 1,$$

in contradiction to (a).

(b) $\implies$ (c) Assume (c) is wrong. Then there exist $M, N \in U$ with $|M| > |N|$, but for all $v \in M \setminus N$ we have $N \cup \{v\} \notin U$. For every subset $\tilde{M} \subseteq M$ with $|\tilde{M}| = |N| + 1$ we then also have $\tilde{M} \in U$. But for all $v \in \tilde{M} \setminus N$ we have $v \in M \setminus N$, which implies $N \cup \{v\} \notin U$ in contradiction to (b).

(c) $\implies$ (a) Assume (a) is wrong. Then there exists a subset $F \subseteq G$ with $\underline{r}(F) < r(F)$, i.e. there exist two bases $N, M \in U$ of F with $|M| > |N|$. According to (c) we can then find a $v \in M$ with $N \cup \{v\} \in U$. Since we then also have $N \cup \{v\} \subseteq F$ this contradicts the choice of N as a basis of F . $\square$

Our next goal is to show that the greedy method applied to optimization problems

$$\min_{M \in U} f(M)$$

with a modular objective function $f(M) = \sum_{v \in M} f(v)$ determines an optimal solution, if the independence set U is a matroid.

But before we do so, let us have a closer look at the objective function: For all $M \in U$ we have $f(M) = \sum_{v \in M} f(v)$. For $v \in M$ with $f(v) > 0$, we know that $M \setminus v$ is also an independent set with smaller objective function value. In a solution of the optimization problem we thus never pick v with $f(v) > 0$ and can therefore eliminate it from the ground set G . In case there exists a $v \in G$ with $v \notin M$ for all $M \in U$, we can also eliminate v from the ground set G , because it does not affect the feasible set or the objective function. Thus, we can focus on the case, where $f(v) \leq 0$ for all $v \in G$.

We often additionally assume that the optimal function value satisfies $f^* < 0$. This is not a significant restriction, because $f(v) \leq 0$ for all $v \in G$ immediately implies $f^* \leq 0$ and in case $f^* = 0$, the empty set would be a solution.

Note, that it is sometimes also possible to reformulate problems, which do not satisfy these conditions, suitably, see Example 6.10.

Theorem 6.8. Let G be a finite ground set and $U \subseteq 2^G$ a nontrivial independence system and $f : 2^G \rightarrow (-\infty, 0]$ a modular objective function. Then the minimal objective function value f^* of f on U and the objective function value g^* determined by the greedy method satisfy

$$f^* \leq g^* \leq f^* \cdot q(G, U)$$

in case $f^* < 0$. Additionally, there exists a modular objective function f such that $g^* = f^* \cdot q(G, U)$.

Proof. Without loss of generality assume $G = \{1, 2, \dots, n\}$ and $f(1) \leq f(2) \leq \dots \leq f(n) \leq 0$. To simplify the formulas, we define $f(n+1) := 0$ and the sets

$$E_i := \{1, \dots, i\} \quad \forall i = 1, \dots, n.$$

For all subsets $F \subseteq G$ we then have

$$\begin{aligned} f(F) &= \sum_{i \in F} f(i) = \sum_{i \in F} \sum_{j=i}^n (f(j) - f(j+1)) \\ &= \sum_{j=1}^n |F \cap E_j| \cdot (f(j) - f(j+1)). \end{aligned}$$

Here we used $f(n+1) := 0$ and switched the order of the two sums.

If $M^* \in U$ is a solution with objective function value f^* , then for all $i = 1, \dots, n$ we have

$$M^* \cap E_i \subseteq M^* \implies M^* \cap E_i \in U \implies |M^* \cap E_i| \leq r(E_i).$$

By construction of the greedy solution M_{greedy} we know that $M_{\text{greedy}} \cap E_i$ is a basis of E_i for all $i = 1, \dots, n$ and thus $|M_{\text{greedy}} \cap E_i| \geq \underline{r}(E_i)$. This implies

$$|M_{\text{greedy}} \cap E_i| \geq \underline{r}(E_i) \cdot \underbrace{\frac{|M^* \cap E_i|}{r(E_i)}}_{\leq 1} = |M^* \cap E_i| \cdot \frac{\underline{r}(E_i)}{r(E_i)} \geq |M^* \cap E_i| \cdot q(G, U).$$

Combining all pieces, we obtain

$$\begin{aligned} g^* &= f(M_{\text{greedy}}) = \sum_{i \in M_{\text{greedy}}} f(i) \\ &= \sum_{j=1}^n |M_{\text{greedy}} \cap E_j| \cdot \underbrace{(f(j) - f(j+1))}_{\leq 0} \\ &\leq \sum_{j=1}^n |M^* \cap E_j| \cdot q(G, U) \cdot (f(j) - f(j+1)) \\ &= q(G, U) \cdot \sum_{j=1}^n |M^* \cap E_j| \cdot (f(j) - f(j+1)) \\ &= q(G, U) \cdot f(M^*) = q(G, U) \cdot f^*. \end{aligned}$$

The second inequality $f^* \leq g^*$ follows directly from the optimality of f^* and the feasibility of M_{greedy} .

To show that there exists a modular objective function with $g^* = f^* \cdot q(G, U)$, let $F \subseteq G$ be a subset with $q(G, U) = \frac{\underline{r}(F)}{r(F)}$. Without loss of generality assume $F = \{1, \dots, m\}$ and let $B = \{1, \dots, p\}$ be a smallest basis of F with $p = |B| = \underline{r}(F)$. Then we define f as $f(i) = -1$ for all $i = 1, \dots, m$ and $f(i) = 0$ for all $i > m$. The greedy method then computes $M_{\text{greedy}} = B$ with $g^* = f(M_{\text{greedy}}) = p = \underline{r}(F)$. And every optimal solution M^* is a largest basis and thus satisfies $f^* = f(M^*) = r(F)$, which implies $g^* = \underline{r}(F) = r(F) \cdot q(G, U) = f^* \cdot q(G, U)$. $\square$

If the optimal function value f^* is negative, the rank quotient $q(G, U)$ thus provides a measure for how bad the greedy solution can be compared to the optimal solution. For matroids we have $q(G, U) = 1$ and the greedy method consequently computes an optimal solution. If instead $q(G, U) < 1$ holds, we have shown that one can find a modular objective function f with

$$g^* = f^* \cdot q(G, U) > f^*.$$

This implies the following result.

Theorem 6.9. Let G be a finite ground set and $U \subseteq 2^G$ a nontrivial independence system. Then U is a matroid if and only if the greedy method computes an optimal solution for all modular objective functions $f : 2^G \rightarrow (-\infty, 0]$ with optimal function value $f^* < 0$.

If U is a matroid, the greedy method thus computes the optimal solution of

$$\min_{M \in U} f(M)$$

for all modular objective functions $f : 2^G \rightarrow (-\infty, 0]$ with optimal function value $f^* < 0$.

Example 6.10. Consider a weighted graph $G = (V, E)$ and define U as all subsets of $G := E$ containing no cycles. Then U is an independence system and the bases of all subsets $F \subseteq G$ correspond to spanning forests² for the subgraphs (V, F) . Since the number of edges in a spanning forest is the same for all spanning forests of (V, F) , the independence system U is a matroid.

Now let $w_e \in \mathbb{R}$ be the weights of the edges e . Then we could define the modular objective function as

$$\tilde{f}(M) = \sum_{e \in M} w_e.$$

However, typically the edge weights w_e are nonnegative and in this case the solution of the optimization problem

$$\min_{M \in U} \tilde{f}(M)$$

would be the empty set with $f^* = 0$. (The greedy algorithm as we defined it in Algorithm 6.3 would still compute a minimum spanning forest, because there we did not enforce that

²The graph (V, F) does not have to be connected and thus does not necessarily contain a spanning tree. But each connected component of (V, F) contains a spanning tree and the disjoint union of trees is called a *forest* (Wald).

Note that the number of edges in the spanning tree of a connected graph is always equal to the number of nodes in the graph minus 1. The number of edges in the spanning forest of a disconnected graph is thus equal to the number of nodes in the graph minus the number of connected components.

only edges e with $f(e) \leq 0$ are added to the set M_{greedy} , but instead focused on computing a base with a small objective function value.)

To fix this problem, let W be the largest occurring edge weight and define the objective function as

$$f(M) = \sum_{e \in M} (w_e - W^* - 1) = \tilde{f}(M) - |M| \cdot (W^* + 1).$$

Then all $f(e) = w_e - W^* - 1$ are negative and thus adding as many as possible to the solution set improves the objective function value. The solution of

$$\min_{M \in U} f(M)$$

is thus a spanning forest with $f^* < 0$.

Note that our modification $w_e - W^* - 1$ did not change the order of the edge weights, i.e. edges with the smallest weights w_e also have the smallest weights $w_e - W^* - 1$. However, our modification does impact sets M with different sizes $|M|$ differently, because in total we subtract $|M| \cdot (W^* + 1)$. Fortunately, all spanning forests have the same number of edges and thus the solution of the modified optimization problem is still a minimum spanning forest.

Thanks to the previous result we know that the greedy method also computes a minimum spanning forest. The resulting algorithm is exactly Kruskal's algorithm.

While e.g. the set of linear independent vectors or acyclic subgraphs are matroids, this assumption is violated for many other important applications such as the knapsack problem, see Example 6.5 or TSP. For these problems the feasible point computed by the greedy method can in general be arbitrarily worse than the optimal solution. However, sometimes it is possible to at least provide a worst-case estimate.

We already developed one such worst-case estimate in Theorem 6.8, where we showed

$$f^* \leq g^* \leq f^* \cdot q(G, U)$$

for modular objective functions satisfying the given assumptions. However, this result is only useful, if the rank quotient $q(G, U)$ is known or can at least be bounded below.

One such bound is based on circuits: If U is a nontrivial independence set, then a minimal (with regards to inclusion) dependent set $K \notin U$ is called a *circuit* (*Kreis*).

Remark 6.11 (optional). A basis is maximal independent set, i.e. when we have a basis B and we add any additional element $v \in G \setminus B$, we obtain a dependent set $B \cup \{v\}$. However, the opposite direction is obviously not true, i.e. if we have a dependent set M and eliminate one element $v \in M$, the result $M \setminus \{v\}$ does not have to be a basis, in fact does not even have to be independent.

Similarly, a circuit is a minimal dependent set, i.e. when we have a circuit C and we eliminate any element $v \in C$, we obtain an independent set $C \setminus \{v\}$. And again, the

opposite direction is not necessarily true. If we have an independent set M and we add one element $v \in G \setminus M$, the resulting set $M \cup \{v\}$ can still be independent. If we have a basis B and add one element $v \in G \setminus B$, the resulting set $B \cup \{v\}$ is dependent, but does not have to be a circuit.

As an example for the last point, consider a knapsack problem with four items, weights $(w_1, w_2, w_3, w_4) = (1, 1, 1, 3)$ and weight limit $W = 4$. Then $M = \{1, 2, 3\}$ is a maximal feasible load, i.e. a basis. The set $M \cup \{4\} = \{1, 2, 3, 4\}$ is a infeasible load, i.e. dependent. However, it is not a circuit, because its subsets $\{2, 3, 4\}$, $\{1, 3, 4\}$ and $\{1, 2, 4\}$ are still infeasible loads, i.e. dependent. In contrast, these three subsets are circuits, because they are dependent/infeasible and removing a arbitrary element causes them to become independent/feasible.

If it is possible to bound the number of circuits contained in sets $M \cup \{v\}$ with $M \in U$, then this bound can be used to compute an estimate for the rank quotient.

Theorem 6.12. (optional) Let G be a finite ground set and $U \subseteq 2^G$ a nontrivial independence system. If for all $M \in U$ and all $v \in G$ the set $M \cup \{v\}$ contains at most p different circuits, then

$$q(G, U) \geq \frac{1}{p}.$$

In this case, we know for $f^* < 0$ that

$$f^* \leq g^* \leq f^* \cdot q(G, U) \leq \frac{f^*}{p}.$$

6.2 Local Search

Consider an optimization problem

$$\min_x f(x) \quad \text{s.t.} \quad x \in X$$

with feasible set X . For every feasible point $x \in X$ we define a (feasible) *neighborhood* (*Nachbarschaft*) $N(x) \subseteq X$. To simplify subsequent discussions, we only consider neighborhoods $N(x)$ with the property $x \notin N(x)$.

Example 6.13. (a) In case $X \subseteq \mathbb{R}^n$, an example for such a neighborhood would be

$$N(x) := \{y \in X \setminus \{x\} \mid \|x - y\| \leq d\}$$

with some radius $r > 0$ and some norm on $\mathbb{R}^n$. However, for discrete optimization problems, this neighborhood is empty, if the radius r is chosen too small.

- (b) Consider a binary optimization problem with feasible points $x \in \{0, 1\}^n$. Then we can use the Hamming distance (Hamming Abstand)

$$\Delta(x, y) = |\{i = 1, \dots, n \mid x_i \neq y_i\}|$$

to define a neighborhood as

$$N(x) := \{y \in X \setminus \{x\} \mid \Delta(x, y) \leq d\}$$

with some maximum distance $d > 0$.

For integer optimization problems, we can use a similar construction, but in some situations might want to not only limit the number of differing components $x_i \neq y_i$ but also their distance $|x_i - y_i|$.

The basic idea of *local search* (*lokale Suche*) is then described in Algorithm 6.14.

Algorithm 6.14. (Basic Local Search)

Choose a feasible initial point $x^0 \in X$ and set $k = 0$.

while there exists a neighbor $\tilde{x} \in N(x^k)$ with $f(\tilde{x}) < f(x^k)$ **do**

 Set $x^{k+1} = \tilde{x}$ and $k \leftarrow k + 1$.

end while

Output: Return x^k .

This method obviously depends heavily on the definition of the neighborhood $N(x)$: The returned point x^* is a local optimum with regards to its neighborhood $N(x^*)$ and thus larger neighborhoods lead to a stronger optimality conditions. On the other hand, we have to determine the best point $\tilde{x}$ in $N(x^k)$, which requires more work for larger neighborhoods. So it is important to find a good balance between the expressiveness of the neighborhood and its searchability.

Good choices of the neighborhood often depend on the structure of the feasible set and possibly the objective function. For TSP, we could for example define the neighborhood of a round trip as all round trips, where two cities are switched.

Taboo Search An downside of local search is that it is easy to get stuck in a local solution, i.e. a point x , which is optimal with regards to its neighborhood $N(x)$, but not globally optimal. To fix this problem, we can always accept the best point $\tilde{x}$ in the neighborhood $N(x^k)$ as the next iterate, even if it does not improve the current point, i.e. if $f(\tilde{x}) \geq f(x^k)$.

However, this modification causes a new problem: If $\tilde{x} \in N(x^k)$, then we often also have $x^k \in N(\tilde{x})$. Thus, if x^k is a local optimum, we would return from $\tilde{x}$ to x^k in the next

iteration and start to cycle between these two points. To avoid this, we can introduce a *taboo list* (*Tabu-Liste*) T , which contains feasible points to which we do not want to return. Theoretically, we should save all prior iterates in T . If this is not possible due to storage limitations, taboo lists with a limited size can also be used.

Due to the modification, the current iterate x^k is not necessarily the best known point anymore, so we have to track the best point x^{opt} separately. A basic version of the resulting *taboo search* (*Tabu-Suche*) can be found in Algorithm 6.15.

Algorithm 6.15. (Basic Taboo Search)

```

Choose a feasible initial point  $x^0 \in X$  and initialize  $T_0 = \{x^0\}$ ,  $x^{\text{opt}} = x^0$  and set  $k = 0$ .
while termination condition not satisfied do
    Determine  $\tilde{x} \in \operatorname{argmin}\{f(y) \mid y \in N(x^k) \setminus T_k\}$ .
    if  $f(\tilde{x}) < f(x^{\text{opt}})$  then
        Update  $x^{\text{opt}} \leftarrow \tilde{x}$ .
    end if
    Update  $x^{k+1} = \tilde{x}$  and  $T_{k+1} \leftarrow T_k \cup \{x^{k+1}\}$ , set  $k \leftarrow k + 1$ .
    If the taboo list  $T_k$  is getting too big, delete suitable elements.
end while
Output: Return the best found solution  $x^{\text{opt}}$ .

```

Possible termination conditions are e.g.

- a maximum number of iterations,
- a maximum computation time (or other type of resource constraint),
- a maximum number of iterations without improvement of x^{opt} ,
- $|f(x^{\text{opt}}) - f^*| \leq \varepsilon$, if the optimal function value f^* is known.

Simulated Annealing In contrast to local search and taboo search in *simulated annealing* (*Simulated Annealing*) the next iterate is a randomly chosen point $\tilde{x} \in N(x^k)$. Depending on the size of the neighborhood $N(x^k)$, this can be easier to realize than computing the best neighboring point. Better points $\tilde{x}$ with $f(\tilde{x}) < f(x^k)$ are always accepted. But worse points are also accepted with a probability, which decreases over the course of the algorithm. Accepting worse points as the next iterate is again intended to prevent the algorithm from getting stuck in a local solution. And the decreasing probability of accepting a worse point is intended to ensure that the algorithm stabilizes in a “good area”. The name simulated annealing is motivated by the the cooling process of metal, which for lower temperatures also becomes more rigid.

A basic version of simulated annealing can be found in Algorithm 6.16. Here, the term $\text{random}(0, 1)$ means that we draw a random number $\alpha \in [0, 1]$, which should be uniformly distributed. In case $f(x^k) < f(\tilde{x})$, the exponent $(f(x^k) - f(\tilde{x}))/T_k$ of the exponential function is negative and thus the exponential function attains a value in $(0, 1)$. If this value of the exponential function is at least close to 1, which is the case if $f(x^k) \approx f(\tilde{x})$ or if T_k is large, then $\tilde{x}$ is accepted with a higher probability.

Algorithm 6.16. (Basic Simulated Annealing)

Choose an initial point $x^0 \in X$, an initial temperature $T_0 > 0$, initialize $x^{\text{opt}} = x^0$ and $k = 0$.

while $T_k > 0$ **do**

 Choose a random point $\tilde{x} \in N(x^k)$.

if $f(\tilde{x}) < f(x^{\text{opt}})$ **then**

 Update $x^{\text{opt}} \leftarrow \tilde{x}$.

end if

if $f(x^k) \geq f(\tilde{x})$ or $\text{random}(0, 1) < e^{(f(x^k) - f(\tilde{x}))/T_k}$ **then**

 Update $x^{k+1} = \tilde{x}$.

end if

 Choose a new temperature $T_{k+1} \leq T_k$ and set $k \leftarrow k + 1$.

end while

Output: Return the best found solution x^{opt} .

In this basic version, we assume that the temperature is decreased to 0 after a finite number of iterations and then we terminate the algorithm. We can also use other termination conditions here, e.g. a maximum number of iterations or a maximum number of iterations without improvement in x^{opt} .

6.3 Genetic Algorithms

The final approach we want to have a look at a heuristic, which is motivated by ideas from genetics. In contrast to the previous methods, here we do not consider one single point per iteration, but instead a *population* (*Population*) of m points $x_1, \dots, x_m$. In each iteration, some pairs of such points x_i, x_j are selected as *parents* (*Eltern*) and used to generate *children* (*Kinder*) via *crossover* (*kreuzen*). The new child points typically inherit some characteristics of their parents. Afterwards, some points in the current population are *mutated* (*mutiert*), i.e. randomly changed. Finally, some kind of *fitness* (*Fitness*) measure f is used to *select* (*selektieren*), which of the points in the population survive into the next iteration.

If we consider a constrained optimization problem

$$\min_x f(x) \quad \text{s.t.} \quad x \in X$$

with feasible set X , then we have to consider the following complication: Even if the entire initial population $x_1, \dots, x_m$ is feasible, the crossover operation and the mutation can generate infeasible points. Some possible approaches to handle this situation are:

- For certain types of constraints. e.g. for box constraints $l \leq x \leq u$, it is easy to define crossover and mutation such that they result in feasible points.
- Otherwise, if we obtain an infeasible point due to crossover or mutation, we can try to repair the infeasibility. However, this should be done in such a way that the resulting feasible points are uniformly distributed in the feasible set and do not favour certain areas.
- Alternatively, we can also accept infeasible points, but incorporate infeasibility in the fitness measure analogously to a penalty term in continuous optimization.

For simplicities sake, Algorithm 6.17 does not explicitly take the feasibility issue into account.

Algorithm 6.17. (Basic Genetic Algorithm)

Choose an initial population $x_1, \dots, x_m$ and initialize $x^{\text{opt}} = \operatorname{argmin}\{f(x_i) \mid i = 1, \dots, m\}$.

while termination condition not satisfied **do**

Parents: Choose several pairs of points in $x_1, \dots, x_m$.

Crossover: Use the pairs of parent points to generate one or more new child points $y_1, y_2, \dots$ each.

Mutation: Change some of the (old/new) points randomly.

Selection: Use a fitness measure f to choose the new population $x_1, \dots, x_m$.

Update x^{opt} to the best feasible point, that occurred during the iteration.

end while

Output: Return x^{opt} .

When choosing parents, it can be beneficial to choose fitter points with a higher probability. In case $f(x_i) < 0$ for all x_i , we could e.g. use $\frac{f(x_i)}{\sum_{j=1}^m f(x_j)} \in (0, 1)$ as probability to choose the point x_i as a parent. But we should always also choose less fit points as parents with a positive probability to prevent the population from contracting to one (possibly only locally optimal) point too fast.

During crossover, we can try to preserve “good” parts of each parent point and combine them to an even better new point. Or we can just randomly combine information from the two parent points to generate a new child point.

During mutation, we can randomly change some part of a point in order to incorporate new information/possibilities into the population.

During selection, the fitter points should survive with a higher probability. However, less fit points should also survive with a positive probability in order to prevent a premature contraction of the population. The goal here is to find a balance between *survival of the fittest* (*Überleben der Stärksten*), which drives the population towards better solutions, and *diversity of species* (*Artenvielfalt*), which keeps options open.

Some possible termination conditions are:

- maximum number of iterations/populations,
- maximum number of iterations without improvement in x^{opt} ,
- $|f(x^{\text{opt}}) - f^*| \leq \varepsilon$, if the optimal function value f^* is known,
- limit on the computation time.

Example 6.18. Consider the knapsack problem

$$\min c^T x \quad \text{s.t.} \quad x \in \{0, 1\}^n, \quad w^T x \leq W$$

with costs $c < 0$, weights $w > 0$ and a weight limit $W > 0$. Then we can represent each load with a vector $x = \{0, 1\}^n$ and generate an initial population by choosing random vectors $x_1, \dots, x_m \in \{0, 1\}^n$. As a fitness function, we can use

$$f(x) := \begin{cases} c^T x - \varepsilon & \text{if } w^T x \leq W, \\ -\varepsilon & \text{else} \end{cases}$$

with $\varepsilon > 0$ small used to ensure that all objective function values remain negative and can thus be used to compute the probabilities below.

Then we can use

$$p(x_i) := \frac{f(x_i)}{\sum_{j=1}^n f(x_j)} \in (0, 1)$$

as probability for a point x_i to be chosen as a parent node.

For crossover, we can take the two parent points and select a (possibly random) crossover point k such that the child point coincides with the first parent point in the first k components and with the second parent point in the last $n - k$ components.

To mutate a point x_i , we can choose a random component of the vector x_i and change it to the other possible value.

Finally, we can use a fitness function analogous to p again to decide, which n points out of the old population and the new child points survive into the next iteration.

6.4 Exercises

Bibliography

- [1] Tobias Achterberg, Thorsten Koch, and Alexander Martin. Branching rules revisited. *Operations Research Letters*, 33:42–54, 2005.
- [2] Sven Oliver Krumke and Hartmut Noltemeier. *Graphentheoretische Konzepte und Algorithmen*. Springer, 2009.
- [3] George L Nemhauser and Laurence A Wolsey. *Integer Programming and Combinatorial Optimization*, volume 191. Springer, 1988.
- [4] Laurence A Wolsey. *Integer Programming*. John Wiley & Sons, 2020.